



Apollo: a flexible, powerful and customisable
freeware package for choice model estimation and
application

version 0.3.8

User manual

www.ApolloChoiceModelling.com

Stephane Hess & David Palma
Choice Modelling Centre
University of Leeds

Release date: 17 May 2026
Manual updated: 29 June 2026

Apollo is licensed under GNU GENERAL PUBLIC LICENSE v2 (GPL-2)
<https://cran.r-project.org/web/licenses/GPL-2>.

Apollo is provided free of charge and comes WITHOUT ANY WARRANTY of any kind. In no event will the authors or their employers be liable to any party for any damages resulting from any use of Apollo.

This manual is licensed under a Creative Commons Attribution-NoDerivatives 4.0 International License
<https://creativecommons.org/licenses/by-nd/4.0/>.



While the Apollo package is the result of many years of development, the core of this work was carried out under the umbrella of the European Research Council (ERC) funded consolidator grant 615596-DECISIONS, followed by the ERC proof of concept grant 875692-APOLLO, and the ERC advanced grant 101020940-SYNERGY.

We are grateful to the many colleagues who provided suggestions and/or tested the code extensively, including Michel Bierlaire, Chiara Calastri, Romain Crasted dit Sourd, Andrew Daly, Jeff Dumont, Joe Molloy and Basil Schmid. We would like to especially thank David Bunch for his work on adapting the BGW algorithm for use with Apollo, Thijs Dekker for his contributions to precursors of Apollo and his guidance on the EM algorithm, Thomas Hancock for his implementation of Decision Field Theory and Annesha Enam for her contributions on MDCEV without an outside good. We are also grateful to Kay Axhausen for making the Swiss public transport route choice dataset available.

Contents

1	Introduction	8
2	Installing <i>Apollo</i>, loading the library and running the code	12
3	Data format and datasets used for examples	14
3.1	RP-SP mode choice dataset: <code>apollo_modeChoiceData</code>	15
3.2	SP route choice dataset: <code>apollo_swissRouteChoiceData</code>	15
3.3	Health attitudes SP: <code>apollo_drugChoiceData</code>	16
3.4	Time use data: <code>apollo_timeUseData</code>	16
4	General code structure and components: illustration for Logit	17
4.1	Initialising the code	19
4.2	Reading and processing the data	21
4.3	Model parameters	21
4.4	Validation and preparing user inputs	24
4.5	Likelihood component: the <code>apollo_probabilities</code> function	25
4.5.1	Initialisation	26
4.5.2	Model definition	26
4.5.3	Function output	30
4.6	Estimation	31
4.7	Reporting and saving results	36
4.8	Extension: iterative coding of utilities	41
5	Other model components	44
5.1	Other RUM-consistent discrete choice models	44
5.1.1	Nested Logit	44
5.1.2	Cross-nested Logit	48
5.2	Non-RUM decision rules for discrete choice	51
5.2.1	Random regret minimisation (RRM)	51
5.2.2	Decision field theory (DFT)	55
5.3	Models for aggregate/shares data or uncertainty in the dependent variable	59
5.4	Models for ranking, rating and continuous dependent variables	60
5.4.1	Exploded Logit	61
5.4.2	Best-worst choice model	64
5.4.3	Ordered Logit and Ordered Probit	64
5.4.4	Normally distributed continuous variables	67
5.5	Discrete-continuous models	68
5.5.1	Multiple Discrete Continuous Extreme Value (MDCEV) model	68
5.5.2	Multiple Discrete Continuous Nested Extreme Value (MDCNEV) model	70

5.6	Extended Multiple Discrete Continuous (eMDC) model	74
5.7	Adding new model types	78
6	Incorporating random heterogeneity	79
6.1	Continuous random coefficients	79
6.1.1	Introduction	79
6.1.2	Example model specification	82
6.1.3	Implementation	83
6.1.4	Estimation	88
6.2	Error components	89
6.3	Discrete mixtures and Latent Class	91
6.4	Combining Latent Class with continuous random heterogeneity	96
6.5	Multi-threading capabilities	96
7	Joint estimation of multiple model components	101
7.1	Joint estimation on RP and SP data	102
7.2	Joint best-worst model	105
7.3	Hybrid choice model	105
8	Alternative estimation approaches	111
8.1	Bayesian estimation	111
8.2	Expectation-maximisation (EM) algorithm	117
8.2.1	EM algorithm for LC model	117
8.2.2	MMNL model with full covariance matrix for random coefficients	122
9	Pre and post-estimation capabilities	126
9.1	Pre-estimation analysis of choices	126
9.2	Reading in a previously saved model object	129
9.3	Starting value search	129
9.4	Calculating model fit with given parameter values	130
9.5	Bootstrap covariance estimation	132
9.6	Likelihood ratio tests against other models	135
9.7	Ben-Akiva & Swait test	136
9.8	Utilities at specified parameter values	136
9.9	Model predictions	137
9.10	Market share recovery for subgroups of data	142
9.11	Comparison of model fit across subgroups of data	144
9.12	Out of sample fit (Cross validation)	145
9.13	Delta method for functions of model parameters	148
9.14	Unconditionals for random parameters	149
9.15	Conditionals for random coefficients	151
9.15.1	Continuous random coefficients	151
9.15.2	Latent class	152
9.16	Summary of results for multiple models	156
10	Debugging	157

11 Frequently asked questions	160
11.1 General	160
11.2 Installation and updating of <i>Apollo</i>	161
11.3 Data	161
11.4 Model specification	162
11.5 Errors and failures during estimation	164
11.6 Model results	166
A <i>Apollo</i> versions: timeline, changes and backwards compatibility	168
B Data dictionaries	201
C Index of example files	205
D Detailed description of model object	210
Bibliography	221
Index: <i>Apollo</i> syntax	222

List of Figures

4.1	General structure of an <i>Apollo</i> model file	18
4.2	Code initialisation	19
4.3	Loading data, selecting a subset and creating an additional variable	21
4.4	Setting names and starting values for model parameters, and fixing some parameters to their starting values	23
4.5	Using <code>apollo_readBeta</code> to load results from an earlier model as starting values	24
4.6	Running <code>apollo_validateInputs</code>	25
4.7	The <code>apollo_probabilities</code> function: example for MNL model	27
4.8	Running <code>apollo_estimate</code> on MNL model	34
4.9	On screen output obtained using <code>apollo_modelOutput</code> for MNL model	39
4.10	On screen output obtained using <code>print(model)</code> for MNL model	41
4.11	On screen output obtained using <code>summary(model)</code> for MNL model	42
4.12	Defining models for large choice sets (<code>MNL_iterative_coding.r</code>)	43
5.1	Nested Logit implementation (extract)	46
5.2	Nested Logit tree structure after estimation	48
5.3	Cross-nested Logit implementation (extract)	51
5.4	Cross-nested Logit structure after estimation	51
5.5	Implementation of random regret MNL model	54
5.6	An example of a decision-maker stopping upon reaching either an internal or external threshold	55
5.7	DFT implementation	58
5.8	FMNL implementation in <i>Apollo</i>	61
5.9	Exploded Logit implementation	63
5.10	Exploded Logit for B-W data	65
5.11	MDCEV implementation without outside good	71
5.12	MDCEV implementation with an outside good	72
5.13	MDCNEV implementation and call to <code>apollo_estimate</code> using scaling	75
6.1	Setting the number of cores	84
6.2	Defining settings for generation of draws	84
6.3	The <code>apollo_randCoeff</code> function	86
6.4	The <code>apollo_probabilities</code> function for a MMNL model	87
6.5	Running <code>apollo_estimate</code> for MMNL using 3 cores	88
6.6	Using error components for heteroskedasticity	89
6.7	Using error components for a pseudo-panel effect	90
6.8	The <code>apollo_lcPars</code> function	92
6.9	Implementing choice probabilities for Latent Class	94

6.10	The <code>apollo_randCoeff</code> and <code>apollo_lcPars</code> functions for a Latent Class model with continuous random heterogeneity	97
6.11	Implementing choice probabilities for Latent Class with continuous random heterogeneity	98
6.12	Running <code>apollo_speedTest</code>	100
7.1	Joint RP-SP model on mode choice data	103
7.2	On screen output for RP-SP model	104
7.3	Hybrid choice model: draws and latent variable	107
7.4	Hybrid choice model with ordered measurement model: defining probabilities	108
7.5	Hybrid choice model with continuous measurement model: zero-centering indicators and defining probabilities	109
8.1	Bayesian estimation in <i>Apollo</i> : model settings	112
8.2	Bayesian estimation in <i>Apollo</i> : estimation process	114
8.3	Bayesian estimation in <i>Apollo</i> : estimation process (parameter chains)	115
8.4	Bayesian estimation in <i>Apollo</i> : output (extracts)	116
8.5	EM algorithm for Latent Class: setup	120
8.6	EM algorithm for Latent Class: estimation	121
8.7	EM algorithm for Mixed Logit: setup	124
8.8	EM algorithm for Mixed Logit: estimation	125
9.1	Running <code>apollo_choiceAnalysis</code> (syntax and excerpt of output)	127
9.2	Running <code>apollo_choiceAnalysis</code> on unlabelled data (syntax and excerpt of output)	128
9.3	Running <code>apollo_searchStart</code>	131
9.4	Running <code>apollo_llCalc</code>	132
9.5	Running <code>apollo_bootstrap</code>	134
9.6	Running <code>apollo_lrTest</code>	135
9.7	Running <code>apollo_basTest</code>	136
9.8	Running <code>apollo_probabilities</code> with <code>functionality="utilities"</code>	137
9.9	Running <code>apollo_prediction</code> : base prediction	140
9.10	Running <code>apollo_prediction</code> : prediction of impact of price change	141
9.11	Running <code>apollo_prediction</code> with a vector of parameters	142
9.12	Running <code>apollo_prediction</code> with a vector of parameters	142
9.13	Running <code>apollo_sharesTest</code>	144
9.14	Running <code>apollo_fitsTest</code>	145
9.15	Running <code>apollo_outOfSample</code>	147
9.16	Calculating prediction RMSE using <code>apollo_outOfSample</code>	148
9.17	Running <code>apollo_deltaMethod</code>	149
9.18	Running <code>apollo_unconditionals</code> and <code>apollo_conditionals</code>	153
9.19	Running <code>apollo_unconditionals</code> and <code>apollo_conditionals</code> for latent class	155
10.1	Example of failure during model estimation	158
10.2	Debugging step 1: testing with <code>functionality="output"</code>	158
10.3	Debugging step 1, part 2: likelihood for individual model components	159
10.4	Debugging step 2: analysing choices for respondents with zero likelihood value	159

List of Tables

B.1	Data dictionary for <code>apollo_modeChoiceData</code>	201
B.2	Data dictionary for <code>apollo_swissRouteChoiceData</code>	202
B.3	Data dictionary for <code>apollo_drugChoiceData</code>	203
B.4	Data dictionary for <code>apollo_timeUseData</code>	204
C.1	Index of example files	206
D.1	Elements inside a model object estimated by <i>Apollo</i>	211

1

Introduction

Choice modelling techniques have been used across different disciplines for over four decades (see [McFadden 2000](#) for a retrospective and [Hess and Daly 2024](#) for recent contributions across fields). For the majority of that time, the number of users of especially the most advanced models was rather small, and similarly, a small number of software packages was used by this community. In the last two decades, the pool of users of choice models has expanded dramatically, in terms of their number as well as the breadth of disciplines covered. At the same time, we have seen the development of new modelling approaches, and gains in computer performance and software availability have given a growing and broader group of users access to ever more advanced models.

These developments have also seen a certain fragmentation of the community in terms of software, which in part runs along discipline lines¹. Notwithstanding the most advanced users who develop their own code (sometimes for their own models), there is first a split between the users of commercial software and those using freeware tools. Commercial packages are generally computationally more powerful but may have more limitations in terms of available model structures or the possibility for customisation. On the other hand, freeware options can have limitations in terms of performance and user friendliness but may benefit from more frequent development to accommodate new model structures.

A further key differentiation between packages is the link between user inputs/interface and the actual underlying methodology. Many existing packages, both freeware and commercial, are black box tools where the user has little or no knowledge of what goes on “under the hood”. While this has made advanced models accessible to a broader group of users, a disconnect between theory and software not only increases the risk of misinterpretations and misspecifications, but can also hide relevant nuances of the modelling process and mistakenly give the impression that choice models are “easy tools”. On the other hand, software that relies on users to code all components from scratch arguably imposes too high a bar in terms of access.

Software also almost exclusively allows the use of only either classical estimation tech-

¹We intentionally do not refer to specific packages, so as not to risk any misrepresentations but also given the growing number of freeware tools, some of which we might not be aware of.

niques or Bayesian techniques. This fragmentation again runs largely in parallel with discipline boundaries and has only served to further contribute to the lack of interaction/dialogue between the classical and Bayesian communities.

A final difference arises in terms of software environment. While commercial software usually provides a custom user interface, freeware options in general (though not exclusively) rely on existing statistical or econometric software and are made available as packages within these. The latter at times means that *freeware* packages are not really free to use (if the host software is not), while there are also cases of software being accessible only in a single operating system (e.g. Windows or Linux/MacOS, not across systems).

The above points served in large part as the motivation for the development of *Apollo*. Our aims were:

Free access: *Apollo* is a completely free package which does not rely on commercial statistical software as a host environment.

Big community: *Apollo* relies on R (R Core Team, 2017), which is very widely used across disciplines and works well across different operating systems.

Transparent, yet accessible: *Apollo* is neither a blackbox nor does it require expert econometric skills. The user can see as much or as little detail of the underlying methodology as desired, but the link between inputs and outputs remains.

Ease of use: *Apollo* combines easy to use R functions with new intuitive functions without unnecessary jargon or complexity.

Modular nature: *Apollo* uses the same code structure independently of whether the simplest Multinomial Logit model is to be estimated, or a complex structure using random coefficients and combining multiple model components.

Fully customisable: *Apollo* provides functions for many well known models but the user is able to add new structures and still make use of the overall code framework.

Beyond discrete choice: *Apollo* incorporates functions not just for commonly used discrete choice models but also for fractional choice models and a family of models that looks jointly at discrete and continuous choices.

Novel structures: *Apollo* goes beyond standard choice models by incorporating the ability to estimate Decision Field Theory (DFT) models, a popular accumulator model from mathematical psychology. In addition, users can code their own models.

Classical and Bayesian: *Apollo* does not restrict the user to either classical or Bayesian estimation but easily allows changing from one to the other.

Easy multi-threading: *Apollo* allows users to split the computational work across multiple processors without making changes to the model code.

Not limited to estimation: *Apollo* provides a number of pre and post-estimation tools, including diagnostics as well as prediction/forecasting capabilities and posterior analysis of model estimates.

While *Apollo* is easy to use, we also remain of the opinion that users of choice modelling software should understand the actual process that happens during estimation. For this reason, the user needs to explicitly include or exclude calls to specific functions that are model and dataset specific. For example, in the case of repeated choice data, the user needs to include a call to a function that takes the product across choices for the same person (`apollo_panelProd`). Or in the case of a Mixed Logit model, the user needs to include a call to a function that averages across draws (`apollo_avgInterDraws` and/or `apollo_avgIntraDraws`). If calls to these functions are missing when needed, or if a user makes a call to a function that should not be used in the specific model, the code will fail,

and provide the user with feedback about why this happened. This is in our view much better than a situation where the software permits users to make mistakes and fixes them behind the scenes.

Users of *Apollo* are asked to acknowledge the use of the software by citing the academic paper:

- Hess, S. & Palma, D. (2019), *Apollo*: a flexible, powerful and customisable freeware package for choice model estimation and application, Journal of Choice Modelling, 32, 100170, doi.org/10.1016/j.jocm.2019.100170

and noting the version of *Apollo* used in their work.

Apollo is the culmination of many years of development of individual choice modelling routines, starting with code developed by Hess while at Imperial College (cf. Hess, 2005) using Ox (Doornik, 2001). This code was gradually transitioned to R at the University of Leeds, with substantial further developments once Palma joined the team in Leeds, bringing with him ideas developed at Pontificia Universidad Católica de Chile (cf. Palma, 2016). No code is an island, and we have been inspired especially by ALogit (ALogit, 2016) and Biogeme (Bierlaire, 2003), and *Apollo* mirrors at least some of their features.

This manual presents an overview of the capabilities of the *Apollo* package and serves as a user manual. It is accompanied online at www.ApolloChoiceModelling.com by numerous example files (some of which are used in this manual) and a number of free to use datasets. Users are also encouraged to visit the online help forum where numerous questions on specification have already been answered. In addition to the detailed information in this manual, users are strongly encouraged to make use of the help files provides for specific functions directly in R, using e.g.

`?apollo_mnl`

for help on the `apollo_mnl` function.

In line with our earlier point about other software, this manual does not include any comparisons with other packages, in terms of capabilities or speed. The code has been widely tested to ensure accuracy. In our view, any speed comparison offers little practical benefit. For simple models, there is a clear advantage for highly specialised code, while, for complex models, any benchmarking is impacted substantially by the specific implementation and degree of optimisation used.

In the remainder of this manual, we do not provide details on common R functions and syntax used in the code, or how to run R code, and the reader is instead referred to R Core Team (2017). For the syntax shown in this manual, it is just worth noting that in R, a line (or part of a line) starting with one or more `#` characters is a comment. We tend to use a single `#` for optional lines that a user can comment in or out, and `###` for actual comments. In addition, two other points are worth raising.

- In complex models, the R syntax file for *Apollo* can become quite large, and a user may wish to split this into separate files, e.g. one for loading and processing the data, one for the actual model definition, etc, and then have a master file which calls the individual files (using `source`).
- For any predefined functions, the order of arguments passed to the function should be kept in the order specified in this manual².

²For example, if a function is defined to take two inputs, namely `dependent` and `explanatory`,

Specific naming conventions have been adopted for functions and inputs to functions. All functions within the code start with the prefix `apollo_`. This is then followed by the “name” of the actual function in a single word, where any new part of the name starts with a capital letter, for example `apollo_modelOutput`. The prefix `apollo_` is also used for a number of key non-function objects in the code, namely:

- the user defined settings `apollo_control`, `apollo_HB` and `apollo_draws`;
- the list of parameters `apollo_beta` and fixed parameters `apollo_fixed`; and
- the automatically generated combined inputs variable `apollo_inputs`.

The functions in *Apollo* take numerous inputs and for ease of programming, these are often combined into a *list* object. The naming convention used for these is to have the name of the function (without the `apollo_` prefix) followed by `_settings`, for example in `modelOutput_settings`. Finally, individual variables/settings do not have a prefix and again use the convention of capitalising the first letter of any new word except for the start, for example in `printDiagnostics`.

Before we proceed, a brief explanation is needed as to our choice of the name *Apollo*. Several existing packages refer to specific models in their name (e.g. *ALogit*, *NLogit*) which is not applicable in our case given the wider set of models we cover. We failed miserably in our efforts to come up with an imaginative acronym like *Biogeme*, and so went back to Greek mythology. The obvious choice would have been *Cassandra*, with her gift of prophecy and the curse that nobody listened to her (a bit like choice modellers trying to sell their ideas to policy makers). Alas, the name has already been used for a large database package, so we resorted to *Apollo*, the Greek god of prophecy who gave this gift to *Cassandra* in the first place. And, as a student more versed in Greek mythology than ourselves told us, it was *Apollo* who slayed *Python*.

The remainder of this manual is organised as follows. The following chapter talks about installation, before Chapter 3 introduce a number of datasets used throughout the manual. Chapter 4 provides an in-depth introduction to the code structure, using the example of a simple Multinomial Logit model. This is followed in Chapter 5 by an overview of other available model components, and a description of how the user can add his/her own models. Chapter 6 covers random heterogeneity, both discrete and continuous while Chapter 7 discusses joint estimation of multiple model components, with a focus on hybrid choice models. Alternative estimation approaches (Bayesian and Expectation-Maximisation) are covered in Chapter 8 with additional pre and post-estimation capabilities discussed in Chapter 9. Chapter 10 looks at debugging an *Apollo* model that fails in estimation. Finally, Chapter 11 addresses a set of frequently asked questions.

A number of appendices are also included. Appendix A summarises changes across different versions of *Apollo*. Appendix B contains data dictionaries, Appendix C a list of the example files and Appendix D an overview of the contents of model objects as produced by *Apollo*.

e.g. `model_prob(dependent,explanatory)`, and the user wants to use `choice` and `utility` as the inputs, then the function can be called as `model_prob(choice,utility)` but not as `model_prob(utility,choice)`. The latter change in order is only possible if the function is called explicitly as `model_prob(explanatory=utility,dependent=choice)`, which is the same as `model_prob(dependent=choice,explanatory=utility)`.

2

Installing *Apollo*, loading the library and running the code

Apollo runs in R, with a minimum R version of 4.3.0. *Apollo* can be installed in two ways. If an internet connection is available, the easiest way to install it is to type the following command into the R console. This will install the latest release from CRAN, and also install all dependencies, i.e. other routines used by the *Apollo* package. When using the CRAN release, we recommend installation using binaries rather than building from source.

```
install.packages("apollo")
```

The second way is to install it from a file, which is most relevant for development versions. A file containing the source code can be obtained at www.ApolloChoiceModelling.com. Then, the following command must be typed into the R console.

```
install.packages("../v0.3.5.tar.gz", repos = NULL, type = "source")
```

where `../apollo_v0.3.5.tar.gz` must be replaced by the correct path to the file in the user's computer, using the version that was downloaded. This will not automatically install dependencies.

The installation of the package does not need to be repeated every time R is started, nor every time a model is to be estimated. Instead, it only needs to be done once for each new release of *Apollo* (unless R itself is updated, in which case the installation must be repeated).

Every time users want to estimate a model, they need to load *Apollo* into memory. This can be achieved by simply running the following line of code in R, or by including it in the source file of each model, prior to running any *Apollo* functions.

```
library(apollo)
```

Users are encouraged to check for updated versions of the package every few months. Updates, when available, can be acquired by simply re-installing the package, i.e. running

```
install.packages("apollo")
```

Installation from CRAN will install the latest release. Previous releases will be available from the software website, where users also have access to versions with new features that are under development prior to a full release. These versions need to be compiled locally, and users require Rtools for this purpose.

Most users will run R from a shell such as RStudio ([RStudio Team, 2015](#)). A full *Apollo* model file, or any other R script, can also be run from the command line, without accessing R directly. This can be useful when running many scripts unattended, or when submitting jobs to a computer cluster. The command to do this changes depending on the operation system and the local directory structure. In Linux, the command is as follows: `R CMD BATCH model.R`. In Windows, the command is for example as follows: `"C:\Program Files\R\R-4.4.2\bin\R.exe" CMD BATCH model.R`. Note that in both cases, the working directory should be set within the model file using the `setwd` function. The output that would normally be printed to the R Terminal will instead be written to a file called `model.Rout`, which can be opened with any plain text editor.

3

Data format and datasets used for examples

Apollo makes use of a format where all relevant information for a given observation is stored in the same row. Using a simple discrete choice context, this would imply that the data for all alternatives is included in the same row, rather than one row per alternative. If the data file contains multiple rows per individual, then these need to be next to each (i.e. contiguous) in the data file.

Many choice modellers refer to this format (one row per observation) as the *wide* format, as opposed to the *long* format, which would have one row per alternative. This terminology is in fact not very helpful as, in the context of repeated measurements data, the term *wide* refers to a format where all measurements for the same person are included in one line. In the one row per observation format in a choice modelling context, there will still be multiple rows for different choices for the same person³.

Analysts whose data is in the one row per alternative format need to reshape it prior to using it in *Apollo*. A function is made available for this purpose, but with capabilities that may not work for every type of data. The function `apollo_longToWide` is called as:

```
database = apollo_longToWide(longData,  
                             longToWide_settings)
```

The function takes a database in the long format as input and returns a reshaped version of this database. The second argument, `longToWide_settings` is a list with the following inputs:

³There are good reasons for why *Apollo* is using the one row per observation format. This format is the more common format in choice modelling, generally uses less space (except if there are big differences in choice set size across observations), and is also more general in allowing for a mixture of different dependent variables in the same data.

longToWide_settings

altColumn: A string giving the name of the column in the long data that contains the names of the alternatives (either numeric or character).

altSpecAtts: A vector of strings that give the names of the columns in the long data with attributes that vary across alternatives within an observation.

choiceColumn: A string giving the name of the column in the long data that contains the choice.

idColumn: A string giving the name of the column in the long data that contains the ID of individuals.

obsColumn: A string giving the name of the column in the long data that contains the observation index.

This chapter presents a number of datasets used throughout the manual and in the online examples. Below, we give brief introductions to the datasets, with details on variable names provided in Appendix B. The datasets are included in the *Apollo* package itself. A user can access a given dataset, say the `apollo_drugChoiceData`, by typing `data("apollo_drugChoiceData", package="apollo")` in the console.

To use any data in *Apollo*, it needs to be stored in an object called `database`, and once *Apollo* is loaded in memory, the user can simply use `database=apollo_drugChoiceData` to load the data. While this is possible for the four datasets included with *Apollo*⁴, the majority of applications will of course rely on users' own datasets, which will be read in from a file, typically of the comma separated volume (*csv*) format, using e.g. `database=read.csv("data.csv",header=TRUE)`. We do not recommend loading data directly from Excel files.

Apollo uses the `data.frame` format for any input data, and if a user provides a `tibble` object instead, *Apollo* will transform this to a `data.frame`.

3.1 RP-SP mode choice dataset: `apollo_modeChoiceData`

Our first resource is a synthetic dataset looking at mode choices for 500 travellers. For each individual, the data contains two revealed preference (RP) inter-city trips, where the possible modes were car, bus, air and rail, and where each individual has at least two of these four modes available to them. The journey options are described on the basis of access time (except for car), travel time and cost, with times in minutes, and costs in £. The data then also contains 14 stated preference (SP) tasks per person, using the same alternatives as those available on the RP journey for that person, but with an additional categorical quality of service attribute for air and rail, taking three levels, namely *no frills*, *wifi available*, or *food available*. For each individual, the dataset also contains information on whether a respondent is female or not, whether the journey was a business trip or not, and the individual's income.

3.2 SP route choice dataset: `apollo_swissRouteChoiceData`

Our second dataset comes from an actual SP survey of public transport route choice conducted in Switzerland (Axhausen et al., 2008). A total of 388 people were faced with 9 choice tasks each between two public transport routes, both using train (leading to 3,492 observations in

⁴The four datasets are available as *csv* files from the *Apollo* website at www.ApolloChoiceModelling.com.

the data). The two alternatives are described on the basis of travel time, travel cost, headway (time between subsequent trains) and the number of interchanges. For each individual, the dataset additionally contains information on income, car availability in the household, and whether the journey was made for commuting, shopping, business or leisure.

3.3 Health attitudes SP: `apollo_drugChoiceData`

Our third dataset is a synthetic dataset looking at drug choices for the treatment of headaches for 1,000 individuals. For each person, the data contains 10 SP tasks, each giving a choice between four alternatives, the first two being products by recognised drug companies while the final two are generic products. In each choice task, a full ranking of the four alternatives is given. The drugs are described in terms of brand (two recognised brands and three generic brands), country of origin (six countries), drug features (three types of features), risk of side effects, and price. The possible levels for the attributes differ between the first two (branded) and last two (generic) alternatives. For each individual, the dataset additionally contains answers to four attitudinal questions as well as information on whether an individual is a regular user, their education and their age.

3.4 Time use data: `apollo_timeUseData`

Our fourth dataset comes from a GPS tracking survey on time use conducted in the UK ([Calastri et al., 2019](#)). A set of 447 individuals completed a digital activity log for up to 14 days, providing 2,826 days of data (first day discarded for each person). For each day, the amount of time spent in each of twelve activities is recorded, as well as some of the individual's characteristics. The activities considered were dropping-off or picking-up, working, going to school, shopping, private business, getting petrol, social or leisure activities, vacation, doing exercise, being at home, travelling, and a last activity grouping the time allocated to other activities by the individual.

4

General code structure and components: illustration for Logit

In this chapter, we provide an introduction to the general capabilities of the *Apollo* package by using the example of a Logit model. In what follows, we use the term Multinomial Logit (MNL) for such models. We are aware that in some disciplines, users refer to MNL models as using decision-maker covariates only (though of course this then only works with labels for alternatives), and using Conditional Logit for models that incorporate characteristics of the alternatives only (McFadden, 1974). In their seminal paper, McFadden and Train (2000) similarly use MNL.

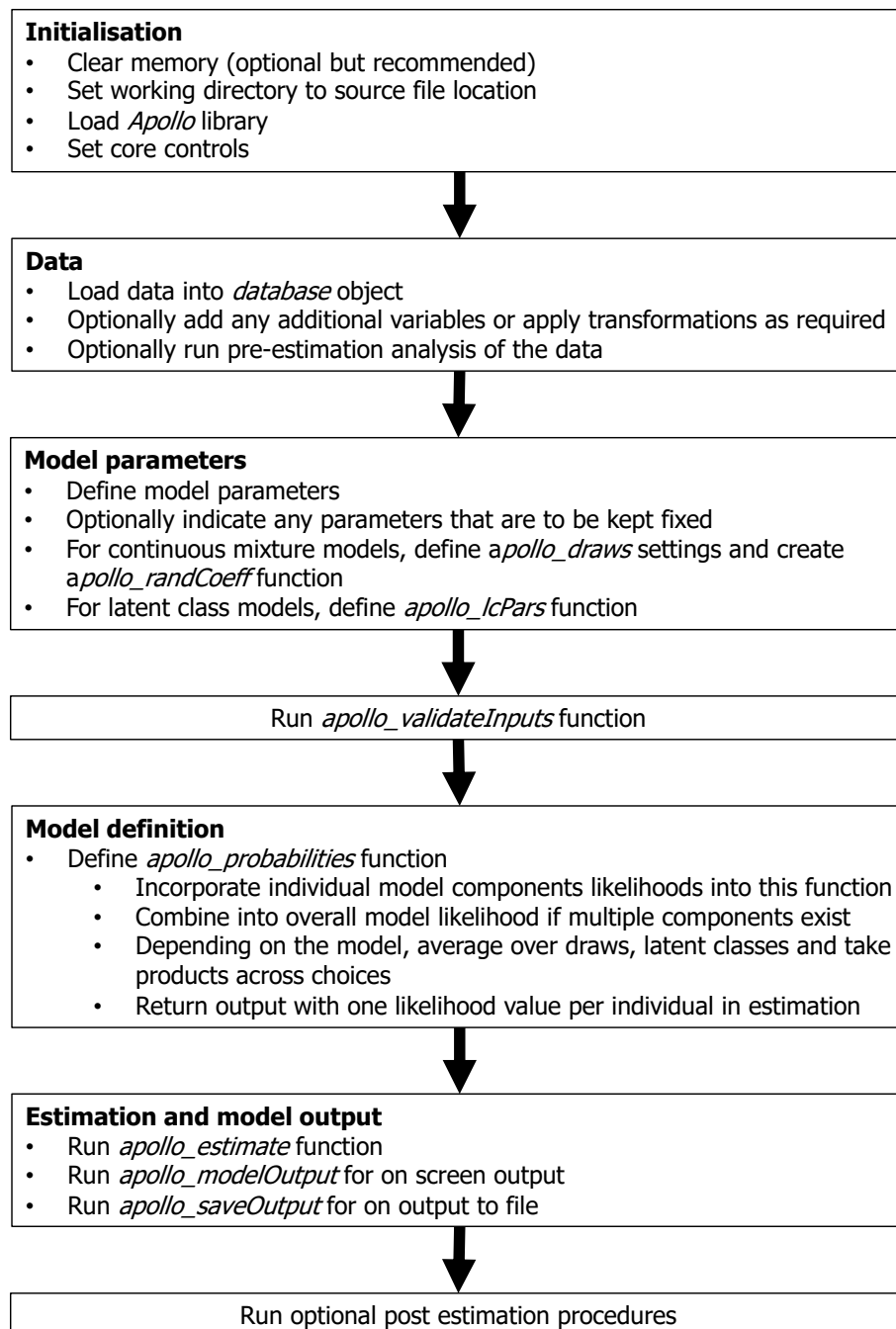
The probability of person n choosing alternative i (out of $j = 1, \dots, J$) in choice situation t is given by:

$$P_{i,n,t} = \frac{e^{V_{i,n,t}}}{\sum_{j=1}^J e^{V_{j,n,t}}}, \quad (4.1)$$

with $V_{i,n,t}$ giving the systematic component of the utility for alternative i for person n in choice situation t .

We apply this to the mode choice stated preference survey introduced in Section 3.1, where we use the SP part of this data, i.e. 14 choices each for 500 individuals. This example is available in the file `MNL_SP_covariates.r` and uses a very detailed specification of the utility function. More barebones examples are also available in `MNL_RP.r` and `MNL_SP.r`, which are models without any socio-demographics, estimated on the RP and SP data, respectively, with a WTP space version in `MNL_SP_WTP_space.r`.

The structure of an *Apollo* model file varies across specifications, but a general overview is shown in Figure 4.1, and we now look at these steps in turn.

Figure 4.1: General structure of an *Apollo* model file

4.1 Initialising the code

The first step in every use of *Apollo* is to initialise the code, as illustrated in Figure 4.2.

```
1  ### Clear memory
2  rm(list = ls())
3
4  ### Load Apollo library
5  library(apollo)
6
7  ### Set working directory (only works in RStudio)
8  apollo_setWorkDir()
9
10 ### Initialise code
11 apollo_initialise()
12
13 ### Set core controls
14 apollo_control = list(
15   modelDescr      = "MNL model with socio-demographics on mode choice SP data",
16   indivID         = "ID",
17   outputDirectory = "output"
18 )
```

Figure 4.2: Code initialisation

In an optional *but strongly recommended* first step, we clear the memory/workspace by using `rm(list = ls())`. We next load the *Apollo* library. The user is strongly encouraged to then set the working directory to be that of the R file, ensuring that output is saved in the appropriate location. When working in RStudio, this can be done using the function `apollo_setWorkDir`, which is called without any arguments and does not return any output variables, i.e.:

```
apollo_setWorkDir()
```

When not working in RStudio, the user can set the working directory using the `setwd` command.

This is followed by calling the `apollo_initialise` function, which ‘detaches’ variables⁵ and makes sure that output is directed to the console rather than a file.

```
apollo_initialise()
```

The user next sets a number of core controls in a list called `apollo_control`, where in our case, we only provide a brief description of the model in `modelName` (for use in the output) and indicate in `indivID` the name (in quotes) of the column in the data which contains the identifier variable for individual decision makers, and set an output directory. Each time, the entry on the left is an *Apollo*-defined variable whose name is not to be changed, and the user provides the value on the right.

User-controlled settings for `apollo_control` include:

⁵In R, a user can ‘attach’ an object, which means that individual components in it can be called by name, e.g. referring to `choice` directly without the prefix of the object it is contained in, such as `database$choice`.

apollo_control: user-controlled settings

calculateLLC: A boolean variable, which, when set to TRUE, means that the log-likelihood at constants is calculated in addition to the log-likelihood at zero (if applicable), and then also used for ρ^2 measures (default is TRUE).

HB: A boolean variable which needs to be set to TRUE for using Bayesian estimation, as discussed in Section 8.1 (default is FALSE).

indivID: A character string giving the name of the column in the database with each decision maker's ID. This is a compulsory setting with no default.

modelDescr: A optional character string giving a description of the model for use in the output files.

modelName: A character string giving the model name, which is to be used also as the name for the output files. This is set automatically when using RStudio.

nCores: An integer setting the number of cores used during estimation discussed, as discussed in Section 6.5 (default is set to 1).

noDiagnostics A boolean variable, which, when set to TRUE, means that no model diagnostics are reported. This setting is provided primarily to avoid excessively verbose output with complex models using many components (cf. Section 7) but will be set to FALSE (default) for most models by most users.

noValidation A boolean variable, which, when set to TRUE, means that no validation checks are performed (default is FALSE).

outputDirectory: The name of a directory to be used for reading input and writing output to. This should be a subdirectory of the working directory, or an absolute path. By default, the output directory is set to the working directory.

panelData: A boolean variable indicating whether the data is to be treated as panel data. This is set automatically to TRUE if multiple observations are present per individual, and FALSE otherwise. If a user sets this to FALSE in the presence of multiple observations per individual, the data will be treated as cross-sectional.

seed: An integer setting the seed used for any random number generation (default is 13).

weights: The name of a variable in the database containing weights for each observation, which can then be used in estimation if also using the function `apollo_weighting` (default is for weights to be missing).

workInLogs: A boolean variable, which, when set to TRUE, means that the logs of probabilities are used when processing probabilities inside `apollo_probabilities`. This can avoid numerical issues with complex models and datasets where there are large numbers of observations per individual. This is only really useful with repeated choice, and slows down estimation and prevents the use of analytical gradients (default is set to FALSE).

Only `indivID`, i.e. the second setting in Figure 4.2 which indicates the individual ID, is a requirement without which the code will not run. For any other settings, the code will use default values when not provided by the user, as illustrated in Figure 4.6 for some relevant omitted settings. A special mention is needed for the `modelName` setting, which gives the name of the model, where any output files will use this name too. It is good practice to use the same name as that of the `.r` file with the model syntax in it, and when running an Apollo model from RStudio, it will by default use that name (unless one is provided by the user). When not using RStudio, the user needs to set `modelName` explicitly in `apollo_control`.

4.2 Reading and processing the data

Figure 4.3 illustrates the process of loading the data, in this case directly from the package. When reading the data from a file (e.g. *csv*⁶), users may need to add the path of the file depending on their local setup and file structure.

In our example, we work with only a subset of the data (in this case removing the RP observations) and create additional variables in the data (in this case a variable with the mean income in the data). Any new variables created by the user, such as mean income in our case, need to be created in the database object rather than the global environment, and this needs to happen prior to validating the user inputs. For any objects that need to be accessible during model estimation but are of a different shape than the database, the user can include these directly in `apollo_inputs`, as discussed in Section 4.4.

Some applications may combine data from multiple files or databases included in packages. The user can either combine the data outside of R or do so inside R using appropriate merging functions, but at the point of validating the user inputs (Section 4.4), all data needs to be combined in a single R data.frame called `database`.

```

1  ### Loading data from package
2  ### if data is to be loaded from a file (e.g. called data.csv),
3  ### the code would be: database = read.csv("data.csv",header=TRUE)
4  database = apollo_modeChoiceData
5  ### for data dictionary, use ?apollo_modeChoiceData
6
7  ### Use only SP data
8  database = subset(database,database$SP==1)
9
10 ### Create new variable with average income
11 database$mean_income = mean(database$income)

```

Figure 4.3: Loading data, selecting a subset and creating an additional variable

4.3 Model parameters

In this simple model, we estimate alternative specific constants (ASCs), mode specific travel time coefficients, a cost and access time coefficient, and dummy coded coefficients for the service quality attribute (a version using effects coding is implemented in `MNL_SP_effects.r`). In addition, we interact the constants with gender, allow for differences in the time and cost sensitivities for business travellers (generic across modes), and incorporate an income elasticity on the cost sensitivity.

With the above, the utilities for the four modes in choice situation t for individual n are given by⁷:

⁶The code is not limited to using *csv* files, and R allows the user to read in tab separated files too, see for example (cf. R Core Team, 2017). We do not recommend loading data directly from Excel files. *Apollo* uses the `data.frame` format for any input data, and if a user provides a `tibble` object instead, *Apollo* will transform this to a `data.frame`.

⁷We use δ for alternative specific constants, β for utility parameters, λ for the elasticity term, x for attributes of the alternatives, and z for person characteristics. The attributes are referred to as tt for travel time, at for access time, tc for cost, and *service* for the service quality attribute. Finally, ε are the extreme value error terms.

$$\begin{aligned}
U_{car,n,t} &= \delta_{car} \\
&+ (\beta_{tt,car} + \beta_{tt,business-interaction} \cdot z_{business,n}) \cdot x_{tt,car,n,t} \\
&+ (\beta_{tc} + \beta_{tc,business-interaction} \cdot z_{business,n}) \cdot \left(\frac{z_{income,n}}{z_{income}} \right)^{\lambda_{income}} \cdot x_{tc,car,n,t} \\
&+ \varepsilon_{car,n,t} \\
U_{bus,n,t} &= \delta_{bus} + \delta_{bus,female-interaction} \cdot z_{female,n} \\
&+ (\beta_{tt,bus} + \beta_{tt,business-interaction} \cdot z_{business,n}) \cdot x_{tt,bus,n,t} \\
&+ \beta_{at} \cdot x_{at,bus,n,t} \\
&+ (\beta_{tc} + \beta_{tc,business-interaction} \cdot z_{business,n}) \cdot \left(\frac{z_{income,n}}{z_{income}} \right)^{\lambda_{income}} \cdot x_{tc,bus,n,t} \\
&+ \varepsilon_{bus,n,t} \\
U_{air,n,t} &= \delta_{air} + \delta_{air,female-interaction} \cdot z_{female,n} \\
&+ (\beta_{tt,air} + \beta_{tt,business-interaction} \cdot z_{business,n}) \cdot x_{tt,air,n,t} \\
&+ \beta_{at} \cdot x_{at,air,n,t} \\
&+ (\beta_{tc} + \beta_{tc,business-interaction} \cdot z_{business,n}) \cdot \left(\frac{z_{income,n}}{z_{income}} \right)^{\lambda_{income}} \cdot x_{tc,air,n,t} \\
&+ \beta_{no\ frills} \cdot (x_{service,air,n,t} == 1) \\
&+ \beta_{wifi} \cdot (x_{service,air,n,t} == 2) \\
&+ \beta_{food} \cdot (x_{service,air,n,t} == 3) \\
&+ \varepsilon_{air,n,t} \\
U_{rail,n,t} &= \delta_{rail} + \delta_{rail,female-interaction} \cdot z_{female,n} \\
&+ (\beta_{tt,rail} + \beta_{tt,business-interaction} \cdot z_{business,n}) \cdot x_{tt,rail,n,t} \\
&+ \beta_{at} \cdot x_{at,rail,n,t} \\
&+ (\beta_{tc} + \beta_{tc,business-interaction} \cdot z_{business,n}) \cdot \left(\frac{z_{income,n}}{z_{income}} \right)^{\lambda_{income}} \cdot x_{tc,rail,n,t} \\
&+ \beta_{no\ frills} \cdot (x_{service,rail,n,t} == 1) \\
&+ \beta_{wifi} \cdot (x_{service,rail,n,t} == 2) \\
&+ \beta_{food} \cdot (x_{service,rail,n,t} == 3) \\
&+ \varepsilon_{rail,n,t},
\end{aligned} \tag{4.2}$$

where all parameters are estimated except for δ_{car} and $\beta_{no\ frills}$, which are both fixed to a value of zero for identification.

In the code, the user needs to define the parameters and their starting values, and also indicate whether any of the parameters are to be kept at their starting values. This process is illustrated in Figure 4.4. We first create an R object of the *named vector* type, called `apollo_beta`, with the name and starting value for each parameter, including any that are later on fixed to their starting values. In our case, we keep two of the parameters, namely `asc_car` and `b_no_frills`, fixed to their starting values by including their names in the character vector `apollo_fixed`, where this vector is kept empty, i.e. `apollo_fixed = c()`, if all parameters are to be estimated. Parameters included in `apollo_fixed` are kept at the value used in `apollo_beta`, which may not be zero.

For complex models especially, it can sometimes be beneficial to read in starting values

```

1  ### Vector of parameters, including any that are kept fixed in estimation
2  apollo_beta=c(asc_car           = 0,
3                asc_bus           = 0,
4                asc_air           = 0,
5                asc_rail          = 0,
6                asc_bus_interaction_female = 0,
7                asc_air_interaction_female = 0,
8                asc_rail_interaction_female = 0,
9                b_tt_car          = 0,
10               b_tt_bus          = 0,
11               b_tt_air          = 0,
12               b_tt_rail         = 0,
13               b_tt_interaction_business = 0,
14               b_access          = 0,
15               b_cost            = 0,
16               b_cost_interaction_business = 0,
17               cost_income_elast = 0,
18               b_no_frills       = 0,
19               b_wifi            = 0,
20               b_food            = 0)
21
22  ### Vector with names (in quotes) of parameters to be kept fixed at their starting value in
23  ↪ apollo_beta, use apollo_beta_fixed = c() if none
   apollo_fixed = c("asc_car", "b_no_frills")

```

Figure 4.4: Setting names and starting values for model parameters, and fixing some parameters to their starting values

from an earlier model⁸, albeit that users should be mindful that this can lead to problems with convergence to the estimates of the old model. This process is made possible by the function `apollo_readBeta`, which is called as:

```

apollo_beta = apollo_readBeta(apollo_beta,
                              apollo_fixed,
                              inputModelName,
                              overwriteFixed)

```

The function returns an updated version of `apollo_beta`. The function takes up to four arguments, as follows:

apollo_readBeta: function inputs

apollo_beta: A named numeric vector of parameter names and values
apollo_fixed: A character vector with the names of any parameters that are not to be estimated (i.e. kept at their starting values)
inputModelName: The name of a previously estimated model, given as a string.
overwriteFixed: An optional boolean variable indicating whether parameters that are not to be estimated should have their starting values overwritten by the input file (set to FALSE by default).

⁸Later on in the manual, we also discuss starting value search processes, in Section 9.3.

To use `apollo_readBeta`, the outputs from the input model need to have been saved in the same directory as the current model file (or in the associated output directory). We illustrate the use of this function in Figure 4.5, where we read in parameters from the earlier SP model without covariates (`MNL_SP.r`) which did not include the socio-demographic effects, thus meaning that only values for the 11 previously estimated parameters were read in, with the fixed parameter `asc_car` kept to the value from `apollo_beta` given the use of `overwriteFixed=FALSE`, where, with `overwriteFixed=TRUE`, the value from the input file would also be used for fixed parameters.

```

1 > apollo_beta=apollo_readBeta(apollo_beta,apollo_fixed,"MNL_SP",overwriteFixed=FALSE)
2 Out of the 19 parameters in apollo_beta, 11 were updated with values from the input file.
3 2 parameters in apollo_beta were kept fixed at their starting values rather than being
4 updated from the input file.

```

Figure 4.5: Using `apollo_readBeta` to load results from an earlier model as starting values

4.4 Validation and preparing user inputs

The final step in preparing the code and data for model estimation or application is to make a call to `apollo_validateInputs`. The function runs a number of checks and produces a consolidated list of model inputs. It is called as:

```
apollo_inputs=apollo_validateInputs()
```

This function takes a number of arguments as options, but looks in the global environment for any not provided by the user. The arguments are:

```
apollo_validateInputs: function inputs
```

apollo_beta: A named numeric vector of parameter names and values
apollo_fixed: A character vector with the names of any parameters that are not to be estimated (i.e. kept at their starting values)
database: Data used by model
apollo_control: Options controlling the running of the code.
apollo_HB: List containing options for Bayesian estimation.
apollo_draws: List of arguments describing the inter and intra individual draws.
apollo_randCoeff: Function used with mixing models to construct the random parameters of a mixing model.
apollo_lcPars: Function used with latent class models to constructs a list of parameters for each latent class.
recycle: If TRUE, an older version of `apollo_inputs` is looked for in the calling environment (parent frame), and any element in that old version created by the user is copied into the new `apollo_inputs` returned by this function. For `recycle=TRUE` to work, the old version of `apollo_inputs` **must** be named `apollo_inputs`. If FALSE, nothing is copied from any older version of `apollo_inputs`. TRUE is the default.
silent: TRUE to keep the function from printing to the console. Default is FALSE.

Most users will call the function without any arguments, and *Apollo* will then look for

the required inputs. This always includes the control settings `apollo_control`, the model parameters `apollo_beta`, the vector with names of fixed parameters `apollo_fixed` and finally the data object `database`. If any of these objects are missing from the global environment, the execution of `apollo_validateInputs` fails. The function also looks for a number of optional objects, namely `apollo_HB`, which is used for Bayesian estimation (cf. Section 8.1), `apollo_draws` and `apollo_randCoeff`, which are used for continuous random coefficients (cf. Section 6.1), and `apollo_lcPars`, which is used for latent class (cf. Section 6.3).

Before returning the list of model inputs, `apollo_validateInputs` runs a number of validation tests on the `apollo_control` settings and the `database`. It sets the `modelName` if not provided (and if using `RStudio`, uses default values for any missing settings, and, in the case of panel data, adds an extra column called `apollo_sequence` which is a running index of observations for each individual in the data. Finally, the code also checks for the presence of multiple rows per individual in the data and accordingly sets `apollo_control$panelData` to `TRUE` or `FALSE`⁹. The running of `apollo_validateInputs` is illustrated in Figure 4.6. The list that is returned, `apollo_inputs`, contains the validated versions of the various objects mentioned above, e.g. `database`.

If the user wants to make use of data outside `database` inside the model (i.e. inside `apollo_probabilities`), then the required object should be included in `apollo_inputs`, and it should include the individuals' ids as a column, using the same name as provided in `apollo_control$indivID`. Let us imagine we want to use matrix `X` inside `apollo_probabilities`.

1. If the object `X` contains information at the individual level (e.g. sociodemographics of each individual), then the first column of matrix `X` should contain the corresponding individual's ID for each row in `X`. This is necessary for *Apollo* to be able to distribute the object `X` across cores when using multi-core estimation. If the object `X` does not contain data associated to each individual, then there is no need to include the ID column.
2. Store the object `X` inside `apollo_inputs` after running `apollo_validateInputs`, by using `apollo_inputs$X=X`.
3. Every time `X` is used inside `apollo_probabilities` (or `apollo_randCoeff` or `apollo_lcPars`) it needs to be called as `apollo_inputs$X`.

This approach is used in the example in Section 4.8.

```

1 > apollo_inputs = apollo_validateInputs()
2 Model name missing in apollo_control, set to "MNL_SP_covariates".
3 Several observations per individual detected based on the value of ID. Setting panelData in
  ↪ apollo_control set to TRUE.
4 All checks on apollo_control completed.
5 All checks on database completed

```

Figure 4.6: Running `apollo_validateInputs`

4.5 Likelihood component: the *apollo_probabilities* function

The core part of the code is contained in the `apollo_probabilities` function, where we show this function for our MNL model in Figure 4.7. An important distinction arises between

⁹In R, elements of a list such as `apollo_control` can be referred to via `apollo_control$panelData`.

`apollo_probabilities` and other functions in *Apollo*. While the other functions we have encountered are part of the package, `apollo_probabilities` needs to be defined by the user as it is specific to the model to be estimated. The function itself is never called by the user, but is used for example by the function for model estimation `apollo_estimate` discussed below. The `apollo_probabilities` function returns probabilities, where the specific format depends on `functionality`, which takes a default value for model estimation, but other values apply for example in prediction, as discussed in Section 9.9. The value used for `functionality` depends on which function makes the call to `apollo_probabilities` and is controlled internally - the user does not need to change this, except in debugging, as explained in Chapter 10.

This function takes three inputs, namely the vector of parameters `apollo_beta`, the list of combined model inputs `apollo_inputs`, and the argument `functionality`.

In the following three subsections, we look at the individual components of the code shown in Figure 4.7.

4.5.1 Initialisation

Any use of the `apollo_probabilities` function begins with a call to `apollo_attach` which enables the user to then call individual elements within for example the database by name, e.g. using `female` instead of `database$female`. This function is called as:

```
apollo_attach(apollo_beta,
              apollo_inputs)
```

The function does not return an object as output and the user does not need to change the arguments for this function. The call to this function is immediately followed by a command instructing R to run the function `apollo_detach` once the code exits `apollo_probabilities`. This ensures that this call is made even if there is an error that leads to a failure (and hence hard exit) from `apollo_probabilities`. This call is made as:

```
on.exit(apollo_detach(apollo_beta,
                      apollo_inputs))
```

With the database now having been attached, all elements within it can be referred to it by name inside `apollo_probabilities` and referring to `database` is no longer permitted, except if referring to the version in `apollo_inputs`, as discussed in Section 4.8.

We next initialise a list (a flexible R object) called `P` which will contain the probabilities for the model, where this is a requirement for any type of model used with the code.

4.5.2 Model definition

With $\varepsilon_{car,n,t}$, $\varepsilon_{bus,n,t}$, $\varepsilon_{air,n,t}$ and $\varepsilon_{rail,n,t}$ in Equation 4.2 being distributed identically and independently (*iid*) across individuals and choice scenarios following a type I extreme value distribution, we obtain an MNL mode (cf. Luce, 1959; McFadden, 1974), with the probability for alternative i in choice task t for person n given by:

$$P_{i,n,t}(\beta) = \frac{z_{avail,i,n,t} \cdot e^{V_{i,n,t}}}{\sum_{j=1}^J z_{avail,j,n,t} \cdot e^{V_{j,n,t}}}, \quad (4.3)$$

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2    ### Attach inputs and detach after function exit
3    apollo_attach(apollo_beta, apollo_inputs)
4    on.exit(apollo_detach(apollo_beta, apollo_inputs))
5
6    ### Create list of probabilities P
7    P = list()
8
9    ### Create alternative specific constants and coefficients using interactions with
10   ↪ socio-demographics
11   asc_bus_value = asc_bus + asc_bus_interaction_female * female
12   asc_air_value = asc_air + asc_air_interaction_female * female
13   asc_rail_value = asc_rail + asc_rail_interaction_female * female
14   b_tt_car_value = b_tt_car + b_tt_interaction_business * business
15   b_tt_bus_value = b_tt_bus + b_tt_interaction_business * business
16   b_tt_air_value = b_tt_air + b_tt_interaction_business * business
17   b_tt_rail_value = b_tt_rail + b_tt_interaction_business * business
18   b_cost_value = ( b_cost + b_cost_interaction_business * business ) * ( income /
19   ↪ mean_income ) ^ cost_income_elast
20
21   ### List of utilities: these must use the same names as in mnl_settings, order is irrelevant
22   V = list()
23   V[["car"]] = asc_car + b_tt_car_value * time_car +
24   ↪ b_cost_value * cost_car
25   V[["bus"]] = asc_bus_value + b_tt_bus_value * time_bus + b_access * access_bus +
26   ↪ b_cost_value * cost_bus
27   V[["air"]] = asc_air_value + b_tt_air_value * time_air + b_access * access_air +
28   ↪ b_cost_value * cost_air + b_no_frills * ( service_air == 1 ) + b_wifi * ( service_air
29   ↪ == 2 ) + b_food * ( service_air == 3 )
30   V[["rail"]] = asc_rail_value + b_tt_rail_value * time_rail + b_access * access_rail +
31   ↪ b_cost_value * cost_rail + b_no_frills * ( service_rail == 1 ) + b_wifi * ( service_rail
32   ↪ == 2 ) + b_food * ( service_rail == 3 )
33
34   ### Define settings for MNL model component
35   mnl_settings = list(
36     alternatives = c(car=1, bus=2, air=3, rail=4),
37     avail = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
38     choiceVar = choice,
39     utilities = V
40   )
41
42   ### Compute probabilities using MNL model
43   P[["model"]] = apollo_mnl(mnl_settings, functionality)
44
45   ### Take product across observation for same individual
46   P = apollo_panelProd(P, apollo_inputs, functionality)
47
48   ### Prepare and return outputs of function
49   P = apollo_prepareProb(P, apollo_inputs, functionality)
50   return(P)
51 }

```

Figure 4.7: The `apollo_probabilities` function: example for MNL model

where β is a vector combining all model parameters, $V_{j,n,t}$ refers to the part of the utility functions in Equation 4.2 that excludes the error term $\varepsilon_{j,n,t}$, and where $z_{avail,j,n,t}$ takes a value of 1 if alternative j is available in choice set t for person n , and 0 otherwise.

In the central part of the `apollo_probabilities` function, the user defines the actual model, where in our example, this is a simple MNL model. No limits on flexibility are imposed on the user with the *Apollo* package. A number of prewritten functions for common models are made available in the package, going beyond MNL, as discussed in Section 5. Additionally, the user can define his/her own models, as discussed in Section 5.7. Finally, this part of the code can contain either a single model, as shown here, or multiple individual model components, as discussed in Section 7.

The `apollo_mnl` function is called via:

```
P[["model"]] = apollo_mnl(mnl_settings,
                          functionality)
```

The function returns probabilities for the model, where depending on `functionality`, this is for the chosen alternative only or for all alternatives. The output of the function is saved in a component of the list `P` where for single component models such as here, this element is called `P[["model"]]`. The function takes as its core input a list called `mnl_settings` which has six possible inputs, of which three are compulsory (`alternatives`, `choiceVar`, and `utilities`). We will now look at these in turn.

apollo_mnl: user-controlled settings

alternatives: A named vector containing the names of the alternatives as defined by the user, and for each alternative, giving the value used in the dependent variable in the data. In our case, these simply go from 1 to 4.

avail: A list containing one element per alternative, using the same names as in **alternatives**. For each alternative, we define the availability either through a vector of values of the same length as the number of observations (i.e. a column from the data) or by a scalar of 1 if an alternative is always available. A user can also set **avail=1** (or omit its use) which implies that all of the alternatives are available for every choice observation in the data.

choiceVar: A vector of length equal to the number of observations, containing the chosen alternative for each observation. In our example, this column is simply called **choice**.

componentName: This is an optional argument of the character type which allows the user to specify a name of the given model component. This is then used in various places in diagnostic tests and model outputs. If omitted, a default is used by *Apollo*. It is recommended the user omits this setting and lets Apollo set it up automatically.

rows: This is an optional argument which is missing by default. It allows the user to specify a vector called **rows** of the same length as the number of rows in the data. This vector needs to use logical statements to identify which rows in the data are to be used for this model. For any observations in the data where the entry in **rows** is set to FALSE, the probability for the model will be set to 1. This means that, for this model, this observation does not contribute to the calculation of the likelihood and hence estimation of the model parameters. It is useful for example in the case of hybrid choice models, a point we return to in Section 7. When omitted from the call to **apollo_mnl**, all rows are used, as in our example in Figure 4.7.

utilities: A list object containing one utility for each alternative, using the same names as in **alternatives**, where any linear or non-linear specification is possible. The contents of **utilities** are complicated and are thus generally defined prior to calling the function, as in Figure 4.7. In our case, we pre-compute the interactions with socio-demographic variables in the lines preceding the definition of the actual utilities, creating for example the new parameter **b_tt_car_value**. This helps keep the code organised, makes it easier to add additional interactions and also avoids unnecessary calculations. The latter point can be understood by noting that in our example, the impact of income and purpose on the cost coefficient is calculated just once and then used in each of the four utilities, rather than being calculated four times. For models with large numbers of alternatives and/or attributes, users may consider iterative coding of utilities, as discussed in Section 4.8.

In the code example, we actually create the utilities **utilities** outside **mnl_settings** first just for ease of coding, but they can similarly be created directly inside the list. What matters if using the former approach is that they are then copied into a component called **utilities** inside **mnl_settings**.

4.5.3 Function output

The final component of the `apollo_probabilities` function prepares the output of the function. This performs further processing of the `P` list, which needs to include an element called `model`, where, in our example, this is the only element in `P`. The specific functions (and their order) to be called in this part of the code depend on the data and model, where once again, the actual inputs to these functions are not to be changed by the user.

In our specific example, the only additional manipulation of the raw probabilities produced by `apollo_mnl` is a call to `apollo_panelProd` which multiplies the probabilities across individual choice observations for the same individual, thus recognising the repeated choice nature of our data. This function is only to be used in the presence of multiple observations per individual. When estimating a model, the code computes the probability for the chosen alternative, say $y_{n,t}$ in choice task t for person n , i.e. $P_{y_{n,t}}$, using Equation 4.3. The contribution by person n to the likelihood function, with a given value for the vector of model parameters β , is then given by:

$$L_n(\beta) = \prod_{t=1}^{T_n} P_{y_{n,t}}, \quad (4.4)$$

where T_n is the number of separate choice situations for person n . This function is called as:

```
P = apollo_panelProd(P,
                      apollo_inputs,
                      functionality)
```

All arguments of this function have been described already. When called in model prediction (cf. Section 9.9), the multiplication across choices is omitted, i.e. the function returns an unmodified version of `P`, with one row per observation.

Independent of the model specification, the function `apollo_probabilities` always ends with the same two commands. First is `apollo_prepareProb` which prepares the output of the function depending on `functionality`, e.g. with different output for estimation and prediction. This is called as:

```
P = apollo_prepareProb(P,
                       apollo_inputs,
                       functionality)
```

This is followed by

```
return(P)
```

which ensures that `P` is returned as the output of `apollo_probabilities`.

We earlier mentioned the possible use of weights by including the setting `weights` in `apollo_control`. If the user wants to use weights, then they must also call the function

`apollo_weighting` prior to `apollo_prepareProb`. This is called as:

```
P = apollo_weighting(P,  
                    apollo_inputs,  
                    functionality)
```

Note that when weights are used, they are applied during estimation, prediction and any other application of the model (calculation of the model null log-likelihood, conditionals, unconditionals, etc.). If the user wants, for example, to estimate without weights but predict with them, then we recommend setting the values of all weights to 1 during estimation, and to their real values during forecasting, by making a change to the `database` and calling `apollo_inputs = apollo_validateInputs()` again.

4.6 Estimation

Now that we have defined our model, we can perform model estimation by calling the function `apollo_estimate` and saving the output from it in an object called `model`. This function uses the `bgw` package (Bunch et al., 1993) and `maxLik` package (Henningsen and Toomet, 2011) for classical estimation, where Bayesian estimation is discussed in Section 8.1. In its simplest form, this function is called via:

```
model = apollo_estimate(apollo_beta,  
                       apollo_fixed,  
                       apollo_probabilities,  
                       apollo_inputs)
```

where we have already covered all four arguments. The function may also be called with an additional argument, namely `estimate_settings`, i.e.:

```
model = apollo_estimate(apollo_beta,  
                       apollo_fixed,  
                       apollo_probabilities,  
                       apollo_inputs,  
                       estimate_settings)
```

The additional input `estimate_settings` is a list which contains a number of settings for estimation. None of these settings is compulsory and default settings will be used for any omitted settings, or indeed all settings when calling `apollo_estimate` without the `estimate_settings` argument. The possible settings to include in this list are:

apollo_estimate: user-controlled settings (part 1)

- bgw_settings:** An optional list of settings for the `bgw` package - for details, see `?bgw_mle`.
- bootstrapSE:** A numeric variable indicating the number of bootstrap samples to calculate standard errors. The default is 0, meaning no bootstrap standard errors will be calculated.
- bootstrapSeed:** A numeric variable indicating the seed for the bootstrap sampling. The default is 24. If changed, it must be a positive integer. This value is only used if `bootstrapSE > 0`. In general, there is no need to change this value. If the user wants to add new repetitions to a completed or interrupted estimation with bootstrap standard errors, the draws used will automatically be different.
- convergenceCheck:** A boolean variable, which, when set to `TRUE`, means that additional convergence checks are carried out when using BGW for estimation, with a second round of estimation carried out if necessary (default is set to `FALSE`).
- constraints:** A character vector defining constraints on the parameters to be applied during estimation. This is only possible with `bfgs`, and several limitations apply^a.
- forceCovarianceMatrix:** A boolean variable, which, when set to `TRUE`, means that the covariance matrix is computed even if estimation has failed (default is set to `FALSE`).
- estimationRoutine:** A character object which can take the values `bfgs` (for the [Broyden 1970](#) - [Fletcher 1970](#) - [Goldfarb 1970](#) - [Shanno 1970](#) algorithm), `bgw` ([Bunch et al., 1993](#)), `bhhh` (for the Berndt-Hall-Hall-Hausman algorithm, [Berndt et al. 1974](#) or `nr` (for the Newton-Raphson algorithm), where the specific syntax is for example `estimationRoutine="bgw"` (default is set to `bgw`).
- hessianRoutine:** A character variable indicating what routine to use for calculating the final Hessian. Possible values are `numDeriv` for the `numDeriv` package ([Gilbert and Varadhan, 2016](#)), `maxLik`, and `none` for no covariance matrix calculation (although the BHHH covariance matrix will be available). We have generally found that `numDeriv` is more reliable than `maxLik` for computing the covariance matrix, but its use may lead to issues with complex mixture models. If `numDeriv` fails, the code reverts to using `maxLik` (default is set to `numDeriv`).
- maxIterations:** An integer setting a maximum on the number of iterations (default = 200).
- maxLik_settings:** A list of additional settings for `maxLik`.
- numDeriv_method:** Method used for numerical differentiation when calculating the covariance matrix. Can be "Richardson" or "simple".
- numDeriv_settings:** A list of optional settings to be passed on to `numDeriv` when this is used for the Hessian (cf. [Gilbert and Varadhan, 2016](#)).
- printLevel:** A numeric variable which can take levels from 0 to 3 and controls the level of detail printed out during estimation, with higher levels meaning more detail (default is set to 3).
- scaleAfterConvergence:** A boolean variable, which, when set to `TRUE`, means that the parameters are rescaled after convergence, before launching a second round of estimation to improve convergence (default is set to `FALSE`). For more details on scaling, see the `scaling` setting below in this list.

^aOnly linear constraints are supported. Only `>`, `<`, and `=` constraints are allowed, and they cannot be mixed, e.g. $\beta_1 \geq 0$ and $\beta_2 = \beta_1$ cannot be applied at the same time. Fixed parameters (those whose value do not change throughout estimation) cannot be included in constraints. When writing constraints, all parameter names must be on the left side of the expression and only numeric values on the right. So, for example, if we want the following constraints: $0 < \beta_1 < 1$ and $\beta_2 > \beta_1$, then we would write them as `constraints = c("b_1 > 0", "b_1 < 1", "-b_1 + b_2 > 0")`. Constraints can also be defined using matrix notation using the `maxLik` ([Henningesen and Toomet, 2011](#)) coding approach.

apollo_estimate: user-controlled settings (part 2)

scaleHessian: A boolean variable, which, when set to TRUE, means that the parameters are rescaled after convergence, prior to the calculation of the Hessian, to reduce the risk of numerical problems (default is set to FALSE). For more details on scaling, see the **scaling** setting below in this list.

scaling: An optional named vector of scalings to be applied to individual parameters during estimation. This can help estimation if the scale of individual parameters at convergence is very different^a. The aim of this process is to have the parameters that are actually used in model estimation, i.e. β_k^* , to be of a similar scale.

silent: A boolean variable, which, when set to TRUE, means that no information is printed to the screen during estimation (default is set to FALSE).

validateGrad: A boolean variable, which, when set to TRUE, means that the analytical gradients (if available) are compared to the numerical ones before estimation (default is set to TRUE).

writeIter: A boolean variable, which, when set to TRUE, means that the values of parameters at each iteration are saved into a file in the working directory, saved as **modelName_iterations.csv** where **modelName** is as defined in **apollo_control**. This allows the user to monitor progress during estimation, which is useful especially for complex models (default is set to TRUE).

^aIn classical estimation, the user can specify scales for individual model parameters. For example, if the unscaled specification involves a component $\beta_k x_k$ in the utility function, and if the user wishes to apply a scale of s_{β_k} , the starting value will be automatically adjusted to $\beta_k^* = \frac{1}{s_{\beta_k}} x_k$, the utility component will be adjusted to $s_{\beta_k} \beta_k^* x_k$ and the maximum likelihood estimation will optimise the value of β_k^* . The final model estimates will be translated to the original scale, i.e. returning estimates for β_k . When using scaling in Bayesian estimation in *Apollo*, not all estimates are returned to their original scale after estimation. Indeed, the scaling is applied to the parameter chains directly, and producing scaled values for the underlying Normals is not convenient. We thus report the scaled outputs only for the non-random parameters, the random parameters after transformation to the actual distributions used, and the posterior means.

Figure 4.8 illustrates what happens when running **apollo_estimate** on our simple MNL model. The code first checks the model specification used inside **apollo_probabilities** and reports some basic diagnostics. The validation and diagnostic steps are skipped if the user has set a value of TRUE for **noValidation** or **noDiagnostics**, respectively, in **apollo_control**. For MNL, the checks include for example ensuring that no unavailable alternatives are chosen. The code also checks if any parameters are included in **apollo_beta** that do not influence the calculation of the model likelihood, or if the probabilities of the model are zero at the starting values. This is followed by the main estimation process¹⁰. After convergence, we proceed with the calculation of the Hessian. Prior to that step, which can take a long time in complex models (and may fail), the code also prints out the final estimates and approximate standard errors from the BHHH matrix.

We can see from Figure 4.8 that the estimation uses minimisation of the negative of the log-likelihood, hence the positive values, which is of course equivalent to maximisation of the log-likelihood itself.

Model estimation creates an object of the **model** type. The contents of this depend on the

¹⁰Note that, as is common with optimisers, we use a minimiser, such that we minimise the negative of the log-likelihood function, which is equivalent to maximising the log-likelihood function.

```

1 > model = apollo_estimate(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 Preparing user-defined functions.
3 Testing likelihood function...
4 Overview of choices for MNL model component :
5
6           car      bus      air      rail
7 Times available    5446.00 6314.00 5264.00 6118.00
8 Times chosen       1946.00 358.00 1522.00 3174.00
9 Percentage chosen overall    27.80   5.11  21.74  45.34
10 Percentage chosen when available 35.73   5.67  28.91  51.88
11
12 Pre-processing likelihood function...
13 Testing influence of parameters
14 Starting main estimation
15
16 BGW using analytic model derivatives supplied by caller...
17
18 Iterates will be written to:
19 output/MNL_SP_covariates_iterations.csv
20
21      it   nf      F          RELDF    PRELDF    RELDX    MODEL stppar
22      0    1 5.598900605e+03
23      1    5 4.944826642e+03 1.168e-01 1.110e-01 1.05e-01  G   7.46e-02
24      2    6 4.834039950e+03 2.240e-02 1.927e-02 1.03e-01  G   5.59e-04
25      ...
26      8   12 4.830944738e+03 8.941e-13 9.713e-13 8.86e-07  S   0.00e+00
27
28 ***** Relative function convergence *****
29
30 Estimated parameters with approximate standard errors from BHHH matrix:
31
32           Estimate      BHHH se BHH t-ratio (0)
33 asc_car          0.000000           NA           NA
34 asc_bus          0.286354      0.598493       0.4785
35 asc_air         -0.903377      0.370540      -2.4380
36 asc_rail        -2.092701      0.349166      -5.9934
37 ...
38 b_no_frills      0.000000           NA           NA
39 b_wifi           1.026711      0.057637     17.8135
40 b_food           0.422072      0.056878       7.4206
41
42 Final LL: -4830.9447
43
44 Calculating log-likelihood at equal shares (LL(0)) for applicable models...
45 Calculating log-likelihood at observed shares from estimation data (LL(c)) for applicable
46 models...
47 Calculating LL of each model component...
48 Calculating other model fit measures
49 Computing covariance matrix using numerical jacobian of analytical gradient.
50 0%...25%...50%...75%...100%
51 Negative definite Hessian with maximum eigenvalue: -2.818749
52 Computing score matrix...

```

Figure 4.8: Running apollo_estimate on MNL model

specific type of model used, as well as on the estimation routine. Possible contents of this are described in more detail in Appendix D.

Model estimation is the most likely step during which failures are encountered when working with the *Apollo* package. These could be either caused by errors in using the R syntax, resulting in generic R error messages, or errors made in the use of the various *Apollo* functions, leading to more specific error or warning messages. Not all warnings will be terminal and the code will continue to run and report warnings after completion. It is in this case entirely possible that the estimation has reached an acceptable solution with the warning messages for example being a result of the estimation process trying parameter values that lead to numeric issues in some iterations.

If a user runs the entire script contained in a model file including any post-estimation processes in one go, then errors during estimation will cause further problems in the steps that follow, but the reporting of those problems will likely become less intuitive further down the line. The user should in that case return to the first error message obtained and identify the cause of this and remedy it in the code. In general, running the code section by section is advisable to avoid this issue as far as possible.

The outcomes of model estimation are saved in a list called `model`, which contains amongst other things the estimates (`model$estimates`) and the classical, robust and BHHH covariance matrices (`model$varcov`, `model$robvarcov` and `model$BHHHvarcov`). Even if estimation fails, the BHHH covariance matrix is returned (if available). A user can also request the full covariance matrix to be calculated even if estimation fails, using `estimateSettings$forceCovarianceMatrix=TRUE`.

The classical covariance matrix is an estimate of the sampling variance, say $\hat{\Omega}_{\hat{\beta}}$, calculated as the inverse of the Fisher information matrix, $I_{\hat{\beta}}$, which itself is the negative of the expected Hessian (matrix of second derivatives) of the log-likelihood function with respect to the model parameters. We have:

$$\hat{\Omega}_{\hat{\beta}} = I_{\hat{\beta}}^{-1} \quad (4.5)$$

$$I_{\hat{\beta}} = -E_Y \left(H_{\hat{\beta}} \right) \quad (4.6)$$

$$H_{\hat{\beta}} = \left(\frac{\partial^2 LL(\beta)}{\partial \beta \partial \beta'} \right)_{\hat{\beta}} \quad (4.7)$$

The matrix $\hat{\Omega}_{\hat{\beta}}$ now gives us the covariance matrix of the estimates at the maximum likelihood estimate (MLE). From $\hat{\Omega}_{\hat{\beta}}$, we can obtain standard errors as the square roots of its diagonal elements, say $\hat{\sigma}_{\hat{\beta}_k} = \sqrt{\hat{\Omega}_{\hat{\beta}}[k, k]}$ for parameter β_k .

The *robust* covariance matrix is obtained using the so called *sandwich* estimator¹¹, where:

$$\Omega_{robust} = \hat{\Omega}_{\hat{\beta}}^T B_{\hat{\beta}} \hat{\Omega}_{\hat{\beta}}, \quad (4.8)$$

where $B_{\hat{\beta},jk}$ is the BHHH matrix (Berndt et al., 1974), which is an approximation of the Hessian, where the element in row j and column k is given by:

$$B_{\hat{\beta},jk} = \sum_{n=1}^N \left(\frac{\partial LL_n(\beta)}{\partial \beta_j} \frac{\partial LL_n(\beta)}{\partial \beta_k} \right)_{\hat{\beta}}. \quad (4.9)$$

¹¹The origin of the term *sandwich estimator* is that Ω forms the bread, while the BHHH matrix B is the filling.

In purely cross-sectional estimation, the sandwich estimator corrects the standard errors for general mis-specification of the model. In a panel specification, it can additionally corrects the standard errors to accommodate the panel nature. It has long been recognised (see e.g. [Louviere and Woodworth, 1983](#)) that individual choices for the same respondent are not independent, and that the information content of N respondents each giving T responses is usually considerably less than that of NT respondents each giving a single response. As a result, the estimates of accuracy given by cross-sectional modelling are incorrect, i.e. such models are likely to produce biased standard errors, with the expectation being a downwards bias. When there is no explicit retention of information between observations, $H_{\hat{\beta}}$ will be identical whether the data is considered as a panel or not. However, $B_{\hat{\beta},jk}$ will differ given the use of derivatives at the likelihood *vs* observation level. Robust standard errors are thus different for the panel case and the non-panel cases, because the specification of an “observation” is different. When using `apollo_panelProd`, the robust standard errors are automatically corrected for the repeated choice nature of the data.

4.7 Reporting and saving results

After completing model estimation, the user can output the results to the console (screen) and/or a set of different output files. Two separate functions are used for this, namely `apollo_modelOutput` for output to the screen, and `apollo_saveOutput` for output to files. These two commands do not return an object as output, i.e. are called without an object to assign the output to. In their default versions, these functions are called with only the `model` object as input, i.e.:

```
apollo_modelOutput(model)
```

and

```
apollo_saveOutput(model)
```

In addition, it is possible to call both functions with an additional argument that is a list of settings, i.e.

```
apollo_modelOutput(model,
                    modelOutput_settings)
```

and

```
apollo_saveOutput(model,
                  saveOutput_settings)
```

The two lists `modelOutput_settings` and `saveOutput_settings` have a number of arguments that are all optional, namely:

apollo_modelOutput and apollo_saveOutput: user-controlled settings

printBHHH: If set to TRUE, the code will output BHHH standard errors as well as robust standard errors. This setting then also affects the reporting of t-ratios, p-values and covariance/correlation matrices.

printChange: If set to TRUE, the changes from the starting values are reported for the estimated parameters (default is FALSE for `apollo_modelOutput` and `apollo_saveOutput`).

printClassical: If set to TRUE, the code will output classical standard errors as well as robust standard errors, computed using the sandwich estimator. This setting then also affects the reporting of t-ratios, p-values and covariance/correlation matrices. If the computation of classical standard errors fails for some parameters, the user is alerted to this even if classical standard errors are not reported (default is TRUE for `apollo_modelOutput` and `apollo_saveOutput`).

printCorr: if set to TRUE, the correlation matrix of parameters is reported (default is FALSE for `apollo_modelOutput` and TRUE for `apollo_saveOutput`).

printCovar: If set to TRUE, the covariance matrix of parameters is reported (default is FALSE for `apollo_modelOutput` and TRUE for `apollo_saveOutput`).

printDataReport: If set to TRUE, a summary of each model's dependant variable is reported (default is TRUE for `apollo_modelOutput` and `apollo_saveOutput`).

printFixed: If set to TRUE, the code will print fixed parameters as well as estimated parameters (default is TRUE for `apollo_modelOutput` and `apollo_saveOutput`).

printFunctions: If set to TRUE, a copy of `apollo_probabilities`, `apollo_randomCoeff`, `apollo_lcPars`, `apollo_control`, `apollo_HB` and `estimate_settings$scaling`, are printed, as well as a list of the methods attempted to calculate the Hessian matrix (default is FALSE for `apollo_modelOutput` and TRUE for `apollo_saveOutput`).

printHBconvergence: If set to TRUE, Geweke convergence tests are printed for HB estimation (default is FALSE for `apollo_modelOutput` and TRUE for `apollo_saveOutput`).

printHBiterations: If set to TRUE, an iteration report is printed for HB estimation (default is TRUE for `apollo_modelOutput` and `apollo_saveOutput`).

printModelStructure: If set to TRUE, a summary of the model structure is reported. Only some models report their model structure: NL, CNL and MDCNEV (default is TRUE for `apollo_modelOutput` and `apollo_saveOutput`).

printOutliers: If set to TRUE, the 20 worst outliers in terms of lowest average probabilities for the chosen alternative are reported (default is FALSE for `apollo_modelOutput` and `apollo_saveOutput`). Alternatively, a scalar can be provide to use instead of 20 to change the number of outliers reported.

printPVal: If set to 0, p-values are not reported. If set to 1, then one-sided p-values are reported. If set to 2, then two-sided p-values are reported (default value is 0 for both `apollo_modelOutput` and `apollo_saveOutput`).

printT1: if set to TRUE, t-ratios against 1 are reported in addition to t-ratios against 0, where this is useful for Nested Logit models and for multipliers (default is FALSE for `apollo_modelOutput` and `apollo_saveOutput`).

The main outputs controlled by the above settings will determine what `apollo_saveOutput` writes into the main output file, which will be called

`modelName_output.txt` where `modelName` is as defined in `apollo_control`. When saving outputs to files, `saveOutput_settings` list has five additional possible settings, namely:

apollo_saveOutput: additional user-controlled settings

saveEst: If set to TRUE, the code will save a *csv* file with the parameter estimates, standard errors and t-ratios, saved as `modelName_estimates.csv` (default is TRUE).

saveCorr: If set to TRUE, a *csv* file will be produced with the correlation matrix, saved as `modelName_robcorr.csv`, where, if `printClassical==TRUE`, a separate file will be produced with the classical correlation matrix, saved as `modelName_corr.csv` (default is TRUE).

saveCov: If set to TRUE, a *csv* file will be produced with the covariance matrix, saved as `modelName_robcovar.csv`, where, if `printClassical==TRUE`, a separate file will be produced with the classical covariance matrix, saved as `modelName_covar.csv` (default is TRUE).

saveHBiterations: If set to TRUE, iterations from HB estimation (cf. Section 8.1) are the saved model object (default is FALSE).

saveModelObject: If set to TRUE, an output file of the *rds* (an R format) will be produced containing the model object, saved as `modelName_model.rds` (default is TRUE).

saveOld: If set to TRUE, existing files are kept with an added OLD suffix. If not, they are overwritten (default is TRUE).

writeF12: If set to TRUE, the code will produce an F12 file, which is an output format used by the ALogit software (ALogit, 2016)^a, saved as `modelName.f12` (default is FALSE).

^aThis is file containing all key model outputs. It is also produced by Biogeme. ALogit provides a shell to compare the results across models using these files, which can come from different estimation packages. See www.alogit.com

It is worth noting that if a user has previously run `apollo_saveOutput` from the same model, the old output files will be renamed rather than overwritten unless `saveOutput_settings$saveOld=FALSE`.

An example of the on screen output is shown in Figure 4.9. For `apollo_saveOutput`, a text file containing output using the above settings will be produced, using a filename corresponding to `apollo_control$modelName`. The default settings imply a more verbose output for the log file as opposed to the on screen output, in addition to files with estimates, covariance matrices etc, unless instructed not to. The majority of the output in Figure 4.9 is self-explanatory.

Apollo reports convergence information, including the eigenvalue of the Hessian that is closest to zero. Small values can indicate convergence issues. A special warning message is displayed if some of the eigenvalues are positive. For `bgw`, “Relative function convergence” is standard convergence.

The output reports starting and final log-likelihood, and where appropriate (model dependent), also the log-likelihood at zero values for all parameters, defined as $LL(0)$; and for a model with constants only, defined as $LL(c)$. A number of goodness-of-fit statistics are included. For discrete choice models, *Apollo* reports the ρ^2 measure and the adjusted ρ^2 measures, calculated both against a model with zero parameters, and model with constants


```

1 > apollo_modelOutput(model)
2 Model name : MNL_SP_covariates
3 Model description : MNL model with socio-demographics on mode choice SP
  ↳ data
4 Model run at : 2025-02-25 11:13:52.963517
5 Estimation method : bgw
6 Model diagnosis : Relative function convergence
7 Optimisation diagnosis : Maximum found
8   hessian properties : Negative definite
9   maximum eigenvalue : -2.818749
10  reciprocal of condition number : 2.9745e-08
11 Number of individuals : 500
12 Number of rows in database : 7000
13 Number of modelled outcomes : 7000
14
15 Number of cores used : 1
16 Model without mixing
17
18 LL(start) : -5598.9
19 LL at equal shares, LL(0) : -8196.02
20 LL at observed shares, LL(C) : -6706.94
21 LL(final) : -4830.94
22 Rho-squared vs equal shares : 0.4106
23 Adj.Rho-squared vs equal shares : 0.4085
24 Rho-squared vs observed shares : 0.2797
25 Adj.Rho-squared vs observed shares : 0.2776
26 AIC : 9695.89
27 BIC : 9812.4
28
29 Estimated parameters : 17
30 Time taken (hh:mm:ss) : 00:00:2.07
31   pre-estimation : 00:00:0.32
32   estimation : 00:00:0.2
33   post-estimation : 00:00:1.56
34 Iterations : 8
35
36 Unconstrained optimisation.
37
38 Estimates:
39
40      Estimate      s.e.    t.rat.(0)    Rob.s.e. Rob.t.rat.(0)
41 asc_car      0.000000      NA          NA          NA          NA
42 asc_bus      0.286354    0.582952    0.4912    0.549513    0.5211
43 asc_air     -0.903377    0.373537   -2.4184    0.361570   -2.4985
44 asc_rail    -2.092701    0.353360   -5.9223    0.351092   -5.9606
45 asc_bus_interaction_female  0.340193    0.132783    2.5620    0.145286    2.3415
46 asc_air_interaction_female  0.268174    0.091507    2.9307    0.095283    2.8145
47 asc_rail_interaction_female  0.189612    0.073759    2.5707    0.078168    2.4257
48 b_tt_car    -0.013107    7.3440e-04  -17.8475    7.6776e-04  -17.0722
49 b_tt_bus    -0.021265    0.001598   -13.3107    0.001518   -14.0090
50 b_tt_air    -0.016578    0.002774   -5.9770    0.002671   -6.2074
51 b_tt_rail   -0.007051    0.001811   -3.8929    0.001765   -3.9959
52 b_tt_interaction_business -0.006234    6.0074e-04  -10.3772    5.9078e-04  -10.5520
53 b_access    -0.021152    0.002865   -7.3842    0.002706   -7.8156
54 b_cost      -0.076188    0.002097   -36.3304    0.002095   -36.3623
55 b_cost_interaction_business  0.033379    0.002739    12.1847    0.002572    12.9786
56 cost_income_elast -0.613821    0.030094   -20.3970    0.030605   -20.0562
57 b_no_frills  0.000000      NA          NA          NA          NA
58 b_wifi      1.026711    0.056142    18.2877    0.057815    17.7587
59 b_food      0.422072    0.055027    7.6703    0.056464    7.4750

```

Figure 4.9: On screen output obtained using `apollo_modelOutput` for MNL model

only. Using the calculation against $LL(0)$ as our example, the basic ρ^2 is calculated as:

$$\rho^2 = 1 - \frac{LL(\hat{\beta})}{LL(0)} \quad (4.10)$$

while the adjusted ρ^2 is given by:

$$\bar{\rho}^2 = 1 - \frac{LL(\hat{\beta}) - K}{LL(0)}, \quad (4.11)$$

where K is the number of estimated parameters.

When computing the ρ^2 against a model at observed shares, the formula is given by:

$$\rho^2(\bar{OS}) = 1 - \frac{LL(\hat{\beta}) - K + J - 1}{LL(\beta_{OS})}, \quad (4.12)$$

where J is the number of alternatives.

In addition, for all models, *Apollo* reports the Akaike Information Criterion (AIC), given by:

$$AIC = -2LL(\hat{\beta}) + 2K, \quad (4.13)$$

with K being number of estimated parameters, and the Bayesian Information Criterion (BIC), given by:

$$BIC = -2LL(\hat{\beta}) + K \ln(N), \quad (4.14)$$

where N is the number of observations (not individuals) in the data.

This is then followed by the number of estimated parameters, the estimation time (divided into subcomponents) and iterations taken, and finally the estimates and covariance matrix information.

In addition to using `apollo_modelOutput`, a user can also obtain simplified output about a model by calling `print(model)` (or simply `model`) or `summary(model)`.

The simplest on screen output can be obtained via:

```
print(model)
```

No arguments other than `model` can be passed to `print(model)`. The output obtained using `print(model)` is shown in Figure 4.10.

A slightly more detailed version of the output is obtained using:

```
summary(model)
```

When called without arguments other than `model`, this produces the output in Figure 4.11, which reports one-sided p -values. For two-sided tests, the function is called as:

```
summary(model, pTwoSided = TRUE)
```

```

1 > print(model)
2 Apollo model summary
3
4 Model name           : MNL_SP_covariates
5 Model description    : MNL model with socio-demographics on mode choice SP data
6 Estimation method    : bgw
7
8 LL(final)           : -4830.94
9 Estimated parameters : 17
10
11 EEstimates:
12             asc_bus             asc_air             asc_rail
13             0.286400           -0.903400           -2.093000
14 asc_bus_interaction_female asc_air_interaction_female asc_rail_interaction_female
15             0.340200             0.268200             0.189600
16             b_tt_car             b_tt_bus             b_tt_air
17             -0.013110           -0.021270           -0.016580
18             b_tt_rail b_tt_interaction_business             b_access
19             -0.007051           -0.006234             -0.021150
20             b_cost b_cost_interaction_business             cost_income_elast
21             -0.076190             0.033380             -0.613800
22             b_wifi             b_food
23             1.027000             0.422100
24
25 For more detailed output, use summary, or apollo_modelOutput for full outputs

```

Figure 4.10: On screen output obtained using `print(model)` for MNL model

4.8 Extension: iterative coding of utilities

In the examples shown in this manual, the user codes the utilities of all alternatives one by one. With very large choice sets, and/or large numbers of attributes, this may not be practical, and a user may create the utilities recursively. We illustrate this in [MNL_iterative_coding.r](#).

For this example, we generate a large dataset with 100 alternatives, and 10 attributes per alternative, leading to a database with 1,000 columns for the attributes, labelled as e.g. `x_j_k` for attribute k for alternative j . In the example, we assume the utility of each alternative contains no alternative specific constant (ASC). Figure 4.12 shows the definition of utilities, availabilities and the names of the alternatives in a recursive way, inside `apollo_probabilities`.

In this example code, we make use of three specific R functions that are especially useful for this context. Firstly, `paste0` creates a string by combining the various elements that are passed to it as input. This string can be used directly inside `utilities` as an index. It can also be used to refer to an actual variable in combination with the function `get`. Finally, the function `setNames` is used to assign the names from `V` to the vector with the codings for the alternatives.

In Figure 4.12, we define availability by explicitly extracting the relevant columns from the database. The availability columns follow a similar naming structure to the attributes, with, for example, `av_30` storing the the availability of alternative 30. Note that when referencing the database we must point to the `apollo_inputs$database`, as opposed to simply `database`. If all alternatives are always available, then we could just skip the definition of availability altogether, and `apollo_mnl` would assume full availability. The same rationale as for using

```

1 > summary(model)
2 Apollo model summary
3
4 Model name           : MNL_SP_covariates
5 Model description    : MNL model with socio-demographics on mode choice SP data
6 Estimation method    : bgw
7 Modelled outcomes    : 7000
8
9 LL(final)           : -4830.94
10 Estimated parameters : 17
11
12
13 Estimates (robust covariance matrix, 1-sided p-values):
14
15               estimate std. error t-ratio p (1-sided)
16 asc_bus          0.28635   0.54951    0.5    0.301
17 asc_air         -0.90338   0.36157   -2.5    0.006 **
18 asc_rail        -2.09270   0.35109   -6.0    1e-09 ***
19 asc_bus_interaction_female  0.34019   0.14529    2.3    0.010 **
20 asc_air_interaction_female  0.26817   0.09528    2.8    0.002 **
21 asc_rail_interaction_female  0.18961   0.07817    2.4    0.008 **
22 b_tt_car        -0.01311   0.00077  -17.1   <2e-16 ***
23 b_tt_bus        -0.02127   0.00152  -14.0   <2e-16 ***
24 b_tt_air        -0.01658   0.00267   -6.2    3e-10 ***
25 b_tt_rail       -0.00705   0.00176   -4.0    3e-05 ***
26 b_tt_interaction_business -0.00623   0.00059  -10.6   <2e-16 ***
27 b_access        -0.02115   0.00271   -7.8    3e-15 ***
28 b_cost         -0.07619   0.00210  -36.4   <2e-16 ***
29 b_cost_interaction_business  0.03338   0.00257   13.0   <2e-16 ***
30 cost_income_elast -0.61382   0.03060  -20.1   <2e-16 ***
31 b_wifi          1.02671   0.05781   17.8   <2e-16 ***
32 b_food          0.42207   0.05646    7.5    4e-14 ***
33 ---
34 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
35
36 For more detailed output, use apollo_modelOutput

```

Figure 4.11: On screen output obtained using `summary(model)` for MNL model

`apollo_inputs$database` also applies to `apollo_inputs$J` and `apollo_inputs$K`, as the values have been stored as objects inside `apollo_inputs`.

```

1  # #####
2  #### DEFINE MODEL PARAMETERS          ####
3  # #####
4
5  ### Vector of parameters, including any that are kept fixed in estimation
6  apollo_beta = setNames(rep(0,K),paste0("beta_",1:K))
7
8  ...
9
10 # #####
11 #### GROUP AND VALIDATE INPUTS          ####
12 # #####
13
14 apollo_inputs = apollo_validateInputs()
15 apollo_inputs$J = J # need to retain J (number of alternatives) for use
16                   # inside apollo_probabilities by putting it into apollo_inputs
17 apollo_inputs$K = K # need to retain K (number of attributes) for use
18                   # inside apollo_probabilities by putting it into apollo_inputs
19
20 # #####
21 #### DEFINE MODEL AND LIKELIHOOD FUNCTION          ####
22 # #####
23
24 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
25
26   ...
27
28   ### List of utilities: these must use the same names as in mnl_settings, order is irrelevant
29   V = list()
30   for(j in 1:apollo_inputs$J)
31     V[[paste0("alt_",j)]] = 0
32     for(k in 1:apollo_inputs$K) V[[paste0("alt_",j)]] = V[[paste0("alt_",j)]] +
33       get(paste0("beta_",k))*get(paste0("x_",j,"_",k))
34
35   ### Define settings for MNL model component
36   mnl_settings = list(
37     alternatives = setNames(1:apollo_inputs$J, names(V)),
38     avail       = setNames(apollo_inputs$database[,paste0("avail_",1:apollo_inputs$J)],
39       ↪ names(V)),
40     choiceVar   = choice,
41     utilities   = V
42   )
43   ...
44 }

```

Figure 4.12: Defining models for large choice sets ([MNL_iterative_coding.r](#))

5

Other model components

In Chapter 4, we gave a detailed overview of the approach to specifying and estimating models in *Apollo*. In this section, we look at the use of other model components, thus replacing the part of the code discussed in Section 4.5.2. We discuss ready-to-use functions for a number of commonly used models. We cover structures belonging to the family of random utility models as well as those that do not. We then look at models that are not for discrete choice, looking at aggregate/shares data, ordered choice data as well as discrete-continuous data. We finally explain how a user can add his/her own model functions.

5.1 Other RUM-consistent discrete choice models

5.1.1 Nested Logit

For the Nested Logit (NL) model (Daly and Zachary, 1978; McFadden, 1978; Williams, 1977), we use the efficient implementation of Daly (1987) but adapt it to the more commonly used version which divides the utilities by the nesting parameter in the within nest probabilities (see the discussions in Train 2009, chapter 4 and Koppelman and Wen 1998). Let us assume we have a nesting structure with three levels and that alternative i falls into nest o_m on the lowest level of nesting, which itself is a member of nest m on the upper level of nesting, with m being in the root nest. Using λ for nesting parameters, we would then have that $0 < \lambda_{o_m} \leq \lambda_m \leq \lambda_r \leq 1$. The probability¹² of person n choosing alternative i in choice situation t is then given by:

$$P_{i,n,t} = P_{m,n,t} P_{(o_m|m),n,t} P_{(i|o_m),n,t} \quad (5.1)$$

¹²For the sake of simplicity of notation, we assume here that all alternatives are available in every choice situation for every person. In the code, we of course allow for departures from this assumption.

where

$$P_{(i|o_m),n,t} = \frac{e^{\left(\frac{V_{i,n,t}}{\lambda_{o_m}}\right)}}{\sum_{j \in o_m} e^{\left(\frac{V_{j,n,t}}{\lambda_{o_m}}\right)}} \quad (5.2)$$

$$P_{(o_m|m),n,t} = \frac{e^{\left(\frac{\lambda_{o_m}}{\lambda_m} I_{o_m,n,t}\right)}}{\sum_{l_m=1}^{M_m} e^{\left(\frac{\lambda_{l_m}}{\lambda_m} I_{l_m,n,t}\right)}} \quad (5.3)$$

$$P_{m,n,t} = \frac{e^{\left(\frac{\lambda_m}{\lambda_r} I_{m,n,t}\right)}}{\sum_{m=1}^M e^{\left(\frac{\lambda_l}{\lambda_r} I_{l,n,t}\right)}} \quad (5.4)$$

with

$$I_{m,n,t} = \log \sum_{l_m=1}^{M_m} e^{\left(\frac{\lambda_{l_m}}{\lambda_m} I_{l_m,n,t}\right)} \quad (5.5)$$

$$I_{o_m,n,t} = \log \sum_{j \in o_m} e^{\left(\frac{V_{j,n,t}}{\lambda_{o_m}}\right)}, \quad (5.6)$$

where $0 < \lambda_{o_m} \leq \lambda_m \leq \lambda_r \leq 1$, and where there are M nests at the upper level, with M_m lower-level nests contained in nest m . For normalisation, we set $\lambda_r = 1$, i.e. using normalisation at the top.

In the efficient implementation of [Daly \(1987\)](#), we then work in logs, where we define a set of elementary alternatives $\mathbf{E}$ and a tree function $\mathbf{t}$. The tree function gives us a set of composite nodes $\mathbf{C} = (\mathbf{t}(j), \mathbf{t}(\mathbf{t}(j)), \dots \forall j \in \mathbf{E})$. For each elementary alternative, there is a single path up to the root r , where, for alternative i , this is given by: $\mathbf{A}(i, r, \mathbf{t}) = (i, \mathbf{t}(i), \mathbf{t}(\mathbf{t}(i)), \dots, r)$. We then have that:

$$\log(P_{i,n,t}) = \sum_{a \in \mathbf{A}(j,r,\mathbf{t})} \frac{1}{\lambda_{\mathbf{t}(a)}} \left(V_{a,n,t} - \widetilde{V_{\mathbf{t}(a),n,t}} \right). \quad (5.7)$$

where $\lambda_{\mathbf{t}(a)}$ is the nesting parameter for the nest that contains a , and for any non-elementary elements a , we have:

$$\widetilde{V_{a,n,t}} = \lambda_a \log \sum_{l \in a} \exp\left(\frac{V_{l,n,t}}{\lambda_a}\right) \quad (5.8)$$

where $l \in a$ gives all the elements contained in a , which can be a mixture of nests and elementary alternatives. For normalisation, we set $\lambda_r = 1$, and for consistency with utility maximisation, we then have that $0 < \lambda_a \leq 1, \forall a$ and $\lambda_a \leq \lambda_{\mathbf{t}(a)}$, i.e. the λ terms in a given chain $\mathbf{A}(j, r, \mathbf{t})$ decrease as we go from the root down the tree.

In the actual user syntax, we adopt an approach inspired by ALogit¹³ ([ALogit, 2016](#)) where a user needs to specify a chain going from the root to each of the elementary alternatives. To illustrate this, we look at an example on the data described in Section 3.1, where we implement a three-level NL (`NL_three_levels.r`). A simpler two-level model is available in `NL_two_levels.r`. Note that *Apollo* does not impose any constraints on the nesting parameters (though a warning will be printed if the values are not in line with theory), and it

¹³www.alogit.com

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2    ...
3
4    ### Specify nests for NL model
5    nlNests      = list(root=1, PT=lambda_PT, fastPT=lambda_fastPT)
6
7    ### Specify tree structure for NL model
8    nlStructure= list()
9    nlStructure[["root"]] = c("car", "PT")
10   nlStructure[["PT"]]   = c("bus", "fastPT")
11   nlStructure[["fastPT"]] = c("air", "rail")
12
13   ### Define settings for NL model
14   nl_settings <- list(
15     alternatives = c(car=1, bus=2, air=3, rail=4),
16     avail       = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
17     choiceVar   = choice,
18     utilities   = V,
19     nlNests     = nlNests,
20     nlStructure = nlStructure
21   )
22
23   ### Compute probabilities using NL model
24   P[["model"]] = apollo_nl(nl_settings, functionality)
25
26   ...
27 }

```

Figure 5.1: Nested Logit implementation (extract)

is the user's role to ensure that the starting values and final estimates are consistent with theory. Example with constraints are given in [NL_two_levels_constraints_on_lambda.r](#) and [NL_three_levels_constraints_on_lambdas.r](#).

In the first level of the tree, alternatives are divided into public transport (PT) alternatives and car, while the PT alternatives are then further split into a nest (fastPT) containing rail and air, where bus is on its own. To estimate this model, we specify two additional parameters compared to the MNL model in Section 4.5.2 in the `apollo_beta` vector, namely `lambda_PT` and `lambda_fastPT`.

Just like `apollo_mnl`, the `apollo_nl` function is called as follows:

```

P[["model"]] = apollo_nl(nl_settings,
                        functionality)

```

The list `nl_settings` contains all the same elements as for MNL, i.e. the compulsory inputs `alternatives`, `choiceVar` and `utilities` and the optional inputs `avail`, `rows` and `componentName`. For Nested Logit, the list `nl_settings` needs to contain the following arguments:

apollo_nl: user-controlled settings

- alternatives:** A named vector containing the names of the alternatives as defined by the user, and for each alternative, giving the value used in the dependent variable in the data.
- avail:** A list containing one element per alternative, using the same names as in **alternatives**. For each alternative, we define the availability either through a vector of values of the same length as the number of observations (i.e. a column from the data) or by a scalar of 1 if an alternative is always available. A user can also set **avail=1** (or omit its use) which implies that all of the alternatives are available for every choice observation in the data.
- choiceVar:** A vector of length equal to the number of observations, containing the chosen alternative for each observation.
- componentName:** This is an optional argument of the character type which allows the user to specify a name of the given model component. This is then used in various places in diagnostic tests and model outputs. If omitted, a default is used by *Apollo*. It is recommended the user omits this setting and lets Apollo set it up automatically.
- nlNests:** A named vector containing the names of the nests and the associated structural parameters λ . For each λ , we give the name of the associated parameter. This list needs to include the **root**, which is the only nest for which the choice of name is not free for the user to determine.
- nlStructure:** A list containing one element per nest, where each element is a vector with the names of the contents of that nest, which can itself be a mix of nests and elementary alternatives.
- rows:** This is an optional argument which is missing by default. It allows the user to specify a vector called **rows** of the same length as the number of rows in the data. This vector needs to use logical statements to identify which rows in the data are to be used for this model. For any observations in the data where the entry in **rows** is set to FALSE, the probability for the model will be set to 1. This means that, for this model, this observation does not contribute to the calculation of the likelihood and hence estimation of the model parameters. It is useful for example in the case of hybrid choice models, a point we return to in Section 7. When omitted from the call, all rows are used.
- utilities:** A list object containing one utility for each alternative, using the same names as in **alternatives**, where any linear or non-linear specification is possible. For models with large numbers of alternatives and/or attributes, users may consider iterative coding of utilities, as discussed in Section 4.8.

In our example, as illustrated in Figure 5.1 (where we do not show the definition of alternatives, availabilities, choices and utilities, as these remain the same as in the MNL model in Section 4.5.2), we have three nests, where this includes the root. The order of elements is of no importance as they are identified by the nest names, yet, for consistency, using the same order as in the model structure which follows is advisable. For each nest, we give the nesting parameter, using the parameter names previously defined in **apollo_beta**. The final step in the definition of the NL model is a call to **apollo_nl** with the appropriate inputs. Extensive checks are performed by this function, notably ensuring that for each alternative, there is exactly one chain from the root to the bottom of the tree.

After estimation, the model reports estimates for all parameters, as for any model,


```

1 Nest: root (1)
2 |-----Alternative: car
3 '-Nest: PT (0.6953)
4   |----Alternative: bus
5   '-Nest: fastPT (0.5862)
6     |-Alternative: air
7     '-Alternative: rail

```

Figure 5.2: Nested Logit tree structure after estimation

but in addition prints out (except if `apollo_control$noDiagnostics==FALSE`) the resulting tree structure with the estimated nesting parameters in brackets (and this is repeated with `apollo_modelOutput` and `apollo_saveOutput` if `printDiagnostics` is set to `TRUE` in their respective settings). In the case of our example, shown in Figure 5.2, we see that, as intended, there is a direct link from the root to the car alternative, while all other alternatives are nested in a public transport nest, with a further layer of nesting for rail and air within that nest. The nesting parameters also follow the required decreasing trend when going from the root down the tree.

5.1.2 Cross-nested Logit

For our implementation of the Cross-nested Logit (CNL) model (Vovsha, 1997), we follow the “Generalised Nested Logit” (GNL) model of Wen and Koppelman (2001), with all nesting parameters freely estimated, and the constraint on the allocation parameters (showing the membership of alternative j in nest m) that $0 \leq \alpha_{j,m} \leq 1, \forall j, m$ and $\sum_m \alpha_{j,m} = 1, \forall j$. Only two-level versions of CNL are available through the `apollo_cnl` function, i.e. one layer of nests below the root, with the membership of a non-root nest being made up entirely of elementary choice alternatives.

In our implementation, each alternative needs to fall into at least one nest on the second level of the tree, where this can be a single alternative nest. We then have M nests, S_1 to S_M , where $\alpha_{j,m}$ represents allocation of alternative j to nest S_m . The probabilities are then given by a sum over nests:

$$P_{i,n,t} = \sum_{m=1}^M P_{S_m,n,t} P_{(i|S_m),n,t} \quad (5.9)$$

where

$$P_{S_m,n,t} = \frac{\left(\sum_{j \in S_m} (\alpha_{j,m} e^{V_{j,n,t}})^{\frac{1}{\lambda_m}} \right)^{\lambda_m}}{\sum_{l=1}^M \left(\sum_{j \in S_l} (\alpha_{j,l} e^{V_{j,n,t}})^{\frac{1}{\lambda_l}} \right)^{\lambda_l}} \quad (5.10)$$

$$P_{(i|S_m),n,t} = \frac{(\alpha_{i,m} e^{V_{i,n,t}})^{\frac{1}{\lambda_m}}}{\sum_{j \in S_m} (\alpha_{j,m} e^{V_{j,n,t}})^{\frac{1}{\lambda_m}}} \quad (5.11)$$

Just like `apollo_mnl` and `apollo_nl`, the `apollo_cnl` function is called as follows:

```

P[["model"]] = apollo_cnl(cnl_settings,
                           functionality)

```

The list `cnl_settings` contains all the same elements as for MNL, along with arguments defining the nesting structure. Specifically, the possible entries in the list `cnl_settings` are:

apollo_cnl: user-controlled settings

- alternatives:** A named vector containing the names of the alternatives as defined by the user, and for each alternative, giving the value used in the dependent variable in the data.
- avail:** A list containing one element per alternative, using the same names as in **alternatives**. For each alternative, we define the availability either through a vector of values of the same length as the number of observations (i.e. a column from the data) or by a scalar of 1 if an alternative is always available. A user can also set **avail=1** (or omit its use) which implies that all of the alternatives are available for every choice observation in the data.
- choiceVar:** A vector of length equal to the number of observations, containing the chosen alternative for each observation.
- cnlNests:** A named vector containing the names of the nests and the associated structural parameters λ . For each λ , we give the name of the associated parameter. Unlike in **apollo_nl**, the **root** is not included for **apollo_cnl** as only two-level structures are used.
- cnlStructure:** A matrix showing the allocation of alternatives to nests, with one row per nest and one column per alternative, using the same ordering as in **alternatives** and **cnlNests**.
- componentName:** This is an optional argument of the character type which allows the user to specify a name of the given model component. This is then used in various places in diagnostic tests and model outputs. If omitted, a default is used by *Apollo*. It is recommended the user omits this setting and lets *Apollo* set it up automatically.
- rows:** This is an optional argument which is missing by default. It allows the user to specify a vector called **rows** of the same length as the number of rows in the data. This vector needs to use logical statements to identify which rows in the data are to be used for this model. For any observations in the data where the entry in **rows** is set to **FALSE**, the probability for the model will be set to 1. This means that, for this model, this observation does not contribute to the calculation of the likelihood and hence estimation of the model parameters. It is useful for example in the case of hybrid choice models, a point we return to in Section 7. When omitted from the call, all rows are used.
- utilities:** A list object containing one utility for each alternative, using the same names as in **alternatives**, where any linear or non-linear specification is possible. For models with large numbers of alternatives and/or attributes, users may consider iterative coding of utilities, as discussed in Section 4.8.

For our implementation example on the simple mode choice data, we define a structure where air is nested together with rail (fast PT), and bus is nested together with rail (ground-based PT), while there is no joint nest membership for bus and air. Finally, car is nested on its own. This means that the only alternative for which allocation parameters need to be estimated is the rail alternative, where we have that $\alpha_{rail,fastPT} + \alpha_{rail,groundPT} = 1$, with both $\alpha_{rail,fastPT}$ and $\alpha_{rail,groundPT}$ being constrained to be between 0 and 1. Imposing constraints directly on the estimation routine is inefficient and can affect the standard error calculations (although we show an example of this in [CNL_constraint_on_alphas.r](#)). We instead recom-

mend the use of a logistic transform ([CNL_logistic_transform_for_alphas.r](#))¹⁴, where, with alternative j having an estimated allocation parameter for M different nests, we have that, for nest m :

$$\alpha_{j,m} = \frac{e^{(\alpha_{0,j,m})}}{\sum_{l=1}^M e^{(\alpha_{0,j,l})}}, \quad (5.12)$$

where a normalisation is required, for example fixing $\alpha_{0,j,1} = 0$. *Apollo* will not estimate a model where, at the starting values, the allocation parameters for a single alternative sum to a value different from 1.

Like in the NL model, we define a vector of names for the nests, `cnlNests`, which defines the names of the nests and the associated structural parameters λ , using the parameter names previously defined in `apollo_beta`.

In our example ([CNL_logistic_transform_for_alphas.r](#)), as illustrated in Figure 5.3 (where we do not show the definition of alternatives, availabilities, choices and utilities, as these remain the same as in the MNL and NL models), we have three nests, one for air and rail (fastPT), one for bus and rail (groundPT), and one for car, which is nested on its own. The nesting parameter for the car nest is set to 1 given this is a single alternative nest. Our CNL implementation is limited to a two-level structure, and all elementary alternatives need to belong to at least one nest below the root, even if these are single alternative nests. This means that all nests defined by the user are automatically positioned below the root and the root is thus not included in the definition of the nest or tree structure given by the user. Note that, as for Nested Logit, *Apollo* does not impose any constraints on the nesting parameters, and it is the user's role to ensure that the starting values and final estimates are consistent with theory.

For the allocation or nest membership parameters, car and bus both fall into one nest exactly, so they have $\alpha_{j,m} = 1$ for the specific nest m they fall into and the remaining ones are set to zero. For rail, we define $\alpha_{rail,fastPT} = \frac{e^{(\alpha_{0,rail,fastPT})}}{e^{(\alpha_{0,rail,fastPT})} + e^{(\alpha_{0,rail,groundPT})}}$, and obviously $\alpha_{rail,groundPT} = 1 - \alpha_{rail,fastPT}$, while we use the normalisation that $\alpha_{0,rail,groundPT} = 0$, by including the parameters `alpha0_rail_fastP` in `apollo_fixed`. The crucial part of the definition of a CNL model is again the actual model structure, which in our code is again called `cnlStructure`, where this is now made up of a matrix with one row per nest, and one alternative per column, where the entry in a given cell corresponds to the appropriate allocation parameter. The order of rows and columns needs to be consistent with the order in `cnlNests` and `alternatives`, respectively.

In the model output, the code reports the resulting tree structure with the estimated allocation and nesting parameters. In the case of our example, shown in Figure 5.4, we see that, as intended, car, bus and air all belong to one nest only, while the estimation has shown that the split for rail is almost 50-50, with the λ parameter being smaller in the fastPT nest. In reporting the allocation parameters, the code uses the final values used inside `cnlStructure`, i.e. after the logistic transform in our example.

¹⁴We also include a version without even these *soft* constraints, in [CNL.r](#).

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2    ...
3
4    ### Specify nests for CNL model
5    cnlNests = list(car=1,groundPT=lambda_groundPT,fastPT=lambda_fastPT)
6
7    ### Specify nest allocation parameters for alternatives included in multiple nests
8    ### logistic transform to ensure values between 0 and 1, and summing to 1
9    alpha_rail_fastPT = exp(alpha_rail_fastPT_logistic)/
10   ↪ (exp(alpha_rail_fastPT_logistic)+exp(alpha_rail_groundPT_logistic))
11   alpha_rail_groundPT = 1 - alpha_rail_fastPT
12
13   ### Specify tree structure, showing membership in nests (one row per nest, one column per
14   ↪ alternative)
15   cnlStructure = matrix(0, nrow=length(cnlNests), ncol=length(V))
16   cnlStructure[1,] = c( 1,  0,  0,  0 ) # car
17   cnlStructure[2,] = c( 0,  1,  0, alpha_rail_groundPT ) # groundPT
18   cnlStructure[3,] = c( 0,  0,  1, alpha_rail_fastPT ) # fastPT
19
20   ### Define settings for CNL model
21   cnl_settings <- list(
22     alternatives = c(car=1, bus=2, air=3, rail=4),
23     avail        = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
24     choiceVar    = choice,
25     utilities     = V,
26     cnlNests     = cnlNests,
27     cnlStructure = cnlStructure
28   )
29
30   ### Compute probabilities using CNL model
31   P[["model"]] = apollo_cnl(cnl_settings, functionality)
32   ...
33 }

```

Figure 5.3: Cross-nested Logit implementation (extract)

```

1  Structure for CNL model component :
2
3  car (alpha) bus (alpha) air (alpha) rail (alpha) lambda
4  car nest    1.000    0.000    0.000    0.0000    1.0000
5  groundPT nest 0.000    1.000    0.000    0.5071    0.5285
6  fastPT nest  0.000    0.000    1.000    0.4929    0.4012

```

Figure 5.4: Cross-nested Logit structure after estimation

5.2 Non-RUM decision rules for discrete choice

In this section, we present the use of two alternatives to RUM in *Apollo*, namely random regret minimisation (RRM) and Decision Field Theory (DFT).

5.2.1 Random regret minimisation (RRM)

The fundamental assumption in regret theory is that what matters is not only the realised outcome but also on what could have been obtained by selecting a different course of action.

This means that the model incorporates anticipated feelings of regret that would be experienced once ex-post decision outcomes are revealed to be “unfavourable”. The value of an alternative can thus only be assigned following a cross-wise evaluation of alternatives, and this is the cause for substantial increases in computational complexity with large choice sets.

In the example here, we use the μ -RRM model of [van Cranenburgh et al. \(2015\)](#), which can simplify to the standard RRM model of [Chorus \(2010\)](#). We define the deterministic regret for alternative j ($j = 1, \dots, J$) for respondent n in choice task t as:

$$R_{i,n,t} = \sum_{k=1}^K \sum_{j \neq i} \mu_k \ln \left(1 + e^{\frac{\beta_k}{\mu_k} (x_{j,n,t,k} - x_{i,n,t,k})} \right) \quad (5.13)$$

where β_k is the coefficient associated with attribute x_k , with $k = 1, \dots, K$. The regret is informed by all the pairwise comparisons, where regret for alternative i increases whenever an alternative $j \neq i$ performs better than i on a given attribute. When using extreme value error terms, RRM models are in fact Logit models, albeit not RUM-consistent models. With the assumption of type *I* extreme value errors, the probability of respondent n choosing alternative i in choice task t , is now simply given by a MNL formula as:

$$P_{i,n,t} = \frac{e^{-R_{i,n,t}}}{\sum_{j=1}^J e^{-R_{j,n,t}}}. \quad (5.14)$$

In RRM, we minimise the regret rather than maximising the utility, and this is achieved by maximising the negative regret in Equation 5.14.

Given the above point, any of the Logit family models in *Apollo* can be used also for regret minimisation, by simply replacing the utilities (i.e. `utilities`) by the negative of regret. The labour intensive part comes in specifying the regret functions for the alternatives, i.e. implementing Equation 5.13. A separate function is available for RRM, with the function `apollo_rrm` called as:

```
P[["model"]] = apollo_rrm(rrm_settings,
                           functionality)
```

The list `rrm_settings` contains several elements we already know for utility based models, such as the `alternatives`, `avail`, `choiceVar`, `rows` and `componentName`. For RRM, the list `rrm_settings` of settings is as follows:

apollo_rrm: additional user-controlled settings

alternatives: A named vector containing the names of the alternatives as defined by the user, and for each alternative, giving the value used in the dependent variable in the data.

avail: A list containing one element per alternative, using the same names as in **alternatives**. For each alternative, we define the availability either through a vector of values of the same length as the number of observations (i.e. a column from the data) or by a scalar of 1 if an alternative is always available. A user can also set **avail=1** (or omit its use) which implies that all of the alternatives are available for every choice observation in the data.

choiceset_scaling: An optional scaling input to deal with varying choice set sizes, with one entry per row in the database, often set to 2 divided by the number of available alternatives.

choiceVar: A vector of length equal to the number of observations, containing the chosen alternative for each observation.

componentName: This is an optional argument of the character type which allows the user to specify a name of the given model component. This is then used in various places in diagnostic tests and model outputs. If omitted, a default is used by *Apollo*. It is recommended the user omits this setting and lets Apollo set it up automatically.

regret_inputs: A named list of regret functions. This should contain one list per attribute, where these lists themselves contain two vectors, namely a vector of attributes (at the alternative level) and parameters (either generic or attribute specific). Zeros can be used for omitted attributes for some alternatives. The order for each attribute needs to be the same as the order in alternatives.

regret_scale: An optional named list of regret scales (i.e. μ_k). This should have the same length as **rrm_settings\$regret_inputs** or be a single entry in the case of a generic scale parameter across regret attributes.

rows: This is an optional argument which is missing by default. It allows the user to specify a vector called **rows** of the same length as the number of rows in the data. This vector needs to use logical statements to identify which rows in the data are to be used for this model. For any observations in the data where the entry in **rows** is set to FALSE, the probability for the model will be set to 1. This means that, for this model, this observation does not contribute to the calculation of the likelihood and hence estimation of the model parameters. It is useful for example in the case of hybrid choice models, a point we return to in Section 7. When omitted from the call, all rows are used.

rum_inputs: Named list of (optional) deterministic utilities of the alternatives to be included in combined RUM-RRM models. Names of elements must match those in alternatives.

An example of an RRM implementation is given in Figure 5.5, where we apply a MNL (i.e. non-nested) version of RRM to the mode choice data from Section 3.1. We use a simpler implementation than in Section 4.5.2, with no socio-demographics. This example is available in [RRM.r](#). The ASCs are used in the RUM part, with the remainder of the model using a RRM specification. We use a μ_{RRM} specification of the model.

In a RUM model, only the attributes applying to a given alternative are used in the utility for that alternative, and the absence of access time for car or service quality for car

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### Create RRM settings
11     rrm_settings=list(
12         alternatives = c(car=1, bus=2, air=3, rail=4),
13         avail        = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
14         choiceVar     = choice,
15         rum_inputs    = list(car = asc_car,
16                             bus = asc_bus,
17                             air = asc_air,
18                             rail = asc_rail),
19         regret_inputs = list(
20             time=list(x=list(time_car, time_bus, time_air, time_rail),
21                       b=list(b_tt_car,b_tt_bus,b_tt_air,b_tt_rail)),
22             cost=list(x=list(cost_car, cost_bus, cost_air, cost_rail),
23                      b=list(b_cost)),
24             access=list(x=list(0, access_bus, access_air, access_rail),
25                        b=list(b_access)),
26             frills=list(x=list(0,
27                               0,
28                               b_no_frills * ( service_air == 1 ) + b_wifi * ( service_air == 2 ) +
29                               ↪ b_food * ( service_air == 3 ),
30                               b_no_frills * ( service_rail == 1 ) + b_wifi * ( service_rail == 2 ) +
31                               ↪ b_food * ( service_rail == 3 )),
32                       b=1)),
33         regret_scale = list(mu_rrm)
34     )
35
36     ### Compute probabilities using RRM model
37     P[["model"]] = apollo_rrm(rrm_settings, functionality)
38
39     ### Take product across observation for same individual
40     P = apollo_panelProd(P, apollo_inputs, functionality)
41
42     ### Prepare and return outputs of function
43     P = apollo_prepareProb(P, apollo_inputs, functionality)
44     return(P)
45 }

```

Figure 5.5: Implementation of random regret MNL model

and bus is of no importance. In a RRM model, we need to create these attributes given that the differences across the alternatives are used for all attributes. We next define the regret function for each alternative, where we here only show the regret for the car alternative, with a corresponding formulation applying for other modes, each time using Equation 5.13. For the alternative specific constants (ASCs), we adopt the convention of entering them directly into the regret function rather than using them in the comparison across alternatives - i.e. they would appear outside the sum across alternatives in Equation 5.13. In the `mnl_settings` list, we now define `utilities` to be the negative of `R` by multiplying each element in `R` by `-1`. We finally make the call to `apollo_mnl`.

5.2.2 Decision field theory (DFT)

Decision field theory (DFT) originates in mathematical psychology (Busemeyer and Townsend, 1992, 1993) and is very different to both RUM and RRM. The key assumption under a DFT model is that the preferences for alternatives update over time. The decision-maker considers the alternatives until they reach an internal threshold (similar to the concept of satisficing, where one of the options is deemed ‘good enough’) or some external threshold (i.e. some time constraint, where a decision-maker stops deliberating on the alternatives as a result of running out of time to make the decision).

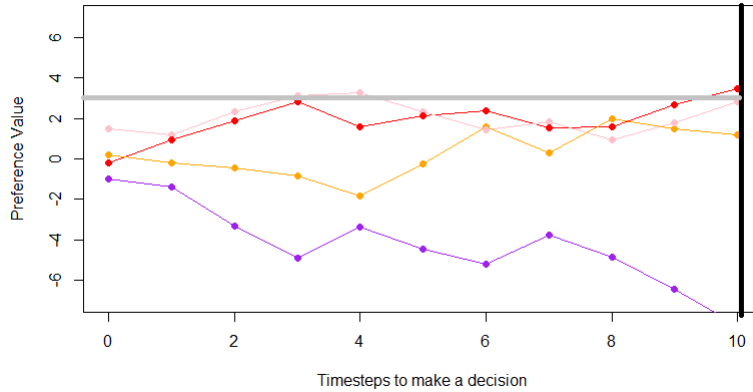


Figure 5.6: An example of a decision-maker stopping upon reaching either an internal or external threshold

An example of a decision process under DFT is given in Figure 5.6. In this particular example, the decision-maker chooses different alternatives if they make their choice after reaching an internal threshold (which is represented by the horizontal line) on the 4th preference updating timestep or if they conclude after 10 steps upon reaching a time threshold. Mathematically, DFT has been operationalised differently depending on whether internal or external thresholds are used. A full specification of DFT with internal thresholds is given by Busemeyer and Townsend (1993), while we focus here on DFT with external thresholds (c.f. Roe et al. (2001) for the first adaptation of DFT with external thresholds for multiple alternatives). For DFT with an external threshold, the preference values update stochastically as a result of the assumption that a decision-maker considers just one attribute of an alternative at each timestep. Consequently, the preference values for each alternative update iteratively:

$$P_t = S \cdot P_{t-1} + V_t, \quad (5.15)$$

where P_t is a column vector containing the preference values of each alternative i at timestep t (not to be confused with a choice task t). S is a feedback matrix with memory and sensitivity parameters (detailed in Equation 5.16) and V_t is a valence vector (Equation 5.17), which varies depending on which attribute is attended to at time t . The feedback matrix used in *Apollo* is based on the definition by [Hotelling et al. \(2010\)](#):

$$S = I - \phi_2 \times \exp(-\phi_1 \times D^2), \quad (5.16)$$

where I is the identity matrix of size n and n is the number of alternatives. The feedback parameter has two free parameters. The first, ϕ_1 , is a sensitivity parameter, which allows for competition between alternatives that are more similar (in terms of attribute values). The second, ϕ_2 , is a memory parameter, which captures whether attributes considered at the start of the deliberation process or attributes considered at the end are more important. Finally, D is some measure of distance between the alternatives. In our code, we use the Euclidean distance for simplicity. Next, the valence vector can be described as:

$$V_t = C \cdot M \cdot W_t + \varepsilon_t, \quad (5.17)$$

where C is a contrast matrix used to rescale the attribute values such that they sum to zero, M is a matrix containing the attribute values for all of the alternatives, $W_t = [0..1..0]'$ with entry $k = 1$ if and only if attribute k is the attribute being attended to by the decision-maker at timestep t , and ε_t is an error term.

The implementation of DFT in *Apollo* allows for two different ways of accounting for the relative importance of attributes. A user may define attribute importance weights w_k , for each attribute, that are to be estimated and which correspond to the likelihood of a decision-maker attending to that attribute k . These however have the limitation that they must sum to one, which consequently requires the user to have a priori knowledge on the directionality of attributes ([Hancock et al., 2018](#)). Alternatively, the analyst may use ‘attribute scaling coefficients’. These have many benefits (see [Hancock et al. 2019](#) for a detailed explanation of these), including, most importantly, avoiding the limitation of having to sum to one. By instead assuming that each attribute is attended to with the same likelihood (all weights, $w_k = 1/n$), the relative importance can instead enter as a set of scaling coefficients, s_k , which are applied to the attributes before they are entered (through M in Equation 5.17) into the calculation of the valence vector at each timestep.

The error term ε is drawn from independent and identically distributed normal draws with mean 0 and a standard deviation which is an estimated parameter. Consequently, the preference values P_t converge to a multivariate normal distribution ([Roe et al., 2001](#)). To calculate the probabilities of alternatives under DFT we thus simply require the expectation and covariance of P_t (ξ_t and Ω_t , respectively). Hence, the probability of choosing alternative j from a set of J alternatives at time t is:

$$P_{DFT} \left[\max_{i \in J} P_t[i] = P_t[j] \right] = \int_{X > 0} \exp \left[-(X - \Gamma)' \Lambda^{-1} (X - \Gamma) / 2 \right] / (2\pi |\Lambda|^{0.5}) dX, \quad (5.18)$$

with X the set of differences between the preference value for the chosen alternatives and each other alternative, $X = [P_t[j] - P_t[1], \dots, P_t[j] - P_t[J]]'$. Additionally, we require transformations of the expectation and covariance, $\Gamma = L\xi_t$, $\Lambda = L\Omega_t L'$, with L a matrix comprised of a column vector of 1s and a negative identity matrix of size $J - 1$ where J is the number of alternatives. The column vector of 1s is placed in the i^{th} column where i is the chosen alternative.

An implementation of DFT is given in `apollo_dft`, which is called as:

```
P[["model"]] = apollo_dft(dft_settings,
                          functionality)
```

where `dft_settings` contains the following elements:

apollo_rrm: additional user-controlled settings

alternatives: A named vector containing the names of the alternatives, as for other discrete choice models.

attrScalings: A list of scaling parameters that are applied to attribute values before they are passed into M in Equation 5.17. These do not need to sum to 1 across the set of attributes. Any attributes included in `attrValues` but missing from `attrScalings` will be ignored. Conversely, any attribute missing in `attrValues` but included in `attrScalings` will be created in `attrValues` but set to zero. Note that `attrScalings` should be set to 1 if `attrWeights` is provided.

altStart: A list containing a starting preference value for each alternative, using the same names as `alternatives`. As with other models, these are generally defined in `apollo_beta`, but could involve interactions with socio-demographics or be randomly distributed across individuals and/or observations.

attrValues: A list with attribute values for alternatives, with one list per alternative, using the names from `alternatives`. Each list contains the attribute values for that alternative, with one entry per attribute, where these are all column vectors with one entry per observation. DFT requires all alternatives to have each of the specified attributes, so by default will set attribute values of zero for any attributes not provided for a given alternative. Note that attributes specified here that are not included in either `attrWeights` or `attrScalings` will be ignored.

attrWeights: A list of weights, one for each attribute. These should sum to one and will be adjusted accordingly if they do not. As mentioned above, attributes included in `attrValues` but missing from `attrWeights` will be ignored. Conversely, attributes missing in `attrValues` but included in `attrWeights` will be created in `attrValues` but set to zero. Note that `attrWeights` should be set to 1 if `attrScalings` is provided.

avail: A list containing availabilities, as for other discrete choice models.

choiceVars: A variable indicting the column in the database which identifies the alternative chosen in a given choice situation, as for other discrete choice models.

componentName: The optional argument giving a name to the model component described already for earlier models.

procPars: A list containing the four DFT ‘process parameters’. These are:

- **error_sd:** the standard deviation of the error term in Equation 5.17.
- **timesteps:** the number of preference updating timesteps (t in Equations 5.15 and 5.17). [`apollo_dft` will automatically adjust the number of timesteps such that there is at least one timestep.
- **phi1** and **phi2:** sensitivity and process parameters from Equation 5.16.

All of these parameters can be entered as single values to be used across the dataset, or can take choice-set dependent values.

rows: The optional rows argument already described for the earlier models.

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2    ...
3    ### Create list of probabilities P
4    P = list()
5
6    ### Create alternative specific constants and coefficients using interactions with
7    ↪ socio-demographics
8    asc_bus_value = asc_bus + asc_bus_interaction_female * female
9    ...
10   b_cost_value = ( b_cost + b_cost_interaction_business * business ) * ( income / mean_income
11   ↪ ) ^ cost_income_elast
12
13   ### List of attribute values
14   attrValues = list()
15   attrValues[["car"]] = list(time=time_car, access=0, cost=cost_car, wifi=0, food=0)
16   ...
17   attrValues[["rail"]] = list(time=time_rail, access=access_rail, cost=cost_rail,
18   ↪ wifi=1*(service_rail == 2), food=1*(service_rail == 3))
19
20   ### List of initial preference values
21   altStart = list()
22   altStart[["car"]] = asc_car
23   ...
24   altStart[["rail"]] = asc_rail_value
25
26   ### List of attribute scaling factors
27   attrScalings = list(time = list(car = b_tt_car_value, bus = b_tt_bus_value, air =
28   ↪ b_tt_air_value, rail = b_tt_rail_value),
29   ...
30   food = b_food)
31
32   ### List of process parameters
33   procPars = list(
34     error_sd = p_error_sd,
35     timesteps = 1+exp(p_timesteps),
36     phi1 = exp(p_phi1),
37     phi2 = exp(p_phi2)/(1+exp(p_phi2))
38   )
39
40   ### Define settings for DFT model component
41   dft_settings <- list(
42     alternatives = c(car=1, bus=2, air=3, rail=4),
43     avail = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
44     choiceVar = choice,
45     attrValues = attrValues,
46     altStart = altStart,
47     attrWeights = 1,          ### Using scaling factors, so attrWeights must be set to 1.
48     attrScalings = attrScalings,
49     procPars = procPars
50   )
51
52   ### Compute choice probabilities using DFT model
53   P[["model"]] = apollo_dft(dft_settings, functionality)
54   ### Take product across observation for same individual
55   P = apollo_panelProd(P, apollo_inputs, functionality)
56   ### Prepare and return outputs of function
57   P = apollo_prepareProb(P, apollo_inputs, functionality)
58   return(P)
59 }

```

Figure 5.7: DFT implementation

An example of a DFT implementation is given in Figure 5.7, where we apply a DFT model with scale parameters to the mode choice data from Section 3.1. We use an identical implementation to that of the MNL model in Section 4.5.2, with the same socio-demographics parameters. This is a key advantage of using scaling parameters (with the weights instead being fixed) in a DFT model, as it allows us to make equivalent adjustments to the parameters. This example is available in `DFT_mode_choice.r`, where a simpler DFT model without covariates applied to the Swiss route choice data is available in `DFT_route_choice.r`.

Values for alternatives without a given attribute (wifi, food and access time for car, for example) are set to zero (and would be automatically set to zero if not initially provided). Additionally, DFT weights are automatically rescaled to sum to one, therefore attribute specific scalings (such as the one for the travel time coefficient in this example) are more efficiently employed through the use of attribute scaling parameters. Consequently, `attrWeights` is set to 1 in `dft_settings`.

Note that DFT process parameters can often cause identification or estimation issues (c.f. Hancock et al. 2019). Consequently, care is required, particularly when estimating DFT models on datasets where the process parameters are unlikely to have an impact, as poor initial starting values for the parameters can result in convergence to poor local optima. Here, we adjust the process parameters to aid estimation. We use exponentials to restrict the number of deliberation timesteps to be greater than 1 and the sensitivity parameter to be positive, and a logistic transform to ensure the memory parameter falls between 0 and 1. Additionally, with this data, we fix `error_sd` by including it in `apollo_fixed`. Finally, it is preferable to use non-zero starting values for all parameters.

5.3 Models for aggregate/shares data or uncertainty in the dependent variable

Let $C_{n,t}$ be the choice vector for individual n in choice situation t , with J alternatives in the choice set, such that $C_{n,t} = \langle s_{1,n,t}, \dots, s_{J,n,t} \rangle$, where $s_{j,n,t}$ is a choice indicator for alternative j . The type of data we have considered thus far is of the type where for each observation, a single alternative is chosen, such that $s_{j,n,t} \in \{0, 1\}, \forall j$ and $\sum s_{j,n,t} = 1$, meaning that if $s_{i,n,t} = 1$, then $s_{j,n,t} = 0, \forall j \neq i$, and still satisfying $\sum_{j=1}^J s_{j,n,t} = 1$.

We now instead consider a situation where $0 \leq s_{j,n,t} \leq 1, \forall j$ while still ensuring that $\sum s_{j,n,t} = 1$, meaning that multiple alternatives can have a non-zero outcome. This type of data can arise for a number of reasons. First, the data might be collected in a setting where individuals allocate their choices across a range of alternatives, in different proportions. Along the same line, data might be collected at an aggregate level, e.g. what share of choices for a given individual (or subgroup of a population) are for a specific alternative. Finally, there could be uncertainty in the choice from the perspective of the analyst, only capturing the deterministic outcome up to a probability.

Let $P_{j,n,t}(\beta)$ be the probability of individual n choosing alternative j in task t , conditional on utility parameters β . The Likelihood of the observed sequence of choices for person n is then given by

$$L_n(\beta) = \prod_{t=1}^{T_n} \prod_{j=1}^J P_{j,n,t}^{s_{j,n,t}}(\beta), \quad (5.19)$$

where, with discrete choice, this of course just collapses to a product across observations of the probability of the chosen alternative, i.e. Equation 4.4. When multiple elements in $C_{n,t}$

are non-zero, the product across all the non-zero elements needs to be used. For MNL, a further simplification arises in that

$$L_n(\beta) = \prod_{t=1}^{T_n} \prod_{j=1}^J P_{j,n,t}^{s_{j,n,t}}(\beta) = \prod_{t=1}^{T_n} \prod_{j=1}^J \left(\frac{e^{V_{j,n,t}}}{\sum_{l=1}^J e^{V_{l,n,t}}} \right)^{s_{j,n,t}} = \prod_{t=1}^{T_n} \frac{e^{\sum_{j=1}^J s_{j,n,t} V_{j,n,t}}}{\sum_{l=1}^J e^{V_{l,n,t}}}, \quad (5.20)$$

where the dependency of V on β is not shown explicitly. This is the Fractional Multinomial Logit (FMNL) model, which is a generalisation of the work of [Papke and Wooldridge \(1996\)](#). The reader will note that this is equivalent to replicating each observation for individual n J times, assigning the choice in each of these replicated rows to one of the J alternatives, and giving a weight to each of these rows that is equal to $s_{j,n,t}$.

In *Apollo*, the FMNL model is implemented in the function `apollo_fmnl`, which is called as follows:

```
P[["model"]] = apollo_fmnl(fmnl_settings,
                           functionality)
```

where the contents of `fmnl_settings` are:

apollo_fmnl: user-controlled settings

alternatives: A character named vector containing the names of the alternatives, where unlike for discrete choice models, no value referring to a choice column needs to be included here.

avail: A list containing availabilities, as for MNL, NL and CNL.

choiceShares: A list containing the names of the variables indicating the column in the database which identify the shares allocated to each alternative.

componentName: The optional argument giving a name to the model component described already for earlier models.

rows: The optional `rows` argument already described for the earlier models.

utilities: A list of utilities, as for MNL, NL and CNL.

The example included for FMNL with *Apollo* uses the time use data described in Section 3.4, where the share of a day allocated to each activity is used as the dependent variable. This example is implemented in `FMNL.r`, and is illustrated in Figure 5.8, where we use constants only in the utilities. The only real difference in comparison with the implementation of MNL models is that the `alternatives` settings is now simply a character vector, while the list `choiceShares` replaces the single vector `choiceVar`.

5.4 Models for ranking, rating and continuous dependent variables

Especially when developing hybrid choice models (cf. Section 7.3, some of the dependent variables in the model will not be of the discrete choice type. We now look at how to model such dependent variables in *Apollo*.

```

1  ##### DEFINE MODEL PARAMETERS
2  apollo_beta=c(asc_dropOff      = 0,
3                ...
4                asc_other        = 0)
5
6  apollo_fixed = c("asc_other")
7
8  ...
9
10 ##### DEFINE MODEL AND LIKELIHOOD FUNCTION
11 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
12
13     ...
14
15     ### List of utilities: these must use the same names as in fmn1_settings, order is irrelevant
16     V = list()
17     V[["dropOff"]    ] = asc_dropOff
18     ...
19     V[["other"]      ] = asc_other
20
21     ### Define settings for MNL model component
22     fmn1_settings = list(
23         alternatives = c("dropOff",
24                         ...
25                         "other"),
26         choiceShares = list(dropOff = t_a01,
27                             ...
28                             other   = t_a12),
29         utilities     = V
30     )
31
32     ### Compute probabilities using FMNL model
33     P[["model"]] = apollo_fmnl(fmn1_settings, functionality)
34
35     ...
36 }

```

Figure 5.8: FMNL implementation in *Apollo*

5.4.1 Exploded Logit

Datasets may include the full ranking for alternatives, in which case an Exploded Logit model can be used. In particular, with J different alternatives for individual n in choice situation t , we may observe the ranking $R_{nt} = \langle R_{nt,1}, \dots, R_{nt,J} \rangle$, where $R_{nt,1}$ is the index for the alternative which is ranked the highest, i.e. the choice in a simple discrete choice setting. Note that this is different from the convention where $R_{nt,j}$ is the rank of alternative j ; here, $R_{nt,j}$ refers to the specific alternative ranked in j^{th} place.

We then have that the probability of the observed ranking is given by:

$$P_{nt} = \prod_{i=1}^{J-1} \frac{e^{\mu_i V_{R_{nt,i}}}}{\sum_{j=i}^J e^{\mu_i V_{R_{nt,j}}}}, \quad (5.21)$$

where this is given by a product of Logit probabilities for all but the last ranking (which is just a single alternative), where the denominator gradually omits alternatives, and where we allow for differences in scale across the stages, with an appropriate normalisation, e.g. $\mu_1 = 1$.

In *Apollo*, the Exploded Logit model is implemented in the function `apollo_el`, which is called as follows:

```
P[["model"]] = apollo_el(el_settings,
                        functionality)
```

where the contents of `el_settings` are a little different from the earlier MNL, NL and CNL models. In particular, we have:

apollo_el: user-controlled settings

alternatives: A named vector containing the names of the alternatives, as for MNL, NL and CNL.

avail: A list containing availabilities, as for MNL, NL and CNL.

choiceVars: A list containing the names of the variables indicating the column in the database which identify the choices at each stage in the ranking, except for the final (worst) alternative. If not all alternatives are available for all individuals, then some of the later rankings will not apply for these individuals, and the user should put a value of -1 in the data for those entries. For example, if a given person only has two out of the four alternatives available, then the third and fourth ranking should be given as -1 in the data for that individual.

componentName: The optional argument giving a name to the model component described already for earlier models.

rows: The optional `rows` argument already described for the earlier models.

scales: An optional argument given by a list, with one entry per stage in the ranking, giving the scale parameter to be used in that stage.

utilities: A list of utilities, as for MNL, NL and CNL.

An example using the Exploded Logit model is given in [EL.r](#), using the drug choice data from [Section 3.3](#). We use a dummy coded specification for the three categorical variables, along with a continuous specification for risk and cost.

The utility for alternative j in choice situation t for individual n is given by:

$$\begin{aligned}
 V_{j,n,t} = & \sum_{s=1}^5 \beta_{brand_s} \cdot (x_{brand_{j,n,t}} == s) \\
 & + \sum_{s=1}^6 \beta_{country_s} \cdot (x_{country_{j,n,t}} == s) \\
 & + \sum_{s=1}^3 \beta_{characteristic_s} \cdot (x_{characteristic_{j,n,t}} == s) \\
 & + \beta_{side_effects} \cdot x_{side_effects_{j,n,t}} \\
 & + \beta_{price} \cdot x_{price_{j,n,t}}
 \end{aligned} \tag{5.22}$$

For the first three rows in Equation 5.22, one of the β parameters in each row is constrained to zero (dummy coded), and not all levels apply for each alternative, as described in [Appendix B](#).

The implementation of the model is shown in [Figure 5.9](#). Special care is required for the qualitative attributes. These are coded as text in the data, and one parameter needs to be

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### List of utilities: these must use the same names as in el_settings, order is irrelevant
11     V = list()
12     V[["alt1"]] = ( b_brand_Artemis*(brand_1=="Artemis") + b_brand_Novum*(brand_1=="Novum")
13                   + b_country_CH*(country_1=="Switzerland") +
14                     ↪ b_country_DK*(country_1=="Denmark") + b_country_USA*(country_1=="USA")
15                   + b_char_standard*(char_1=="standard") + b_char_fast*(char_1=="fast acting") +
16                     ↪ b_char_double*(char_1=="double strength")
17                   + b_risk*side_effects_1
18                   + b_price*price_1)
19
20     ...
21     V[["alt4"]] = ( b_brand_BestValue*(brand_4=="BestValue") +
22                   ↪ b_brand_Supermarket*(brand_4=="Supermarket") + b_brand_PainAway*(brand_4=="PainAway")
23                   + b_country_USA*(country_4=="USA") + b_country_IND*(country_4=="India") +
24                     ↪ b_country_RUS*(country_4=="Russia") + b_country_BRA*(country_4=="Brazil")
25                   + b_char_standard*(char_4=="standard") + b_char_fast*(char_4=="fast acting")
26                   + b_risk*side_effects_4
27                   + b_price*price_4 )
28
29     ### Define settings for exploded logit
30     el_settings = list(
31       alternatives = c(alt1=1, alt2=2, alt3=3, alt4=4),
32       avail       = list(alt1=1, alt2=1, alt3=1, alt4=1),
33       choiceVars  = list(best, second_pref, third_pref),
34       utilities   = V,
35       scales      = list(scale_1,scale_2,scale_3)
36     )
37
38     ### Compute exploded logit probabilities
39     P[["model"]]=apollo_el(el_settings, functionality)
40
41     ### Take product across observation for same individual
42     P = apollo_panelProd(P, apollo_inputs, functionality)
43
44     ### Prepare and return outputs of function
45     P = apollo_prepareProb(P, apollo_inputs, functionality)
46     return(P)
47 }

```

Figure 5.9: Exploded Logit implementation

associated with each level, where we impose an appropriate normalisation in `apollo_fixed` to set the parameter for one level to zero for each attribute. The levels that are included in the utility functions differ across alternatives, as reflected in the design of the survey (cf. Table B.3). The key different from an MNL model arises in the inclusion of `choiceVars` instead of `choiceVar` in `el_settings` where this differs from giving a single preferred alternative for each observation and instead giving one column for each stage in the ranking except for the final stage. We also provide scale parameters for these three stages in `el_settings$scales`, where the scale for the first stage is normalised to 1 (by including `scale_1` in `apollo_fixed`).

5.4.2 Best-worst choice model

Many stated choice surveys ask respondents for the most and least preferred alternatives, otherwise known as best-worst or BW (Lancsar et al., 2013). Although there is evidence that the behaviour in these two stages is not necessarily symmetrical (Giergiczny et al., 2017), such data is commonly analysed jointly, using in the simplest form a model where the probability for a given person n in choice task t is given by:

$$P_{n,t} = \frac{e^{V_{b_{n,t}}}}{\sum_{j=1} e^{V_{j,n,t}}} \cdot \frac{e^{-\mu_w V_{w_{n,t}}}}{\sum_{j \neq b_{n,t}} e^{-\mu_w V_{j,n,t}}}, \quad (5.23)$$

where $b_{n,t}$ is the most preferred alternative for respondent n in choice situation t while $w_{n,t}$ is the least preferred option. The above specification assumes that the best option is chosen first, and the worst is then chosen from the remaining set of alternatives, where the alternative with the lowest utility has the highest probability of being chosen (given the multiplication of the utilities by -1). A scale difference between the two stages is allowed for with the estimation of μ_w . This approach is using an exploded logit model with just two stages, and a negative scale for the *worst* stage. An alternative approach for coding best-worst in *Apollo* is discussed in Section 7.2.

An example using the above exploded approach to best-worst data is given in `BW_EL.r`, using the drug choice data from Section 3.3, and using a specification in line with the EL model in Section 5.4.1. The implementation of the model is shown in Figure 5.10, where we multiply the scale for the *worst* stage by -1 (to reflect a minimisation of utility), and where the `apollo_el` function now only uses two stages.

The reader will note that this typical approach to best-worst data makes an assumption of order of the process, with best chosen first, and then worst out of the remaining alternatives. The reverse might of course apply, or a simultaneous process might be used. An implementation of the latter is included in `BW_simultaneous.r`.

5.4.3 Ordered Logit and Ordered Probit

For ordinal dependent variables, the function `apollo_ol` provides an implementation of the Ordered Logit model, while `apollo_op` provides an implementation of the Ordered Probit model. These models are used where, with $Y_{n,t}$ being the observed value for the dependent variable for the t^{th} observation for individual n , $Y_{n,t}$ can take S different possible values, going from $s = 1, \dots, S$.

```

1  ### Vector of parameters, including any that are kept fixed in estimation
2  apollo_beta = c(...,
3                  scale_best      = 1,
4                  scale_worst     = 1)
5
6  ### Vector with names (in quotes) of parameters to be kept fixed at their starting value in
7  ↪ apollo_beta, use apollo_beta_fixed = c() if none
8  apollo_fixed = c("b_brand_Artemis", "b_country_USA", "b_char_standard", "scale_best")
9
10 # ##### DEFINE MODEL AND LIKELIHOOD FUNCTION #####
11 # #####
12
13 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
14
15     ...
16
17     ### Define settings for exploded logit
18     el_settings = list(
19         alternatives = c(alt1=1, alt2=2, alt3=3, alt4=4),
20         avail       = list(alt1=1, alt2=1, alt3=1, alt4=1),
21         choiceVars  = list(best, worst),
22         utilities    = V,
23         scales      = list(scale_best, -scale_worst)
24     )
25
26     ...
27
28     return(P)
29 }

```

Figure 5.10: Exploded Logit for B-W data

Ordered Logit

In an Ordered Logit (OL) model, the probability of observing value s is given by:

$$P_{Y_{n,t}=s} = \frac{e^{\tau_s - V_{n,t}}}{1 + e^{\tau_s - V_{n,t}}} - \frac{e^{\tau_{s-1} - V_{n,t}}}{1 + e^{\tau_{s-1} - V_{n,t}}} \quad (5.24)$$

The likelihood of the observed value $Y_{n,t}$ is then given by:

$$L_{Y_{n,t}} = \sum_{s=1}^S \delta_{(Y_{n,t}=s)} \left[\frac{e^{\tau_s - V_{n,t}}}{1 + e^{\tau_s - V_{n,t}}} - \frac{e^{\tau_{s-1} - V_{n,t}}}{1 + e^{\tau_{s-1} - V_{n,t}}} \right], \quad (5.25)$$

where, for normalisation, we set $\tau_S = +\infty$ and $\tau_0 = -\infty$, such that the probability of $Y_{n,t} = 1$ is given by $\frac{e^{\tau_1 - V_{n,t}}}{1 + e^{\tau_1 - V_{n,t}}}$ while the probability of $Y_{n,t} = S$ is given by $1 - \frac{e^{\tau_{S-1} - V_{n,t}}}{1 + e^{\tau_{S-1} - V_{n,t}}}$. In our notation, $V_{n,t}$ is the utility used inside the Ordered Logit model, which will be a function of characteristics of the decision maker and the scenario that the dependent variable relates to.

For an example using `apollo_ol`, see the section on hybrid choice models (Section 7.3), or the standalone implementation in [OL.r](#). In *Apollo*, the `apollo_ol` function is called as

follows:

```
P[["model"]] = apollo_ol(ol_settings,
                        functionality)
```

where the contents of `ol_settings` are a little different from the earlier MNL, NL and CNL models. In particular, we have:

apollo_ol and apollo_op: user-controlled settings

coding: An optional argument of numeric or character vector type which is only required as an input if the dependent variable does not use an incremental coding from 1 to a value equal to the number of possible values for the dependent variable Y . This can be used both if the dependent variable is numeric in the data but not monotonic or with unequal increment, or if the dependent variable is given in string format.

componentName: The optional argument giving a name to the model component described already for earlier models.

outcomeOrdered: A variable indicating the column in the database which identifies the level selected for the ordinal variable in each observation.

rows: The optional `rows` argument already described for the earlier models.

tau: A list containing the thresholds that are used in the model. The thresholds can be either scalars (i.e. the same across all observations), vectors (one value per observation) or matrices or cubes if allowing for random heterogeneity in the thresholds. The list should have one fewer element than the number of possible values for the dependent variable Y . Extreme thresholds at $-\infty$ and $+\infty$ are added automatically by *Apollo*.

utility: A numeric vector containing the explanatory variable used in the Ordered Logit model, i.e. the utility in Equation 5.24.

Ordered Probit

In an Ordered Probit (OP) model, the probability of observing value s is given by:

$$\begin{aligned} P_{Y_{n,t}=s} &= P(\tau_{s-1} < V_{n,t} + \varepsilon < \tau_s) \\ &= P(\varepsilon < \tau_s - V_{n,t}) - P(\varepsilon < \tau_{s-1} - V_{n,t}) \\ &= \Phi(\tau_s - V_{n,t}) - \Phi(\tau_{s-1} - V_{n,t}) \end{aligned} \quad (5.26)$$

The likelihood of the observed value $Y_{n,t}$ is then given by:

$$L_{Y_{n,t}} = \sum_{s=1}^S \delta_{(Y_{n,t}=s)} [\Phi(\tau_s - V_{n,t}) - \Phi(\tau_{s-1} - V_{n,t})], \quad (5.27)$$

where, for normalisation, we set $\tau_S = +\infty$ and $\tau_0 = -\infty$, such that the probability of $Y_{n,t} = 1$ is given by $\Phi(\tau_1 - V_{n,t})$ while the probability of $Y_{n,t} = S$ is given by $1 - \Phi(\tau_{S-1} - V_{n,t})$. In our notation, $V_{n,t}$ is the utility used inside the Ordered Probit model, which will be a function of characteristics of the decision maker and the scenario that the dependent variable relates to.

For an example using `apollo_op`, see [OP.r](#). In *Apollo*, the `apollo_op` function is called as follows:

```
P[["model"]] = apollo_op(op_settings,
                        functionality)
```

where the contents of `op_settings` are the same as in `ol_setting`.

5.4.4 Normally distributed continuous variables

For continuous dependent variables (or ordinal dependent variables that are treated as continuous) the function `apollo_normalDensity` is available, which is an implementation of the Normal probability density function. This implies that the probability of observing the specific value for the dependent variable Y in situation t for person n is given by:

$$P(Y_{n,t}) = \frac{\phi\left(\frac{Y_{n,t} - X_{n,t} - \mu}{\sigma}\right)}{\sigma}, \quad (5.28)$$

where $X_{n,t}$ is the explanatory variable used, μ and σ are the estimated means and standard deviations, and ϕ is the standard Normal density function.

For an example using `apollo_normalDensity`, see the section on hybrid choice models (Section 7.3), or the standalone implementation in [normal_density.r](#). The `apollo_normalDensity` is called as follows:

```
P[["model"]] = apollo_normalDensity(normalDensity_settings,
                                    functionality)
```

where the contents of `normalDensity_settings` now include:

`apollo_normalDensity`: user-controlled settings

componentName: The optional argument giving a name to the model component described already for earlier models.
mu: The parameter used as the mean for the Normal density.
outcomeNormal: A variable indicating the column in the database which contains the value for the dependent variable in each observation.
rows: The optional `rows` argument already described for the earlier models.
sigma: The parameter used as the standard deviation for the Normal density.
xNormal: A numeric vector containing the explanatory variable used in Equation 5.28.

5.5 Discrete-continuous models

While choice modelling is generally best known for the study of the choice between mutually exclusive alternatives, a large body of research has also looked at the joint choice of multiple alternatives and the *consumption* of different quantities of each of these. Especially the family of Multiple Discrete Continuous Extreme Value models has received extensive interest in recent years, and two of these models are implemented in *Apollo*.

5.5.1 Multiple Discrete Continuous Extreme Value (MDCEV) model

The MDCEV model (Bhat, 2008) is a representation of a multiple discrete-continuous decisions process. Such a process consist of choosing one or more elements from a set of alternatives, and then choosing a non-negative amount of each of the chosen elements. Examples of such a process are consumption (what products or services to buy and how much of each), and time use (what activities to engage with and for how long). More formally, the MDCEV model is a stochastic implementation of the classical consumer maximization processes, where consumers allocate resources (e.g. their income) in a way that maximizes their utility. This problem can be formulated as follows:

$$\begin{aligned} \text{Max}_{x_j \forall j} \quad & \sum_{j=1}^J \frac{\gamma_j}{\alpha_j} \psi_j \left(\left(\frac{x_j}{\gamma_j} + 1 \right)^{\alpha_j} - 1 \right) \\ \text{subject to} \quad & \sum_{j=1}^J x_j p_j = B \end{aligned} \tag{5.29}$$

$$\psi_j = \exp(V_j + \varepsilon_j), \tag{5.30}$$

where J is the number of alternatives, x_j is the amount consumed of product j , and p_j is the unit price or cost of alternative j , and B is the budget available to the individual for consumption. The term ε_j is an independent and identically distributed random disturbance following a *Gumbel*(0, σ) distribution. Finally, α_j and γ_j are parameters determining satiation, while V_j determines each alternative's base utility (i.e. its marginal utility at zero consumption).

The probability of an observed vector of consumptions is then given by:

$$\begin{aligned} & P(x_1^*, x_2^*, \dots, x_M^*, 0, \dots, 0) \\ &= \frac{1}{p_1} \frac{1}{\sigma^{M-1}} \left(\prod_{m=1}^M f_m \right) \left(\sum_{m=1}^M \frac{p_m}{f_m} \right) \left(\frac{\prod_{m=1}^M e^{V_m/\sigma}}{\left(\sum_{j=1}^J e^{W_j/\sigma} \right)^M} \right) (M-1)!, \end{aligned} \tag{5.31}$$

where $f_i = \frac{1-\alpha_i}{x_i^* + \gamma_i}$ and $W_j = V_j + (\alpha_j - 1) \log\left(\frac{x_j^*}{\gamma_j} + 1\right) - \log(p_j)$, and x_j^* is the observed (optimum) consumption of product j .

A revised formulation as shown in Equation 5.32 is obtained when an outside good is included among the alternatives. An outside good is a product that is consumed by all individuals in the sample. The outside good usually represents an aggregate measure of the consumption of all products that are not of interest for the study. For example, if a study focuses on use of leisure time, the outside good might be all activities that are not leisure (such as sleeping, work, travelling, etc.), while the inside goods (i.e. all alternatives that are

not the outside good) could deal with leisure in a more detailed way (e.g. going to the park, hiking, going to the cinema, meeting friends, etc.).

$$P(x_1^*, x_2^*, \dots, x_M^*, 0, \dots, 0) = \frac{1}{\sigma^{M-1}} \left(\prod_{m=1}^M f_m \right) \left(\sum_{m=1}^M \frac{p_m}{f_m} \right) \left(\frac{\prod_{m=1}^M e^{V_i/\sigma}}{\left(\sum_{j=1}^J e^{W_j/\sigma} \right)^M} \right) (M-1)!, \quad (5.32)$$

The function `apollo_mdcev` calculates the loglikelihood of an MDCEV model, using equation 5.31 if no outside good is provided and uses equation 5.32 if an outside good is provided. An example of a function call, as well as a definition of its arguments follow. The function is called as:

```
P[["model"]] = apollo_mdcev(mdcev_settings,
                             functionality)
```

The list `mdcev_settings` contains the following objects:

apollo_mdcev: user-controlled settings (part 1)

- alpha:** List containing the α parameter for each alternative, using the names from **alternatives**.
- alternatives:** Character vector containing the name of all alternatives. If one of these alternatives is called **outside**, it will automatically be used as the outside good.
- avail:** List of availabilities, using the names from **alternatives**. Each element can be scalar (0 or 1) or a vector detailing availability for each observation.
- budget:** Vector with the amount of the resource (e.g. money or time) available for each observation. It must be equal to the total consumption of that observation.
- componentName:** The optional argument giving a name to the model component. It is recommended not to include this argument and instead let Apollo set it up automatically.
- continuousChoice:** List of continuous consumption, using the names from **alternatives**. Each element must be a vector of length N (number of observations) indicating the amount consumed.
- cost:** List containing the cost or price of each alternative, using the names from **alternatives**. Each element can be a scalar if the price does not change across observations, or a vector of length equal to the number of observations in the data, detailing the price for each observation.
- gamma:** List containing the γ parameter for each alternative, using the names from **alternatives**, excluding any outside good.
- nRep:** Optional argument. Number of simulations used to calculate the forecast. Unlike MNL and other models, the MDCEV model relies on simulation to generate its forecast. For each repetition, a full set of random components for each observation in the sample are drawn, and the optimal consumption is calculated for each observation. With this setting, the user can choose how many repetitions are used, before calculating the average across them, which becomes the forecast. Default is 100.
- outside:** Optional argument with the name of an outside good. This is not needed if one of the alternatives is already called **outside**.

apollo_mdcev: user-controlled settings (part 2)

rawPrediction: Optional argument. It is a logical (boolean) value. If TRUE, then the forecast will not average across draws, instead returning a three-dimensional array of $nObs \times nAlt \times nRep$ where each x-y slice is the forecast for each draw. If FALSE (the default), the forecast will be the average across repetitions.

rows: The optional **rows** argument already described for the earlier models.

sigma: Scalar representing the scale parameter of the error term. If there is no price variation across products, this should be fixed to 1.

utilities: A list of length J (i.e. number of alternatives), containing the deterministic part of the base utility of each alternative. The outside good should have $V=0$ if included.

As discussed at length by [Bhat \(2008\)](#), different profiles exist for normalisation of a MDCEV model, either using a generic α and alternative-specific γ parameters, an α parameter only for the outside good (and set to zero for others) along with alternative-specific γ parameters, or alternative specific α terms with $\gamma = 1$ for all goods. In our examples below, we use a generic α and alternative-specific γ parameters. Other profiles can be implemented by simply changing which parameters are generic and which are alternative specific, and making some α terms equal to zero, as appropriate.

We include two examples of the MDCEV model on the time use data described in Section 3.4. The first example, `MDCEV_no_outside_good.r` does not include an outside good. We illustrate this in Figure 5.11.

We begin by defining the names of the alternatives, availabilities and continuous consumptions, where we turn minutes into hours. We then create the list of utilities, where, in our example, these include alternative specific constants only, where we fix δ_{home} to zero for identification. This is followed by the definition of a generic α parameter, which is constrained to be below 1 by using a logistic transform, with $\alpha = \frac{1}{1+e^{-\alpha_{base}}}$, and the set of γ parameters, where these are alternative-specific in our case. We finally define the costs, set the budget to 24 hours, and make the call to `apollo_mdcev`. In this example, we also fix `sig` to its starting value of 1 in `apollo_fixed`.

The second example, `MDCEV_with_outside_good.r`, groups together some alternatives to create an outside good. It also incorporates socio-demographics in the utility function, though not in the α and γ terms, which is however also possible in *Apollo*. We illustrate this example in Figure 5.12.

To create the new activities, we sum some of the activities up after reading in the data, where we create an outside good by combining time spent travelling with time spent at home. We also create a generic leisure activity. The remainder of the specification is no different in principle from that in Figure 5.12 with the exception of there being an alternative called `outside`, and with using more detailed utility functions.

5.5.2 Multiple Discrete Continuous Nested Extreme Value (MDCNEV) model

The MDCNEV is an extension to the MDCEV model, proposed by [Pinjari and Bhat \(2010a\)](#). It incorporates correlation between alternatives, in a similar way to the Nested Logit (NL), where correlation can be introduced by nesting, i.e. grouping alternatives that are correlated among them. The implementation of MDCNEV in *Apollo* allows for only a single level of

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2  ...
3  ### Define individual alternatives
4  alternatives = c("dropOff",
5                  ...
6                  "other")
7
8  ### Define availabilities
9  avail = list(dropOff = 1,
10              ...
11              other    = 1)
12
13  ### Define continuous consumption for individual alternatives
14  continuousChoice = list(dropOff = t_a01/60,
15                          ...
16                          other    = t_a12/60)
17
18  ### Define utilities for individual alternatives
19  V = list()
20  V[["dropOff"]] = delta_dropOff
21  ...
22  V[["other"]] = delta_other
23
24  ### Define alpha parameters
25  alpha = list(dropOff = 1 / (1 + exp(-alpha_base)),
26              ...
27              other    = 1 / (1 + exp(-alpha_base)))
28
29  ### Define gamma parameters
30  gamma = list(dropOff = gamma_dropOff,
31              ...
32              other    = gamma_other)
33
34  ### Define costs for individual alternatives
35  cost = list(dropOff = 1,
36              ...
37              other    = 1)
38
39  ### Define settings for MDCEV model
40  mdcev_settings <- list(alternatives = alternatives,
41                        avail         = avail,
42                        continuousChoice = continuousChoice,
43                        V             = V,
44                        alpha         = alpha,
45                        gamma         = gamma,
46                        sigma         = sig,
47                        cost           = cost,
48                        budget        = 24)
49
50  ### Compute probabilities using MDCEV model
51  P[["model"]] = apollo_mdcev(mdcev_settings, functionality)
52  ...
53  }

```

Figure 5.11: MDCEV implementation without outside good


```

1 database = read.csv("apollo_timeUseData.csv",header=TRUE)
2
3 ### Create consumption variables for combined activities
4 database$t_outside = rowSums(database[,c("t_a01", "t_a06", "t_a10", "t_a11", "t_a12")]) #
5 ↪ outside good: time spent at home and travelling
6 database$t_leisure = rowSums(database[,c("t_a07", "t_a08", "t_a09")])
7 ...
8 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
9   ...
10  ### Define individual alternatives
11  alternatives = c("outside",
12                  ...
13                  "leisure")
14
15  ### Define availabilities
16  avail = list(outside = 1,
17               ...
18               leisure = 1)
19
20  ### Define continuous consumption for individual alternatives
21  continuousChoice = list(outside = t_outside/60,
22                           ...
23                           leisure = t_leisure/60)
24
25  ### Define utilities for individual alternatives
26  V = list()
27  V[["outside"]] = 0
28  ...
29  V[["leisure"]] = delta_leisure + delta_leisure_wknd*weekend
30
31  ### Define alpha parameters
32  alpha = list(outside = 1 / (1 + exp(-alpha_base)),
33               ...
34               leisure = 1 / (1 + exp(-alpha_base)))
35
36  ### Define gamma parameters
37  gamma = list(work = gamma_work,
38               ...
39               leisure = gamma_leisure)
40
41  ### Define costs for individual alternatives
42  cost = list(outside = 1,
43               ...
44               leisure = 1)
45
46  ...
47 }

```

Figure 5.12: MDCEV implementation with an outside good

nesting and is also only valid for models with an outside good, i.e. a product that is consumed in every observation. The likelihood function of the model is as follows.

$$\begin{aligned}
 & P(x_1^*, x_2^*, \dots, x_M^*, 0, \dots, 0) \\
 &= |J| \frac{\prod_{i \in \text{chosen alts}} e^{\frac{V_i}{\theta_i}}}{\prod_{s=1}^{S_M} \left(\sum_{i \in \text{sthnest}} e^{\frac{V_i}{\theta_s}} \right)^{q_s}} \\
 & \cdot \sum_{r_1=1}^{q_1} \dots \sum_{r_s=1}^{q_s} \dots \sum_{r_{S_M}=1}^{q_{S_M}} \left\{ \prod_{s=1}^{S_M} \left[\frac{\left(\sum_{i \in \text{sthnest}} e^{\frac{V_i}{\theta_s}} \right)^{\theta_s}}{\sum_{j=1}^{S_j} \left\{ \left(\sum_{i \in \text{sthnest}} e^{\frac{V_i}{\theta_s}} \right)^{\theta_s} \right\}} \right]^{q_s - r_s + 1} \left(\prod_{s=1}^{S_M} \text{sum}(X_{r,s}) \right) \left(\sum_{s=1}^{S_M} (q_s - r_s + 1) - 1 \right)! \right\}, \quad (5.33)
 \end{aligned}$$

For a detailed explanation of the values in the equation, see [Pinjari and Bhat \(2010a\)](#).

The `apollo_mdcnev` function is called as:

```
P[["model"]] = apollo_mdcnev(mdcnev_settings,
                             functionality)
```

Aside from the previously defined contents in `mdcnev_settings`, we now have two additional inputs, namely:

apollo_mdcnev: user-controlled settings (part 1)

- alpha:** List containing the α parameter for each alternative, using the names from **alternatives**.
- alternatives:** Character vector containing the name of all alternatives. If one of these alternatives is called **outside**, it will automatically be used as the outside good.
- avail:** List of availabilities, using the names from **alternatives**. Each element can be scalar (0 or 1) or a vector detailing availability for each observation.
- budget:** Vector with the amount of the resource (e.g. money or time) available for each observation. It must be equal to the total consumption of that observation.
- componentName:** The optional argument giving a name to the model component. It is recommended not to include this argument and instead let Apollo set it up automatically.
- continuousChoice:** List of continuous consumption, using the names from **alternatives**. Each element must be a vector of length N (number of observations) indicating the amount consumed.
- cost:** List containing the cost or price of each alternative, using the names from **alternatives**. Each element can be a scalar if the price does not change across observations, or a vector of length equal to the number of observations in the data, detailing the price for each observation.
- gamma:** List containing the γ parameter for each alternative, using the names from **alternatives**, excluding any outside good.

apollo_mdcnev: user-controlled settings (part 2)

mdcnevNests: A named vector containing the names of the nests and the associated structural parameters θ . For each θ , we give the name of the associated parameter. Unlike in `apollo_n1`, the `root` is not included for `apollo_cn1` as only two-level structures are used.

mdcnevStructure: A matrix showing the allocation of alternatives to nests, with one row per nest and one column per alternative, using the same ordering as in `alternatives` and `mdcnevStructure`. Element (i, j) should take value 1 if alternative j belongs to nest i , and zero otherwise.

outside: Optional argument with the name of an outside good. This is not needed if one of the alternatives is already called `outside`.

rows: The optional `rows` argument already described for the earlier models.

utilities: A list of length J (i.e. number of alternatives), containing the deterministic part of the base utility of each alternative. The outside good should have $V=0$ if included.

The example `MDCNEV.r` is a nested version of the model with an outside good used in Figure 5.12, i.e. `MDCEV_with_outside_good.r`. The model uses two nests, one for *mandatory* activities (work, school, private) and one for *optional* activities (all others, including the outside good). In Figure 5.13, we only show the part of the code that differs from the standard MDCEV model. We define two nests, and assign the appropriate θ parameter to each, where in our example, `theta_optional` is further fixed to 1 via `apollo_fixed` as its estimate was not significantly different from 1. We then describe the allocation of alternatives to nests using a matrix of ones and zeros, with one row per nest and one column per alternative, where each alternative falls into exactly one nest. Finally, we make the call to `apollo_mdcnev`. For this model, we use scaling of some of the parameters in model estimation given the earlier findings of very diverse scales for the individual parameters in the corresponding simple MDCEV model, i.e. `MDCEV_with_outside_good.r`.

5.6 Extended Multiple Discrete Continuous (eMDC) model

The eMDC model [Palma and Hess \(2022\)](#) is a discrete-continuous model that (i) allows for true complementarity and substitution patterns between alternatives, (ii) does not require the budget to be defined, allowing for model estimation even if the budget is unknown, and (iii) does not include a random error term in the base utility of the outside good.

Property (i) is relevant as many MDC models lack complementarity, and they only incorporate substitution through budget effects. To see this more clearly, consider two products: a can of Coca-Cola and a can of Pepsi. If consumption of Coca-Cola increases, consumption of Pepsi will decrease for two reasons: first, because there is less money available to buy it, and secondly, because the consumption of Coca-Cola discourages the consumption of Pepsi (because they serve the same need). Most MDC models capture only the first effect, while eMDC can capture both.

Property (ii) is optional, and the model can be estimated either with or without specifying the budget. If the budget is not specified, it is assumed to be very large. This is useful for cases where the budget is very hard to define, for example when modelling distance travelled by different modes (as there is no obvious upper limit), or when modelling demand for a small

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3  ...
4
5  ### Define nesting structure
6  mdcnevNests = list(mandatory = theta_mandatory,
7                    optional  = theta_optional)
8
9  mdcnevStructure = matrix(0, nrow=length(mdcnevNests), ncol=length(V))
10 ###
11      outside work school shopping private leisure
12 mdcnevStructure[1,] = c( 0, 1, 1, 0, 1, 0) # mandatory
13 mdcnevStructure[2,] = c( 1, 0, 0, 1, 0, 1) # optional
14
15 ...
16
17 ### Define settings for MDCNEV model
18 mdcnev_settings <- list(...)
19      mdcnevNests      = mdcnevNests,
20      mdcnevStructure  = mdcnevStructure)
21
22 ### Compute probabilities using MDCNEV model
23 P[["model"]] = apollo_mdcnev(mdcnev_settings, functionality)
24 ...
25 }

```

Figure 5.13: MDCNEV implementation and call to apollo_estimate using scaling

set of products whose cost is much smaller than individuals' income.

Property (iii) allows the models to have a tractable likelihood function, but prevents the model from capturing common random shocks to demand. This is somewhat attenuated, however, thanks to the inclusion of complementarity and substitution in the model.

The eMDC model assumes that individuals maximise their utility through solving the classical consumer utility maximisation problem, as follows.

$$\begin{aligned}
 \text{Max}_{x_n} \quad & u_0(x_{n0}) + \sum_{j=1}^J \psi_{nj} \gamma_j \log \left(\frac{x_{nj}}{\gamma_j} + 1 \right) + \sum_{j=1}^{J-1} \sum_{l=j+1}^J \delta_{jl} (1 - e^{-x_{nj}}) (1 - e^{-x_{nl}}) \\
 \text{s.t.} \quad & x_{n0} p_{n0} + \sum_{j=1}^J x_{nj} p_{nj} = B_n
 \end{aligned} \tag{5.34}$$

Where n indexes individuals and j alternatives. x_{nj} is the consumed amount of alternative j by individual n . ψ_{nj} is the base utility of alternative j for individual n , i.e. a measure of how attractive the alternative is. γ_j is the satiation parameter of alternative j , and δ_{jl} is the complementarity and substitution parameter between alternatives j and l . If $\delta_{jl} > 0$, then there is complementarity between alternatives j and l . If $\delta_{jl} < 0$, then there is substitution between alternatives j and l . If $\delta_{jl} = 0$, then there is no complementarity nor substitution between alternatives j and l . p_{nj} is the price or cost of alternatives, while B_n is the budget available to individual n .

The functional form of $u_0(x_{n0})$ will depend on whether the budget is observed or not. If the budget is observed, i.e. we know the value of B_n , then we use $u_0(x_{n0}) = \psi_{n0} \log(x_{n0})$. If

the budget is not observed, then we use $u_0(x_{n0} = \psi_{n0}x_{n0})$.

To guarantee the base utilities being bigger than zero, we parametrise them as follows: $\psi_{n0} = e^{\alpha z_{n0}}$ and $\psi_{nj} = e^{\beta_j z_{nj} + \varepsilon_{nj}}$, where z are explanatory variables, usually characteristics of individual n or attributes of alternative j ; α and β are parameters to be estimated; and ε_{nj} is a random error term following a normal distribution with mean zero and standard deviation equal to σ , to be estimated. Under these conditions, the likelihood function can be derived, reaching different forms depending on whether B_n is observed or not.

When B_n is **observed**, the likelihood function is as follows. Subscript n is omitted for clarity.

$$\begin{aligned}
L(x_j) &= |J| \prod_{j=1}^J \phi_{\sigma}(-W_j)^{I_{x_j>0}} \Phi_{\sigma}(-W_j)^{I_{x_j=0}} \\
W_j &= z_j \beta_j - \log\left(\frac{x_j}{\gamma_j} + 1\right) - \log\left(\frac{\psi_0}{x_0} p_j - e^{-x_j} \sum_{l \neq j} \delta_{jl} (1 - e^{-x_l})\right) \\
J_{ii} &= \frac{1}{x_i + \gamma_i} + \frac{\frac{\psi_0}{x_0^2} p_i^2 + E_i}{\frac{\psi_0}{x_0} p_i - E_i} \\
J_{ij} &= \frac{\frac{\psi_0}{x_0^2} p_i p_j - \delta_{ij} e^{-x_i} e^{-x_j}}{\frac{\psi_0}{x_0} p_i - E_i} \\
E_i &= e^{-x_i} \sum_{l \neq i} \delta_{il} (1 - e^{-x_l})
\end{aligned} \tag{5.35}$$

Where $|J|$ is the determinant of the Jacobian of vector $-W$, which only includes those W_j where $x_j > 0$. The elements of the Jacobian are defined by expression J_{ii} and J_{ij} for elements in the diagonal and in rows (i, j) , respectively. The function $I_{x_j>0}$ takes value 1 if $x_j > 0$ and zero otherwise.

When B_n is **not observed**, the likelihood function is analogous to the previous one, but with different definitions for W_j , J_{ii} and J_{ij} .

$$\begin{aligned}
W_j &= z_j \beta_j - \log\left(\frac{x_j}{\gamma_j} + 1\right) - \log\left(\psi_0 p_j - e^{-x_j} \sum_{l \neq j} \delta_{jl} (1 - e^{-x_l})\right) \\
J_{ii} &= \frac{1}{x_i + \gamma_i} + \frac{E_i}{\psi_0 p_i - E_i} \\
J_{ij} &= \frac{-\delta_{ij} e^{-x_i} e^{-x_j}}{\psi_0 p_i - E_i}
\end{aligned} \tag{5.36}$$

In *Apollo*, the eMDC model can be estimated using function `apollo_emdc`. The appropriate version of the model will be used depending on the provision of the budget, or the absent of it. The function is called as follows.

```
P[["model"]] = apollo_emdc(emdc_settings,
                             functionality)
```

The list `emdc_settings` contains the following objects.

`apollo_emdc`: user-controlled settings

- avail**: Optional argument. A list indicating the availability of alternatives. Each element of the list should be a vector as long as the number of observations, with elements equal to zero (0) or FALSE to denote the alternative is unavailable, and one (1) or TRUE to denote availability. If omitted, full availability is assumed.
- budget**: Optional argument. Budget (B_n). Must be bigger than the expenditure on all inside goods. If omitted, the model with unobserved budget is used (second formulation above).
- cost**: Optional argument. Price of each product (p_{nj}). It should be provided as a list with as many elements as alternatives. Each element should be a vector as long as the number of observations, containing the (larger than zero) price for that product, faced by the decision maker in that observation. If omitted, a cost of 1 is assumed for all alternatives across all observations.
- continuousChoice**: Amount consumed of each alternative (x_{nj}). Outside good (x_{n0}) must not be included. Should be provided as a list with as many elements as alternatives. Each element must be named as the corresponding alternative, and be a vector as long as the number of observations.
- delta**: Complementarity/substitution parameter (δ_{jl}). Should be given as a lower triangular numeric matrix, or list of lists, with each element a component of the matrix in row, column order.
- gamma**: Satiation parameter of each product (γ_j). Named list of numeric vectors.
- nRep**: Number of repetitions used when predicting, as prediction is based on simulation. Must be a positive integer larger than zero. Default is 100.
- sigma**: Optional numeric vector or scalar. Standard deviation of the error term (σ). Default is one.
- timeLimit**: Optional positive scalar. Maximum amount of seconds the optimiser is allowed to spend calculating a prediction before setting the predicted value to NA.
- tol**: Optional positive scalar. Tolerance of the prediction algorithm (see paper).
- utilities**: Named list of numeric vectors (or matrices or arrays). Log of the base utility of each product ($\beta_j z_{nj}$).
- utilityOutside**: Optional numeric vector (or matrix or array). Log of the base utility of the outside good (αz_{n0}). Must be normalised to 0 for at least one individual. Default is 0 for every observation.

We include two examples of the eMDC model using the time-use dataset. The first example, `eMDC_with_budget.r` uses the model with observed budget, while the second example `eMDC_without_budget.r` uses the model without the budget.

5.7 Adding new model types

As already mentioned, users of *Apollo* are not restricted to those models for which functions are available in the code. Any model that yields a probability for an outcome can be used in the code and parameters for the model can be estimated using either classical or Bayesian estimation. The advantage of the predefined functions is of course that they run a large number of checks to avoid issues with mis-specification and produce output for different user needs. The level of these checks and output flexibility that a user implements for new models will vary as a function of the user's needs.

The user has the option of either creating new functions in R that are defined outside `apollo_probabilities` much in the same way as for example `apollo_mnl`, or to simply code the probabilities for a model inside `apollo_probabilities`. An example of the latter approach is shown in Section 8.2.

Clearly, coding models as separate functions is preferable in terms of reusability as well as code organisation. Users who are interested in coding their own functions should inspect the source code for some of the implemented functions for guidance, for example using `apollo_ol` as a simple start. For user defined models to be compatible with *Apollo*, a number of simple basic requirements need to be fulfilled. In particular, the function needs to take `functionality` as an argument to be able to produce different output depending on the value passed to it for `functionality`. Not all possible values discussed for `functionality` in this manual need to be implemented for new models, but essential capabilities include the ability to deal with the following four settings for `functionality`:

estimate: Return the probabilities for each row in the data, using a vector, matrix or cube (array) depending on the presence of random coefficients (cf. Section 6.1).

output: Same as `functionality="estimate"`.

validate: Run checks on the inputs and returns the loglikelihood (same as "estimate") if all inputs are correct.

zero_LL: Return the log-likelihood of the model component with all parameters at zero, or returns NA if not applicable for given model.

If the models are to be compatible with random coefficients, they furthermore need to be able to produce probabilities as three-dimensional arrays, as discussed in Section 6.1. Additionally, if the models are components of an overall model from which predictions are to be made (cf. Section 9.9), then output from the function is also needed with `functionality==prediction`, even if returning NA for that model component.

6

Incorporating random heterogeneity

In this chapter, we describe how to use the *Apollo* package to incorporate random coefficients. We look first at continuous random heterogeneity before looking at Discrete Mixtures (DM) and Latent Class (LC) models, and also a combination of the two. Finally, we discuss multi-core estimation, which is beneficial for models with random heterogeneity. In this section, we use the simple binary public transport route choice SP data described in Section 3.2.

6.1 Continuous random coefficients

6.1.1 Introduction

The *Apollo* package allows for a very general use of continuous random coefficients. The code works for models allowing for intra-individual mixing (i.e. heterogeneity across observations for the same individual), inter-individual mixing (i.e. heterogeneity across individual people), as well as a mixture of the two. For background, we provide a brief recap of the discussions in [Hess and Train \(2011\)](#) on this topic.

In cross-sectional data, we would have a sample of N individuals, indexed as $n = 1, \dots, N$, where each individual is observed to face only one choice situation. Let β_n be a vector of the true, but unobserved taste coefficients for consumer n . We assume that $\beta_n \forall n$ is *iid* over consumers with density $g(\beta | \Omega)$, where Ω is a vector of parameters of this distribution, such as the mean and variance. Let y_n be the alternative chosen by consumer n , such that $P_n(y_n | \beta)$ gives the probability of the observed choice for consumer n , conditional on β . The Mixed Logit probability of consumer n 's chosen alternative is

$$P_n(y_n | \Omega) = \int_{\beta} P_n(y_n | \beta) g(\beta | \Omega) d\beta. \quad (6.1)$$

The log-likelihood function is then given by:

$$\text{LL}(\Omega) = \sum_{n=1}^N \ln \left(\int_{\beta} P_n(y_n | \beta) g(\beta | \Omega) d\beta \right), \quad (6.2)$$

Since the integrals do not take a closed form, they are approximated by simulation. The simulated log-likelihood is:

$$\text{SLL}(\Omega) = \sum_{n=1}^N \ln \left(\frac{1}{R} \sum_{r=1}^R P_n(y_n | \beta_{r,n}) \right). \quad (6.3)$$

where $\beta_{r,n}$ gives the r^{th} draw (out of R) from $g(\beta | \Omega)$ for individual n . Different draws are used for the N consumers, for a total of NR draws.

When we have multiple observations per individual, we typically make the assumption that sensitivities vary across people, but stay constant across individuals. We would then have that the likelihood of the sequence of choices for person n is given by:

$$P_n(\Omega) = \int_{\beta} \prod_{t=1}^{T_n} P_{n,t}(y_{n,t} | \beta) g(\beta | \Omega) d\beta, \quad (6.4)$$

where $y_{n,t}$ is the alternative chosen by individual n in choice situation t . Note that, since the same sensitivities apply to all choices by a given consumer, the integration over the density of β applies to all the consumer's choices combined, rather than each one separately.

The log-likelihood function for the observed choices is then:

$$\text{LL}(\Omega) = \sum_{n=1}^N \ln \left(\int_{\beta} \left[\prod_{t=1}^{T_n} (P_{n,t}(y_{n,t} | \beta)) \right] g(\beta | \Omega) d\beta \right). \quad (6.5)$$

The simulated LL (SLL) is:

$$\text{SLL}(\Omega) = \sum_{n=1}^N \ln \left(\frac{1}{R} \sum_{r=1}^R \left[\prod_{t=1}^{T_n} (P_{n,t}(y_{n,t} | \beta_{r,n})) \right] \right). \quad (6.6)$$

Note that in this formulation, the product over choice situations is calculated for each draw; the product is averaged over draws; and *then* the log of the average is taken. The SLL is the sum over consumers of the log of the average (across draws) of products. The calculation of the contribution to the SLL function for consumer n involves the computation of RT_n Mixed Logit probabilities.

Instead of utilising the panel nature of the data, the model could be estimated *as if* each choice were from a different consumer. That is, the panel data could be *treated as if* they were cross-sectional. The objective function is similar to Equation 6.2 except that the multiple choice situations by each consumer are represented as being for different individuals:

$$\text{LL}(\Omega) = \sum_{n=1}^N \sum_{t=1}^{T_n} \ln \left(\int_{\beta} P_{n,t}(y_{n,t} | \beta) g(\beta | \Omega) d\beta \right), \quad (6.7)$$

where the integration across the distribution of taste coefficients is applied to each choice, rather than to each consumer's sequence of choices. This function is simulated as:

$$\text{SLL}(\Omega) = \sum_{n=1}^N \sum_{t=1}^{T_n} \ln \left(\frac{1}{R} \sum_{r=1}^R P_{n,t}(y_{n,t} | \beta_{r,t,n}) \right). \quad (6.8)$$

where $\beta_{r,t,n}$ is the r^{th} draw from $g(\beta | \Omega)$ for choice situation t for individual n . Different draws are used for the T_n choice situations for consumer n , as well as for the N consumers.

Consumer n 's contribution to the SLL function utilises RT_n draws of β rather than R draws as in Equation 6.6, but involves the computation of the same number of Logit probabilities as before, namely, RT_n . The difference is that the averaging across draws is performed before taking the product across choice situations.

We now generalise the specification on panel data to include intra-personal taste heterogeneity in addition to inter-personal heterogeneity. Let $\beta_{n,t} = \alpha_n + \gamma_{n,t}$ where α_n is distributed across consumers but not over choice situations for a given consumer, and $\gamma_{n,t}$ is distributed over choice situations as well as consumers. That is, α_n captures inter-personal variation in tastes while $\gamma_{n,t}$ captures intra-personal variation. Their densities are denoted as $f(\alpha)$ and $h(\gamma)$, respectively,¹⁵ where their dependence on underlying parameters, contained collectively in Ω , is suppressed for convenience.

The LL function is given by:

$$LL(\Omega) = \sum_{n=1}^N \ln \left[\int_{\alpha} \left(\prod_{t=1}^{T_n} \left(\int_{\gamma} P_{n,t}(y_{n,t} \mid \alpha, \gamma) h(\gamma) d\gamma \right) \right) f(\alpha) d\alpha \right]. \quad (6.9)$$

The two levels of integration create two levels of simulation, which can be specified as:

$$SLL = \sum_{n=1}^N \ln \left[\frac{1}{R} \sum_{r=1}^R \left(\prod_{t=1}^{T_n} \frac{1}{K} \sum_{k=1}^K (P_{n,t}(y_{n,t} \mid \alpha_{r,n}, \gamma_{k,t,n})) \right) \right]. \quad (6.10)$$

This simulation uses R draws of α for consumer n , along with $K T_n$ draws of γ . Note that, in this specification, the same draws of γ are used for all draws of α . That is, $\gamma_{k,t,n}$ does not have an additional subscript for r . The total number of evaluations of a Logit probability for consumer n is equal to $R K T_n$, compared to RT_n when there is only inter-personal variation.

The *Apollo* package allows the user to incorporate continuous random heterogeneity for all types of models. In a model using `apollo_mnl` inside `apollo_probabilities`, we would thus obtain a Mixed Multinomial Logit (MMNL) model, while, with a CNL core, i.e. `apollo_cnl`, we would have a Mixed CNL model. Users can similarly specify and estimate mixed MDCEV models (an example is included in `Mixed_MDCEV.r`, and clearly also hybrid choice structures, as described in Section 7). It is straightforward to combine continuous random heterogeneity with deterministic heterogeneity for individual parameters, as shown in our example. There are very few limits imposed on what parameters can incorporate continuous random heterogeneity, opening up the use of error components for correlation across alternatives and heteroskedasticity (cf. Section 6.2). The parameters (in the models made available with *Apollo*) for which random heterogeneity is not allowed are:

- the allocation parameters α in a CNL model
- the σ parameter in a MDCEV model
- the θ parameters in a MDCNEV model

A very flexible implementation is used that minimises the changes in the code that are required to introduce random coefficients or to change between the different layers of integration. In particular, the package works with arrays in three dimensions. For a model without continuous random coefficients, the likelihood for a model (prior to multiplying across observations for the same individual) is contained in a column vector of length O , where O is the number of observations in the data. If we introduce continuous random heterogeneity across

¹⁵The mean of β_n is captured in α_n such that the mean of $\gamma_{n,t}$ is zero.

individual people, with typically multiple observations per person, the likelihood is given by a $O \times R_1$ matrix, with one row per observation, and one column per draw from the random coefficients, where we use R_1 draws per random coefficient and per individual. Here, the same draws would be reused across the T_n rows for a given individual n , meaning that we would have N sets of draws, where N is the number of individuals. In the presence of additional heterogeneity across observations for the same respondent, the likelihood becomes a cube of dimensions $O \times R_1 \times R_2$, where in this third dimension, different draws are used across different observations for the same individual. As described by [Hess and Train \(2011\)](#), a given inter-individual draw is then associated with multiple intra-individual draws. If only intra-individual heterogeneity is used, the cube collapses to an array of dimensions $O \times 1 \times R_2$, i.e. a matrix but with columns going into the third dimension rather than second dimension. Depending on the type of heterogeneity (inter and inter) present in the model, different operations are required in terms of averaging across draws and multiplying across choices, and we discuss these in detail in our example below.

A number of guidelines are appropriate at this stage:

- If a user has panel data, i.e. multiple observations for at least some of the individuals, inter-individual draws are used for variation across individuals, and intra-individual draws for variation across observations for the same individual.
- If a user has cross-sectional data, i.e. only one observation per individual, only inter-individual draws should be used.
- If a user has panel data, but uses `panelData=FALSE` in `apollo_control`, the data will be treated as cross-sectional, and only inter-individual draws should be used.

6.1.2 Example model specification

In what follows, we show the specification of a MMNL model with various levels of heterogeneity on the route choice data described in Section 3.2. We specify the utility of alternative j for individual n in choice situation t in willingness to pay space as:

$$V_{n,j,t} = \delta_j + \beta_{TC,n} (\beta_{VTT,n,t} TT_{n,j,t} + TC_{n,j,t} + \beta_{VHW,n} HW_{n,j,t} + \beta_{VCH} CH_{n,j,t}), \quad (6.11)$$

where $TT_{n,j,t}$, $TC_{n,j,t}$, $HW_{n,j,t}$ and $CH_{n,j,t}$ refer to the travel time, travel cost, headway and interchanges attributes, respectively, for alternative j in choice situation t for individual n . The treatment in terms of deterministic and random heterogeneity differs across the various parameters, as we will now explain in turn:

- Alternative specific constants (ASC) are included to capture any left-right bias in the survey, where we set $\delta_2 = 0$ for normalisation. No random or deterministic heterogeneity is incorporated for δ_1 .
- The travel cost coefficient $\beta_{TC,n}$ multiplies the entire remainder of the utility function, meaning that our model produces direct estimates of willingness-to-pay (WTP) measures through working in WTP space ([Train and Weeks, 2005](#)). We use a negative log-uniform distribution (cf. [Hess et al., 2017](#)) for this coefficient, capturing inter-individual heterogeneity only, with

$$\beta_{TC,n} = -\exp(a_{\log(\beta_{TC})} + b_{\log(\beta_{TC})} \cdot \xi_{tc,n}), \quad (6.12)$$

where $\xi_{tc,n}$ follows a uniform distribution across individuals (but is constant across choices for the same individual), and $a_{\log(\beta_{TC})}$ and $b_{\log(\beta_{TC})}$ are the offset and range, respectively, for the Uniform distribution used for the log of β_{TC} .

- The value of travel time parameter $\beta_{VTT,nt}$ gives a direct estimate of the monetary valuation of travel time (VTT). We use a very flexible distribution for this coefficient. We begin with a lognormal distribution at the inter-individual level, but add additional heterogeneity across choices for the same individual, a semi non-parametric term to allow for deviation from the lognormal distribution at the individual level (Fosgerau and Mabit, 2013), and a deterministic multiplier to allow for differences between business and non-business travellers. We have:

$$\begin{aligned} \beta_{VTT,n,t} = \exp[& \mu_{\log(\beta_{VTT})} \\ & + \sigma_{\log(\beta_{VTT}),inter} \cdot \xi_{tt,n} \\ & + \sigma_{\log(\beta_{VTT}),inter,2} \cdot \xi_{tt,n}^2 \\ & + \sigma_{\log(\beta_{VTT}),intra} \cdot \xi_{tt,nt}] \\ & \cdot (\gamma_{VTT,business} \cdot x_{business,n} + (1 - x_{business,n})), \end{aligned} \quad (6.13)$$

where $\mu_{\log(\beta_{VTT})}$ is the estimated mean for the log of β_{VTT} , $\sigma_{\log(\beta_{VTT}),inter}$ and $\sigma_{\log(\beta_{VTT}),inter,2}$ are the standard deviation and first additional Fosgerau and Mabit (2013) polynomial term at the inter-individual level, multiplying the inter-individual level standard normally distributed $\xi_{tt,n}$ error term and its square, respectively, and $\sigma_{\log(\beta_{VTT}),intra}$ captures additional intra-individual heterogeneity by multiplying a standard normally distributed error term which also varies observations for the same individual, $\xi_{tt,nt}$. Finally $\gamma_{VTT,business}$ is a multiplier for business travellers, for whom $x_{business,n} = 1$. The subscript t on $\beta_{VTT,n,t}$ reflects the fact that β_{VTT} is distributed across individuals and across choices.

- The value of headway parameter $\beta_{VHW,n}$ again follows a lognormal distribution only at the inter-individual level, but with correlation with the inter-individual heterogeneity in the value of travel time parameter $\beta_{VTT,nt}$, such that:

$$\beta_{VHW,n} = \exp(\mu_{\log(\beta_{VHW})} + \sigma_{\log(\beta_{VHW})} \cdot \xi_{hw,n} + \sigma_{\log(\beta_{VHW},\beta_{VTT})} \cdot \xi_{tt,n}), \quad (6.14)$$

where $\xi_{hw,n}$ again follows a standard Normal distribution across individuals, and where the reuse of $\xi_{tt,n}$ from the $\beta_{VTT,nt}$ definition allows us to capture correlation between $\beta_{VHW,n}$ and $\beta_{VTT,nt}$ through the estimate $\sigma_{\log(\beta_{VHW},\beta_{VTT})}$. If the parameter $\sigma_{\log(\beta_{VHW},\beta_{VTT})}$ is positive, we get positive correlation between the distribution of $\beta_{VTT,n,t}$ and $\beta_{VHW,n}$, meaning that people who are more sensitive to travel time are also more sensitive to headway, and vice versa, while a negative estimate would imply negative correlation, meaning that people who are more sensitive to travel time are less sensitive to headway, and vice versa. The actual correlation is complicated in this case because of the semi-non parametric term and the lognormals, but can be calculated empirically from the draws produced by `apollo_unconditionals`.

- The value of interchanges parameter β_{VCH} is estimated without any heterogeneity, hence the lack of subscript.

We use this highly complex specification with a view to illustrating both the flexibility of the code and the ease of implementation of complex models.

6.1.3 Implementation

We explain the implementation of the model from Section 6.1.2 in four simple steps. We do not revisit obvious steps such as the definition of parameters to estimate. The model

is implemented in `MMNL_wtp_space_inter_intra.r`, with simpler Mixed Logit models also available in `MMNL_preference_space.r` and `MMNL_preference_space_correlated.r`.

Settings

For models with mixing, it is useful for a user to set `nCores` to a value larger than 1 in `apollo_control`, a point we return to in Section 6.5. We would thus for example have:

```
1 apollo_control = list(
2   ...
3   nCores       = 4,
4   ...
5 )
```

Figure 6.1: Setting the number of cores

Draws

The second step concerns the generation of draws for random distributions. In our case, we need to produce uniformly distributed inter-individual draws for $\xi_{tc,n}$, normally distributed inter-individual draws for $\xi_{tt,n}$ and $\xi_{hw,n}$, and normally distributed intra-individual draws for $\xi_{tt,n,t}$. Draws are generated by *Apollo* using the settings defined in a list called `apollo_draws`. This process happens during `apollo_validateInputs`. The setup used for this is illustrated in Figure 6.2.

```
1 apollo_draws = list(
2   interDrawsType = "halton",
3   interNDraws    = 100,
4   interUnifDraws = c("draws_tc_inter"),
5   interNormDraws = c("draws_hw_inter", "draws_tt_inter"),
6   intraDrawsType = "mlhs",
7   intraNDraws    = 100,
8   intraUnifDraws = c(),
9   intraNormDraws = c("draws_tt_intra")
10 )
```

Figure 6.2: Defining settings for generation of draws

In `apollo_draws`, the user needs to create settings for the type of draws, both for inter (`interDrawsType`) and intra-individual (`intraDrawsType`) draws. Seven pre-defined types of draws are available in *Apollo*, namely:

- pmc** for pseudo-Monte Carlo draws;
- halton** for Halton draws (Halton (1960), not recommended for more than five random coefficients, (cf. Bhat, 2003));
- mlhs** for MLHS draws (Hess et al., 2006);
- sobol** for Sobol draws (Sobol, 1967);
- sobolOwen** for Sobol draws with Owen scrambling (Owen, 1995);
- sobolFaureTezuka** for Sobol draws with Faure-Tezuka scrambling (Faure and Tezuka, 2000); and
- sobolOwenFaureTezuka** for Sobol draws with both Owen and Faure-Tezuka scrambling

While the type of draws used can differ between the inter and intra-individual sets of draws, multiple sets of draws within either category (i.e. inter or intra) will come from the same type. In our case, we use Halton draws for the inter-individual draws and MLHS draws for the intra-individual draws. The use of Halton draws is possible here given the low number of random coefficients, but Halton draws are not advised for more than 5 random coefficients given colinearity issues (cf. [Bhat, 2003](#)). When using the same type of draws for both inter and intra-individual draws, different parts of the sequence (e.g. primes for Halton) are used for the two types.

The user needs to next specify how many draws are to be used per individual for inter-individual draws, and per observation for intra-individual draws. This is set via `interNDraws` and `intraNDraws`, respectively. The number can differ between these two dimensions of integration. We use 100 inter-individual draws per parameter and per individual, and 100 intra-individual draws per parameter and per choice situation. If only inter-individual draws are to be used, then a setting of `intraNDraws = 0` is used, with a corresponding approach for intra-individual draws only. Alternatively, these settings can be omitted by the user.

Finally, the user needs to define the actual random disturbances or sets of draws, by giving each set of draws a name which can be used later in the model specification, and by determining whether the draws are Normally or Uniformly distributed, by including their names in `interNormDraws` and `interUnifDraws`, respectively, in the case of inter-individual draws, and `intraNormDraws` and `intraUnifDraws`, respectively, in the case of intra-individual draws. These two distributions (standard Normal and Uniform between 0 and 1) are used as the base for any other distributions later in the code. All the draws in our example follow standard Normal distributions, except $\xi_{tc,n}$, which comes from a Uniform distribution between 0 and 1. A user can either specify empty vectors for any settings that are not in use, such as `intraUnifDraws = c()` in our case, or omit these settings entirely.

Some users may want additional flexibility to combine different types of draws or to generate their own draws. This is possible in *Apollo* by giving the name of a user generated object in `apollo_draws$interDrawsType` and/or `apollo_draws$intraDrawsType` instead of providing one of the seven specific types of draws listed above. Using the example from [Figure 6.2](#), let us assume the user wants to provide his/her own draws for inter-individual mixing, but continue to use the *Apollo* generated MLHS draws for intra-individual mixing. In that case, the user needs to replace `halton` by for example `ownInterDraws`, where this is a list, with one element per random set of draws. Each entry in the list needs to have a name, where this same set of names is then used across `interUnifDraws` and `interNormDraws` to instruct the code to either leave the draws untransformed or apply an inverse Normal CDF. The draws provided in the list `ownInterDraws` should thus be uniformly distributed. The user also still needs to specify `interNDraws` and `intraNDraws`. Each element of the list of draws provided by the user (`ownInterDraws` in our example) should be a matrix containing the user-generated draws. In the case of inter-individual draws, each matrix must have one row per individual in the database and `interNDraws` columns, while, for intra-individual draws, the matrix must have one row per observation in the database, and `intraNDraws` columns.

Random coefficients

The third step concerns the actual definition of those coefficients in the model that follow a random distribution. For this, the code includes an additional function defined outside the `apollo_probabilities` function, namely `apollo_randCoeff`. Just as with `apollo_probabilities`, this is a function that the user does not call but which the user

defines.

```

1  apollo_randCoeff = function(apollo_beta, apollo_inputs){
2    randcoeff = list()
3
4    randcoeff[["b_tc"]] = -exp( mu_log_b_tc
5                               + sigma_log_b_tc_inter      * draws_tc_inter )
6
7    randcoeff[["v_tt"]] = ( exp( mu_log_v_tt
8                               + sigma_log_v_tt_inter      * draws_tt_inter
9                               + sigma_log_v_tt_inter_2    * draws_tt_inter ^ 2
10                              + sigma_log_v_tt_intra      * draws_tt_intra )
11                          * ( gamma_vtt_business      * business + ( 1 - business ) ) )
12
13    randcoeff[["v_hw"]] = exp( mu_log_v_hw
14                              + sigma_log_v_hw_inter      * draws_hw_inter
15                              + sigma_log_v_hw_v_tt_inter * draws_tt_inter )
16
17    return(randcoeff)
18  }

```

Figure 6.3: The `apollo_randCoeff` function

This function takes `apollo_beta` and `apollo_inputs` as inputs and generates a new list which contains the random coefficients, incorporating any deterministic effects too. This step is shown in Figure 6.3, where the correspondence with Equations 6.12 to 6.14 should be clear. The contents of `apollo_randCoeff` will vary across model specifications, only the first line (`randCoeff = list()`) and final line (`return(randCoeff)`) are to remain as in the example.

Model definition

The final step consists of adapting the `apollo_probabilities` function to work with random coefficients. This step is in essence the easiest as the writing of the utility functions and probabilities is equivalent to the approach used in the models without random heterogeneity. This is possible thanks to having defined the actual random coefficients in the `apollo_randCoeff` function which means that the user can now simply use the elements contained in the `randCoeff` list.

We illustrate this in Figure 6.4. As we can see, we still define the model as MNL, as this is the model structure conditional on the random coefficients. The only distinction with the earlier MNL example is that we make calls to two additional functions towards the end of `apollo_probabilities`. In the MNL example in Figure 4.7, we made a call to `apollo_panelProd`, which takes the product across choices for the same individual, before preparing the probabilities for output using `apollo_prepareProb`. In our MMNL model, the probabilities are however not now given by a vector with one value per choice task, but a cube with one column per inter-individual draw in the second dimension, and one column per intra-individual draw in the third dimension. The actual log-likelihood function for our model is thus given by:

$$L(\Omega) = \prod_{n=1}^N \int_{\xi_{tc,n}} \int_{\xi_{tt,n}} \int_{\xi_{hw,n}} \prod_{t=1}^{T_n} \int_{\xi_{tt,n,t}} P_{y_{n,t}} d\xi_{tt,n,t} d\xi_{hw,n} d\xi_{tt,n} d\xi_{tc,n}, \quad (6.15)$$

The two layers of integration need to be approximated using numerical simulation, where


```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Function initialisation: do not change the following three commands
4      ### Attach inputs and detach after function exit
5      apollo_attach(apollo_beta, apollo_inputs)
6      on.exit(apollo_detach(apollo_beta, apollo_inputs))
7
8      ### Create list of probabilities P
9      P = list()
10
11     ### List of utilities: these must use the same names as in mnl_settings, order is irrelevant
12     V = list()
13     V[["alt1"]] = asc_1 + b_tc*(v_tt*tt1 + tc1 + v_hw*hw1 + v_ch*ch1)
14     V[["alt2"]] = asc_2 + b_tc*(v_tt*tt2 + tc2 + v_hw*hw2 + v_ch*ch2)
15
16     ### Define settings for MNL model component
17     mnl_settings = list(
18         alternatives = c(alt1=1, alt2=2),
19         avail        = list(alt1=1, alt2=1),
20         choiceVar    = choice,
21         utilities     = V
22     )
23
24     ### Compute probabilities using MNL model
25     P[["model"]] = apollo_mnl(mnl_settings, functionality)
26
27     ### Average across intra-individual draws
28     P = apollo_avgIntraDraws(P, apollo_inputs, functionality)
29
30     ### Take product across observation for same individual
31     P = apollo_panelProd(P, apollo_inputs, functionality)
32
33     ### Average across inter-individual draws
34     P = apollo_avgInterDraws(P, apollo_inputs, functionality)
35
36     ### Prepare and return outputs of function
37     P = apollo_prepareProb(P, apollo_inputs, functionality)
38     return(P)
39 }

```

Figure 6.4: The `apollo_probabilities` function for a MMNL model

different functions are used for simulation at the inter-individual and intra-individual level. These two functions, `apollo_avgIntraDraws()` and `apollo_avgInterDraws` are called as:

```

P = apollo_avgIntraDraws(P,
                        apollo_inputs,
                        functionality)

```


and

```
P = apollo_avgInterDraws(P,
                          apollo_inputs,
                          functionality)
```

In our example, we first average across intra-individual draws, using `apollo_avgIntraDraws`. We then take the product over choices, using `apollo_panelProd`, before averaging across the inter-individual draws using `apollo_avgInterDraws` to obtain a column vector once again, with one row per individual. We finally call `apollo_prepareProb`.

Returning to our earlier point, while the example here is for a MMNL model, i.e. a mixture of a MNL kernel, it is similarly possible to use for example a Mixed Nested Logit model, and in *Apollo*, this is straightforward by replacing `apollo_mnl` with `apollo_nl` and defining appropriate additional arguments.

6.1.4 Estimation

The estimation of a continuous Mixed Logit model uses the same routine `apollo_estimate` as our other models, and the code automatically finds the draws and random coefficients in `apollo_inputs`. This is illustrated in Figure 6.5, where we use 3 cores in estimation, and where the use of a MNL kernel inside the MNL model is made clear by the output.

```
1 > model = apollo_estimate(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 ...
3 Overview of choices for MNL model component :
4           alt1    alt2
5 Times available    3492.00 3492.00
6 Times chosen      1734.00 1758.00
7 Percentage chosen overall    49.66  50.34
8 Percentage chosen when available    49.66  50.34
9
10 Pre-processing likelihood function...
11 Creating cluster...
12 Preparing workers for multithreading...
13
14 Testing influence of parameters
15 Starting main estimation
16
17 BGW using analytic model derivatives supplied by caller...
18
19
20 Iterates will be written to:
21 output/MMNL_wtp_space_inter_intra_iterations.csv
22   it   nf    F      RELDF    PRELDF    RELDX    MODEL stppar
23   0    1 2.406919551e+03
24   1    3 2.242875974e+03 6.815e-02 2.857e-02 3.81e-02    G   4.31e+00
25   ...
```

Figure 6.5: Running `apollo_estimate` for MMNL using 3 cores

6.2 Error components

The main focus when using random parameters in choice models is to introduce heterogeneity in sensitivities to individual attributes. However, random parameters can similarly be used as “error components”, with a view to introducing for example heteroskedasticity or correlation across alternatives. An introduction to the use of error components is given in Train (2009, chapter 6.3). With error components, special care is required in relation to identification and normalisation, an issue ignored by many and discussed in detail by Walker et al. (2007).

The use of error components is straightforward in *Apollo*, and simply involves the use of random parameters that do not multiply an attribute¹⁶. We show two examples of code here, one of them for introducing heteroskedasticity, and the other for capturing the so called pseudo panel effect.

An example of how to introduce heteroskedasticity is included in `ECL_preference_space_heteroskedasticity.r`, which is an extension of `MMNL_preference_space.r`. This involves adding a $N(0, \sigma_{hsk})$ term to the first utility. We show only those parts of the affected code in Figure 6.6. The user needs to create a new set of draws and a random component that creates the $N(0, \sigma_{hsk})$ term, called `ec` in our example, which is then added to the utility of the first alternative.

```

1  apollo_beta = c(...)
2      sigma_hsk      = 0.01)
3
4  apollo_draws = list(
5      ...
6      interNormDraws = c("draws_tt", "draws_tc", "draws_hw", "draws_ch", "draws_hsk"),
7      ...
8  )
9
10 apollo_randCoeff = function(apollo_beta, apollo_inputs){
11     ...
12     randcoeff[["hsk"]] = sigma_hsk * draws_hsk
13     ...
14 }
15
16 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
17     ...
18     V[["alt1"]] = b_tt * tt1 + b_tc * tc1 + b_hw * hw1 + b_ch * ch1 + hsk
19     ...
20 }

```

Figure 6.6: Using error components for heteroskedasticity

Another popular use of error components aims to capture an individual-specific effect that creates correlation across choices for the same respondent, the so called pseudo-panel effect. It is clearly impossible for identification reasons to add the same error components to all J alternatives. As for example reported by Yáñez et al. (2011), it has become common practice to instead include the same error component in $J - 1$ of the utility functions. Unfortunately, such a specification now introduces correlation across those $J - 1$ alternatives, as well as heteroskedasticity. Adding an error component to just a single alternative is no better, while

¹⁶This is the *traditional* use of error components, although there are also cases where error components multiply attributes, such as in Hess et al. (2007b)

randomly varying the omitted error component across respondents (cf. [Yáñez et al., 2011](#)) is also problematic, as it creates random variations in the correlation and/or heteroskedasticity structure across individuals. An alternative is to use an approach first put forward by [Hess et al. \(2008\)](#) which consists of adding *iid* (across respondents but not observations) error components to *all* of the alternatives, where this model is identified in panel data given the independent nature of the error components. We illustrate this process in Figure 6.7, with the code available in [ECL_preference_space_panel_effect.r](#). We now require two separate sets of draws, and create two independently but identically distributed $N(0, \sigma_{panel})$ terms in `apollo_randCoeff`. These use different draws for each error component, but the same parameter `sigma_panel` for the standard deviation. One term is then included in each utility function.

```

1  apollo_beta = c(...)
2      sigma_panel = 0)
3
4  apollo_draws = list(
5      ...
6      interNormDraws = c("draws_tt", "draws_tc", "draws_hw", "draws_ch", "draws_alt1", "draws_alt2"),
7      ...
8  )
9
10 apollo_randCoeff = function(apollo_beta, apollo_inputs){
11     ...
12     randcoeff[["ec_alt1"]] = sigma_panel * draws_alt1
13     randcoeff[["ec_alt2"]] = sigma_panel * draws_alt2
14     ...
15 }
16
17 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
18     ...
19     V[["alt1"]] = b_tt * tt1 + b_tc * tc1 + b_hw * hw1 + b_ch * ch1 + ec_alt1
20     V[["alt2"]] = b_tt * tt2 + b_tc * tc2 + b_hw * hw2 + b_ch * ch2 + ec_alt2
21     ...
22 }

```

Figure 6.7: Using error components for a pseudo-panel effect

6.3 Discrete mixtures and Latent Class

Apollo offers the same degree of flexibility with Latent Class and Discrete Mixture models as with continuous mixture models. Unlike with continuous mixtures, the α parameters in CNL, the σ parameters in MDCEV, and the θ parameters in MDCNEV can also vary across classes.

In a Latent Class model, heterogeneity is accommodated by making use of separate classes with different values for the vector β in each class. With S classes, we have S instances of β , say β_1 to β_S , with the possibility of some elements staying fixed across some classes. Individual n belongs to class s with probability $\pi_{n,s}$, where $0 \leq \pi_{n,s} \leq 1 \forall s$ and $\sum_{s=1}^S \pi_{n,s} = 1$.

Let $P_{i,n,t}(\beta_s)$ give the probability of respondent n choosing alternative i in choice situation t , conditional on n falling into class s , where $P_{i,n,t}$ is typically specified as a MNL model, but this is not a requirement in theory or in *Apollo*. Indeed, there is now a substantial body of work using different model structures (often based on different decision rules) in different classes (cf. [Hess et al., 2012](#)).

The unconditional (on s) choice probability is then given by:

$$P_{i,n,t}(\beta_1, \dots, \beta_S) = \sum_{s=1}^S \pi_{n,s} P_{i,n,t}(\beta_s) \quad (6.16)$$

In the presence of repeated choice data, it is natural to perform the mixing at the level of individual people, and we then have that the probability of the sequence of choices/observations for person n is given by:

$$L_n(\beta) = \sum_{s=1}^S \pi_{n,s} \prod_{t=1}^{T_n} P_{y_{n,t}}(\beta_s) \quad (6.17)$$

where $\beta = \langle \beta_1, \dots, \beta_S \rangle$, and where $y_{n,t}$ gives the alternative chosen by person n in choice situation t .

In the most basic version, the class allocation probabilities $\pi_{n,s}$ are constant across respondents, i.e. $\pi_{n,s} = \pi_s \forall n$. The real flexibility of the model arises when linking class allocation to socio-demographics, where we use a class allocation model, with typically an underlying Logit structure, such that:

$$\pi_{n,s} = \frac{e^{\delta_s + g(\gamma_s, z_n)}}{\sum_{l=1}^S e^{\delta_l + g(\gamma_l, z_n)}}, \quad (6.18)$$

where δ_s is an offset, and γ_s is a vector of parameters capturing the influence of the vector of individual characteristics z_n on the class allocation probabilities. For normalisation, δ_s is fixed to 0 for one of the S classes, as is γ_s . In a model with constant class allocation probabilities across individuals, we would only estimate the vector of constants δ .

To illustrate the implementation of Latent Class models in *Apollo*, we provide an example on the Swiss route choice data also used for the Mixed Logit model in Section 6.1.2. We develop a model with two classes, where all four marginal utility parameters (time, cost, headway and interchanges) vary across the classes, but where the ASCs are kept fixed across classes. For the class allocation model, we use two socio-demographic characteristics, namely

whether an individual was on a commute journey or not, and whether they had a car available to them. This example is implemented in [LC_with_covariates.r](#), where a simpler Latent Class model without covariates is available in [LC_no_covariates.r](#).

The development of a Latent Class model in *Apollo* consists of two steps, which we now look at it turn.

Defining Latent Class parameters

We first implement a function called `apollo_lcPars`, which performs a role analogous to `apollo_randCoeff` for continuous mixtures. This is thus another function that is not called by the user but which is developed by the user for the specific model that is to be used. Like `apollo_randCoeff`, this function takes `apollo_beta` and `apollo_inputs` as inputs and generates a new list which contains the parameters that vary across classes as well as the class allocation probabilities. The contents of `apollo_lcPars` will vary across model specifications, only the first line (`lcPars = list()`) and final line (`return(lpars)`) are to remain as in the example.

The implementation for our example is shown in Figure 6.8. We create a list called `lcPars` which contains the values for the different parameters across classes, as well as the class allocation probabilities. As can be seen from Figure 6.8, we first produce one element in the list for each of the four marginal utility coefficients. Each one of these elements is a list in itself, and contains the values for the coefficients across the two classes. If more classes are to be used, more entries are added into each one of these lists, where the possibility exists of keeping the values constant for some parameters across some or all of the classes (in which case the number of values still needs to be the same as the number of classes, but some of them are repeated). Note that for parameters that are kept constant across all (i.e. not just some) of the classes, such as the ASCs in our example, there is no need (though also no harm) to include them in `lcPars`.

```

1  apollo_lcPars=function(apollo_beta, apollo_inputs){
2    lcpars = list()
3    lcpars[["beta_tt"]] = list(beta_tt_a, beta_tt_b)
4    lcpars[["beta_tc"]] = list(beta_tc_a, beta_tc_b)
5    lcpars[["beta_hw"]] = list(beta_hw_a, beta_hw_b)
6    lcpars[["beta_ch"]] = list(beta_ch_a, beta_ch_b)
7
8    ### Utilities of class allocation model
9    V=list()
10   V[["class_a"]] = delta_a + gamma_commute_a*commute + gamma_car_av_a*car_availability
11   V[["class_b"]] = delta_b + gamma_commute_b*commute + gamma_car_av_b*car_availability
12
13   ### Settings for class allocation models
14   classAlloc_settings = list(
15     classes      = c(class_a=1, class_b=2),
16     utilities    = V
17   )
18   lcpars[["pi_values"]] = apollo_classAlloc(classAlloc_settings)
19
20   return(lcpars)
21 }
```

Figure 6.8: The `apollo_lcPars` function

We next calculate the class allocation probabilities, i.e. $\pi_{n,s}$, $\forall n, s$. We use a MNL model

for the class allocation probabilities, for which we use the function `apollo_classAlloc`, which functions like `apollo_MNL`, but returns probabilities for all classes, and thus not use a choice variable. The `apollo_classAlloc` function is called via:

```
lcpars[["pi_values"]] = apollo_classAlloc(classAlloc_settings)
```

The function takes as its sole input a list called `classAlloc_settings` which has four possible inputs, of which only `utilities` is compulsory. We will now look at these in turn.

`apollo_classAlloc`: user-controlled settings

avail: A list containing one element per class, using the same names as in `classes`. For each alternative, we define the availability either through a vector of values of the same length as the number of observations (i.e. a column from the data) or by a scalar of 1 if an alternative is always available. A user can also set `avail=1` (or omit its use) which implies that all of the alternatives are available for every choice observation in the data.

classes: A named vector containing the names of the classes. When not defined, the names from the utilities will be used automatically.

rows: The optional `rows` argument, as previously defined for MNL.

utilities: A list object containing one utility for each class, using the same names as in `classes`.

In our example, both classes are available for all individuals in the data, and the `avail` setting is thus omitted.

Model definition

We next turn to the calculation of the actual Latent Class choice probabilities, a process that is illustrated in Figure 6.9. As can be seen, we first create a generic version of `mnl_settings` that contains those settings which will be constant across classes, namely the alternatives, availabilities and choice variable.

We then incorporate a loop over classes (where the number of classes is given by the number of entries in `pi_values`), where we calculate the utilities for the two alternatives in each class, using the appropriate values for those parameters that vary across classes. In each class, we update `mnl_settings` to use the utilities in that specific class, and we also define a name for each class in the `componentName` setting, where we use the `paste0` function in R to combine the string `Class_` with the index for the class. This step is not compulsory but helps with interpreting the outputs. We finally create one element in the `P` list for each class, where we again give these a name using `paste0` rather than just an index. Again, this is not compulsory, and if a simple index is given (i.e. using `P[[s]]`), the class-specific fits will be referred to as `component_1`, `component_2`, etc. The reader will note that in class s , we are using the coefficients for that class (e.g. `beta_tc[[s]]` uses the s^{th} element in `beta_tc` created in `apollo_lcPars`), and the call to `apollo_mnl` in each class uses the appropriate utilities for that class as these are updated inside the overall `mnl_settings` using `mnl_settings$utilities = V` in each step of the loop. In our example, we calculate the within class probabilities using a MNL model, where it would again also be possible to use different models inside the Latent Class structure, e.g. Nested Logit. In preparation for the averaging across classes, we take the product across choices for the same individual in each class, using `apollo_panelProd`, in line with Equation 6.17.

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### Define settings for MNL model component that are generic across classes
11     mnl_settings = list(
12         alternatives = c(alt1=1, alt2=2),
13         avail        = list(alt1=1, alt2=1),
14         choiceVar    = choice
15     )
16
17     ### Loop over classes
18     for(s in 1:2){
19
20         ### Compute class-specific utilities
21         V=list()
22         V[["alt1"]] = asc_1 + beta_tc[[s]]*tc1 + beta_tt[[s]]*tt1 + beta_hw[[s]]*hw1 +
23         ↪ beta_ch[[s]]*ch1
24         V[["alt2"]] = asc_2 + beta_tc[[s]]*tc2 + beta_tt[[s]]*tt2 + beta_hw[[s]]*hw2 +
25         ↪ beta_ch[[s]]*ch2
26
27         mnl_settings$utilities    = V
28
29         ### Compute within-class choice probabilities using MNL model
30         P[[paste0("Class_",s)]] = apollo_mnl(mnl_settings, functionality)
31
32         ### Take product across observation for same individual
33         P[[paste0("Class_",s)]] = apollo_panelProd(P[[paste0("Class_",s)]], apollo_inputs
34         ↪ ,functionality)
35     }
36
37     ### Compute latent class model probabilities
38     lc_settings = list(inClassProb = P, classProb=pi_values)
39     P[["model"]] = apollo_lc(lc_settings, apollo_inputs, functionality)
40
41     ### Prepare and return outputs of function
42     P = apollo_prepareProb(P, apollo_inputs, functionality)
43     return(P)
44 }

```

Figure 6.9: Implementing choice probabilities for Latent Class

We now have the likelihoods in each class, i.e. for class s , we have $L_{n,s} = \prod_{t=1}^{T_n} P_{y_{n,t}}(\beta_s)$. The remaining step is to take the weighted average across classes, i.e. $\sum_{s=1}^S \pi_{n,s} L_{n,s}$. This is achieved by the `apollo_lc` function, which uses the within class probabilities contained in the S existing elements of P , multiplies each one by the appropriate class allocation probability

in `pi_values`, and then sums across classes. This function is called as:

```
P[["model"]] = apollo_lc(lc_settings,
                        apollo_inputs,
                        functionality)
```

The list `lc_settings` contains two elements, namely:

apollo_lc: user-controlled settings

classProb: A list of class allocation probabilities, which can be either scalars (if constant across people), vectors (if using only deterministic heterogeneity) or matrices or cubes (if including continuous random heterogeneity in the class allocation probabilities, a point we return to in Section 6.4).

componentName: This is an optional argument of the character type which allows the user to specify a name of the given model component. This is then used in various places in diagnostic tests and model outputs. If omitted, a default is used by *Apollo*. It is recommended the user omits this setting and lets *Apollo* set it up automatically.

inClassProb: A list of in class probabilities, i.e. the $L_{n,s}$ for different classes. These need to already have all continuous random heterogeneity averaged out and contain one entry per individual, i.e. having been multiplied across observations for the same individual.

The output from this function is the actual Latent Class model probability in Equation 6.17 and is stored in the `model` component of the list `P`.

One final point to note here relates to the class specific model components. In the discussion here, the probabilities for individual classes are stored inside `P`, in our case as `P[["Class_1"]]` and `P[["Class_2"]]`, while the probability for the model is stored in `P[["model"]]`. The output from model estimation will then also report the log-likelihood from the individual classes. When using a Latent Class model inside a hybrid structure, the within-class probabilities should however be stored in a separate list from `P`, a point we return to in Section 7.

The focus in our discussion so far has been on Latent Class models rather than Discrete Mixtures (cf. Hess et al., 2007a). In a Discrete Mixture model, we have S_k values for parameter β_k , where the number of possible values S_k can vary across parameters (and can be 1 for some). A weight $\pi_{n,k,s}$ is assigned to the s^{th} value for β_k for person n , with $\sum_{s=1}^{S_k} \pi_{n,k,s} = 1, \forall n, k$. We thus have $\sum_{k=1}^K S_k$ possible values across the K different β parameters, and each combination is possible in the model. This effectively means that the Discrete Mixture model can be written as a Latent Class model with $\prod_{k=1}^K S_k$ classes, where, for example, the first class might use the first value for each of the coefficients, and thus have a class allocation probability $\pi_{n,s}^* = \prod_{k=1}^K \pi_{n,k,1}$. Discrete mixtures can thus be estimated using software for Latent Class, including in *Apollo*, and an example is given in DM.r. The use of Discrete Mixture models leads to a larger number of parameters, as we now have separate $\pi_{n,k,s}$ for different β parameters, as well as generally a larger number of overall classes (and hence a more complex likelihood function) given that $S^* = \prod_{k=1}^K S_k$. Latent class models also provide a more natural way of capturing correlation in the heterogeneity across different coefficients.

6.4 Combining Latent Class with continuous random heterogeneity

Apollo also allows users to combine continuous random heterogeneity with Latent Classes (cf. [Greene and Hensher, 2013](#)). Continuous heterogeneity can be allowed for both in the within-class probabilities and in the class membership probabilities. Specifically, let us assume that the vector π is distributed according to $f(\pi | \Omega_\pi)$ where Ω_π is a vector of parameters, while the vector β_s , which contains the parameters for the within-class model in class s is distributed according to $g_s(\beta_s | \Omega_{\beta_s})$, where Ω_{β_s} is a vector of parameters, and where $\Omega_\beta = \langle \Omega_{\beta_1}, \dots, \Omega_{\beta_S} \rangle$. We then have:

$$L_n(\Omega_\pi, \Omega_\beta) = \int_\pi \sum_{s=1}^S \pi_{n,s} \left(\int_{\beta_s} \prod_{t=1}^{T_n} P_{y_{n,t}}(\beta_s) g_s(\beta_s | \Omega_{\beta_s}) d\beta_s \right) f(\pi | \Omega_\pi) d\pi. \quad (6.19)$$

The integration across the distribution for heterogeneity in the within-class model is thus carried out prior to averaging across classes, while the integration across the distribution for heterogeneity in the class-allocation model is carried out outside the averaging across classes. For estimation, this implies averaging across draws in two distinct places, as we will now illustrate.

We extend the model from Section 6.3 by allowing the travel time coefficient to follow a negative lognormal distribution, with separate parameters in the two classes. This model is included in is available in `LC_MMNL.r`.

The specification of the random parameters is illustrated in Figure 6.10. We have that `draws_tt` are standard Normal draws defined in `apollo_draws`. We use a negative Lognormal distribution for `tt_a` and `tt_b`. These random time coefficients are then also used inside `apollo_lcPars` when defining `lcPars[["tt"]]`.

The definition of the model probabilities differs from that of the simple Latent Class model in Figure 6.9 in only two ways. In particular, as seen in Figure 6.11, in line with Equation 6.19, we now average across the random draws in the within class likelihoods via `P[[s]] = apollo_avgInterDraws(P[[s]], apollo_inputs, functionality)`, after taking the product across observations for the same individual using `apollo_panelProd`. This gives one likelihood for the observed choices for each person within each class. We then perform the weighted summation across classes using `P[["model"]] = apollo_lc(lc_settings, apollo_inputs, functionality)`.

6.5 Multi-threading capabilities

Apollo allows for multi-threaded estimation for classical estimation¹⁷, leading to significant estimation speed improvements for some models. This can easily be activated by specifying the number of threads to use in `apollo_control$nCores`. The recommended number of threads is equal to the number of available processor cores in the machine minus one, which can be determined by typing `parallel::detectCores()` in the R console. The use of multi-threaded estimation comes with some restrictions.

¹⁷When using Bayesian estimation, the reliance on `RSGBH` means only single core processing is possible.

```

1  apollo_draws = list(
2    interDrawsType="halton",
3    interNDraws=500,
4    interNormDraws=c("draws_tt")
5  )
6
7  apollo_randCoeff = function(apollo_beta, apollo_inputs){
8    randcoeff = list()
9
10   randcoeff[["tt_a"]] = -exp(log_tt_a_mu + log_tt_a_sig*draws_tt)
11   randcoeff[["tt_b"]] = -exp(log_tt_b_mu + log_tt_b_sig*draws_tt)
12
13   return(randcoeff)
14 }
15
16 apollo_lcPars = function(apollo_beta, apollo_inputs){
17   lcpars = list()
18   lcpars[["tt"]] = list(tt_a, tt_b)
19   lcpars[["tc"]] = list(tc_a, tc_b)
20   lcpars[["hw"]] = list(hw_a, hw_b)
21   lcpars[["ch"]] = list(ch_a, ch_b)
22
23   V=list()
24   V[["class_a"]] = delta_a + gamma_commute_a*commute + gamma_car_av_a*car_availability
25   V[["class_b"]] = delta_b + gamma_commute_b*commute + gamma_car_av_b*car_availability
26
27   classAlloc_settings = list(
28     alternatives = c(class_a=1, class_b=2),
29     avail        = 1,
30     V            = V
31   )
32   lcpars[["pi_values"]] = apollo_classAlloc(classAlloc_settings)
33
34   return(lcpars)
35 }

```

Figure 6.10: The `apollo_randCoeff` and `apollo_lcPars` functions for a Latent Class model with continuous random heterogeneity

apollo_probabilities can only access its arguments: In other words, the likelihood can only use data stored inside `apollo_beta` and `apollo_inputs`, where the latter combines `database`, `apollo_control`, `draws`, `apollo_randCoeff` and `apollo_lcPars`. All other variables created by the user in the global environment before estimation cannot be accessed. This issue is easily avoided by creating any new variables inside the `database` object prior to calling `apollo_validateInputs`, which is good practice anyway.

Data splitting: The dataset is split among several threads, so statistics such as the mean, maximum and minimum of variables, among others, will not be reliably calculated during estimation when using multi-threading. To avoid this issue, any such statistic (for example the mean income in our MNL example in Section 4.2) need to be calculated before estimation and saved as a new variable inside `database`¹⁸.

¹⁸To illustrate this issue, in our earlier example in Section 4.5.2, we created a variable called `mean_income` inside the `database`, prior to calling `apollo_validateInputs`, by calling `database$mean_income = mean(database$income)`. This ensures that with multi-threading, the same mean income would be used in each core, while, if the variable had been created inside `apollo_probabilities`, a different mean income

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### Define settings for MNL model component that are generic across classes
11     mnl_settings = list(
12         alternatives = c(alt1=1, alt2=2),
13         avail        = list(alt1=1, alt2=1),
14         choiceVar     = choice
15     )
16
17     ### Loop over classes
18     for(s in 1:2){
19         ### Compute class-specific utilities
20         V=list()
21         V[["alt1"]] = asc1 + tc[[s]]*tc1 + tt[[s]]*tt1 + hw[[s]]*hw1 + ch[[s]]*ch1
22         V[["alt2"]] = asc2 + tc[[s]]*tc2 + tt[[s]]*tt2 + hw[[s]]*hw2 + ch[[s]]*ch2
23
24         mnl_settings$utilities = V
25
26         ### Compute within-class choice probabilities using MNL model
27         P[[paste0("Class_",s)]] = apollo_mnl(mnl_settings, functionality)
28
29         ### Take product across observation for same individual
30         P[[paste0("Class_",s)]] = apollo_panelProd(P[[paste0("Class_",s)]], apollo_inputs
31         ↪ ,functionality)
32
33         ### Average across inter-individual draws within classes
34         P[[paste0("Class_",s)]] = apollo_avgInterDraws(P[[paste0("Class_",s)]], apollo_inputs,
35         ↪ functionality)
36     }
37
38     ### Compute latent class model probabilities
39     lc_settings = list(inClassProb = P, classProb=pi_values)
40     P[["model"]] = apollo_lc(lc_settings, apollo_inputs, functionality)
41
42     ### Prepare and return outputs of function
43     P = apollo_prepareProb(P, apollo_inputs, functionality)
44     return(P)
45 }

```

Figure 6.11: Implementing choice probabilities for Latent Class with continuous random heterogeneity

Increased memory consumption: Memory consumption is increased when using multi-threading. This is because the dataset and draws (usually the biggest objects in memory) need to split, and copied into several threads.

Speed gains are dependent on the model: In general, models using few iterations that each take a long time will benefit the most. This applies to models using big datasets

would have been used across cores.

or a large number of draws in the case of mixture models. Speed gains also decrease with the number of threads used. For small models, speed gains due to multi-threading might be negligible, or even negative due to overhead.

To help decide how many cores to use, we provide the function `apollo_speedTest`, which calculates the loglikelihood function several times using different number of threads and draws, and reports both the calculation time and the memory usage. This function is called as:

```
apollo_speedTest(apollo_beta,
                 apollo_fixed,
                 apollo_probabilities,
                 apollo_inputs,
                 speedTest_settings)
```

The final argument, `speedTest_settings`, is optional and allows the user to change the following settings:

`apollo_speedTest`: user-controlled settings

nCoresTry: A vector with the number of threads to try (default is to try all cores present in the machine).
nDrawsTry: A vector with the number of draws to try (default is `c(100, 200, 500)`). Note that this may need to be reduced for very complex models if memory issues arise.
nRep: An integer setting the number of times `apollo_probabilities` is calculated for each possible pair of elements from `nDrawsTry` and `nCoresTry` (default is 10), ensuring stable results for the calculation of runtimes.

We illustrate the use of this function in Figure 6.12 for example `MMNL_wtp_space_inter_intra.r`, which shows a big benefit especially by using a second core. When running `apollo_speedTest`, progress and results of the test are printed to the console. Each row displays the set-up, progress, and results of a given configuration. The first column (*nCores*) indicates the number of computational threads in use, i.e. how many processor cores are being used simultaneously by R. The second (*inter*) and third (*intra*) columns indicate the number of inter-individual and intra-individual draws used. The third column (*progress*) indicates the progress of the test for each set-up, each dot representing 10% of the repetitions requested. The fifth column (*sec/LLCal*) indicates the average time in seconds required to complete one evaluation of the `apollo_probabilities` function. The sixth and last column (*RAM(MB)*) presents a lower bound of the memory required to evaluate the `apollo_probabilities` function. After completing the test, results are summarised in a table indicating the time required to evaluate `apollo_probabilities` under each configuration, as well as in a plot. In addition, the outputs are returned by the function as a list.

```

1 > apollo_speedTest(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs,
2   ↪ speedTest_settings)

```

nCores	___Draws___			progress	sec/	
	inter	intra	LLCall		RAM(MB)	
1	50	50	0.62	1660.1		
2	50	50	0.30	2255.5		
3	50	50	0.24	2419.8		
1	75	75	1.30	1993.2		
2	75	75	0.63	2921.5		
3	75	75	0.45	3085.8		
1	100	100	2.07	2459.4		
2	100	100	1.19	3853.9		
3	100	100	0.77	4018.3		

Summary of results (sec. per call to LL function)

	draws50	draws75	draws100
cores1	0.6204	1.2978	2.0731
cores2	0.2958	0.6347	1.1914
cores3	0.2377	0.4467	0.7701

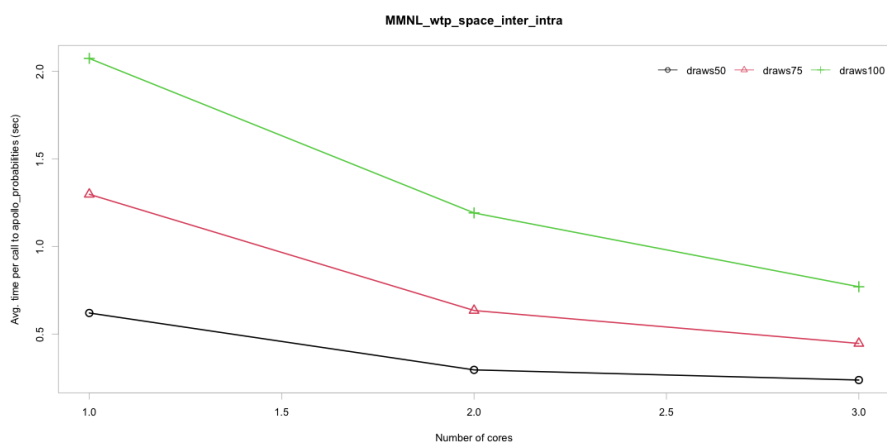


Figure 6.12: Running `apollo_speedTest`

7

Joint estimation of multiple model components

The use of models made up of several separate components is made possible by the function `apollo_combineModels`, which is called as follows:

```
P = apollo_combineModels(P,  
                           apollo_inputs,  
                           functionality,  
                           components,  
                           asList)
```

This function takes the list `P` which contains several individual model components and produces a combined model. There is no limit on the number of subcomponents. The obvious case is estimation, where, with $L_{n,m}$ giving the likelihood of model component m for person n , the overall likelihood for person n is given by $L_n = \prod_{m=1}^M L_{n,m}$ (not showing here the presence of any integration over random terms, which would be carried out outside the product). The function `apollo_combineModels` creates the `model` object inside `P` as the product across individual components - when working with multiple model components, the individual components should thus not be called `model` themselves. The optional argument `components` allows the user to instruct the function to only use a subset of the components, while the optional argument `asList` (set to `TRUE` by default) allows the user to drop all components after taking the product (if `asList` is set to `FALSE`).

The most widely used case in recent years of models with multiple components is that of hybrid choice models. Before we turn to that example, we illustrate the joint estimation capabilities of *Apollo* by looking at two simpler cases of combining two models, namely the case of joint estimation on RP and SP data, and the estimation on best-worst data.

One important point should be mentioned already here. The function `apollo_combineModels` uses all elements inside the list `P`. Thus, the user needs to be careful that only components that should be multiplied together for the overall model are

included in `P`. In general, this will always be the case. However, a distinction arises in the presence of Latent Class models. As discussed in Section 6.3, for Latent Class, the user needs to first create the within-class models, before the weighted average of these is taken by `apollo_lc`, using the class allocation probabilities as weights. The within-class probabilities are in the simplest case stored inside `P`, as in our example in Section 6.3, and this has benefits in terms of showing the within-class likelihood in the output and facilitating the calculation of posteriors (cf. Section 9.15.2). However, when a Latent Class model is one of several components in a model, the use of `apollo_combineModels` would mean that the within-class probabilities are treated as a separate model component in creating the combined likelihood across models. To avoid this, the user should make use of the `components` to specify the relevant components.

7.1 Joint estimation on RP and SP data

The example `MNL_RP_SP.r` uses the mode choice data we already covered for the earlier MNL model (cf. Section 4.5.2) but combines the RP and SP data allowing for scale differences between the two (Bradley and Daly, 1996; Hensher et al., 1998). To achieve this, we incorporate separate scale parameters μ_{RP} and μ_{SP} to the vector parameters to be estimated, where the former is kept fixed to 1 for normalisation.

The basic setup is the same as in Section 4.5.2 with the exception that we omit `database = subset(database, database$SP==1)` used earlier in Figure 4.3 as we now utilise the entire sample. The earlier part of the code remains the same as in Figure 4.7 and is largely omitted here for conciseness of presentation - this includes the definition of `mu_RP` and `mu_SP` in `apollo_beta`, and the inclusion of `mu_RP` in `apollo_fixed`.

The key differences arise in the `apollo_probabilities` function, and are illustrated in Figure 7.1. The definition of the utilities remains the same, with the difference that they are now calculated for all rows in the data, i.e. for RP rows as well as SP rows. In the example used here, the service quality attributes are coded as zero for the RP data and thus do not enter into the utility calculation for these rows.

We first calculate the probabilities for RP choices. The definition of `mnl_settings_RP` differs from that in the SP model in Figure 4.7 in that we multiply the utilities by the RP scale parameter, μ_{RP} , e.g. $V_{i,n,t,RP} = \mu_{RP}V_{i,n,t}$. RP probabilities should only be calculated for RP rows in the data, and we thus include `rows=(RP==1)`, meaning that for SP rows in the data, the probability of RP choices is simply fixed to 1 so as not to contribute to model estimation. We then make the call to `apollo_mnl`, saving the output not in `P[["model"]]` which is reserved for the overall model, but in a component called `P[["RP"]]`. For the SP part of the data, the definition of `mnl_settings_SP` is similar to before, but multiplying the utilities by `mu_SP`, and using `rows = (SP==1)`. We then calculate the probabilities for the SP rows in the data by calling `apollo_mnl` once more and storing its output in `P[["SP"]]`.

The probability for the combined model is obtained by multiplying the RP and SP components together in `P[["model"]]`, which is the component used for estimation. Rather than doing this manually, we use `P = apollo_combineModels(P, apollo_inputs, functionality)`, as this function also prepares different output depending on the setting of `functionality`, allowing the use of joint models also in prediction, for example. If panel data is being used, meaning there are multiple observations per individual, the multiplication of likelihoods across all observations from the same respondent (i.e. the call to `apollo_panelProd`) should usually happen after

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### Create alternative specific constants and coefficients using interactions with
11     ↪ socio-demographics
12     asc_bus_value = asc_bus + asc_bus_interaction_female * female
13     ...
14
15     ### List of utilities (before applying scales): these must use the same names as in
16     ↪ mnl_settings, order is irrelevant
17     V = list()
18     V[["car"]] = asc_car + b_tt_car_value * time_car + b_cost_value * cost_car
19     ...
20
21     ### Compute probabilities for the RP part of the data using MNL model
22     mnl_settings_RP = list(
23         alternatives = c(car=1, bus=2, air=3, rail=4),
24         avail       = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
25         choiceVar   = choice,
26         utilities    = list(car = mu_RP*V[["car"]],
27                             ...
28                             rail = mu_RP*V[["rail"]]),
29         rows        = (RP==1)
30     )
31
32     P[["RP"]] = apollo_mnl(mnl_settings_RP, functionality)
33
34     ### Compute probabilities for the SP part of the data using MNL model
35     mnl_settings_SP = list(
36         alternatives = c(car=1, bus=2, air=3, rail=4),
37         avail       = list(car=av_car, bus=av_bus, air=av_air, rail=av_rail),
38         choiceVar   = choice,
39         utilities    = list(car = mu_SP*V[["car"]],
40                             ...
41                             rail = mu_SP*V[["rail"]]),
42         rows        = (SP==1)
43     )
44
45     P[["SP"]] = apollo_mnl(mnl_settings_SP, functionality)
46
47     ### Combined model
48     P = apollo_combineModels(P, apollo_inputs, functionality)
49
50     ### Take product across observation for same individual
51     P = apollo_panelProd(P, apollo_inputs, functionality)
52
53     ### Prepare and return outputs of function
54     P = apollo_prepareProb(P, apollo_inputs, functionality)
55     return(P)
56 }

```

Figure 7.1: Joint RP-SP model on mode choice data


```

1  Model name : MNL_RP_SP
2  Model description : RP-SP model on mode choice data
3  Model run at : 2025-03-24 15:33:48.764811
4  Estimation method : bgw
5  Model diagnosis : Relative function convergence
6  Optimisation diagnosis : Maximum found
7      hessian properties : Negative definite
8      maximum eigenvalue : -12.026772
9      reciprocal of condition number : 3.03709e-08
10 Number of individuals : 500
11 Number of rows in database : 8000
12 Number of modelled outcomes : 8000
13      RP : 1000
14      SP : 7000
15
16 Number of cores used : 1
17 Model without mixing
18
19 LL(start) : -9366.88
20 LL (whole model) at equal shares, LL(0) : -9366.88
21 LL (whole model) at observed shares, LL(C) : -7792.08
22 LL(final, whole model) : -5802.64
23 Rho-squared vs equal shares : 0.3805
24 Adj.Rho-squared vs equal shares : 0.3786
25 Rho-squared vs observed shares : 0.2553
26 Adj.Rho-squared vs observed shares : 0.2538
27 AIC : 11641.29
28 BIC : 11729.63
29
30 LL(0,RP) : -1170.86
31 LL(final,RP) : -971.24
32 LL(0,SP) : -8196.02
33 LL(final,SP) : -4831.4
34
35 Estimated parameters : 18
36 Time taken (hh:mm:ss) : 00:00:4.23
37     pre-estimation : 00:00:1.01
38     estimation : 00:00:0.41
39     post-estimation : 00:00:2.82
40 Iterations : 12
41
42 Unconstrained optimisation.
43
44 Estimates:
45      Estimate      s.e.    t.rat.(0)    Rob.s.e. Rob.t.rat.(0)
46 ...
47 mu_RP      1.000000      NA         NA         NA         NA
48 mu_SP      1.994744    0.126408    15.7802    0.122879    16.2334

```

Figure 7.2: On screen output for RP-SP model

combining the two model components (i.e. after calling `apollo_combineModels`).

A subset of the model output is shown in Figure 7.2. We see that the model reports the joint log-likelihood as well as the subcomponents for the two separate model components, and we obtain a scale parameter for the SP data (μ_{SP}) which is significantly larger than 1. It should be noted that in models with multiple components, the output in terms of diagnostics (cf. Figure 4.8) can become quite verbose as this information is reported for each model component, and a user may thus set `noDiagnostics` to `FALSE` in `apollo_control`. The diagnostics are reported in the order that the model components appear in the overall structure, in this case RP before SP.

7.2 Joint best-worst model

In Section 5.4.2, we showed how to estimate an exploded logit model on Best-Worst data. The use of the `apollo_el` function allows the user to specify different scales for the best and worst stages, but not differences in the marginal utility parameters. The use of a different model specification for the separate stages is possible by specifying separate models and using `apollo_combineModels`.

An equivalent specification to that from Section 5.4.2 is included in `BW_joint_model.r`. This model only allows for differences in the scale parameter between best and worst. A different approach is implemented in `BW_joint_model_diff_params.r`, allowing for differences in all parameters between the stages. A user could then compare the fit of these two specifications to understand whether differences extend beyond scale effects, and then use the Delta method for comparing pairs of parameters between stages.

7.3 Hybrid choice model

We next turn to the use of *Apollo* for hybrid choice models (see [Abou-Zeid and Ben-Akiva, 2014](#), for a recent overview), where we look at a simple implementation of a model with a single latent variable on the drug choice data described in Section 3.3. We use a dummy coded specification for the three categorical variables, along with a continuous specification for risk and cost. We specify a structural model for the latent variable that uses the three socio-demographic characteristics included in the data, and then use this latent variable in the utilities for the two branded alternatives as well as in the measurement models for the four attitudinal indicators. In our example, we do not incorporate additional random heterogeneity not linked to the latent variable, but this is entirely straightforward to do by including additional terms in `apollo_randCoeff`, as shown in example `Hybrid_with_OL_and_MMNL.r`. Similarly, it is possible to combine latent variables with Latent Class structures, as shown in example `Hybrid_LC_with_OL.r`.

Two different versions of the simple hybrid choice model are provided. The first of these, `Hybrid_with_OL.r` uses an Ordered Logit model for the indicators, as discussed by [Daly et al. \(2012b\)](#). The second example, `Hybrid_with_continuous_measurement.r` uses the common simplification of treating the indicators as being normally distributed. We will now look at these two models in turn.

Specifically, we have that the latent variable for individual n is given by:

$$\alpha_n = \gamma' z_n + \eta_n, \quad (7.1)$$

where z_n is a vector combining the three socio-demographic variables for individual n , γ is a vector of estimated parameters capturing the impact of these variables on α_n and η_n is a random disturbance which follows a standard Normal distribution across individuals, i.e. $\eta_n \sim N(0, 1)$. Notice we are normalising the standard deviation of the latent variable, and not the coefficient of the first measurement equation (ζ_1 , see equations 7.4 and 7.5).

The utility for alternative j in choice situation t for individual n is given by:

$$\begin{aligned}
 V_{j,n,t} = & \sum_{s=1}^5 \beta_{brand_s} \cdot (x_{brand_{j,n,t}} == s) \\
 & + \sum_{s=1}^6 \beta_{country_s} \cdot (x_{country_{j,n,t}} == s) \\
 & + \sum_{s=1}^3 \beta_{characteristic_s} \cdot (x_{characteristic_{j,n,t}} == s) \\
 & + \beta_{side_effects} \cdot x_{side_effects_{j,n,t}} \\
 & + \beta_{price} \cdot x_{price_{j,n,t}} \\
 & + \lambda \cdot \alpha_n \cdot (j \leq 2).
 \end{aligned} \tag{7.2}$$

For the first five rows, the same specification as in Section 5.4.1 and Section 7.2 is used, with dummy coding for the categorical variables and a continuous treatment of risk and price. Finally, the inclusion of the latent variable, i.e. $\lambda \cdot \alpha_n$ only applies to the first two alternatives, i.e. the branded products. We thus get that the likelihood of the observed sequence of T_n choices for person n , conditional on β and α_n , is given by:

$$L_{C_n}(\beta, \alpha_n) = \prod_{t=1}^{T_n} \frac{e^{V_{j_{n,t}^*, t}}}{\sum_{j=1}^4 e^{V_{j,n,t}}}, \tag{7.3}$$

where $j_{n,t}^*$ is the alternative chosen by respondent n in task t .

The latent variable is also used to explain the value of the four attitudinal questions, where two different specifications are used in our example.

With the Ordered Logit model, we have that:

$$L_{I_n, ordered}(\tau, \zeta, \alpha_n) = \prod_{i=1}^4 \left(\sum_{s=1}^S \delta_{(I_n, i=s)} \left[\frac{e^{\tau_{i,s} - \zeta_i \alpha_n}}{1 + e^{\tau_{i,s} - \zeta_i \alpha_n}} - \frac{e^{\tau_{i,s-1} - \zeta_i \alpha_n}}{1 + e^{\tau_{i,s-1} - \zeta_i \alpha_n}} \right] \right), \tag{7.4}$$

where ζ_i is an estimated parameter that measures the impact of α_n on the attitudinal indicator I_i , and $\tau_{i,\cdot}$ is a vector of threshold parameters for this indicator.

With the continuous measurement model, we instead have that:

$$L_{I_n, normal}(\sigma, \zeta, \alpha_n) = \prod_{i=1}^4 \frac{1}{\sqrt{2\pi\sigma_i^2}} e^{-\frac{(I_{n,i} - \bar{I}_i - \zeta_i \alpha_n)^2}{2\sigma_i^2}} \tag{7.5}$$

where ζ_i is an estimated parameter that measures the impact of α_n on the attitudinal indicator I_i , and σ_i is an estimated standard deviation. By subtracting the mean of the indicator across

the sample, i.e. using $I_{n,i} - \bar{I}_i$, we avoid the need to estimate the mean of the normal density. This is most easily done as a data transformation straight after loading the data, as shown in [Hybrid_with_continuous_measurement.r](#).

The combined log-likelihood for the model is then given by:

$$LL(\gamma, \zeta, \tau, \beta) = \sum_{n=1}^N \log \int_{\eta_n} L_{C_n}(\beta, \alpha_n) L_{I_n, \text{ordered}}(\tau, \zeta, \alpha) \phi(\eta_n) d\eta_n, \quad (7.6)$$

with the ordered model, where we would replace $L_{I_n, \text{ordered}}(\tau, \zeta, \alpha)$ by $L_{I_n, \text{normal}}(\sigma, \zeta, \alpha)$ for the continuous measurement model (Equation 7.5 instead of Equation 7.4). This log-likelihood function requires integration over the random component in the latent variable, where this integral is then approximated using numerical simulation.

For conciseness, we do not here reproduce the obvious parts of the code relating to the definition of parameters or basic settings. In Figure 7.3, we start by creating 100 inter-individual standard Normal draws for η based on Halton draws (this low number of draws is used only as an example, the user should also use more draws for estimation). We then define a single random component inside `apollo_randCoeff`, where this is for the latent attitude α_n , in line with Equation 7.1, which includes deterministic heterogeneity through the inclusion of socio-demographic effects.

```

1  ### Set parameters for generating draws
2  apollo_draws = list(
3    interDrawsType="halton",
4    interNDraws=100,
5    interNormDraws=c("eta")
6  )
7
8  ### Create random parameters
9  apollo_randCoeff=function(apollo_beta, apollo_inputs){
10    randcoeff = list()
11
12    randcoeff[["LV"]] = gamma_reg_user*regular_user + gamma_university*university_educated +
13    ↪ gamma_age_50*over_50 + eta
14
15    return(randcoeff)
16  }
```

Figure 7.3: Hybrid choice model: draws and latent variable

Figure 7.4 shows the implementation of the hybrid model in the `apollo_probabilities` function for the example with an ordered measurement model, i.e. [Hybrid_with_OL.r](#). We create a list `P` which will in the end have five components, namely the probabilities of the four measurement models and the probabilities from the choice model. We first compute the probabilities for the four Ordered Logit measurement models, one for each attitudinal statement, where these explain the values for the attitudinal indicators as a function of the latent variable, as detailed in Equation 7.4. For details on the syntax of, refer to 5.4.3. One point to note here is the inclusion of `rows=(task==1)` which ensures that the measurement model is only used once for each attitudinal statement and for each individual, rather than contributing to the overall model likelihood in each row for that person. This is in line with the rows in the data being for choice tasks, and the answers to attitudinal questions being repeated

```

1  apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
2
3      ### Attach inputs and detach after function exit
4      apollo_attach(apollo_beta, apollo_inputs)
5      on.exit(apollo_detach(apollo_beta, apollo_inputs))
6
7      ### Create list of probabilities P
8      P = list()
9
10     ### Likelihood of indicators
11     ol_settings1 = list(outcomeOrdered = attitude_quality,
12                        utility         = zeta_quality*LV,
13                        tau             = list(tau_quality_1, tau_quality_2, tau_quality_3,
14                        ↪ tau_quality_4),
15                        rows           = (task==1))
16     ...
17     P[["indic_quality"]] = apollo_ol(ol_settings1, functionality)
18     P[["indic_quality"]] = apollo_panelProd(P[["indic_quality"]] , apollo_inputs,
19     ↪ functionality)
20     ...
21     ### Likelihood of choices
22     ### List of utilities: these must use the same names as in mnl_settings, order is irrelevant
23     V = list()
24     V[["alt1"]] = ( ...
25                  + lambda*LV )
26     V[["alt2"]] = ( ...
27                  + lambda*LV )
28
29     ### Define settings for MNL model component
30     mnl_settings = list(
31         alternatives = c(alt1=1, alt2=2, alt3=3, alt4=4),
32         avail       = list(alt1=1, alt2=1, alt3=1, alt4=1),
33         choiceVar    = best,
34         utilities    = V
35     )
36
37     ### Compute probabilities for MNL model component
38     P[["choice"]] = apollo_mnl(mnl_settings, functionality)
39     P[["choice"]] = apollo_panelProd(P[["choice"]] , apollo_inputs, functionality)
40
41     ### Likelihood of the whole model
42     P = apollo_combineModels(P, apollo_inputs, functionality)
43
44     ### Average across inter-individual draws
45     P = apollo_avgInterDraws(P, apollo_inputs, functionality)
46
47     ### Prepare and return outputs of function
48     P = apollo_prepareProb(P, apollo_inputs, functionality)
49     return(P)
50 }

```

Figure 7.4: Hybrid choice model with ordered measurement model: defining probabilities

in the data in each row. For memory efficiency gains, when working with multi-component models with mixing, we suggest that the user calls `apollo_panelProd` immediately after each call to a probability calculation such as `apollo_ol`, `apollo_normalDensity` or `apollo_mnl`, rather than waiting until after `apollo_combineModels`.

```

1  ### Load data
2  database <- read.csv("apollo_drugChoiceData.csv")
3
4  database$attitude_quality=database$attitude_quality-mean(database$attitude_quality)
5  database$attitude_ingredients=database$attitude_ingredients-mean(database$attitude_ingredients)
6  database$attitude_patent=database$attitude_patent-mean(database$attitude_patent)
7  database$attitude_dominance=database$attitude_dominance-mean(database$attitude_dominance)
8
9  ...
10
11 # #####
12 #### DEFINE MODEL AND LIKELIHOOD FUNCTION #####
13 # #####
14
15 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
16
17   ...
18   ### Likelihood of indicators
19   normalDensity_settings1 = list(outcomeNormal = attitude_quality,
20                                 xNormal       = zeta_quality*LV,
21                                 mu            = 0,
22                                 sigma         = sigma_qual,
23                                 rows          = (task==1))
24   ...
25   normalDensity_settings4 = list(outcomeNormal = attitude_dominance,
26                                 xNormal       = zeta_dominance*LV,
27                                 mu            = 0,
28                                 sigma         = sigma_domi,
29                                 rows          = (task==1))
30   P[["indic_quality"]]    = apollo_normalDensity(normalDensity_settings1, functionality)
31   P[["indic_quality"]]    = apollo_panelProd(P[["indic_quality"]], apollo_inputs,
32   ↪ functionality)
33   ...
34 }

```

Figure 7.5: Hybrid choice model with continuous measurement model: zero-centering indicators and defining probabilities

We next turn to the calculation of the probabilities for the choice model component of the hybrid model. The definition of `alternatives`, `avail` and `choiceVar` is as before. The core part of the utility functions is as in Figure 5.9, with the addition that the latent variable α_n is introduced into the utilities for the first two alternatives only, multiplied by a common parameter (λ), as shown in Equation 7.2. We then also call `apollo_panelProd` for the choice component.

The list `P` now contains five individual components, and the call to `apollo_combineModels` combines these into a joint model, prior to multiplying across choices for the same individual using `apollo_panelProd` and averaging across draws using `apollo_avgInterDraws`.

In `Hybrid_with_continuous_measurement.r`, we use a continuous measurement model

for the indicators. In line with Equation 7.5, we wish to avoid the estimation of the means for the latent variable. This is achieved by zero-centering the indicators, a process that needs to take place at the database level, prior to the call to `apollo_validateInputs` to ensure that these new variables are identical across cores in a multi-core setting. We show this part of the code in Figure 7.5, along with the part of `apollo_probabilities` which changes, which is only the treatment of the indicators, where we now use the function `apollo_normalDensity`, with details available in Section 5.4.4.

8

Alternative estimation approaches

8.1 Bayesian estimation

Apollo allows the user to replace classical estimation by Bayesian estimation, for all models. We do not provide details here on Bayesian theory but instead refer the reader to [Lenk \(2014\)](#) and the references therein. Bayesian estimation in *Apollo* makes use of the functions from the now archived **RSGHB** package¹⁹, directly coded into *Apollo*. The key advantage for the user is that *Apollo* provides a wrapper around **RSGHB** so that the syntax in `apollo_probabilities` does not change when a user moves from classical to Bayesian estimation. In addition, *Apollo* implements a large number of checks, most notably ensuring that all parameters defined by the user actually impact on the likelihood of the model. It also reports whether **RSGHB** applied any censoring to the probabilities.

To explain the process, we now look at the estimation of a Mixed Logit version of the mode choice model from Section 4.5.2, which is included in `HB_MMNL.r`. A simpler MNL example is included in `HB_MNL.r` while a more complex example, using Bayesian estimation of a Hybrid choice model, is included in `Hybrid_with_OL_and_MMNL_bayesian.r`.

We use Normal distributions for the three ASCs, negative Lognormal distributions for the time and cost coefficients, censored Normal distributions (with negative values fixed to zero) for the wifi and food parameters, and non-random parameters for all other terms.

The first steps in the model definition are shown in Figure 8.1, where we define the individual coefficients and their starting values in `apollo_beta`. In Bayesian estimation, the values given here are the means of the underlying Normal distributions. As a result, we use starting values of -3 for the underlying mean of the Lognormally distributed coefficients, i.e. the mean of the logarithm of the coefficients. The definition of `apollo_fixed` is also as in the MNL model. However, only parameters that are non-random across individuals can be kept fixed via `apollo_fixed`. **RSGHB** also allows users to have random parameters where the mean and/or standard deviation are fixed, as discussed below in the list of settings.

We next create a list called `apollo_HB`, containing settings for the Bayesian estimation of the model. The only requirement when using Bayesian estimation in *Apollo* is that this list

¹⁹The user is referred to the documentation in [Dumont and Keller \(2019\)](#) for **RSGHB**-specific settings.


```

1  ### Vector of parameters, including any that are kept fixed in estimation
2  apollo_beta=c(asc_car           = 0,
3                asc_bus           = 0,
4                asc_air           = 0,
5                asc_rail          = 0,
6                asc_bus_interaction_female = 0,
7                asc_air_interaction_female = 0,
8                asc_rail_interaction_female = 0,
9                b_tt_car          = -3,
10               b_tt_bus          = -3,
11               b_tt_air          = -3,
12               b_tt_rail         = -3,
13               b_tt_interaction_business = 0,
14               b_access          = -3,
15               b_cost            = -3,
16               b_cost_interaction_business = 0,
17               cost_income_elast = 0,
18               b_no_frills       = 0,
19               b_wifi            = 0,
20               b_food            = 0)
21
22  ### Vector with names (in quotes) of parameters to be kept fixed at their starting value in
23  ↪ apollo_beta, use apollo_beta_fixed = c() if none
24  apollo_fixed = c("asc_car", "b_no_frills")
25
26  ##### HB settings
27
28  apollo_HB = list(
29    hbDist      = c(asc_car           = "NR",
30                   asc_bus           = "N",
31                   asc_air           = "N",
32                   asc_rail          = "N",
33                   asc_bus_interaction_female = "NR",
34                   asc_air_interaction_female = "NR",
35                   asc_rail_interaction_female = "NR",
36                   b_tt_car          = "LN-",
37                   b_tt_bus          = "LN-",
38                   b_tt_air          = "LN-",
39                   b_tt_rail         = "LN-",
40                   b_tt_interaction_business = "NR",
41                   b_access          = "LN-",
42                   b_cost            = "LN-",
43                   b_cost_interaction_business = "NR",
44                   cost_income_elast = "NR",
45                   b_no_frills       = "NR",
46                   b_wifi            = "CN+",
47                   b_food            = "CN+"),
48    gNCREP      = 50000, # burn-in iterations
49    gNEREP      = 20000, # post burn-in iterations
50    gINFOSKIP   = 500)

```

Figure 8.1: Bayesian estimation in *Apollo*: model settings

must contain an element called `hbDist`. It can also include any other setting as described in the documentation of the `RSGHB` package (for more info, type `?RSGHB::doHB` in the R console). The following is a non-exhaustive list of the most relevant setting to be included in `apollo_HB`:

Selection of settings for `apollo_HB`

constraintNorm: a character vector with constraints on random coefficients. For example, `c("b1>b2", "b1<0")` indicates that all draws of parameter `b1` must be bigger than the corresponding draw of `b2`, and that all draws from `b1` should be smaller than zero. Supported constraints are of the form `"b1>b2"`, `"b1<b2"`, `"b1>0"`, and `"b1<0"`, where `b1` and `b2` are the names of parameters. Constraints can also be expressed using numerical coding of the parameters as described in the documentation of the `RSGHB` package.

fixedA: Named numeric vector. Contains the names and fixed mean values of random parameters. For example, `c(b1=0)` fixes the mean of parameter `b1` to zero.

fixedD: Named numeric vector. Contains the names and fixed variance of random parameters. For example, `c(b1=1)` fixes the variance of `b1` to zero.

gFULLCV: whether the full variance-covariance structure should be used for random parameters (TRUE by default).

gNCREP: number of burn-in iterations to use prior to convergence (default=10⁵).

gNEREP: number of iterations to keep for averaging after convergence has been reached (default=10⁵).

gINFOSKIP: number of iterations between printing/plotting information about the iteration process (default=250).

hbDist: This is the only mandatory setting to be included in `apollo_HB`. It is a vector giving the name of each model parameter and indicating the distribution to be used. This replaces the vector `gdlist` in `RSGHB`, which requires the user to use numeric coding for distributions. All parameters in `apollo_beta` should be included in the `hbDist` vector. There are seven possible distributions, as follows.

"CN+": normally distributed random parameters, bounded below at 0.

"CN-": normally distributed random parameters, bounded above at 0.

"JSB": Johnson SB distributed random parameters.

"LN+": positive lognormally distributed random parameters.

"LN-": negative lognormally distributed random parameters;.

"N": normally distributed random parameters.

"NR": non-random (*fixed*) parameters. This setting should also be used for parameters that are included in `apollo_fixed`.

Additional settings can be included in `apollo_HB` as described in the documentation of the `RSGHB` package. For more information type `?RSGHB::doHB` in the R console. Settings `modelname`, `gVarNamesFixed`, `gVarNamesNormal`, `gDIST`, `svN` and `FC` should *not* be included in `apollo_HB`, as these are automatically set by *Apollo*. *Apollo* also by default sets `nodiagnosics=TRUE` in the settings for `RSGHB` unless otherwise instructed by the user, thus avoiding the need for the user to confirm the settings prior to continuing estimation.

The `apollo_probabilities` function is exactly the same as for the MNL model shown in Figure 4.7 and is thus not reproduced here. When using Bayesian estimation, the use of `apollo_avgInterDraws` and `apollo_avgIntraDraws` does not apply even in the presence of random coefficients and these functions should not be used. In addition, the call

```

1 > model = apollo_estimate(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2
3 -----
4           Number of Individuals:           500
5           Number of Observations:         7000
6           Prior variance:                 2
7           Target Acceptance (Fixed):       0.3
8           Target Acceptance (Normal):      0.3
9           Degrees of Freedom:             5
10          Avg. Number of Observations per Individual: 14
11          Initial Log-Likelihood: -19242.47502
12
13 -----
14          Fixed Parameters Start
15          asc_bus_interaction_female      0
16          ...
17          cost_income_elast              0
18
19 -----
19          Random Parameters Start Dist.
20          asc_bus      0      N
21          ...
22          b_food      0      CN+
23
24 -----
25          ...
26
27 -----
28          Iteration:           70000
29
30 -----
31          RHO (Fixed): 2.552501553e-06
32          Acceptance Rate (Fixed):      0.27
33          RHO (Normal):  0.3597337714
34          Acceptance Rate (Normal):     0.274
35          Parameter RMS:  0.8041928485
36          Avg. Variance:  0.6332235963
37          Log-Likelihood: -4679.250243
38          RLH:           0.5321134338
39
40 -----
41          Fixed Parameters      Estimate
42          asc_bus_interaction_female: 0.155267594
43          ...
44          cost_income_elast: -0.657755249
45
46 -----
46          Random Parameters      Estimate
47          asc_bus: -1.26844325
48          ...
49          b_food:  0.04980724
50
51 -----
51          Time per iteration: 0.0497 secs
52          Time to completion: 0 mins
53
54 -----
54          Estimation complete.
55          WARNING: RSGHB has censored the probabilities. Please note that in
56          at least some iterations RSGHB has avoided numerical issues by
57          left censoring the probabilities. This has the side effect of
58          zero or negative probabilities not leading to failures!

```

Figure 8.2: Bayesian estimation in *Apollo*: estimation process

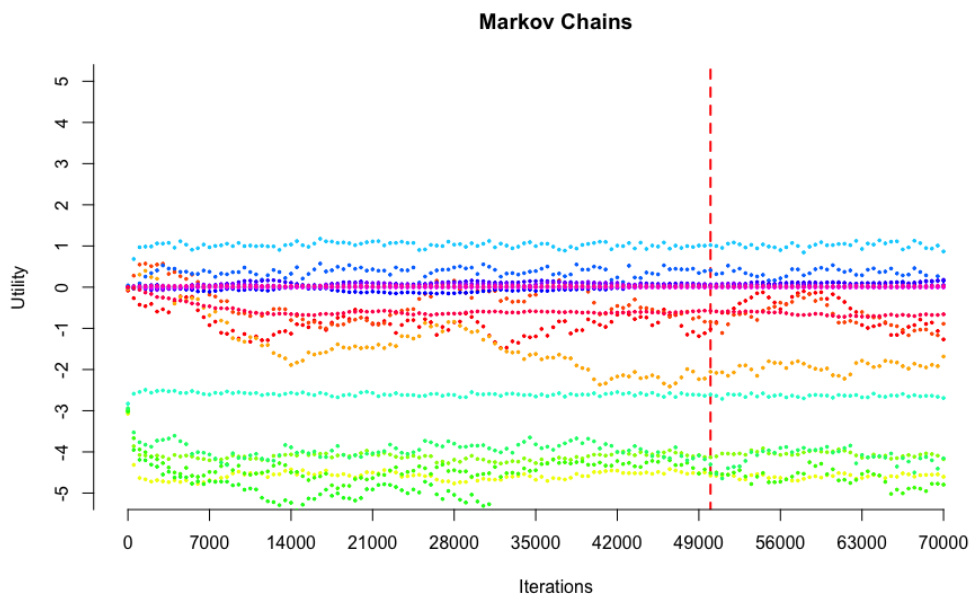


Figure 8.3: Bayesian estimation in *Apollo*: estimation process (parameter chains)

to `apollo_panelProd` should not be made as `RSGHB` automatically groups together observations for the same individual.

The call to `apollo_estimate` is made in exactly the same way as with classical estimation. The estimation process is illustrated in Figure 8.2 for the text output and Figure 8.3 for a graphical output of the chains. In the text output, we show the first and final iteration, where this also highlights the way in which `RSGHB` confirms the distributions used at the outset. Note that `RSGHB` refers to non-random parameters as *fixed*, even if their values are estimated - the terminology refers to them being fixed across individuals.

The post-estimation output from a model using Bayesian estimation is substantially different from that with classical estimation, and is summarised in Figure 8.4 for on screen output using the default setting. The early information on model name etc is the same as with classical estimation. This is followed by the calculation of model fit statistics assuming parameters to be equal to the average across the post burn-in draws (iterations). Next, a summary of the parameter chains for non-random coefficients is presented, and then the posterior distributions and sample level distributions of random parameters. The sample level distributions are shown both with the distributional transforms as well as for the underlying Normals.

Several additional outputs can be generated, as can be seen in the helpfiles for `apollo_modelOutput` and `apollo_saveOutput`. This for example includes convergence reports for the parameter chains, where these use the Geweke test (Geweke, 1992). In classical estimation, *Apollo* creates an object `estimate` in the `model` list created after estimation, containing the final parameter values. When using Bayesian estimation, `model$estimate` is also produced, combining non-random parameters with individual specific posteriors for random parameters, where these are replicated across observations for the same individual. This allows the use of `apollo_prediction` and `apollo_llCalc` on such outputs, where care is of course required in interpretation of outputs based on posterior means.

```

1 > apollo_modelOutput(model)
2 Model name : HB_MMNL
3 Model description : HB model on mode choice SP data, mix of random and
  ↳ non-random parameters
4 Estimation method : Hierarchical Bayes
5
6 Classical model fit statistics were calculated at parameter values obtained using averaging
  ↳ across
7 the post burn-in iterations.
8 LL(start) : -19242.48
9 LL at equal shares, LL(0) : -8196.02
10 LL at observed shares, LL(C) : -6706.94
11 LL(final) : -4916.64
12
13 Summary of parameter chains
14
15 Non-random coefficients
16
17      Mean      SD
18 asc_car      0.0000      NA
19 asc_bus_interaction_female  0.1298 0.0403
20 asc_air_interaction_female  0.2237 0.0497
21 asc_rail_interaction_female  0.0974 0.0563
22 b_tt_interaction_business -0.0080 0.0008
23 b_cost_interaction_business  0.0268 0.0038
24 cost_income_elast      -0.6535 0.0248
25 b_no_frills      0.0000      NA
26
27 Results for posterior means for random coefficients
28      Mean      SD
29 asc_bus -1.6322 0.0379
30 asc_air -0.5863 0.0438
31 asc_rail -1.4929 0.0489
32 b_tt_car -0.0107 0.0008
33 b_tt_bus -0.0151 0.0013
34 ...
35 b_wifi 1.0211 0.1515
36 b_food 0.4231 0.1073
37
38 Summary of distributions of random coefficients (after distributional transforms)
39      Mean      SD
40 asc_bus -1.6315 0.4994
41 asc_air -0.5832 0.3951
42 asc_rail -1.4875 0.3825
43 b_tt_car -0.0107 0.0025
44 b_tt_bus -0.0151 0.0040
45 ...
46 b_wifi 1.0189 0.4417
47 b_food 0.4181 0.3780
48
49 Upper level model results for mean parameters for underlying Normals
50      Mean      SD
51 asc_bus -1.6323 0.4147
52 asc_air -0.5865 0.1862
53 asc_rail -1.4930 0.1735
54 b_tt_car -4.5601 0.0637
55 b_tt_bus -4.2271 0.0762
56 ...
57 b_wifi 1.0186 0.0629
58 b_food 0.3639 0.0905

```

Figure 8.4: Bayesian estimation in *Apollo*: output (extracts)

8.2 Expectation-maximisation (EM) algorithm

Apollo allows the user to estimate models using Expectation - Maximisation (EM) algorithms. These are iterative algorithms where the updating of the parameters is usually achieved through the maximization of a simplified version of the model likelihood function. EM algorithms do not provide standard errors for the parameter estimates. To obtain them, a Maximum Likelihood estimation with the EM estimated parameters as the starting values is typically run afterwards. This guarantees quick convergence and standard errors for all parameters. For a detailed discussion of EM algorithms, see [Train \(2009, ch. 14\)](#).

The precise steps of these algorithms change depending on the kind of model, making them hard to generalize and implement in a flexible way. *Apollo* includes EM routines for two types of models, namely Latent Class (LC) models where all parameters vary across classes, and Mixed Multinomial Logit (MMNL) models where all parameters are random, with a full covariance matrix being estimated. These functions were added in *Apollo* v0.2.0 and users interested in the manual coding of these routines are referred to the manual for earlier versions of *Apollo*.

8.2.1 EM algorithm for LC model

Apollo incorporates the function `apollo_lcEM` for latent class models, where this can be used for models with any functional form inside the latent classes (i.e. not just MNL), as long as all parameters vary across classes (i.e. no generic parameters). The function `apollo_lcEM` is called via:

```
model = apollo_lcEM(apollo_beta,
                    apollo_fixed,
                    apollo_probabilities,
                    apollo_inputs,
                    lcEM_settings,
                    estimate_settings)
```

where we have already covered the first four arguments, as well as the final one. The further optional argument `lcEM_settings` can contain the following entries:

lcEM_settings: user-controlled settings

EMmaxIterations: An integer setting a maximum number of iterations of the EM algorithm before stopping (default is 100).

postEM: A scalar which determines the number of tasks performed by this function after the EM algorithm has converged. Can take values 0, 1 or 2 only. If value is 0, only the EM algorithm will be performed, and the results will be a model object without a covariance matrix (i.e. estimates only.). If value is 1, after the EM algorithm the covariance matrix of the model will be calculated as well, and the result will be a model object with a covariance matrix. If value is 2, after the EM algorithm, the estimated parameter values will be used as starting value for a maximum likelihood estimation process, which will render a model object with a covariance matrix. Performing maximum likelihood estimation after the EM algorithm is useful, as there may be room for further improvement (default is 2).

silent: A boolean variable, which, when set to TRUE, means that no information is printed to the screen during estimation (default is set to FALSE).

stoppingCriterion: A numeric convergence criterion. The EM process will stop when improvements in the log-likelihood fall below this value (default is 10^{-5}).

In addition to ensuring that all parameters vary across classes (or are included in `apollo_fixed`, the only other requirement for using the `apollo_lcEM` function is that inside the `apollo_probabilities` function, the loop over classes is defined as

```
for(s in 1:length(pi_values)){
  ...
}
```

We first describe the EM estimation of a choice model with S different classes. The conditional choice probability of class s (i.e. the in-class probability) is determined by a MNL model. All preference parameters β are allowed to vary across classes (β_s for class s). The class allocation is determined by a MNL model using covariates Z_n such as an individual's income, interacted with a vector of parameters γ_s in class s , causing the allocation probabilities $\pi_{n,s}$ to change from one individual to the next. Estimation is achieved through an iterative five step process detailed below (cf. Train, 2009, ch. 14), drawing also on Bhat (1997).

1. Definition of starting values for γ_s^0 and β_s^0 parameters, where β_s^0 should be different across classes.
2. Calculate the likelihood of the whole model, using $\gamma^0 = \langle \gamma_1^0, \dots, \gamma_S^0 \rangle$ (which gives $\pi^0 = \langle \pi_1^0, \dots, \pi_S^0 \rangle$) and $\beta^0 = \langle \beta_1^0, \dots, \beta_S^0 \rangle$. Store this likelihood as L_0 .
3. Calculate class allocation probabilities conditional on observed choices for each class $s \in \{1, \dots, S\}$, using the following expression.

$$h_{n,s}^0 = \frac{\pi_{n,s}^0(\gamma^0)L_{n,s}(\beta_s^0)}{\sum_{s=1}^S \pi_{n,s}^0(\gamma^0)L_{n,s}(\beta_s^0)} \quad (8.1)$$

where $L_{n,s}(\beta_s^0)$ is the likelihood of the observed choice for individual n assuming class s , and where $\pi_{n,s}^0(\gamma^0)$ is the class allocation probability for individual n for class s , using γ^0 as parameters.

4. Update the parameters γ used in the class allocation model by maximising the allocation probabilities weighted by $h_{n,s}^0$.

$$\gamma^1 = \underset{\gamma}{\operatorname{argmax}} \left(\sum_{n=1}^N \sum_{s=1}^S h_{n,s}^0 \log(\pi_{n,s}) \right), \quad (8.2)$$

where this shows the maximisation of the log-likelihood of this component, where the likelihood is given by $\prod_{n=1}^N \prod_{s=1}^S \pi_{n,s}^{h_{n,s}^0}$ (cf. [Bhat, 1997](#)).

5. Update the parameters β_s for the within class model for each class by estimating separate weighted MNL models. Estimation of each MNL model can be done using Maximum Likelihood, using $h_{n,s}^0$ as weights.

$$\beta_s^1 = \underset{\beta_s}{\operatorname{argmax}} \left(\sum_{n=1}^N h_{n,s}^0 \log(L_{n,s}) \right) \forall s \quad (8.3)$$

6. Calculate the likelihood of the whole model, using $\gamma^1 = \langle \gamma_1^1, \dots, \gamma_S^1 \rangle$, which gives $\pi^1 = \langle \pi_1^1, \dots, \pi_S^1 \rangle$, and $\beta^1 = \langle \beta_1^1, \dots, \beta_S^1 \rangle$ and store this as L_1 . If $L_1 - L_0 < c$, where c is a convergence limit, say 10^{-5} , then convergence has been reached. If convergence is not achieved, set $\gamma^0 = \gamma^1$ and $\beta^0 = \beta^1$, and return to step 2.

We illustrate this example using a model similar to `LC_with_covariates.r`, i.e. the two-class LC model on the Swiss route choice data, with the difference being that the ASCs are now also class-specific. This example is available in `EM_LC_with_covariates.r`. The implementation is shown in Figure 8.5, with the estimation process in Figure 8.6. An example without covariates is available in `EM_LC_no_covariates.r`.


```

1  ##### DEFINE MODEL PARAMETERS
2  apollo_beta = c(asc_1_a      = 0,
3                  asc_1_b      = 0,
4                  ...)
5
6  ##### DEFINE LATENT CLASS COMPONENTS
7  apollo_lcPars=function(apollo_beta, apollo_inputs)
8  {
9      lcpars = list()
10     lcpars[["asc_1"]] = list(asc_1_a, asc_1_b)
11     ...
12     return(lcpars)
13 }
14
15 ##### DEFINE MODEL AND LIKELIHOOD FUNCTION
16 apollo_probabilities=function(apollo_beta, apollo_inputs, functionality="estimate"){
17     ...
18     ### Loop over classes
19     for(s in 1:length(pi_values)){
20
21         ### Compute class-specific utilities
22         V=list()
23         V[['alt1']] = asc_1[[s]] + beta_tc[[s]]*tc1 + beta_tt[[s]]*tt1 + beta_hw[[s]]*hw1 +
24         ↪ beta_ch[[s]]*ch1
25         V[['alt2']] = asc_2      + beta_tc[[s]]*tc2 + beta_tt[[s]]*tt2 + beta_hw[[s]]*hw2 +
26         ↪ beta_ch[[s]]*ch2
27
28         mnl_settings$utilities = V
29
30         ### Compute within-class choice probabilities using MNL model
31         P[[paste0("Class_",s)]] = apollo_mnl(mnl_settings, functionality)
32
33         ### Take product across observation for same individual
34         P[[paste0("Class_",s)]] = apollo_panelProd(P[[paste0("Class_",s)]], apollo_inputs
35         ↪ ,functionality)
36     }
37
38     ### Compute latent class model probabilities
39     lc_settings = list(inClassProb = P, classProb=pi_values)
40     P[["model"]] = apollo_lc(lc_settings, apollo_inputs, functionality)
41
42     ### Prepare and return outputs of function
43     P = apollo_prepareProb(P, apollo_inputs, functionality)
44     return(P)
45 }

```

Figure 8.5: EM algorithm for Latent Class: setup

```

1 > model=apollo_lcEM(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 The use of apollo_lcEM has a number of requirements. No checks are run for these, so the
3 user needs to ensure these conditions are met by their model:
4 1: This function is only suitable for single component models, i.e. no use of
5   apollo_combineModels or manual multiplication of model components.
6 2: Any parameters that vary across classes need to be included in the definition of random
7   parameters in apollo_lcPars.
8 3: The entries in the lists in apollo_lcPars need to be individual parameters, not
9   functions thereof.
10
11 Validating inputs of likelihood function (apollo_probabilities)
12
13 Initialising EM algorithm
14 Starting iteration: 1
15 Current LL : -1755.505
16 ...
17 Starting iteration: 35
18 Current LL : -1558.427
19 New LL      : -1558.427
20 Improvement: 9.141917e-06
21
22 EM algorithm stopped: improvements in LL smaller than convergence criterion.
23 Continuing with classical estimation...
24
25 Starting main estimation
26 Initial function value: -1558.427
27 ...
28 iter   1 value 1558.426644
29 final  value 1558.426644
30 converged
31 Estimated parameters:
32             Estimate
33 asc_1_a      -0.08345
34 ...
35 gamma_car_av_b  0.00000
36
37 Computing covariance matrix using numerical methods (numDeriv).
38 0%...25%...50%...75%...100%
39 Negative definite Hessian with maximum eigenvalue: -8.782019
40 Computing score matrix...
41
42 Summary of class allocation for LC model component :
43     Mean prob.
44 Class_1      0.4702
45 Class_2      0.5298
46
47 Calculating LL(0)...
48 Calculating LL of each model component...

```

Figure 8.6: EM algorithm for Latent Class: estimation

8.2.2 MMNL model with full covariance matrix for random coefficients

In this subsection, we describe the EM estimation of a MMNL model in which all parameters are random and where we estimate a full covariance matrix between them. More formally, we assume the preference parameters to follow a joint random normal distribution $\beta \sim N(\mu, \Sigma)$, where transformations to other distributions are straightforward, as explained below.

The iterative process is described below (cf. [Train, 2009](#), chapter 11).

1. Define starting values for μ^0 and Σ^0 .
2. Generate R multivariate draws for each individual n , say $\beta_{n,r}^0$ for draw r , where $\beta_{n,r}^0$ contains one value for each random parameter.
3. Calculate the model likelihood at the individual level for each draw, i.e. $L_0 = L_{n,r}(\beta_{n,r}^0)$.
4. Calculate weights for each draw and for each individual using the following expression.

$$w_{n,r}^0 = \frac{L_{n,r}}{\frac{\sum_r L_{n,r}}{R}} \quad \forall r, n \quad (8.4)$$

5. Update the means of the random parameters.

$$\mu^1 = \frac{w_{n,r}^0 \beta_{n,r}^0}{RN} \quad (8.5)$$

Where N is the number of individuals in the sample.

6. Update the covariance matrix of the random parameters, given by

$$\Sigma^1 = \frac{w_{n,r}^0 \Sigma_{n,r}^0}{RN}, \quad (8.6)$$

where this requires calculating the covariance matrix $\Sigma_{n,r}^0$ at the individual draw level, given by:

$$\Sigma_{n,r}^0 = (\beta_{n,r}^0 - \mu^1) \cdot (\beta_{n,r}^0 - \mu^1)'. \quad (8.7)$$

7. Calculate the likelihood of the whole model and check for convergence. Convergence is achieved when the change on this likelihood is smaller than an pre-defined value. If convergence is not achieved, return to step 2.

Apollo incorporates the function `apollo_mixEM` for MMNL models, where this can be used for models with all parameters being random, and a full covariance matrix being estimated. The function `apollo_mixEM` is called via:

```
model = apollo_mixEM(apollo_beta,
                      apollo_fixed,
                      apollo_probabilities,
                      apollo_inputs,
                      lcEM_settings,
                      estimate_settings)
```

A specific consideration applies in that for `apollo_mixEM`, the entries in `apollo_beta` need to be provided in a specific order. With K random parameters, the first K entries need to be

the means for the underlying Normals, followed by the elements of the lower triangle of the Cholesky matrix, by row.

The further optional argument `mixEM_settings` can contain the following entries:

mixEM_settings: user-controlled settings

EMmaxIterations: An integer setting a maximum number of iterations of the EM algorithm before stopping (default is 100).

postEM: A scalar which determines the number of tasks performed by this function after the EM algorithm has converged. Can take values 0, 1 or 2 only. If value is 0, only the EM algorithm will be performed, and the results will be a model object without a covariance matrix (i.e. estimates only.). If value is 1, after the EM algorithm the covariance matrix of the model will be calculated as well, and the result will be a model object with a covariance matrix. If value is 2, after the EM algorithm, the estimated parameter values will be used as starting value for a maximum likelihood estimation process, which will render a model object with a covariance matrix. Performing maximum likelihood estimation after the EM algorithm is useful, as there may be room for further improvement (default is 2).

silent: A boolean variable, which, when set to TRUE, means that no information is printed to the screen during estimation (default is set to FALSE).

stoppingCriterion: A numeric convergence criterion. The EM process will stop when improvements in the log-likelihood fall below this value (default is 10^{-5}).

transforms: A list, with one entry per parameter, showing the inverse transform to return from beta to the underlying Normal. E.g. if the first parameter is specified as negative lognormal inside `apollo_randCoeff`, then the entry in transforms should be `transforms[[1]]=function(x) log(-x)`. If any transforms are used, then they need to be provided for all random parameters, even untransformed ones.

Figures 8.7 and 8.8 presents code implementing this algorithm for the EM analogue of `MMNL_preference_space_correlated.r`, i.e. using a MMNL model with correlated negative Lognormals on the Swiss route choice data. This example is implemented in `EM_MMNL.r`. The core steps to focus on are the order of parameters in `apollo_beta` and the inverse transforms for a negative Lognormal in `mixEM_settings$transforms`.

```

1  ##### DEFINE MODEL PARAMETERS
2  apollo_beta = c(mu_log_b_tt      ==-3,
3                  mu_log_b_tc      ==-3,
4                  ...
5                  sigma_log_b_hw_ch = 0,
6                  sigma_log_b_ch    = -0.8298)
7
8
9  ##### DEFINE RANDOM COMPONENTS
10 apollo_randCoeff = function(apollo_beta, apollo_inputs)
    randcoeff = list()
    randcoeff[["b_tt"]] = -exp( mu_log_b_tt
                               + sigma_log_b_tt      * draws_tt )
    randcoeff[["b_tc"]] = -exp( mu_log_b_tc
                               + sigma_log_b_tt_tc * draws_tt
                               + sigma_log_b_tc      * draws_tc )
    randcoeff[["b_hw"]] = -exp( mu_log_b_hw
                               + sigma_log_b_tt_hw * draws_tt
                               + sigma_log_b_tc_hw * draws_tc
                               + sigma_log_b_hw      * draws_hw )
    randcoeff[["b_ch"]] = -exp( mu_log_b_ch
                               + sigma_log_b_tt_ch * draws_tt
                               + sigma_log_b_tc_ch * draws_tc
                               + sigma_log_b_hw_ch * draws_hw
                               + sigma_log_b_ch      * draws_ch )

    return(randcoeff)

11
12 ##### EM ESTIMATION
13 mixEM_settings = list(transforms=list(function(x) log(-x),
14                                       function(x) log(-x),
15                                       function(x) log(-x),
16                                       function(x) log(-x)))

```

Figure 8.7: EM algorithm for Mixed Logit: setup

```
1 > model=apollo_mixEM(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 The use of apollo_mixEM has a number of requirements. No checks are run for these, so the
3 user needs to ensure these conditions are met by their model:
4 1:This function is only suitable for single component models, i.e. no use of
5   apollo_combineModels or manual multiplication of model components.
6 2:All parameters need to be random, and a full covariance matrix needs to be
7   estimated/specified in apollo_randCoeff.
8 3:All random parameters need to be based on Normal distributions or transformations
9   thereof.
10 4:With K random parameters, the order of the elements in 'apollo_beta' needs to be as
11   follows: K means for the underlying Normals, followed by the elements of the lower
12   triangle of the Cholesky matrix, by row.
13
14 Initialising EM algorithm
15 Validating inputs of likelihood function (apollo_probabilities)
16
17 Initial LL: -2026.718
18
19 Starting iteration: 1
20 Current LL: -2026.718
21 New LL: -1983.751
22 Improvement: 42.96719
23
24 ...
25
26 Starting iteration: 91
27 Current LL: -1410.822
28 New LL: -1410.84
29 Improvement: -0.01809646
30
31 EM algorithm stopped: improvements in LL smaller than convergence criterion.
32 Continuing with classical estimation...
```

Figure 8.8: EM algorithm for Mixed Logit: estimation

9

Pre and post-estimation capabilities

A large number of additional functions are provided in *Apollo* to allow the user to analyse the results after estimation. We will now look at these various functions in turn.

The outputs from these functions are not saved in the model output files, and it is then helpful for a user to dump the additional output to a text file. This is made possible by the function `apollo_sink`, which is called as:

```
apollo_sink()
```

A call to this function produces a new text file using the name of the current model. Outputs in the console are then also written into this text file. Writing to file is stopped via an additional call to `apollo_sink`.

9.1 Pre-estimation analysis of choices

With labelled choice data (or even unlabelled data where there may be strong left-right effects), it can be useful to analyse the choices before model estimation to determine whether the characteristics of individuals choosing specific alternatives differ across alternatives. This is made possible by the function called `apollo_choiceAnalysis`, which is called as follows:

```
apollo_choiceAnalysis(choiceAnalysis_settings,  
                      apollo_control,  
                      database)
```

where `choiceAnalysis_settings` has the following contents:

choiceAnalysis_settings: user-controlled settings

alternatives: A named vector containing the names of the alternatives and their identifier in `choiceVar` (see below), as in e.g. an MNL model. Note that these need not necessarily be the alternatives as defined in the model, but could e.g. relate to cheapest/most expensive ones.

avail: A list containing one element with availabilities per alternative, as in e.g. an MNL model, but where reference to database needs to be made, as we are operating outside `apollo_probabilities`, e.g. by using `database$avail_a`, for the availability stored in variable `avail_a` (cf. Figure 9.1). Note that these need not necessarily be the alternatives as defined in the model, but could e.g. relate to cheapest/most expensive. If availabilities are not provided, full availability is assumed.

choiceVar: A vector of length equal to the number of observations, containing the chosen alternative for each observation. Note that these need not necessarily be the alternatives as defined in the model, but could e.g. relate to cheapest/most expensive.

explanators: A dataframe containing a set of variables, one per column and one row per choice observation, that are to be used to analyse the choices. This could include explanatory variables describing the alternatives but is most useful for characteristics of the decision makers. As these definitions are outside `apollo_probabilities`, reference to the database again needs to be made, e.g. `database$z` for explanatory variable `z` (cf. Figure 9.1).

rows: This is an optional argument which is missing by default. It allows the user to specify a vector called `rows` of the same length as the number of rows in the data. This vector needs to use logical statements to identify which rows in the data are to be used for the analysis of choices.

```

1 choiceAnalysis_settings <- list(
2   alternatives = c(car=1, bus=2, air=3, rail=4),
3   avail       = list(car=database$av_car, bus=database$av_bus, air=database$av_air,
4   ↪ rail=database$av_rail),
5   choiceVar   = database$choice,
6   explanators = database[,c("female", "business", "income")],
7   rows        = (database$SP==1)
8 )
9
10 > apollo_choiceAnalysis(choiceAnalysis_settings, apollo_control, database)
11
12      car      bus      air      rail
13 Explanator 1 (female), mean when alt is chosen: 0.4717 0.4581 0.4823 0.4776
14 Explanator 1 (female), mean when alt is not chosen: 0.5057 0.4731 0.4698 0.4837
15 Explanator 1 (female), t-test (mean if chosen - mean if not chosen) -2.410 -0.550 0.8200 -0.470
16 ...
17 Outputs of apollo_choiceAnalysis saved to output/MNL_SP_covariates_choiceAnalysis.csv

```

Figure 9.1: Running `apollo_choiceAnalysis` (syntax and excerpt of output)

An example application of this function is included in the MNL model estimated on the SP mode choice data (`MNL_SP_covariates.r`), where we show the results for the first explanator in 9.1. The function returns the output to the screen and also produces a *csv* file with the

same output. In a given block, i.e. for one explanator, the three rows alternative contain, for each alternative, the mean value for the given explanatory variable for those choices where the alternative is chosen, the mean value where it is not chosen (but available), and the test statistic for the two-sample t-test comparing the means in these two groups (where the null hypothesis states that the difference between the means is equal to 0, and the alternative hypothesis says that it is different from zero). These value are also returned silently by the function, so they can be stored in a variable, by using e.g.:

```
output = apollo_choiceAnalysis(choiceAnalysis_settings,
                               apollo_control,
                               database)
```

In the example shown here and included in `MNL_SP_covariates.r`, the alternatives that are used correspond to those used in the actual model. At times, it may be useful to look at the way specific types of alternatives are chosen differently than others, for example looking at the way the cheapest option in a task is chosen in unlabelled data. The user can in that case define his/her own alternatives for use with `apollo_choiceAnalysis`. Examples of this are included in `MMNL_preference_space.r` and `Hybrid_with_OL.r`. Figure 9.2 shows the former of these two, looking at the choice between the cheap and expensive option in the case of unlabelled data.

```
1 choiceAnalysis_settings <- list(
2   alternatives = c(cheap=1, expensive=2),
3   choiceVar    = (1*((database$choice==1)*(database$tc1<=database$tc2)
4   ↪ +(database$choice==2)*(database$tc1>=database$tc2))
5   ↪ +2*((database$choice==1)*(database$tc1>=database$tc2)
6   ↪ +(database$choice==2)*(database$tc1<=database$tc2))),
7   explanators  = database[,c("car_availability", "hh_inc_abs", "business")]
8 )
9
10 > apollo_choiceAnalysis(choiceAnalysis_settings, apollo_control, database)
11 Availabilities not provided for 'apollo_choiceAnalysis', so full availability is assumed.
12
13      cheap expensive
14
15 Explanator 1 (car_availability), mean when alt is chosen:      0.3597    0.3975
16 Explanator 1 (car_availability), mean when alt is not chosen:  0.3975    0.3597
17 Explanator 1 (car_availability), t-test (mean if chosen - mean if not chosen) -2.3100    2.3100
18
19      cheap expensive
20
21 Explanator 2 (hh_inc_abs), mean when alt is chosen:      75114.76  77861.38
22 Explanator 2 (hh_inc_abs), mean when alt is not chosen:  77861.38  75114.76
23 Explanator 2 (hh_inc_abs), t-test (mean if chosen - mean if not chosen)    -1.83     1.83
24
25      cheap expensive
26
27 Explanator 3 (business), mean when alt is chosen:         0.0639    0.1208
28 Explanator 3 (business), mean when alt is not chosen:     0.1208    0.0639
29 Explanator 3 (business), t-test (mean if chosen - mean if not chosen)    -5.8500    5.8500
```

Figure 9.2: Running `apollo_choiceAnalysis` on unlabelled data (syntax and excerpt of output)

9.2 Reading in a previously saved model object

As mentioned in Section 4.7, the call to `apollo_saveOutput` (with default settings) saves the `model` object in a `.rds` file. It is then possible to read this in as a model object using the function `apollo_loadModel` which is called as:

```
oldModel = apollo_loadModel(modelName)
```

where `modelName` needs to be replaced by the name (as a string, i.e. with quotation marks) of previously run model, for which the output was saved in the current directory. If exact paths are not given, the function will look for model files in the output directory and the working directory automatically. The output from this function is a model object, which in this case is saved into `oldModel`. The benefit of this function is that it is then easy for a user to return to a previously estimated model, and compute additional output with the estimates from that model and having access to the full covariance matrix without needing to reestimate the model. An example is included in `MMNL_wtp_space_inter_intra.r`.

9.3 Starting value search

In classical estimation, convergence to the global maximum of the likelihood function is not guaranteed by any optimization algorithm. While this is not a problem for simple linear in attributes MNL models due to their concave likelihood function, it might be for other more complex models, such as Mixed Logit or Latent Class models. A popular approach to reduce the probability of reaching a poor local maximum is to start the optimization process from several different candidate points (i.e. sets of parameters), and keep the solution with the highest likelihood. However, this approach is very computationally intensive. To reduce its cost, algorithms have been proposed to dynamically eliminate unpromising candidates.

The function `apollo_searchStart` implements a simplified version of the algorithm proposed by Bierlaire et al. (2010), where the main difference in our implementation lies in the fact that `apollo_searchStart` uses only two out of three tests on the candidates described by Bierlaire et al. (2010). The implemented algorithm has the following steps, where these use a number of inputs, which we will define below:

1. Randomly draw `nCandidates` candidate starting values from an interval given by the user.
2. Label all candidates with a valid log-likelihood (LL) as *active*.
3. Apply `bgwIter` iterations of the BGW algorithm to each active candidate.
4. Apply the following tests to each candidate:
 - (a) Has the BGW search converged?
 - (b) Are the candidate parameters after BGW closer than `dTest` from any other candidate with higher LL?
 - (c) Is the LL of the candidate after BGW further than `distLL` from a candidate with better LL, and is its gradient smaller than `gTest`?
5. Mark any candidates for which at least one of these tests is passed as *inactive*.
6. If more than one candidate remains as *active*, go back to step 3 for the remaining *active* candidates.

The `apollo_searchStart` function is called as follows:

```
apollo_beta = apollo_searchStart(apollo_beta,
                                apollo_fixed,
                                apollo_probabilities,
                                apollo_inputs,
                                searchStart_settings)
```

The function returns an updated list of starting values. The list `searchStart_settings` has the following contents:

mixEM_settings: user-controlled settings

apolloBetaMax: a vector of the maximum possible value for each parameter (default is `apollo_beta+0.1`).

apolloBetaMin: a vector of the minimum possible value for each parameter (default is `apollo_beta-0.1`).

bgwIter: the maximum number of BGW iterations to apply to each candidate in each iteration of the main algorithm (default is 20).

dTest: the tolerance of test 4(b) described above (default is 1).

gTest: the tolerance for the gradient in test 4(c) described above (default is 10^{-3}).

llTest: the tolerance for the LL in test 4(c) described above (default is 3).

maxStages: the maximum number of iterations of the algorithm, i.e. maximum number of times the algorithm jumps from step 6 to 3 described above (default is 5).

nCandidates: the number of initial candidates (default is 100).

smartStart: if `TRUE`, the Hessian of `apollo_probabilities` is calculated at `apollo_beta`, and the initial candidates are drawn with a higher probability from the area where the Hessian indicates an improvement in the likelihood. This adds a significant amount of time to the initialisation of the algorithm (default is `FALSE`).

The performance of the function varies across models and datasets and depends on the settings used. In particular, we advise to adjust the `bgwIter`, `dTest`, `distLL` and `gTest` parameters to suit each user's particular model characteristics, as their default values might not be suitable for some model specifications. The running of the function is illustrated in Figure 9.3 for example `LC_no_covariates.r`, where only a small part of the output is shown.

9.4 Calculating model fit with given parameter values

Especially with complex models, it can be useful for testing purposes to calculate the log-likelihood of the model (and subcomponents) for given parameter values, before or after estimation. This is made possible by the function `apollo_llCalc`, which is called as follows:

```
apollo_llCalc(apollo_beta,
              apollo_probabilities,
              apollo_inputs)
```

```

1 > apollo_beta=apollo_searchStart(apollo_beta, apollo_fixed,apollo_probabilities,
  ↪ apollo_inputs)
2 ...
3 Creating initial set of 100 candidate values.
4 Calculating LL of candidates 0%...50%...100%
5
6 Stage 1, 100 active candidates.
7 Estimating 20 BGW iteration(s) for each active candidate.
8 Candidate.....LLstart.....LLfinish.....GradientNorm...Converged
9      1      -2067      -1562      0.388      0
10     2      -2430      -1579      194.3      0
11 ...
12    100      -2023      -1563      56.99      0
13 Candidate 1 dropped.
14 - Failed test 1: Too close to 31, 34, 56, 69, 87 in parameter space.
15 Candidate 2 dropped.
16 ...
17 Candidate 100 dropped.
18 - Failed test 1: Too close to 1, 8, 9, 12, 26, 31, 33, 34, 41, 52, 53, 56, 57, 61, 69, 72, 77,
  ↪ 80, 81, 82, 87, 96, 97 in
19 parameter space.
20 Best candidate so far (LL=-1551.9)
21 ...
22 ...
23 Stage 4, 2 active candidates.
24 Estimating 20 BGW iteration(s) for each active candidate.
25 Candidate.....LLstart.....LLfinish.....GradientNorm...Converged
26      5      -1552      -1552      0      1
27     19      -1552      -1552      0.061      1
28 Best candidate so far (LL=-1551.9)
29 Value
30 asc_1    -0.0523
31 asc_2     0.0000
32 beta_tt_a -0.0628
33 beta_tt_b -0.2804
34 beta_tc_a -0.0877
35 beta_tc_b -1.9030
36 beta_hw_a -0.0432
37 beta_hw_b -0.0511
38 beta_ch_a -1.0440
39 beta_ch_b -2.4868
40 delta_a    0.8216
41 delta_b     0.0000

```

Figure 9.3: Running apollo_searchStart

where we illustrate this in Figure 9.4 for the case of the hybrid choice model ([Hybrid_with_OL.r](#)) from Section 7.3. It should be noted that when calling this function, the format of `apollo_beta` needs to be compatible with what is used inside `apollo_probabilities`. For classical estimation, no changes are needed, and even with random coefficients, the full distribution with numerical simulation of the log-likelihood would then be used. However, for Bayesian estimation, `apollo_beta` would need to be passed to `apollo_llCalc` as a list, with one element per parameter, where each of these has as many values as there are observations in the data. Of course, this would imply that the log-likelihood is calculated at a point value for each parameter and observation, not recognising the random distributions.

```

1 > apollo_llCalc(apollo_beta, apollo_probabilities, apollo_inputs)
2 Calculating LL of each model component...Done.
3 $model
4 [1] -51873.29
5
6 $indic_quality
7 [1] -3526.404
8
9 $indic_ingredients
10 [1] -4303.376
11
12 $indic_patent
13 [1] -3661.424
14
15 $indic_dominance
16 [1] -4110.703
17
18 $Class_1
19 [1] -13255.42
20
21 $Class_2
22 [1] -13380.71
23
24 $choice
25 [1] -13314.15

```

Figure 9.4: Running `apollo_llCalc`

Note that `apollo_llCalc` uses the copy of the `database` stored within `apollo_inputs`, and not the version of `database` that may be stored in the user environment (i.e. the Global Environment). If the user wishes to calculate the log-likelihood for a modified `database` (for example, to study how the log-likelihood changes in a new scenario) we recommend doing it in three stages. First, modify `database` in whatever way is needed. Then update `apollo_inputs` by running `apollo_inputs = apollo_validateInputs()`, and finally call `apollo_llCalc`.

9.5 Bootstrap covariance estimation

Apollo also allows the user to use bootstrap estimation. Given a number of repetitions, this function generates as many new samples as requested, by sampling individuals (i.e. blocks of observations) *with replacement* from the original dataset. Parameters are then estimated for each of these new samples. Finally, the covariance matrix of the sequence of estimated

parameters is calculated. This matrix is in itself an estimator of the covariance matrix of the parameter estimates.

The function `apollo_bootstrap` implements the process described above. It is called as follows:

```
apollo_bootstrap(apollo_beta,
                 apollo_fixed,
                 apollo_probabilities,
                 apollo_inputs,
                 estimate_settings,
                 bootstrap_settings)
```

The only new input here is `bootstrap_settings`, which has the following contents:

mixEM_settings: user-controlled settings

nRep: Number of bootstrap samples to use (default is 30).

samples: An optional numeric matrix or data.frame with as many rows as observations in the `database`, and as many columns as number of repetitions wanted. Each column represents a re-sample, and each element must be a 0 or a positive integer representing the number of times that row is used in that given sample. If this argument is provided, then `nRep` is ignored. Note that this allows sampling at the observation rather than the individual level.

seed: An optional positive integer used as seed for the bootstrap sampling generation process. Default is 24. It is only used if `samples` is NA. Changing the seed allows drawing new samples when re-starting a bootstrap process. This is useful when a bootstrap process has been interrupted, or when additional repetitions are needed.

`apollo_bootstrap` returns a list with three elements. The first element is a matrix containing the estimated parameters in each repetition, with as many rows as repetitions, and as many columns as parameters. The second element is the correlation matrix between the estimated parameters across repetitions. The third element is a vector with the log-likelihood at convergence in each repetition.

`apollo_bootstrap` saves to disk a file called `name_bootstrap_params.csv`, where `name` is the name of the model as defined in `apollo_control$modelName`. This file contains the estimates from each of the `nRep` estimation runs, as well as the log-likelihoods. It also saves a file `name_bootstrap_samples.csv` which contains information on the samples and a file called `name_bootstrap_vcov.csv` containing the covariance matrix. If the files already exist, for example because the process got interrupted, the new repetitions will be added to the existing files. So, if the function is run twice with the default settings, the `name_bootstrap_params.csv` file will contain 60 sets of parameters. In addition, the function prints to screen the covariance matrix. The running of the function is illustrated in Figure 9.5 for example `LC_no_covariates.r`.

The function can also be called automatically during estimation, as described in Section 4.6, by adjusting the setting `estimate_settings$bootstrapSE`.

```

1 > apollo_bootstrap(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 30 new datasets will be constructed by randomly sampling 388 individuals with replacement from
  ↳ the
3   original dataset.
4 Preparing bootstrap.
5 Parameters and LL in each repetition will be written to: Apollo_example_18_bootstrap_params.csv
6 Vectors showing sampling rate for each observation in each repetition written to:
7   Apollo_example_18_bootstrap_samples.csv
8
9 Estimation cycle 1 (3492 obs)
10 Estimation results written to file.
11 ...
12 Estimation cycle 30 (3492 obs)
13 Estimation results written to file.
14
15 Finished bootstrap runs.
16 Parameters and LL for each repetition written to: Apollo_example_18_bootstrap_params.csv
17 Vectors showing sampling rate for each observation in each repetition written to:
18   Apollo_example_18_bootstrap_samples.csv
19 Covariance matrix of parameters written to: Apollo_example_18_bootstrap_vcov.csv
20
21 Mean LL across runs: -1553.99
22 Mean parameter values across runs:
23     Estimate
24 asc_1      -0.0407
25 asc_2       0.0000
26 beta_tt_a  -0.0875
27 beta_tt_b  -0.1201
28 beta_tc_a  -0.1049
29 beta_tc_b  -0.7484
30 beta_hw_a  -0.0403
31 beta_hw_b  -0.0494
32 beta_ch_a  -0.9999
33 beta_ch_b  -2.0265
34 delta_a    0.1838
35 delta_b    0.0000
36
37 Covariance matrix across runs:
38     asc_1  beta_tt_a  beta_tt_b ...
39     asc_1  3.119e-03 -4.156e-04 -0.0006079 ...
40 beta_tt_a -4.156e-04  1.032e-03 -0.0015649 ...
41 beta_tt_b -6.079e-04 -1.565e-03  0.0070312 ...
42 ...
43     delta_a
44     asc_1 -6.761e-05
45 beta_tt_a  5.163e-03
46 beta_tt_b -2.302e-02
47 beta_tc_a  2.088e-03
48 beta_tc_b -2.137e-01
49 beta_hw_a -1.398e-03
50 beta_hw_b  1.458e-04
51 beta_ch_a -1.168e-01
52 beta_ch_b -1.524e-02
53     delta_a  2.610e-01
54 Bootstrap processing time: 5.735613 mins

```

Figure 9.5: Running `apollo_bootstrap`

9.6 Likelihood ratio tests against other models

A core step in many model fitting exercises is the comparison of models of different levels of complexity. When comparing two models where one model is a more general version of a base model, i.e. the *base* model is nested within the *general* model, a likelihood-ratio test can be used to compare the two models (cf. Train, 2009, Section 3.8.2.).

This test is implemented in the function `apollo_lrTest`, which is called as follows:

```
apollo_lrTest(model1,
              model2)
```

```
1 > apollo_lrTest("../1 MNL/output/MNL_SP_covariates", model)
2               LL par
3 MNL_SP_covariates -4830.94 17
4 NL_two_levels    -4774.55 18
5 Difference         56.39  1
6
7 Likelihood ratio test-value: 112.78
8 Degrees of freedom:        1
9 Likelihood ratio test p-value: 2.411e-26
10 > apollo_lrTest("../1 MNL/output/MNL_SP_covariates", "NL_two_levels")
11               LL par
12 MNL_SP_covariates -4830.94 17
13 NL_two_levels    -4774.55 18
14 Difference         56.39  1
15
16 Likelihood ratio test-value: 112.78
17 Degrees of freedom:        1
18 Likelihood ratio test p-value: 2.411e-26
```

Figure 9.6: Running `apollo_lrTest`

The function `apollo_lrTest` is flexible in the input it receives for `model1` and `model2`. It can either receive the names of models (i.e. as a character string) for which the output has previously been saved to disk, or the name of model objects stored in memory, i.e. as produced by `apollo_estimate` or loaded via `apollo_loadModel`. A mix of the two is also permissible, i.e. one previously stored model output along with one kept in memory. The `apollo_lrTest` function will determine whether the models have been estimated on the same data (in terms of observations and alternatives), and whether one model uses fewer parameters and has a lower fit. The function is not able to determine whether one model specification is in fact nested within the other, i.e. the user needs to decide whether a likelihood ratio test is appropriate. The order in which the two models are given is not relevant, as `apollo_lrTest` will reorder them in the appropriate way.

In Figure 9.6, we illustrate the function by comparing the MNL model from Section 4.5.2, i.e. `MNL_SP_covariates.r`, and the NL model from Section 5.1.1, i.e. `NL_two_levels.r`. We show both the version where the outputs for both models have been saved earlier²⁰ as well as the version where the more general model has just been run and still exists in the R environment. This function is not suitable when for example comparing a joint model with

²⁰Note that if exact paths not given, the function will look in the output directory and the working directory.

two separate models (e.g. RP-SP vs separate RP and SP models) and the user in that case needs to calculate the LR test manually, which is of course trivial.

9.7 Ben-Akiva & Swait test

A well known example of a non-nested test for model comparison is the Ben-Akiva & Swait test (Ben-Akiva and Swait, 1986), which is based on adjusted ρ^2 . Specifically, we have that

$$P(\bar{\rho}_1^2 - \bar{\rho}_2^2 \geq z) \leq \Phi \left[- (2zLL_0 + df_1 - df_2)^{\frac{1}{2}} \right], \quad (9.1)$$

where z is the observed difference in adjusted ρ^2 , Φ is the cumulative standard normal distribution, and df_1 and df_2 are the number of parameters for models 1 and 2. The two models need to both be discrete choice, and estimated on the same data.

This test is implemented in *Apollo* using the function `apollo_basTest`, which is called as:

```
apollo_basTest(model1,
               model2)
```

The function `apollo_basTest` is flexible in the input it receives for `model1` and `model2`. It can either receive the names of models (i.e. as a character string) for which the output has previously been saved to disk, or the name of model objects stored in memory, i.e. as produced by `apollo_estimate`. A mix of the two is also permissible, i.e. one previously stored model output along with one kept in memory. The `apollo_basTest` function will determine whether the models have been estimated on the same data, and whether a ρ^2 measure is available for both models. The order in which the two models are given is not relevant, as `apollo_basTest` will reorder them in the appropriate way. It then calculates the p-value for the Ben-Akiva & Swait test.

In Figure 9.7, we illustrate the function by comparing the three-level NL model from Section 5.1.1, i.e. `NL_three_levels.r`, and the CNL model from Section 5.1.2, i.e. `CNL.r`. We show here the version where the more general model has just been run and still exists in the R environment, but the function similarly works with loading results from two previously run models.

```
1 > apollo_basTest("NL_three_levels",model)
2
3      LL0      LL par adj.rho2
4 NL_three_levels -8196.02 -4770.48 19 0.4156
5 CNL             -8196.02 -4742.25 20 0.4190
6 Difference      0.00      28.23 1 0.0034
7
8 p-value for Ben-Akiva & Swait test: 2.496e-14
```

Figure 9.7: Running `apollo_basTest`

9.8 Utilities at specified parameter values

In some cases, a user may want to output the utilities calculated at specific values for the model parameters, most typically at the final estimates. This is supported in *Apollo* via the

“utilities” setting for functionality. This feature is implemented for the majority of models. The user can obtain this output via a direct call to `apollo_probabilities` with the appropriate input and setting.

The application of this functionality to the `MNL_RP_SP.r` mode choice model is illustrated in Figure 9.8. In this example, we calculate the utilities at the final parameter estimates, returning a list of two lists, one for the RP data (1,000 rows per alternative) and one for the SP data (7,000 rows per alternative). Note that the returned utilities are calculated for all rows in the data where `rows==TRUE`, where for unavailable alternatives, the utility is set to NA.

```

1  ### calculate utilities at parameter estimates
2  > V=apollo_probabilities(apollo_beta=model$estimate, apollo_inputs, functionality="utilities")
3  ### dimensions of V
4  > lapply(V,summary)
5  $RP
6      Length Class  Mode
7  car   1000  -none- numeric
8  bus   1000  -none- numeric
9  air   1000  -none- numeric
10 rail  1000  -none- numeric
11
12 $SP
13      Length Class  Mode
14 car   7000  -none- numeric
15 bus   7000  -none- numeric
16 air   7000  -none- numeric
17 rail  7000  -none- numeric
18
19 ### look at summary of utilities for RP model component
20 > lapply(V$RP,summary)
21 $car
22      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   -5.591  -3.979  -3.473  -2.910  -2.882    0.000

```

Figure 9.8: Running `apollo_probabilities` with `functionality="utilities"`

9.9 Model predictions

A core capability of *Apollo* is that it covers model application (i.e. prediction) in addition to estimation. This is implemented in the function `apollo_prediction`. The function is called as follows:

```

forecast = apollo_prediction(model,
                             apollo_probabilities,
                             apollo_inputs,
                             prediction_settings)

```

The majority of these arguments have been discussed already. The only additional new argument is `prediction_settings`, which is an optional list that can contain two entries:

prediction_settings: user-controlled settings

modelComponent: name of the model component for which predictions are requested. This is an optional additional argument for models with multiple components, and needs to be the name (string) of one of the elements in the list `P` used inside `apollo_probabilities`. If not provided, predictions are returned in a list, with one element per model component.

nRep: number of Monte Carlo iterations used for models that require simulation for prediction (e.g. MDCEV) (default is 100).

runs: number of runs of the prediction algorithm to use with random draws from the set of parameters (default=1)

silent: if TRUE, this function will not print any output to screen.

simChoice: if TRUE, simulated choices are added to the output. FALSE by default.

summary: If TRUE (default), a summary of the prediction is printed to screen.

When not including the setting for `runs`, the use of `apollo_prediction` produces a list with one `data.frame` per model component (for multi-component models), or a single `data.frame` for single-component models. The `data.frame(s)` returned have one row per observation, and the following sets of columns:

- ID of the individual
- Index of observations for the individual
- Set of J columns with predicted probabilities, one per alternative
- Probability for the chosen alternative

The output of `apollo_prediction` is slightly different for MDCEV models, including the following columns.

- ID of the individual
- Index of observations for the individual
- Set of J columns with predicted continuous consumption, one per alternative
- Set of J columns with the s.d. of the predicted continuous consumption, one per alternative
- Set of J columns with predicted probability of (discrete) consumption, one per alternative
- Set of J columns with the s.d. of the predicted probability of (discrete) consumption, one per alternative
- Set of J columns with predicted expenditure, one per alternative
- Set of J columns with the s.d. of the predicted expenditure, one per alternative

When including a value for `runs` that is larger than 1, `apollo_prediction` returns a list. The first element in the list will be the matrix of prediction at the parameter estimates, using the format above. This element is called `at_estimates`. The second element in the list, called `draws`, is a 3-dimensional array, with dimensionality *number of observations* x $(J + 1)$ or $6J$ x $runs$. Each slide on the first two dimensions has the same columns described above, except for the ID of the individual and the index of the observation (i.e. first two columns in the list above are missing). Each of these predictions come from different draws from the sets of parameter estimates.

Note that `apollo_prediction` uses the copy of the `database` stored within `apollo_inputs`, and not the version of `database` that may be stored in the user environment (i.e. the Global Environment). If the user wishes to calculate predictions for a modified `database` (for example, to study how predictions change in a new scenario) we recommend doing it in three stages. First, modify `database` in whatever way is needed. Then update `apollo_inputs` by running `apollo_inputs = apollo_validateInputs()`, and finally call `apollo_prediction`.

The application of this function to the `MNL_SP_covariates.r` mode choice model is illustrated in Figure 9.9, while Figure 9.10 shows how to look at changes in choices following a change in an explanatory variable, as well as how elasticities can be calculated. We first run a prediction using the model on the original data, in addition to 30 runs of the prediction on estimates drawn from the asymptotic distribution of the parameter estimates, where these are drawn using the full covariance matrix. This allows us to get an indication of the level of uncertainty in the predictions. This process shows the aggregated predicted demand (summing probabilities across observations) at the model estimates, as well as the mean probabilities, and also 95% confidence interval for these, based on the additional repetitions.

We next run a prediction (without separate runs for confidence interval computation) for a scenario where the cost of rail is increased by 1% (after which we reverse that change in the data). As expected, this leads to a drop in the demand for rail, and an increase in the demand for other modes.

We next compare the before and after probabilities at the level of individual observations. For the base predictions, we first need to extract the first element of the list, i.e. the predictions at the actual model estimates. We then look at changes for individual people in the data, summaries of changes, including for subsets of the data, before computing elasticities.

The output of `apollo_prediction` will depend on the underlying model component, but will always include the ID and choice situation index for each row when `runs=1`. In particular:

- For MNL, NL, CNL, DFT, OL and OP models, `apollo_prediction` will return the probability of the chosen alternative, as well as the probability of each alternative at the observation level (rather than person level). In particular, these models return a list containing one vector per alternative plus an additional vector for the chosen alternative, where each vector is as long as the number of observations in the `database`, and contain the probability of that alternative. In the presence of continuous random heterogeneity, the draws are averaged out before presenting the results..
- The discrete continuous models MDCEV and MDCNEV do not return probabilities, but instead expected values of consumption for each alternative at the observation level. In particular, they return a matrix detailing the expected (continuous) consumption for each alternative, and a proxy for the probability of consuming each alternative (discrete choice), as well as the standard deviations for both of these measurements. These outputs are calculated using the efficient forecasting method proposed by Pinjari and Bhat 2010b, and its modification for the MDCNEV model by Calastri et al. 2017. These methods are based on simulation (100 repetitions are used by default), and can therefore be computationally demanding. The probability of consuming each alternative is calculated as the percentage of simulation repetitions in which the alternative is consumed, and is not calculated using an analytical formula. Again, in the presence of continuous random coefficients, the results are averaged across draws. If using the optional argument `runs` in `prediction_settings`, the output will present standard deviations across runs for both the mean predictions and the standard deviations due

```

1  ### Use the estimated model to make predictions
2  > predictions_base = apollo_prediction(model, apollo_probabilities, apollo_inputs,
   ↪ prediction_settings=list(runs=30))
3  Running predictions from model using parameter estimates...
4  Running predictions across draws from the asymptotic distribution for maximum likelihood
   ↪ estimates.
5  Predicting for set of draws 1/30...
6  ...
7  Predicting for set of draws 30/30...
8
9  Aggregated prediction
10      at MLE Sampled mean Sampled std.dev. Quantile 0.025 Quantile 0.975
11  car    1946      1936.0         29.62      1890.4      1989
12  bus     358      361.2         17.07       324.7       390
13  air    1522     1519.9         28.05     1469.8      1568
14  rail   3174     3183.0         32.90     3106.9     3233
15
16  Average prediction
17      at MLE Sampled mean Sampled std.dev. Quantile 0.025 Quantile 0.975
18  car 0.27800    0.27657    0.004232    0.27006    0.28409
19  bus 0.05114    0.05159    0.002439    0.04639    0.05571
20  air 0.21743    0.21712    0.004007    0.20997    0.22398
21  rail 0.45343    0.45471    0.004700    0.44384    0.46183
22
23  The output from apollo_prediction is a list with two elements: a data.frame containing the
   ↪ predictions at the
24  estimated values, and an array with predictions for different values of the parameters drawn
   ↪ from their
25  asymptotic distribution.

```

Figure 9.9: Running apollo_prediction: base prediction

to the use of draws inside the [Pinjari and Bhat \(2010b\)](#) algorithm.

- The **EL** (Exploded Logit) and **Normal Density** models do not return any predictions, as it is not evident what precise outcome would be the most useful for the biggest share of *Apollo* users.

```

1 > ### Now imagine the cost for rail increases by 1%
2 > database$cost_rail = 1.01*database$cost_rail
3 > ### Rerun predictions with the new data
4 > apollo_inputs = apollo_validateInputs()
5 > predictions_new = apollo_prediction(model, apollo_probabilities, apollo_inputs)
6 Running predictions from model using parameter estimates...
7 Prediction at model estimates
8           car    bus    air    rail
9 Aggregate 1967.39 362.97 1535.10 3134.55
10 Average    0.28  0.05  0.22  0.45
11
12 The output from apollo_prediction is a matrix containing the predictions at the estimated values.
13 > ### Return to original data
14 > database$cost_rail = 1/1.01*database$cost_rail
15 > apollo_inputs = apollo_validateInputs()
16 > ### work with predictions at estimates
17 > predictions_base=predictions_base[["at_estimates"]]
18 > ### Compute change in probabilities
19 > change=(predictions_new-predictions_base)/predictions_base
20 > ### Not interested in chosen alternative now, so drop last column
21 > change=change[,-ncol(change)]
22 > ### First two columns (change in ID and task) also not needed
23 > change=change[,-c(1,2)]
24 > ### And person 9, who has all 4 modes available
25 > change[database$ID==9,]
26           car    bus    air    rail
27 113 0.010361642 0.010361642 0.010361642 -0.018685858
28 ...
29 126 0.015476820 0.015476820 0.015476820 -0.006015604
30 > ### Summary of changes (possible presence of NAs for unavailable alternatives)
31 > summary(change)
32           car    bus    air    rail
33 Min.   :0.0000 Min.   :0.0000 Min.   :0.0000 Min.   : -0.1032
34 1st Qu.:0.0071 1st Qu.:0.0071 1st Qu.:0.0061 1st Qu.: -0.0308
35 Median :0.0132 Median :0.0141 Median :0.0125 Median : -0.0149
36 Mean   :0.0138 Mean   :0.0152 Mean   :0.0144 Mean   : -0.0213
37 3rd Qu.:0.0197 3rd Qu.:0.0216 3rd Qu.:0.0203 3rd Qu.: -0.0064
38 Max.   :0.0566 Max.   :0.0719 Max.   :0.0936 Max.   : 0.0000
39 NA's   :1554  NA's   :686  NA's   :1736  NA's   :882
40 > ### Look at mean changes for subsets of the data, ignoring NAs
41 > colMeans(change,na.rm=TRUE)
42           car    bus    air    rail
43 0.01383349 0.01521381 0.01439664 -0.02132545
44 > colMeans(subset(change,database$business==1),na.rm=TRUE)
45           car    bus    air    rail
46 0.01246518 0.01281052 0.01070142 -0.01098996
47 > colMeans(subset(change,database$business==0),na.rm=TRUE)
48           car    bus    air    rail
49 0.01451238 0.01643555 0.01604528 -0.02640501
50 > ### Compute own elasticity for rail:
51 > log(sum(predictions_new[,6])/sum(predictions_base[,6]))/log(1.01)
52 [1] -1.257122
53 > ### Compute cross-elasticities for other modes
54 > log(sum(predictions_new[,3])/sum(predictions_base[,3]))/log(1.01)
55 [1] 1.098745
56 > log(sum(predictions_new[,4])/sum(predictions_base[,4]))/log(1.01)
57 [1] 1.386171
58 > log(sum(predictions_new[,5])/sum(predictions_base[,5]))/log(1.01)
59 [1] 0.86075

```

Figure 9.10: Running apollo_prediction: prediction of impact of price change

Instead of providing `apollo_prediction` with a model object, the user can also call the function with a vector of parameter values as the first argument. An example of this is included in `MNL_SP_covariates.r`, as illustrated in Figure 9.11, where we make a prediction from a vector of parameters obtained from estimation, but with a 10% increase in cost sensitivity.

```

1  ### Make predictions with a higher cost sensitivity
2  > apollo_beta = model$estimate
3  > apollo_beta["b_cost"] = 1.1 * apollo_beta["b_cost"]
4  > predictions_v2 = apollo_prediction(apollo_beta, apollo_probabilities, apollo_inputs)
5  Running predictions using vector of parameters provided by caller...
6
7  Prediction at user provided parameters
8      car    bus    air    rail
9  Aggregate 2040.25 432.67 1397.67 3129.41
10 Average    0.29  0.06  0.20  0.45
11
12 The output from apollo_prediction is a matrix containing the predictions at the estimated values.

```

Figure 9.11: Running `apollo_prediction` with a vector of parameters

Finally, a user can also ask for simulated choices to be added to the output from `apollo_prediction`, where these will be produced according to the probabilities. An example of this is included in `MNL_SP_covariates.r`, as illustrated in Figure 9.12.

```

1  > ### Simulate choices
2  > simulated_choices = apollo_prediction(apollo_beta, apollo_probabilities, apollo_inputs,
3  ↪ prediction_settings=list(simChoice=TRUE))
4  Running predictions using vector of parameters
5  provided by caller...
6
7  Prediction at user provided parameters
8      car    bus    air    rail
9  Aggregate 2040.25 432.67 1397.67 3129.41
10 Average    0.29  0.06  0.20  0.45
11
12 The output from apollo_prediction is a matrix
13 containing the predictions at the estimated
14 values.
15 > table(simulated_choices$simChoice)
16
17 car bus air rail
2039 444 1354 3163

```

Figure 9.12: Running `apollo_prediction` with a vector of parameters

9.10 Market share recovery for subgroups of data

With labelled choice data (or even unlabelled data where there may be strong left-right effects), it can be useful to test after model estimation how well the choice shares in the data are recovered by the model. With a full set of ASCs, a linear in attributes MNL model will perfectly recover market shares at the sample level (see e.g. Train, 2009, Section 2.6.1.). This is however likely not the case in subsets of the data, or indeed for more complex models, and

this test can thus be a useful input for model refinements. The function `apollo_sharesTest` is based on the “apply” tables approach in ALogit (ALogit, 2016). It can be used for any of the discrete choice models implemented in *Apollo*, and is called as follows:

```
apollo_sharesTest(model,
                  apollo_probabilities,
                  apollo_inputs,
                  sharesTest_settings)
```

The list `sharesTest_settings` has the following components:

`sharesTest_settings`: user-controlled settings

alternatives: A named vector containing the names of the alternatives as defined by the user, and their respective code as used in `choiceVar` (see below).

choiceVar: A variable indicating the alternative chosen in each row of the data. This is usually a column (variable) in the database. As we are defining this outside `apollo_probabilities`, we must use the `database$` prefix, e.g. `database$choice`.

subsamples: The list `subsamples` is an optional input which contains one vector for each subset of the data to be used in the test, with each vector having as many elements as observations in the data. Each observation can be included in multiple subsets, i.e. the sum of the values across column vectors in `subsamples` may exceed 1.

modelComponent: The name of the model component for which predictions are requested. This argument is required for models with multiple components, and needs to be the name (string) of one of the elements in the list `P` used inside `apollo_probabilities`.

newAlts: Optional list describing the new alternatives to be used by `apollo_sharesTest`. This should have as many elements as new alternatives, with each entry being a matrix of 0-1 entries, with one row per observation, and one column per alternative used in the model.

newAltsOnly: An optional boolean variable, which, if set to `TRUE`, means that results will only be printed for the ‘new’ alternatives defined in `newAlts`, not the original alternatives used in the model. Set to `FALSE` by default.

The function produces one table per column in `subsamples`, along with an overall table for the entire sample. In each table, the code reports the number of times an alternative is chosen in the data, the number of times the model predicts it to be chosen, the difference between prediction and data, and a t-ratio and p-value for this difference. An example application of this function to the SP mode choice data is included in `MNL_SP_covariates.r`. As we can see from Figure 9.13, in our example, the model significantly overpredicts the rate at which business travellers choose bus and underpredicts the rate at which they choose air. A revised model specification may thus incorporate interactions in these ASCs for business travellers. In addition to screen output, `apollo_sharesTest` also invisibly returns the output so it can be saved into a data.frame, using e.g.:


```

1 > sharesTest_settings = list()
2 > sharesTest_settings[["alternatives"]] = c(car=1, bus=2, air=3, rail=4)
3 > sharesTest_settings[["choiceVar"]] = database$choice
4 > sharesTest_settings[["subsamples"]] = list(business=(database$business==1),
5                                             leisure=(database$business==0))
6
7 > apollo_sharesTest(model, apollo_probabilities, apollo_inputs, sharesTest_settings)
8 Running share prediction tests
9 Prediction test for group: business (2310 observations)
10
11      car    bus    air    rail  All
12 Times chosen (data)    366.000  8.000 771.000 1165.000 2310
13 Times chosen (prediction) 350.442 24.347 739.725 1195.486 2310
14 Diff (prediction-data)  -15.558 16.347 -31.275  30.486    0
15 t-ratio    -1.093  3.463  -1.846  1.613   NA
16 p-val       0.274  0.001  0.065  0.107   NA
17
18 Prediction test for group: leisure (4690 observations)
19
20      car    bus    air    rail  All
21 Times chosen (data)    1580.000 350.000 751.000 2009.000 4690
22 Times chosen (prediction) 1595.552 333.651 782.280 1978.517 4690
23 Diff (prediction-data)   15.552 -16.349  31.280 -30.483    0
24 t-ratio     0.606 -1.075  1.601  -1.159   NA
25 p-val       0.545  0.282  0.109  0.246   NA
26
27 Prediction test for group: All data (7000 observations)
28
29      car    bus    air    rail  All
30 Times chosen (data)    1946.000 358.000 1522.000 3174.000 7000
31 Times chosen (prediction) 1945.994 357.998 1522.005 3174.003 7000
32 Diff (prediction-data)  -0.006 -0.002  0.005  0.003    0
33 t-ratio     0.000  0.000  0.000  0.000   NA
34 p-val       1.000  1.000  1.000  1.000   NA

```

Figure 9.13: Running `apollo_sharesTest`

```

sharesTest_output = apollo_sharesTest(model,
                                       apollo_probabilities,
                                       apollo_inputs,
                                       sharesTest_settings)

```

9.11 Comparison of model fit across subgroups of data

An additional function is implemented to compare the performance of the estimated model for different subsets of the data. The function `apollo_fitsTest` can be used for any models

estimated in *Apollo*, and is called as follows:

```
apollo_fitsTest(model,
                 apollo_probabilities,
                 apollo_inputs,
                 fitsTest_settings)
```

The list `fitsTest_settings` has the following component:

fitsTest_settings: user-controlled settings

subsamples: The list `subsamples` is an optional input which contains one vector for each subset of the data to be used in the test, with each vector having as many elements as observations in the data. Each observation can be included in multiple subsets, i.e. the sum of the values across column vectors in `subsamples` may exceed 1.

The function calculates various statistics for the log-likelihood, as illustrated in Figure 9.14 for the mode choice MNL model `MNL_SP_covariates.r`, where the last row in the output compares the mean log-likelihood in the specific subsample to the mean in all other subsamples. Users need to exercise caution when using this function in the case where the choice set size varies across individuals in a manner that is correlated with the subgroups as the prediction performance for individuals with smaller choice sets will be likely to be larger, all else being equal.

```
1 > fitsTest_settings = list()
2 > fitsTest_settings[["subsamples"]] = list()
3 > fitsTest_settings$subsamples[["business"]] = database$business==1
4 > fitsTest_settings$subsamples[["leisure"]] = database$business==0
5
6 > apollo_fitsTest(model,apollo_probabilities,apollo_inputs,fitsTest_settings)
7
8      All data business leisure
9 Min LL per obs      -1.45    -1.29    -1.45
10 Mean LL per obs     -0.69    -0.59    -0.74
11 Median LL per obs   -0.70    -0.61    -0.74
12 Max LL per obs     -0.01    -0.01    -0.06
13 SD LL per obs       0.28     0.28     0.27
mean vs mean of all others      NA     0.14    -0.14
```

Figure 9.14: Running `apollo_fitsTest`

9.12 Out of sample fit (Cross validation)

A common method to test for overfitting of a model is to measure its fit on a sample not used during estimation, i.e. measuring out-of-sample fit. A simple way to do this is to split the available dataset into two parts: an estimation sample, and a validation sample. The model of interest is estimated using only the estimation sample, and then those estimated parameters are used to measure the fit of the model (e.g. the log-likelihood of the model) on the validation sample. Doing this with only one validation sample may however lead to biased results, as a particular validation sample need not be representative of the population.

One way to minimise this issue is to randomly draw several pairs of estimation and validation samples from the complete dataset, and apply the procedure to each pair. This also allows the calculation of a confidence interval for the out-of-sample measure of fit.

The function `apollo_outOfSample` implements the process described above. It is called as follows:

```
apollo_outOfSample(apollo_beta,
                   apollo_fixed,
                   apollo_probabilities,
                   apollo_inputs,
                   estimate_settings,
                   outOfSample_settings)
```

The only new input here is `outOfSample_settings`, which has the following contents:

outOfSample_settings: user-controlled settings

nRep: Number of times a different pair of estimation and validation sets are to be extracted from the full database (default is 30).

rmse: Character matrix with two columns. Used to calculate Root Mean Squared Error (RMSE) of prediction. The first column must contain the names of observed outcomes in the database. The second column must contain the names of the predicted outcomes as returned by `apollo_prediction`. If omitted or NULL, no RMSE is calculated. This only works for models with a single component.

samples: An optional numeric matrix or data.frame with as many rows as observations in the database, and as many columns as number of repetitions wanted. Each column represents a re-sample, and each element must be a 0 if the observation should be assigned to the estimation sample, or 1 if the observation should be assigned to the prediction sample. If this argument is provided, then **nRep** and **validationSize** are ignored. Note that this allows sampling at the observation rather than the individual level.

validationSize: Size of the validation sample. It can be provided as a fraction of the whole database (number between 0 and 1), or a number of individuals (number bigger than 1). The splitting of the database is done at the individual level, not at the observation level (default is 0.1).

`apollo_outOfSample` writes to disk a file called `name_outOfSample_params.csv`, where `name` is the name of the model as defined in `apollo_control`. This file contains the estimates from each of the **nRep** estimation runs, as well as the estimation and out of sample log-likelihoods. It also writes a file called `name_outOfSample_samples.csv` which contains information on the samples. In addition, the function prints to screen the average per observation log-likelihood for each subsample, both for the estimation sample and the holdout sample, as well as the percentage difference between them, and calculates an average across the runs. This output is also returned invisibly by the function. The running of the function is illustrated in Figure 9.15 using the model in example `LC_no_covariates.r`.

Before running, `apollo_outOfSample` will look for existing `name_outOfSample_params.csv` and `name_outOfSample_samples.csv` files in the working

```

1 > apollo_outOfSample(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
2 10 separate runs will be conducted, each using a random subset of 90% for estimation and the
  ↳ remainder
3   for validation.
4 Number of individuals
5 - for estimation : 349
6 - for forecasting : 39
7 - in sample (total): 388
8 Preparing loop.
9 Estimated parameters and log-likelihoods for each sample will be written to:
10  output/LC_no_covariates_outOfSample_params.csv
11 The matrix defining the observations used in each repetition will be written to:
12  output/LC_no_covariates_outOfSample_samples.csv
13
14 Estimation cycle 1 (3141 obs.)
15 Estimation results written to file.
16 Estimation cycle 2 (3141 obs.)
17 Estimation results written to file.
18 Estimation cycle 3 (3141 obs.)
19 Estimation results written to file.
20 Estimation cycle 4 (3141 obs.)
21 Estimation results written to file.
22 Estimation cycle 5 (3141 obs.)
23 Estimation results written to file.
24 Estimation cycle 6 (3141 obs.)
25 Estimation results written to file.
26 Estimation cycle 7 (3141 obs.)
27 Estimation results written to file.
28 Estimation cycle 8 (3141 obs.)
29 Estimation results written to file.
30 Estimation cycle 9 (3141 obs.)
31 Estimation results written to file.
32 Estimation cycle 10 (3141 obs.)
33 Estimation results written to file.
34 Processing time: 1.131586 mins
35
36 Outputs of out of sample testing:
37      LL per obs in estimation sample LL per obs in validation sample % difference
38 1          -0.4464          -0.4641          -3.96
39 2          -0.4505          -0.4296           4.64
40 3          -0.4514          -0.4209           6.75
41 4          -0.4492          -0.4391           2.25
42 5          -0.4480          -0.4171           6.88
43 6          -0.4388          -0.5329          -21.44
44 7          -0.4468          -0.4620          -3.41
45 8          -0.4474          -0.4197           6.20
46 9          -0.4443          -0.4856          -9.30
47 10         -0.4456          -0.4354           2.29
48 Average          -0.4468          -0.4506          -0.91

```

Figure 9.15: Running `apollo_outOfSample`

directory. If they are found, and are consistent with the model at hand, then new repetitions will be added to those files. This means, for example, that if the function is run twice, the output files will hold results for 60 repetitions. This is a useful feature in case the cross-validation process gets interrupted, or if the user wants to increase the number of repetitions a posteriori.

The use of setting `rmse` is mostly useful for discrete-continuous models (e.g. MDCEV, eMDC). It allows measuring the RMSE of the prediction, additional to comparing the log-likelihood of the models across samples. For example, if we have dependent variables `t1`, `t2`, and `t3` in the `database`, corresponding to the time spent sleeping, eating, and working, respectively, and we want to measure their error in prediction, we could use code as the one in figure 9.16.

```

1 > apollo_probabilities = function(...){
2   ...
3   continuousChoice = list(
4     sleep = t1,
5     eat    = t2,
6     work   = t3)
7   ...
8   return(P)
9 }
10
11 rmse = matrix(c("t1", "sleep",
12                "t2", "eat",
13                "t3", "work"),nrow=3,ncol=2,byrow=TRUE)
14 apollo_outOfSample(apollo_beta, apollo_fixed, apollo_probabilities,
15                    apollo_inputs, outOfSample_settings=list(rmse=rmse))

```

Figure 9.16: Calculating prediction RMSE using `apollo_outOfSample`

9.13 Delta method for functions of model parameters

A key use of estimates from choice models is the calculation of functions of these estimates, for example in the form of ratios of coefficients, leading to marginal rates of substitution, and in the case of a cost coefficient being used as the denominator, willingness-to-pay (WTP) measures. It is then important to be able to calculate standard errors for these derived measures, where this can be done straightforwardly and accurately with the Delta method, as discussed by [Daly et al. \(2012a\)](#), which is not an approximation, but an exact calculation. The function `apollo_deltaMethod` is implemented for this purpose, and is called as follows:

```
apollo_deltaMethod(model,
                   deltaMethod_settings)
```

The list `deltaMethod_settings` has the following component²¹:

²¹In earlier versions of *Apollo*, the user could only use a subset of functions of attributes, using the setting `operation`. This is still possible, but the use of `expression` is now recommended.

deltaMethod_settings: user-controlled settings

allPairs: A setting to instruct *Apollo* to compute standard errors for all ratios of parameters and all differences between parameters. These are written to file and returned invisibly as function output. This is an optional setting, where FALSE is used by default when not provided.

expression: A character vector defining the expression or expressions which is/are to be evaluated. Each expression is an arbitrary function of the estimated parameters, as text, for example using `vtt="b1/b2*60"`. An expression can only contain model parameters (estimated or fixed), numeric values, and operands. An expression cannot contain explanatory variables from the database (e.g. *income*). At least one of the parameters used needs to not have been fixed in estimation. If the user does not provide a name for an expression, then the expression itself is used in the output.

printPVal: A setting to instruct *Apollo* to also report *p*-values for the calculated expressions. This setting uses TRUE or 1 for printing *p*-values for a one-sided test, 2 for printing *p*-values for a two-sided test, and FALSE for not printing *p*-values. This is an optional setting, where FALSE is used by default when not provided.

varcov: Type of variance-covariance matrix to use in calculations. This is an optional setting which can take values "classical", "robust" and "bootstrap", where "robust" is used by default.

An example application of this function is included in `MNL_SP_covariates.r`, and is illustrated in Figure 9.17 for the car value of travel time, i.e. the ratio between the car travel time and cost coefficients (in both minutes and hours) as well as for the difference between the car and rail travel time coefficients. The values here are all calculated for an individual in the base socio-demographic group.

```

1 > deltaMethod_settings=list(expression=c(VTT_car_min="b_tt_car/b_cost",
2 +                                     VTT_car_hour="60*b_tt_car/b_cost",
3 +                                     b_tt_diff_car_rail="b_tt_car-b_tt_rail"))
4 > apollo_deltaMethod(model, deltaMethod_settings)
5 Running Delta method computation for user-defined function:
6
7      Expression  Value Robust s.e. Rob t-ratio (0)
8      VTT_car_min  0.1720    0.0097      17.70
9      VTT_car_hour 10.3222    0.5831      17.70
10     b_tt_diff_car_rail -0.0061    0.0019      -3.18

```

Figure 9.17: Running `apollo_deltaMethod`

9.14 Unconditionals for random parameters

After model estimation, it may be useful to an analyst to have at their disposal the actual values used for random coefficients, especially if these included interactions with socio-demographics or (non-linear) transforms that may lead to a requirement for simulation to calculate moments, as in the semi-non-parametric approach of Fosgerau and Mabit 2013 used in Section 6.1.2.

The function `apollo_unconditionals` is called as follows:

```
unconditionals = apollo_unconditionals(model,  
                                       apollo_probabilities,  
                                       apollo_inputs,  
                                       obsLevel)
```

For continuous random coefficients, the function produces a list as output, with one element per random coefficient, where each element is a matrix for inter-individual draws, and a cube with inter and intra-individual draws. Each time, there is one row per individual, rather than one row per observation. The outputs from this function can then readily be used for summary statistics or to produce plots. An example of this is included in `MMNL_wtp_space_inter_intra.r`, and also illustrated in the discussion of conditionals in Section 9.15.1.

For latent class models, the `apollo_unconditionals` function produces a list which has one element for each model parameter that varies across classes, where these are given by lists, with one element per class. The entry for each class could be either a scalar (if fixed coefficients are used inside the classes), a vector (if interactions with socio-demographics are used), or a matrix or cube if continuous heterogeneity is also incorporated. For the latter two cases, there is one row per individual, rather than one row per observation. The final component in the list produced by `apollo_unconditionals` is a list containing the class allocation probabilities, with one element per class, where these could again be scalars, vectors, matrices or cubes, depending on the extent of heterogeneity allowed for by the user. An example of this is included in `LC_with_covariates.r`, and also illustrated in the discussion of conditionals in Section 9.15.2.

Note that `apollo_unconditionals` uses the copy of the `database` stored within `apollo_inputs`, and not the version of `database` that may be stored in the user environment (i.e. the Global Environment). If the user wishes to calculate the unconditionals for a modified `database` (for example, to study how unconditionals change in a new scenario) we recommend doing it in three stages. First, modify `database` in whatever way is needed. Then update `apollo_inputs` by running `apollo_inputs = apollo_validateInputs()`, and finally call `apollo_unconditionals`.

If a user calls the function `apollo_unconditionals` on a model combining continuous mixtures with latent classes, a list is returned, with the first element being the unconditionals for the continuous mixture part, and the second element being the unconditionals for the latent class part.

Finally, if the user calls `apollo_unconditionals` on a model without continuous mixtures nor latent classes, the function will produce an error message stating that it is only applicable to latent class or continuous mixture models.

The final argument, `obsLevel`, is optional, and is set to `FALSE` by default. When set to `TRUE`, it ensures that the random draws are returned at the observation rather than person level for continuous mixtures even if only inter-individual draws were used. This is useful when random coefficients are interacted with observation-specific variables.

9.15 Conditionals for random coefficients

There is extensive interest by choice modellers in posterior model parameter distributions, as discussed in Train (2009, chapter 11) for continuous mixture models and Hess (2014) for Latent Class. We implement functions for this for both continuous Mixed Logit and Latent Class models, but not for models combining continuous mixtures with latent class.

The calculation of posteriors for models with continuous random heterogeneity is implemented in the function `apollo_conditionals`, which is called as follows:

```
conditionals = apollo_conditionals(model,
                                   apollo_probabilities,
                                   apollo_inputs)
```

We now look separately at the output from this function depending on the model type.

9.15.1 Continuous random coefficients

Let β give a vector of taste coefficients that are jointly distributed according to $f(\beta | \Omega)$, where Ω is a vector of distributional parameters that is to be estimated from the data. Let Y_n give the sequence of observed choices for respondent n (which could be a single choice), and let $L(Y_n | \beta)$ give the probability of observing this sequence of choices with a specific value for the vector β . Then it can be seen that the probability of observing the specific value of β given the choices of respondent n is equal to:

$$L(\beta | Y_n) = \frac{L(Y_n | \beta) f(\beta | \Omega)}{\int_{\beta} L(Y_n | \beta) f(\beta | \Omega) d\beta} \quad (9.2)$$

The integral in the denominator of Equation 9.2 does not have a closed form solution, such that its value needs to be approximated by simulation. This is a simple (albeit numerically expensive) process, where as an example the mean for the conditional distribution for respondent n can be shown to be given by:

$$\widehat{\beta}_n = \frac{\sum_{r=1}^R [L(Y_n | \beta_r) \beta_r]}{\sum_{r=1}^R L(Y_n | \beta_r)}, \quad (9.3)$$

where β_r with $r = 1, \dots, R$ are independent multi-dimensional draws²² with equal weight from $f(\beta | \Omega)$ at the estimated values for Ω . Here, $\widehat{\beta}_n$ gives the most likely value for the various marginal utility coefficients, conditional on the choices observed for respondent n .

It is important to stress that the conditional estimates for each respondent themselves follow a random distribution, and that the output from Equation 9.3 simply gives the expected value of this distribution. As such, a distribution of the output from Equation 9.3 across respondents should not be seen as a conditional distribution of a taste coefficient across respondents, but rather a distribution of the means of the conditional distributions (or conditional means) across respondents. Here, it is similarly possible to produce a measure of the

²²The term *independent* relates to independence across different multivariate draws, where the individual multivariate draws allow for correlation between univariate draws.

conditional standard deviation, given by:

$$\widetilde{\beta}_n = \sqrt{\frac{\sum_{r=1}^R \left[L(Y_n | \beta_r) (\beta_r - \widehat{\beta}_n)^2 \right]}{\sum_{r=1}^R L(Y_n | \beta_r)}}, \quad (9.4)$$

with $\widehat{\beta}_n$ taken from Equation 9.3.

The calculation of posteriors for models with continuous random heterogeneity using the function `apollo_conditionals` produces a list object with one component per continuous random coefficient (element defined in `apollo_randCoeff`). Each of these components is a *data.frame* with one row per individual, containing the ID for that individual, the mean of the posterior distribution for that individual for the coefficient in question, and the standard deviation.

As `apollo_conditionals` uses the contents of `apollo_randCoeff`, any socio-demographic interactions included in `apollo_randCoeff` will also be included in the calculation for the conditionals, where, if these vary across observations for the same individual, they will be averaged across observations. Similarly, any intra-individual random heterogeneity will also be averaged out. This function is only applicable for models using classical estimation, and also only models where `apollo_control$workInLogs==FALSE`.

Note that `apollo_conditionals` uses the copy of the `database` stored within `apollo_inputs`, and not the version of `database` that may be stored in the user environment (i.e. the Global Environment). If the user wishes to calculate the conditionals for a modified `database` (for example, to study how conditionals change in a new scenario) we recommend doing it in three stages. First, modify `database` in whatever way is needed. Then update `apollo_inputs` by running `apollo_inputs = apollo_validateInputs()`, and finally call `apollo_conditionals`.

Figure 9.18 illustrates the use of this function for the value of travel time coefficient in the Swiss route choice MMNL (`MMNL_wtp_space_inter_intra.r`) example, where we show how the conditional means can then for example also be used in regression analysis against characteristics of the individual, as discussed by Train (2009, chapter 11), in our case showing a significant impact of income on the conditionals for the VTT. Note that as `apollo_conditionals` produces one value per individual, we also need to reduce the dimensionality of the income variable to one per individual, using `apollo_firstRow`²³. We also include a comparison with the unconditionals.

9.15.2 Latent class

It is similarly possible to calculate a number of posterior measures from Latent Class models. A key example comes in the form of posterior class allocation probabilities, where the posterior

²³The function `apollo_firstRow` returns the first row per individual from any object passed to it. The function is called as follows:

```
output = apollo_firstRow(x,
                        apollo_inputs)
```

The first element (in this example called `x`) needs to have the same number of rows as the `database` object included in `apollo_inputs`.

```

1 > unconditionals <- apollo_unconditionals(model,apollo_probabilities, apollo_inputs)
2 Unconditional distributions computed
3 > conditionals <- apollo_conditionals(model,apollo_probabilities, apollo_inputs)
4 Your model contains intra-individual draws which will be averaged over for conditionals!
5 Calculating conditionals...
6 > mean(unconditionals[["v_tt"]])
7 [1] 0.4306562
8 > sd(unconditionals[["v_tt"]])
9 [1] 0.5427685
10 > summary(conditionals[["v_tt"]])
11      ID      post.mean      post.sd
12 Min.   : 2439   Min.   :0.1377   Min.   :0.03458
13 1st Qu.:15308  1st Qu.:0.2650   1st Qu.:0.11324
14 Median :18533  Median :0.3304   Median :0.14906
15 Mean   :22181  Mean   :0.4166   Mean   :0.19566
16 3rd Qu.:21948  3rd Qu.:0.4618   3rd Qu.:0.22599
17 Max.   :84525  Max.   :3.2228   Max.   :2.35940
18 > income_n = apollo_firstRow(database$hh_inc_abs, apollo_inputs)
19 > summary(lm(conditionals[["v_tt"]][,2]~income_n))
20 Call:
21 lm(formula = conditionals[["v_tt"]][, 2] ~ income_n)
22
23 Residuals:
24      Min       1Q   Median       3Q      Max
25 -0.32739 -0.15374 -0.07878  0.06241  2.77554
26
27 Coefficients:
28             Estimate Std. Error t value Pr(>|t|)
29 (Intercept) 3.513e-01  2.839e-02  12.371  < 2e-16 ***
30 income_n     8.536e-07  3.211e-07   2.659  0.00817 **
31 ---
32 Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
33
34 Residual standard error: 0.2805 on 386 degrees of freedom
35 Multiple R-squared:  0.01798,    Adjusted R-squared:  0.01544
36 F-statistic: 7.068 on 1 and 386 DF,  p-value: 0.008174

```

Figure 9.18: Running `apollo_unconditionals` and `apollo_conditionals`

probability of individual n for class s is given by:

$$\widehat{\pi_{ns}} = \frac{\pi_{ns} L_n(\beta_s)}{L_n(\beta, \pi_n)}, \quad (9.5)$$

where $L_n(\beta_s)$ gives the likelihood of the observed choices for individual n , conditional on class s .

To explain the benefit of these posterior class allocation probabilities, let us assume that we have calculated for each class in the model a given measure $w_s = \frac{\beta_{s1}}{\beta_{s2}}$, i.e. the ratio between the first two coefficients. Using $\overline{w_n} = \sum_{s=1}^S \pi_{ns} w_s$ simply gives us a sample level mean for the measure w for an individual with the specific observed characteristics of person n . These characteristics (in terms of socio-demographics used in the class allocation probabilities) will however be common to a number of individuals who still make different choices, and the most likely value for w for individual n , conditional on his/her observed choices, can now be calculated as $\widehat{w_n} = \sum_{s=1}^S \widehat{\pi_{ns}} w_s$.

Finally, it might also be useful to produce a profile of the membership in each class. From the parameters in the class allocation probabilities, we know which class is more or less likely to capture individuals who possess a specific characteristic, but this is not taking into account the multivariate nature of these characteristics. Let us for example assume that a given socio-demographic characteristic z_c is used in the class allocation probabilities, with associated parameter γ_c , and using a linear parameterisation in Equation 6.18. We can then calculate the likely value for z_c for an individual in class s as:

$$\widehat{z_{cs}} = \frac{\sum_{n=1}^N \widehat{\pi_{ns}} z_{cn}}{\sum_{n=1}^N \widehat{\pi_{ns}}}, \quad (9.6)$$

where we again use the posterior probabilities to take into account the observed choices. Alternatively, we can also calculate the probability of an individual in class s having a given value κ for z_c by using:

$$P(\widehat{z_{cs}} = \kappa) = \frac{\sum_{n=1}^N \widehat{\pi_{ns}}(z_{cn} = \kappa)}{\sum_{n=1}^N \widehat{\pi_{ns}}}. \quad (9.7)$$

The calculation of posteriors for Latent Class models using the function `apollo_conditionals` produces a *data.frame*, with one row per individual, one column with the individual id, and one column per class, containing the individual-specific posterior class allocation probabilities. Note again that `apollo_conditionals` uses the copy of the `database` stored within `apollo_inputs`, and not the version of `database` that may be stored in the user environment (i.e. the Global Environment). If the user wishes to calculate the conditionals for a modified `database` (for example, to study how conditionals change in a new scenario) we recommend doing it in three stages. First, modify `database` in whatever way is needed. Then update `apollo_inputs` by running `apollo_inputs = apollo_validateInputs()`, and finally call `apollo_conditionals`.

Figure 9.19 illustrates the use of this for the Swiss mode choice LC model (`LC_with_covariates.r`). We first produce the output from the `apollo_unconditionals_lc` function to compare to the conditionals later on, and also calculate the value of travel time (VTT) in each class, e.g. $VTT_a = \frac{\beta_{t,a}}{\beta_{c,a}}$, where $\beta_{t,a}$ and $\beta_{c,a}$ are the time and cost coefficients, respectively, in class a . We then calculate the unconditional VTT obtained by taking the weighted average across classes, where this varies across individuals as the class allocation probabilities do, i.e. $VTT_n = \pi_{n,a} VTT_a + \pi_{n,b} VTT_b$. We next calculate the conditional class allocation probabilities using `apollo_conditionals`. As can be seen from the output, the means of the conditionals is identical to the mean of the unconditionals, but the range is much wider. Similarly, when we calculate the conditional VTT, we see a wider range for that too.

We finally use the conditional class allocation probabilities to calculate some posterior statistics for class membership. To do this, we first retain only one value for the two socio-demographic variables `commute` and `car_availability` for each individual (by using `apollo_firstRow`) to make the dimensionality the same as the conditionals, before using the formula in Eq. 9.6 to calculate the most likely value for these two variables for individuals in the two classes, given the posterior class allocation probabilities. These posteriors class allocation probabilities can of course then also be used in regression.

```

1 > unconditionals = apollo_unconditionals(model,apollo_probabilities,apollo_inputs)
2 Unconditional distributions computed
3 > vtt_class_a = unconditionals[["beta_tt"]][[1]]/unconditionals[["beta_tc"]][[1]]
4 > vtt_class_b = unconditionals[["beta_tt"]][[2]]/unconditionals[["beta_tc"]][[2]]
5 > vtt_unconditional = unconditionals[["pi_values"]][[1]]*vtt_class_a +
   unconditionals[["pi_values"]][[2]]*vtt_class_b
6 > ### Compute conditionals
7 > conditionals = apollo_conditionals(model,apollo_probabilities, apollo_inputs)
8 Calculating conditionals...
9 > summary(conditionals)
10      ID      X1      X2
11 Min.   : 2439   Min.   :0.000003   Min.   :0.0000
12 1st Qu.:15308   1st Qu.:0.151568   1st Qu.:0.1209
13 Median :18533   Median :0.381030   Median :0.6190
14 Mean   :22181   Mean   :0.483881   Mean   :0.5161
15 3rd Qu.:21948   3rd Qu.:0.879087   3rd Qu.:0.8484
16 Max.   :84525   Max.   :1.000000   Max.   :1.0000
17 > summary(as.data.frame(unconditionals[["pi_values"]]))
18      class_a      class_b
19 Min.   :0.3924   Min.   :0.4140
20 1st Qu.:0.4467   1st Qu.:0.4140
21 Median :0.4467   Median :0.5533
22 Mean   :0.4839   Mean   :0.5161
23 3rd Qu.:0.5860   3rd Qu.:0.5533
24 Max.   :0.5860   Max.   :0.6076
25 > vtt_conditional=conditionals[,2]*vtt_class_a+conditionals[,3]*vtt_class_b
26 > summary(vtt_unconditional)
27      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
28 0.4221 0.4544 0.4544 0.4765 0.5374 0.5374
29 > summary(vtt_conditional)
30      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
31 0.1885 0.2787 0.4153 0.4765 0.7118 0.7838
32 > ### Take first value of covariates for each person
33 > commute_n = apollo_firstRow(database$commute, apollo_inputs)
34 > car_availability_n = apollo_firstRow(database$car_availability, apollo_inputs)
35 > ### Compute posterior values for covariates
36 > post_commute=colSums(commute_n*conditionals)/colSums(conditionals)
37 > post_car_availability=colSums(car_availability_n*conditionals)/colSums(conditionals)
38 > post_commute
39      ID      X1      X2
40 0.2724300 0.2629843 0.3077379
41 > post_car_availability
42      ID      X1      X2
43 0.3685971 0.4465379 0.3154211
44 > post_commute[2:3]
45      X1      X2
46 0.2629843 0.3077379
47 > post_car_availability[2:3]
48      X1      X2
49 0.4465379 0.3154211

```

Figure 9.19: Running apollo_unconditionals and apollo_conditionals for latent class

9.16 Summary of results for multiple models

It is often useful to produce an output file combining the estimates from multiple models run on the same data. This is facilitated by the function `apollo_combineResults`. This function allows the user to combine the results from a number of models (which can be larger than 2) for which the outputs have all been saved in the same directory. The function is called as follows:

```
apollo_combineResults(combineResults_settings)
```

where the list `combineResults_settings` has the following contents:

`combineResults_settings`: user-controlled settings

estimateDigits: number of digits used for estimates (default set to 4).
modelNames: a vector of model names, e.g. `c("MNL_RP", "MNL_SP", "MNL_RP_SP")`. If this argument is not given, all models within the directory are used.
printClassical: if set to `TRUE`, the code will save classical standard errors as well as robust standard errors, computed using the sandwich estimator (cf. [Huber, 1967](#)). This setting then also affects the reporting of t-ratios and p-values (default is `FALSE`).
pDigits: number of digits used for p-values (default set to 2).
printPVal: if set to `TRUE`, p-values are saved (default is `FALSE`).
printT1: if set to `TRUE`, t-ratios against 1 are saved in addition to t-ratios against 0 (default is `FALSE`).
sortByDate: sort models by date of estimation. If `FALSE`, order is given by user or alphabetical if using all files (default is `TRUE`).
tDigits: number of digits used for t-ratios (default set to 2).

The function produces a *csv* file with the name `model_comparison_time` where **time** is a numerical value defined by the current date and time. The file contains for each model the name, the model description, the number of individuals and observations, the number of estimated parameters, as well as four model fit statistics, namely the final log-likelihood, the adjusted ρ^2 measure, the AIC and the BIC. Note that not all these measures will be reported for all models, e.g. ρ^2 is only calculated for discrete choice models. The actual model outputs are then included in a number of columns where this depends on the level of detail requested by the user as described above (e.g. including classical t-ratios).

The function can be called as `apollo_combineResults()`, i.e. without any arguments. In that case, the default settings are used for all arguments, and all model files within the directory are combined into the output. An example of how to call this function is included in [CNL.r](#), combining the MNL, NL and CNL results.

10

Debugging

Especially with advanced models, it is easy to make mistakes that lead to failures, most commonly in estimation. A first check is to ensure that all the pre-estimation parts of the code were run and that the memory was cleared before running the code. If that has been done, but the model still fails, then the issue is likely in `apollo_probabilities`. While *Apollo* does spot and report some errors, for others, the calculations will simply fail. This is simply a result of the fact that not all user and data errors can be anticipated by programmers. Debugging a model then becomes an important skill.

A good approach to debugging consists of following these three steps:

Step 1: Find the location of the problem

Step 2: Find out why it fails

Step 3: Solve the problem

To illustrate the process of debugging, [Hybrid_with_OL_bugged_example.r](#) is a version of the hybrid choice model with ordered measurement models that contains a bug. Figure 10.1 shows that the calculation of the initial log-likelihood fails for a large number of individuals. The first step is to identify the location of the failure. Unless *Apollo* reports an error message relating to specific coding errors, the source of the problem is usually to do with calculations leading to numerical problems, either probabilities at zero or not a real number. The function `apollo_probabilities` uses an argument `functionality` as input, which can take different values that are controlled by the functions that call `apollo_probabilities`. The value most useful for debugging is "output" as this produces the likelihood for individual model components as well as the overall model.

The first step is to directly call `apollo_probabilities` from the console, with `functionality="output"`. This is illustrated in Figure 10.2. The output from this process is a list, with one element per model component, as well as one for the overall model. Each of these elements is a vector with the contribution to the likelihood function for each individual for that model component. To get some initial insights, we look at the likelihood for the overall model for first 30 individuals. This clearly shows us that we obtain a likelihood that is zero for many individuals, which will then lead to a failure to calculate the initial log-likelihood.

```

1  ### Estimate model
2  > model = apollo_estimate(apollo_beta, apollo_fixed, apollo_probabilities, apollo_inputs)
3  Log-likelihood calculation fails at starting values!
4  Affected individuals:
5      ID  LL
6       1 -Inf
7       8 -Inf
8      11 -Inf
9      14 -Inf
10     22 -Inf
11     33 -Inf
12     ...
13    992 -Inf
14   1000 -Inf
15 Error in apollo_estimate(apollo_beta, apollo_fixed, apollo_probabilities, :
16   Log-likelihood calculation fails at values close to the starting values!

```

Figure 10.1: Example of failure during model estimation

```

1  ### Calculate likelihood at the individual level for all model components
2  > L=(apollo_probabilities(apollo_beta, apollo_inputs, functionality="output"))
3  ### Inspect values for first 30 individuals for likelihood for overall model
4  > L$model[1:30]
5
6      1      2      3      4      5      6      7
7  0.000000e+00 1.341288e-08 4.041379e-10 5.510659e-10 3.272463e-09 8.181131e-10 1.740158e-07
8      8      9     10     11     12     13     14
9  0.000000e+00 7.542543e-09 2.078465e-09 0.000000e+00 4.518188e-09 4.535531e-08 0.000000e+00
10     15     16     17     18     19     20     21
11 1.252506e-10 3.889364e-10 1.301222e-09 5.643655e-10 1.918924e-10 4.877235e-08 3.367860e-10
12     22     23     24     25     26     27     28
13 0.000000e+00 1.321571e-09 1.889289e-08 2.140313e-09 1.779905e-09 9.902113e-10 5.407789e-09
14     29     30
15 1.666442e-07 3.595164e-09

```

Figure 10.2: Debugging step 1: testing with `functionality="output"`

Our model in this example is a multi-component structure, and it is useful to first understand whether the failures are limited to some (or maybe only one) of the model components. For this, we look at the summaries of the likelihoods for individual components, as shown in Figure 10.3. In a single component model, this step would be skipped. We can clearly see that in our case, the only model component in which the likelihood takes on a value of zero for some individuals is the measurement model for the fourth attitudinal statement, i.e. the model component called `indic_dominance`.

The final step in debugging is to now investigate who the individuals are that are affected by this failure, and whether their choices differ in a systematic way from those of other individuals. This is illustrated in Figure 10.4. We first retrieve the IDs for those individuals where the likelihood for the overall model fails, before looking at the dependent variable for those individuals and for any affected model components. In our case, the only affected model component is one of the measurement model for attitudinal questions, and for this, we only need to retain the first row for each individual (given that these answers are repeated across the rows for different choice tasks). We clearly see that all concerned individuals always give level 5 for the final indicator, and this provides us with the clue to the source of the problem.

```

1  ### Understand which model component is affected by failures
2  > summary(L$indic_quality)
3      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
4      0.1458 0.1487 0.1562 0.2481 0.3931 0.3962
5  > summary(L$indic_ingredients)
6      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
7      0.1447 0.1485 0.1559 0.2416 0.3929 0.3962
8  > summary(L$indic_patent)
9      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
10     0.1454 0.1485 0.1559 0.2385 0.3929 0.3962
11 > summary(L$indic_dominance)
12      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
13     0.0000 0.1482 0.3042 0.2604 0.3934 0.3962
14 > summary(L$choice)
15      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
16  5.107e-07 6.018e-07 9.501e-07 3.961e-06 2.083e-06 3.459e-05

```

Figure 10.3: Debugging step 1, part 2: likelihood for individual model components

The measurement model that was used gave a probability of zero for any observation where the highest level (5) was chosen, meaning that the starting value used for the upper threshold was too large (it was erroneously set to 50 while for all other indicators, a value of 2 was used).

[illegible]

Figure 10.4: Debugging step 2: analysing choices for respondents with zero likelihood value

In this section, we have looked at a very specific case of failure, where this was a clear user error. There are many reasons why a model could fail, and in other cases, it could be due to the data, or the size of the model. The majority of issues manifest themselves through zero (or negative) probabilities, either for some observations or some individuals, leading to a failure to calculate the log-likelihood. There are three broad cases where this happens:

- The zero/negative probabilities could arise for some individual observations, where this becomes clear by looking at e.g. the output of `apollo_mnl`, where probabilities are at the observation level. This may be related to data or starting values.
- The zero/negative probabilities could also arise at the level of individuals, where they may arise only after calling `apollo_panelProd`, where many small numbers will get multiplied together, potentially leading to a product of zero. This issue can often be resolved by using `workInLogs=TRUE` in `apollo_control`, which works with the sum of log-probabilities, rather than the product of probabilities.
- Finally, in models with multiple components (e.g. hybrid choice), problems may apply to some models only, as seen in our example, but the product across models that all produce probabilities that are greater than 0 may similarly lead to problems in case of many components, where `workInLogs=TRUE` may again help.

11

Frequently asked questions

This chapter addresses some frequently asked questions (FAQ), divided into some broad categories. Many other questions by individual users have been answered in the online forum (access via www.apollochoicemodelling.com), and readers are invited to check existing posts there before raising a new question. In addition, users are again encouraged to make use of the help files provides for specific functions directly in R, using e.g.

```
?apollo_mnl
```

Finally, many common failures arise due to users not having the latest version of *Apollo* installed on their system. Users are encouraged to check for updated versions of the package every few months. Updates, when available, can be acquired by simply re-installing the package, i.e. running:

```
install.packages("apollo")
```

11.1 General

What is *Apollo*?

Apollo is a software package for the R programming language. It is a set of tools to aid with the estimation and application of choice models in R. Users are able to write their own model functions or use a mix of already available ones. Random heterogeneity, both continuous and discrete and at the level of individuals and observations, can be incorporated for all models. There is support for both standalone models and hybrid model structures. Both classical and Bayesian estimation is available, and multiple discrete continuous models are covered in addition to discrete choice. Multi-threading processing is supported for estimation. A large number of pre and post-estimation routines, including for computing posterior (individual-level) distributions, are available. For examples, a manual, and a support forum, visit www.ApolloChoiceModelling.com. For more information on choice models see [Train \(2009\)](#) and [Hess and Daly \(2024\)](#) for an overview of the field.

11.2 Installation and updating of *Apollo*

How do I install *Apollo*?

Type `install.packages("apollo")` in the R console and press enter.

How do I update *Apollo*?

You update *Apollo* by re-installing it. Type `install.packages("apollo")` in the R console and press enter.

Why do I get an error during installation that one of the packages is not available for the R version I am running?

You are likely running an old version of R. Update R to the latest version and re-install *Apollo*. You can get the latest version of R from <https://cran.r-project.org/>

Why does installation work on my home laptop/desktop computer but fail in my company laptop/desktop computer

Often computers from big organisations will install R packages in shared libraries (that is in folders in the private company network). R does not like its libraries to be in shared folders. As a general recommendation, always install packages in local libraries, i.e. in a folder in the local hard drive. You can see your active libraries by typing `.libPaths()`. This will list the active libraries. If the local library is, for example, the second one in the list, you should keep only that one by typing `.libPaths(.libPaths()[2])`. Then you should try installing *Apollo* again.

11.3 Data

Why do I get the error message `Error in file(file, "rt") : cannot open the connection when trying to open the data?`

This is most often caused by the user forgetting to set the working directory meaning that R cannot find the data file. Or there could be a typo in the name of the data file.

Can I use “long” formatted data in *Apollo*?

No. *Apollo* requires data to be in a “wide” format, meaning that all necessary information to calculate the likelihood of a single observation should be contained in a single row of the data. In more practical terms, for an MNL model, this means that attributes for all alternatives should be contained in each row. This format is the more common format in choice modelling, uses less space, and is also more general in allowing for a mixture of different dependent variables in the same data.

Is there a way to transform data in “long” format into “wide” format?

Yes, *Apollo* makes available a function for this called `apollo_longToWide`.

Can I use a list of dummy variables to represent the choice? For example, for a choice between three alternatives, with the third chosen use variables `alt1=0`, `alt2=0`, `alt3=1`?

No. *Apollo* requires the user to encode the choice in a single variable. In this case, it would be a variable (for example called “choice”) that could take only three values (for example 1, 2 or 3). This is easily created in R on the basis of separate dummy variables.

Can *Apollo* estimate models with aggregate share data?

From version 0.2.7 onwards, the `apollo_fmnl` function can be used for estimating models on aggregate share data using a Fractional Multinomial Logit model.

Can I model “dual response” survey data with *Apollo*?

Yes, this is possible. We recommend beginning by modelling both questions separately,

and if there is evidence of the parameters being similar, then estimate them jointly using a scale parameter between them. See [BW_joint_model.r](#) to learn how to conduct joint estimation in *Apollo*.

11.4 Model specification

What distributions are possible for random coefficients in *Apollo*?

There are no limits imposed on distributional assumptions. The user can specify whatever distributions they want to use. Distributions can be coded as transformations of either Normal or Uniform draws. While transformations of Normal draws can be used for Lognormal, Censored or Truncated Normals and Johnson SB, Uniform distributions open up even broader scope as an inverse cumulative distribution function can be applied to the Uniform draws for a wide set of possible distributions.

How can I capture the panel structure of my data in *Apollo*?

The treatment of panel data depends completely on the model being used. Whenever the data contains multiple choices per individual, the analyst needs to use the function `apollo_panelProd` to group them together in estimation, except if using the setting `apollo_control$panelData=FALSE`, in which case the data will be treated as if all observations came from separate individuals. In models without any random heterogeneity, such as MNL, there is no explicit modelling of the correlation across choices for the same individual. All that will happen by using `apollo_panelProd` is that the calculation of the robust standard errors recognises that the choices come from the same individual. In models with random heterogeneity, such as Mixed Logit, the analyst can more explicitly account for the panel structure, for example by specifying that the heterogeneity in any random taste coefficients is across individuals, not within individuals, and/or by including an explicit pseudo-panel effect error component. These issues are discussed in detail in the manual.

How can I avoid writing each utility function separately if I have tens or hundreds of alternatives?

If the utilities all use the same structure but with different attributes for each alternative, then the utility functions (and availabilities) can be written iteratively. An example of this is provided in [MNL_iterative_coding.r](#)

What starting values should I use for the thresholds in my Ordered Logit-/Ordered Probit model?

The thresholds need to be different from each other, and monotonically increasing. If the thresholds are too wide, extreme ratings will obtain very low or zero probabilities. If the thresholds are too narrow, extreme ratings will obtain very large probabilities. Either of these can lead to estimation failures. Some trial and error is often required, but a good starting point is to have thresholds symmetrical around zero, going to extreme values of ± 3 .

How many draws should I use to estimate my models with random components?

There is no correct answer to this question. More draws is always better. The likelihood of the model is given by an integral without a closed form solution, and the simulation based approach only offers an approximation to this integral. Using a low number of draws means that the approximation to this integral is poor. In simple words, it means that the model we *are* estimating is not the one we *think* we are estimating. The parameter estimates will then be biased for the model we actually specified.

But the log-likelihood of my model is better with fewer draws, so isn't that good?

The fact that the log-likelihood is better with fewer draws does not justify the use of fewer draws. It is simply a reflection of the fact that fewer draws offers a poor approximation to the real model. Once the number of draws is increased to a sufficient number, the model fit will stay much more stable for further increases.

Why does my model converge with a low number of draws, but fails with a high number of draws?

The fact that the model does not converge with a high number of draws shows that there is a problem with the model. It is known that using a low number of draws can mean a model that is overspecified still converges and can give every impression of being identified ([Chiou and Walker, 2007](#)).

Can I at least use fewer draws if I use quasi-Monte Carlo draws?

In theory, yes, but again, more is better. Care is also required in deciding which type of draws to use. Halton draws are an excellent option for models with a low number of random parameters, but the colinearity issues with Halton draws means they should not be used with more than say 5 random components.

To keep estimation cost under control, can I use a low number of draws in my specification search and then reestimate the final model with a large number?

This is unfortunately rather common practice, but it is misguided to think this is a good approach or that it solves the issues arising with using low numbers of draws. The fact that low numbers are used during the specification search means that the approximation to the integral is poor at that stage. This in turn means that the decisions that lead to the final model specification may themselves be biased. While the final specification is then estimated robustly with a large number of draws, it may in fact be inferior to a specification that would have been obtained by using a high number throughout the specification search.

Does *Apollo* allow me to separate scale heterogeneity from preference heterogeneity?

The notion that it is possible to separate out scale heterogeneity from other heterogeneity is a myth, as discussed at length by [Hess and Train \(2017\)](#). Many models can allow for scale heterogeneity, but no model can separate it from preference heterogeneity, and there is no need to do so.

So can *Apollo* estimate the GMNL model?

The GMNL model is in fact not a new model or a more general model. It is simply a Mixed Logit model with a very particular set of constraints applied to it. It is not more general than Mixed Logit, which is the most general RUM model (cf. [McFadden and Train, 2000](#)). Given that *Apollo* allows full flexibility, users can of course specify the heterogeneity in a Mixed Logit model using GMNL-style constraints, but should be mindful when it comes to interpretation of the results given the above points.

So how about scale adjusted Latent Class (SALC)?

A SALC model is affected by the same issues as discussed by [Hess and Train \(2017\)](#) for GMNL. It is not possible to separate out scale heterogeneity from other heterogeneity. Users of *Apollo* can produce a SALC specification, which is simply a two layer Latent Class model, by using $S_1 \cdot S_2$ classes, allowing for S_1 sets of β parameters and S_2 sets of μ (scale) parameters, with an appropriate normalisation, and with the $S_1 \cdot S_2$ classes

using all combinations of β and μ scales model will be more general than a Latent Class model with S_1 classes with different β , but less general than a model with $S_1 \cdot S_2$ classes with different β .

11.5 Errors and failures during estimation

Why does *Apollo* complain that some function arguments are missing or incorrect?

There are different reasons for this, but the most likely cause is that the analyst has used the wrong order of arguments. For the predefined functions, the order of arguments passed to the function should be kept in the order specified for the function. For example, if a function is defined to take two inputs, namely `dependent` and `explanatory`, e.g. `model_prob(dependent,explanatory)`, and the user wants to use `choice` and `utility` as the inputs, then the function can be called as `model_prob(choice,utility)` but not as `model_prob(utility,choice)`. The latter change in order is only possible if the function is called explicitly as `model_prob(explanatory=utility,dependent=choice)`, which is the same as `model_prob(dependent=choice,explanatory=utility)`.

Why is my estimation failing with the message “Log-likelihood calculation fails at starting values”?

This happens when, at the starting values, the likelihood of the model is zero or cannot be calculated for at least some people in the data. *Apollo* will report the IDs of these individuals. Three common reasons exist for this problem:

Are the starting values feasible/appropriate for the model?

The most common reason for the initial likelihood calculation to fail is a problem with the values used in `apollo_beta`. The starting values of some parameters may be invalid. For example, the model may be dividing by a parameter with an initial value equal to zero. Also, different models have different requirements, for example the structural parameters in nested and Cross-nested Logit models should be different from zero; the α parameters in Cross-nested Logit models should be between zero and one; the γ and σ parameters in MDCEV should be greater than zero; the thresholds in Ordered Logit and Probit models should be different and monotonically increasing; the variance of linear regressions should be positive; etc. Another potential cause (less common than the previous one) is that starting values are too poor, leading to an initial likelihood too close to zero, which in turn leads to an infinite log-likelihood. To avoid this, the user should look for better starting values, either by estimating a simpler model and using those estimates as starting values, or using the `apollo_searchStart` function.

Does the data contain many observations for each person and/or does the model use several components (i.e. hybrid choice)?

When multiplying together the probability of many individual observations at the person level (using `apollo_panelProd`), it is possible that the product becomes too close to zero for R to store it as a number. The same can happen when combining many individual model components using `apollo_combineModels`. The risk of this is greater in case of models with low probabilities, such as in the case of large choicesets or many observations per individual. A solution to this problem is to set `workInLogs=TRUE` in `apollo_control`. This ensures that all calculations are made with the logarithms of probabilities, avoiding the issue of multiplying many small numbers. The use of this

setting is however only recommended when necessary as it will slow down estimation and also prevents the use of analytical gradients.

Does the model use lognormal distributions for random coefficients?

The value of a lognormally distributed coefficient is given by $\beta = \exp(\mu_{\ln(\beta)} + \sigma_{\ln(\beta)}\xi)$, or $\beta = -\exp(\mu_{\ln(-\beta)} + \sigma_{\ln(-\beta)}\xi)$ in the case of a negative lognormal distribution. The estimated parameters thus relate to the mean and standard deviation of the logarithm of β (or the logarithm of $-\beta$). A common mistake is to start $\mu_{\ln(\beta)}$ at zero, just as in the case of a normally distributed β . With the exponential, this will lead to a large starting value for β , which can result in numerical problems. In the case of lognormally distributed coefficients, it is thus advisable to use a large negative value for the starting value of the mean of the logarithm of the coefficient, e.g. starting $\mu_{\ln(\beta)}$ at something like -3 or lower, as this would imply starting the median of β close to zero.

Why do I get an error saying that one of my parameters does not influence the likelihood, even though I am using it in a utility function?

There may be several reasons for this. A common mistake is that, when writing utilities (or any other code statement) across multiple lines, the link between lines is missing and only the first one is considered. To split a statement across multiple lines, the incomplete lines should finish with an operator. For example:

```
U[["A"]] = b0 + b1*x1
+ b2*x2
```

will ignore the effect of $b2*x2$. Instead, it should be:

```
U[["A"]] = b0 + b1*x1 +
b2*x2
```

or

```
U[["A"]] = (b0 + b1*x1
+ b2*x2)
```

It could also be that the attribute associated with the parameter does not vary across the utility functions, or that the same constant is included in all utility functions. Much less common, it could be that the starting probabilities in your model are so small that due to rounding errors, they are equal to zero, and changes in parameter values also lead to small probabilities not different from zero. This is more likely to happen in complex models with many observations per individual. In this case, we recommend (1) to begin by estimating a simple model (e.g. constants only) to obtain better starting values, and (2) to set the option `apollo_control$workInLogs=TRUE`. This last option will increase numerical precision at the expense of estimation speed, and also prevents the use of analytical gradients.

Estimation of my model failed after a long time. Have I lost all the information?

In most cases, *Apollo* will produce a *csv* file with the parameter values at each iteration in the working directory, using the name given to the model inside `apollo_control$modelName`.

Why does my estimation fail, saying the maximum number of iterations has been reached?

The default number of iterations for estimation is set to 200. This can be increased in `estimate_settings$maxIterations`. In general however, if a model has not converged after 200 iterations, this could be a sign of problems with the model. Inspecting the iterations file produced during estimation can help diagnose if there is a problem or if

more iterations are required.

11.6 Model results

Why am I getting Inf or NaN for standard errors?

There are several main reasons why this could happen:

The model could have theoretical identification issues

To diagnose these issues is not easy, as requirements change depending on the particular structure of the model. For example, in random utility models, only differences in utility matters, so the constant of at least one alternative must be fixed to zero. Similarly, a normalisation is required for categorical variables. In Hybrid choice models, the variance of each structural equation (or the slope of one measurement equation per latent variable) should be fixed to one.

The model could be too complex for the data, leading to empirical identification issues

Many users fall into the trap of believing that choice models are easy tools and that the most complex model should always be used. Instead, analyst should always begin by estimating the simplest possible model and moving progressively towards more complex formulations. This will help troubleshooting any potential identification problems. In Mixed Logit for example, analyst should always start by introducing only a few random coefficients, leaving the rest as fixed, and progressively making the model more general.

The calculation of numerical derivatives could lead to some zero probabilities

Especially with complex models, the calculation of the numerical derivatives can be affected by a small number of calculations leading to zero probabilities. Greater stability can in this case be obtained by using bootstrapping for estimating the standard errors (i.e. setting `estimate_settings$bootstrapSE=TRUE`), obviously at the cost of increased estimation time. Similarly, depending on the data used, setting `workInLogs=TRUE` in `apollo_control` could help, but at the expense of estimation speed, and it also prevents the use of analytical gradients.

Why is my estimate of the standard deviation negative?

This happens if a random coefficient is coded as `randCoeff[["beta"]]=mu+sigma*draws` where `draws` is a random variate that is symmetrical around zero.

So should I constrain the standard deviation to be positive?

There is no reason for doing so. The results will be the same if the random variate `draws` is symmetrical around zero. Imposing constraints will also make estimation harder. And of course, if a user wants to allow for correlation between individual coefficients, then the parameters multiplying the draws need to be able to be positive or negative.

Why are my structural/nesting parameters greater than one or smaller than zero in Nested or Cross-nested Logit?

Apollo does not automatically constrain the structural parameters in Nested (NL) and Cross-nested (CNL) Logit models to be between zero and one. If, after estimation, the structural parameters are outside of this interval, this could be evidence of the nesting structure not being supported by the data. The user should then try a different nesting

structure.

So should I constrain them to be between 0 and 1?

In general, we do not recommend imposing constraints. If unconstrained estimation yields an estimate outside the bounds of the interval of acceptable values, then it is highly likely that the use of constraints will lead to an estimate that goes to one of the bounds of the interval that the parameter is constrained in. The model fit will be inferior too and the real problem will simply be masked by the constraints.

So how about constraints on other parameters, such as standard deviations or γ parameters in MDCEV?

It is common practice to use exponential transforms for parameters that are only allowed to be positive, e.g. using $\gamma = \exp(\gamma_0)$, with γ_0 being estimated. We have found that this often slows down estimation and does not necessarily lead to the same solution as unconstrained estimation even if the latter finds an acceptable solution.

The reason for the problem is that small changes to γ_0 will lead to large changes in γ , making estimation difficult, especially with numerical derivatives.

How do I calculate hit rates for my model in *Apollo*?

We made the decision not to include hit rates in *Apollo* outputs. They really offer a very distorted view of results. Models give probabilities in prediction. If, for each task, an analyst just looks at what alternative has the highest probability, then they are ignoring the error term in the model. To put it succinctly, imagine you have a case with 2 alternatives, and we have 2 models. Model A gives a probability of 51% for the chosen alternative in 70% of cases in model 1, but a probability of only 10% in the remaining 30% of cases. Model B gives a probability of 49% for the chosen alternative in 70% of cases in model 1, but a probability of 90% in the remaining 30% of cases. Using a hit rate would give model A a figure of 70%, and model B a figure of 30%. But clearly model B is far superior. This is why choice modellers work with probabilities of correct prediction instead if they want a percent measure like this. That would give 0.387 for model A but 0.613 for model B.



Apollo versions: timeline, changes and backwards compatibility

Version 0.0.6 (13 March 2019)

This is the first fully functioning release of *Apollo*.

Version 0.0.7 (8 May 2019)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Inputs changed for `apollo_choiceAnalysis`

Functions affected: `apollo_choiceAnalysis`

Detailed description: inputs changed so function can be called prior to

`apollo_validateInputs`

Backwards compatibility of code: function call changed from version 0.0.7 onwards

Constraints for classical estimation

Functions affected: `apollo_estimate`

Detailed description: *Apollo* now allows the user to include a list called `constraints` in `estimate_settings` for use with BFGS for classical model estimation.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Scaling of parameters during model estimation

Functions affected: `apollo_estimate`

Detailed description: scaling of model parameters can be used during estimation

Backwards compatibility of code: no backwards compatibility issues for existing functions

Validation output

Functions affected: `apollo_estimate`

Detailed description: *Apollo* no longer reports that all pre-estimation checks were passed for a model component and instead only reports if there are an issues.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Bayesian estimation produces `model$estimate`

Functions affected: `apollo_estimate`, `apollo_prediction`, `apollo_llCalc`

Detailed description: until version 0.0.6, Bayesian estimation in *Apollo* did not produce a `model$estimate` output. We have retained the various existing outputs, but in addition, `model$estimate` is now produced, combining non-random parameters with individual specific posteriors for random parameters. This now allows the user to use `apollo_prediction` and `apollo_llCalc` on such outputs, where care is of course required in interpretation of outputs based on posterior means.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.0.8 (9 September 2019)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Bootstrap estimation added

Functions affected: `apollo_bootstrap`, `apollo_estimate`

Detailed description: the user can now perform bootstrap estimation. This can also be called directly with `apollo_estimate` during estimation

Backwards compatibility of code: new function from version 0.0.8 onwards, new optional arguments for `apollo_estimate`, but function called in the same way

Allow user to use subset of rows for analysis of choices

Functions affected: `apollo_choiceAnalysis`

Detailed description: An additional `rows` argument can be entered into `choiceAnalysis_settings`.

Backwards compatibility of code: optional argument added from version 0.0.8 onwards, but function called in the same way

Outputs changed for `apollo_choiceAnalysis`

Functions affected: `apollo_choiceAnalysis`

Detailed description: outputs changed so that t-test value is reported instead of p-value, and order of outputs is changed

Backwards compatibility of code: outputs changed from version 0.0.8 onwards, but function called in the same way

Allow user to change name and location of outside good in MDCEV and MDCNEV

Functions affected: `apollo_mdcev` and `apollo_mdcnev`

Detailed description: An additional `outside` argument can be entered into `mdcev_settings` and `mdcnev_settings` with the name of the outside good which can now differ from `outside`. It also no longer needs to be in first position in the list of alternatives.

Backwards compatibility of code: optional argument added from version 0.0.8 onwards, but function called in the same way

No need to define superfluous γ for outside good in MDCEV and MDCNEV

Functions affected: `apollo_mdcev` and `apollo_mdcnev`

Detailed description: The user no longer needs to create a `gamma` term for the outside good.

Backwards compatibility of code: function called in the same way

Chosen unavailable alternatives have a likelihood of zero

Functions affected: `apollo_mnl`, `apollo_mdcev`, `apollo_mdcnev`

Detailed description: MNL, MDCEV and MDCNEV now return a likelihood equal to zero for chosen alternatives that are not available. This change is only relevant if `apollo_control$noValidation` is TRUE.

Backwards compatibility of code: new likelihood values for unavailable chosen alternatives on MNL, MDCEV and MDCNEV models from version 0.0.8 onwards

Allow user to specify number of outliers to report

Functions affected: `apollo_modelOutput`, `apollo_saveOutput`

Detailed description: In addition to specifying TRUE/FALSE for `printOutliers`, the user can provide the number of outliers to report (instead of the default of 20).

Backwards compatibility of code: optional argument added from version 0.0.8 onwards, but function called in the same way

Ability to define estimation/validation subsets for `apollo_outOfSample`

Functions affected: `apollo_outOfSample`

Detailed description: the user can now provide a matrix or `data.frame` describing which observations are to be used in the estimation and validation subsets

Backwards compatibility of code: optional argument added from version 0.0.8 onwards, but function called in the same way

Individual IDs and choice scenario numbers added in predictions

Functions affected: `apollo_prediction`

Detailed description: The output from `apollo_prediction` now includes the IDs and choice observation numbers as the first two columns.

Backwards compatibility of code: function called in the same way

Version 0.0.9 (23 October 2019)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Additional diagnostic message for HB estimation

Functions affected: `apollo_estimate`

Detailed description: the `RSGHB` package used for Bayesian estimation left censors likelihood values at the individual level to avoid numerical issues. This has the undesired side effect of mis-specified models still running, and a warning message is now displayed when censoring has been used

Backwards compatibility of code: function called in the same way

Pre-estimation tests to ensure all parameters affect likelihood function

Functions affected: `apollo_estimate`

Detailed description: unless `apollo_control$noDiagnostics==TRUE`, a pre-estimation check is used to ensure that there are no parameters in `apollo_beta` for which changes do not lead to changes in the model likelihood

Backwards compatibility of code: function called in the same way

Version 0.1.0 (16 March 2020)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

R version requirement changed to 3.6.0

Functions affected: all of *Apollo*

Detailed description: users running *Apollo* version 0.1.0 require at a minimum R version 3.6.0

Backwards compatibility of code: no changes to actual functions

Improved error reporting and printing

Functions affected: majority of functions

Detailed description: warnings are now generally displayed at the point they apply rather than after estimation. In addition, numerous new checks have been implemented for many of the functions.

Backwards compatibility of code: no backwards compatibility issues

Referring to database inside `apollo_probabilities` no longer allowed

Functions affected: all functions that call `apollo_probabilities`

Detailed description: the user is not allowed to refer to `database` by name inside `apollo_probabilities`. There is no reason for doing so as the `database` is attached and all elements therein can be referred to directly

Backwards compatibility of code: this should not affect any users. The only requirement is to now use the `get` function when attempting to retrieve an object whose name is put together via `paste0`

Can define names for individual model components

Functions affected: all existing functions for models, i.e. `apollo_mnl` etc

Detailed description: the user can now include an optional additional argument `componentName` in the settings for individual models. This is then used in the reporting of outputs as well as in any error messages

Backwards compatibility of code: no backwards compatibility issues

Tests for zero probabilities at starting values for individual model components

Functions affected: all existing functions for models, i.e. `apollo_mnl` etc

Detailed description: unless `apollo_control$noValidation==TRUE`, *Apollo* now checks the likelihood of individual model components in addition to the overall model and prints a warning if probabilities are zero for some individuals while estimation is not started if probabilities are zero for all. This is relevant for latent class models, where it is permissible to have zero probabilities for some individuals in some classes, but where initial zero probabilities for all individuals in a class are likely to highlight problems.

Backwards compatibility of code: no backwards compatibility issues

Ability to sort results by date

Functions affected: `apollo_combineResults`

Detailed description: an additional option `sortByDate` has been included. When set to

TRUE, the models in the summary file will be sorted by the date when the model was estimated (default set to TRUE)

Backwards compatibility of code: no backwards compatibility issues

Improved memory usage with multi-core estimation

Functions affected: `apollo_estimate`

Detailed description: Memory requirements for multi-core estimation have been reduced substantially compared to previous versions

Backwards compatibility of code: no backwards compatibility issues

Constraints in HB estimation can now use names of parameters

Functions affected: `apollo_estimate` with `apollo_control$HB==TRUE`

Detailed description: the user can now use names of parameters when creating constraints for HB estimation rather than relying on the numeric coding from RSGHB

Backwards compatibility of code: no backwards compatibility issues for existing functions as old format for input still permitted

Smallest absolute eigenvalue of Hessian reported

Functions affected: `apollo_estimate`, `apollo_modelOutput`, `apollo_saveOutput`

Detailed description: *Apollo* reports the eigenvalue of the Hessian that is closest to zero. Small values can indicate convergence issues. A special warning message is displayed if some of the eigenvalues are positive.

Backwards compatibility of code: no backwards compatibility issues

Check whether class allocation probabilities for latent class sum to 1

Functions affected: `apollo_lc`

Detailed description: New check to ensure that class allocation probabilities for latent class sum to 1

Backwards compatibility of code: no backwards compatibility issues

Calculation of $LL(0)$

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: The calculation of the log-likelihood at zero has been improved for some models, as has the reporting of it. Where this measure does not apply, *Apollo* now reports “Not applicable” instead of “NA”

Backwards compatibility of code: no backwards compatibility issues

Ordered probit added

Functions affected: `apollo_op`

Detailed description: the user can now use ordered probit models via the function `apollo_op`

Backwards compatibility of code: no backwards compatibility issues

Out of sample testing can add to existing runs

Functions affected: `apollo_outOfSample`

Detailed description: `apollo_outOfSample` checks whether output files already exists and adds to those if the number of runs requested is larger than what is already stored in

these files

Backwards compatibility of code: no backwards compatibility issues

Prevent use of some functions for HB models

Functions affected: `apollo_panelProd`, `apollo_avgInterdraws`, `apollo_avgIntraDraws`, `apollo_deltaMethod`, `apollo_conditionals`, `apollo_unconditionals`, `apollo_lcConditionals`, `apollo_lcUnconditionals`

Detailed description: Many of the *Apollo* functions are for classical estimation only, and their use previously led to failures. Their use is now prevented when `apollo_control$HB==TRUE`.

Backwards compatibility of code: the call to these functions was previously ignored and will now lead to a failure in any files they are still included in. The appropriate lines need commenting out.

Inputs changed for `apollo_prediction` and added ability to calculate standard errors

Functions affected: `apollo_prediction`

Detailed description: the `apollo_prediction` function now takes `prediction_settings` as an input, where this is the new location for including `modelComponent`. In addition, an optional setting called `runs` has been included that computes standard errors across multiple prediction runs based on different draws from the estimates and covariance matrix for the model parameters

Backwards compatibility of code: no backwards compatibility issues for existing functions as old format for input still permitted

Output files no longer overwritten

Functions affected: `apollo_saveOutputs`

Detailed description: the `apollo_saveOutput` function now checks whether output files for the model already exists and changes their names (by including `OLD` in the name) rather than overwriting them

Backwards compatibility of code: no backwards compatibility issues

Output file for starting value search simplified

Functions affected: `apollo_searchStart`

Detailed description: The output file for `apollo_searchStart` was simplified from v0.0.9 to v0.1.0. Now it only records starting candidate values, and their loglikelihoods throughout the stages, but not their values at each stage, as it used to.

Backwards compatibility of code: no backwards compatibility issues

Subsetting of data when some variables are factors

Functions affected: all functions making use of the `database`

Detailed description: If a dataset contains **factors**, the use of subsetting of the data in **R** would still retain levels that no longer apply. This is a feature of **R** which can have unintended consequences and we thus eliminate any missing levels.

Backwards compatibility of code: no backwards compatibility issues

Version 0.1.1 (unreleased version, 15 September 2020)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Additional checks on draws

Functions affected: all functions with mixing

Detailed description: Apollo now ensures that intra-individual draws are only used in the presence of multiple observations per individual.

Backwards compatibility of code: no backwards compatibility issues

Additional checks for exploded logit

Functions affected: `apollo_el`

Detailed description: The `apollo_el` now ensures that no alternative is chosen more than once across stages and ensures that any obsolete stages have the choice variable coded as `-1`.

Backwards compatibility of code: no backwards compatibility issues

Parameter specific constraints in HB estimation can now use names of parameters

Functions affected: `apollo_estimate` with `apollo_control$HB==TRUE`

Detailed description: the user can now use names of parameters when creating constraints for individual parameters in HB estimation rather than relying on the numeric coding from `RSGHB`

Backwards compatibility of code: no backwards compatibility issues for existing functions as old format for input still permitted

Additional flexibility for thresholds in ordered logit and ordered probit

Functions affected: `apollo_ol` and `apollo_op`

Detailed description: The thresholds for ordered logit and ordered probit are now given as a list, with one entry per threshold, thus allowing the user to have thresholds varying across observations/individuals, with possible additional random heterogeneity.

Backwards compatibility of code: no backwards compatibility issues as the function can still be called with the thresholds as a simple vector

Check for use of reserved names

Functions affected: `apollo_validateInputs`

Detailed description: For internal consistency, some core names, such as `alternatives`, `avail` and `sigma` are now protected and cannot be used in the data, `apollo_beta`, `apollo_lcPars` and `apollo_randCoeff`.

Backwards compatibility of code: backwards compatibility for some model files, but can be easily addressed by changing names

Version 0.2.0 (19 October 2020)

General

Various bug fixes, major improvements to efficiency, stability and reporting of user errors.

Ben-Akiva & Swait test added

Functions affected: `apollo_basTest`

Detailed description: the user can now perform the Ben-Akiva & Swait test

Backwards compatibility of code: new function from version 0.2.0 onwards

New outputs for bootstrap, and changes to storage of repetitions

Functions affected: `apollo_bootstrap`

Detailed description: `apollo_bootstrap` now returns a list which contains two components, namely a list of estimated parameters and likelihood function in each repetition, and the covariance matrix. In addition, when called an additional time, the function will always add new repetitions to old files, if they exist (it won't "complete" the requested number).

Backwards compatibility of code: format of output has changed for post-processing
backwards compatibility issues

`apollo_inputs` is no longer automatically updated when calling post-processing functions

Functions affected: `apollo_conditionals`, `apollo_lcConditionals`,
`apollo_lcUnconditionals`, `apollo_llCalc`, `apollo_prediction`,
`apollo_unconditionals`

Detailed description: In previous versions, `apollo_inputs` would be updated when calling post-processing functions. Now *Apollo* instead checks if any inputs have changed and alerts the user to revalidate `apollo_inputs`.

Backwards compatibility of code: examples where changes to the data were made post estimation require a call to `apollo_validateInputs`

Model description added to output of `apollo_combineResults`

Functions affected: `apollo_combineResults`

Detailed description: The csv file produced by `apollo_combineResults` now includes the model description defined in `apollo_control` by the user

Backwards compatibility of code: Output changed, but no backwards compatibility issues

Estimation time split into separate components in output

Functions affected: `apollo_modelOutput`, whether using `apollo_estimate` or `apollo_estimateHB`

Detailed description: The time taken by `apollo_modelOutput` and `apollo_estimate` is now split into pre-processing, estimation, and post-processing time

Backwards compatibility of code: Output changed, but no backwards compatibility issues

`apollo_fitsTest` uses log-likelihood instead of probabilities

Functions affected: `apollo_fitsTest`

Detailed description: In previous versions of *Apollo*, the function `apollo_fitsTest` used

the probability of correct prediction, which was not appropriate for non-discrete choice models and also not for multi-component models. The function now uses the log-likelihood instead. The function also only does this for the overall model, i.e. no longer allowing the user to look separately at subcomponents of a joint model.
Backwards compatibility of code: Output changed.

ID included in outputs of `apollo_lcConditionals`

Functions affected: `apollo_lcConditionals`

Detailed description: The individual-specific value for `apollo_control$indivID` is now included as the first column of the output of `apollo_lcConditionals`

Backwards compatibility of code: Output changed.

New function for EM algorithm for latent class models

Functions affected: `apollo_lcEM`

Detailed description: Function for automatically using the EM algorithm for latent class models rather than requiring the user to code this for a specific model

Backwards compatibility of code: New function. Users interested in how to code this manually are referred to earlier manuals.

New function for EM algorithm for mixed logit

Functions affected: `apollo_mixEM`

Detailed description: Function for automatically using the EM algorithm for mixed logit models with a full covariance matrix and all parameters being random

Backwards compatibility of code: New function. Users interested in how to code this manually are referred to earlier manuals.

New input options for likelihood ratio test, and enhanced output

Functions affected: `apollo_lrTest`

Detailed description: *Apollo* now allows the user to apply a likelihood ratio test using outputs from two models that are stored in memory (rather than one model needing to have been saved to disk). The restricted model also no longer needs to be given first, as *Apollo* will determine the order of the two models. The `apollo_llTest` function now also prints the likelihoods of each model as well as their difference.

Backwards compatibility of code: No backwards compatibility issues.

User can specify number of replications to use in MDCEV forecasting

Functions affected: `apollo_mdcev` and `apollo_mdcev`

Detailed description: The user can specify the number of replications using the optional argument `nRep`

Backwards compatibility of code: No backwards compatibility issues.

New optional settings for model output

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: The setting `printPVal` can now take three values: 0 (no p-val), 1 (one-sided), or 2 (two-sided). The setting `printDiagnostics` was split into two settings: `printDataReport` (logical) to print the summary of the dependant variable, and `printModelStructure` (logical) to print data on nesting structures. A new setting

`printFunctions` is included to print a copy of `apollo_probabilities`, `apollo_control`, scaling (for estimation and Hessian), Hessian routines attempted, `apollo_lcPars` and `apollo_randCoeff`.

Backwards compatibility of code: No backwards compatibility issues as old input still accepted and new inputs are optional.

Changes to storage of repetitions for out of sample prediction

Functions affected: `apollo_outOfSample`

Detailed description: When called an additional time `apollo_outOfSample` will always add new repetitions to old files, if they exist (it won't "complete" the requested number).

Backwards compatibility of code: format of output has changed for post-processing
no backwards compatibility issues

Improved output for predictions

Functions affected: `apollo_prediction`

Detailed description: A summary of the predictions are printed in a nice way, with special handling of MDCEV models. Automatic creation of confidence intervals when using repeated sampling.

Backwards compatibility of code: Output changed

Data no longer gets sorted by ID

Functions affected: `apollo_validateData`

Detailed description: *Apollo* no longer sorts the data by `apollo_control$ID`. Sorting is not required by the user either, but all observations from same individual need to be contiguous in the data.

Backwards compatibility of code: *Apollo* will now ask the user to group together data for the same individual. Some older models may thus require adjustment to the data.

Allow working in logs for cross-sectional data

Functions affected: `apollo_validateInputs`

Detailed description: Previous versions of *Apollo* allowed the use of `apollo_control$workInLogs` only with panel data.

Backwards compatibility of code: No backwards compatibility issues.

Changes to reserved names

Functions affected: `apollo_validateInputs`

Detailed description: The restrictions imposed in version 0.1.1 for reserved names have been relaxed, and replaced by a check that no names from `apollo_beta`, `apollo_randCoeff`, `database`, `apollo_draws` or `apollo_lcPars` are redefined elsewhere.

Backwards compatibility of code: Possible changes needed to old model files.

Ability to use a subset of model components in `apollo_combineModels`

Functions affected: `apollo_combineModels`

Detailed description: The user can now specify a subset of model components to multiply.

Backwards compatibility of code: New optional argument.

Version 0.2.1 (28 October 2020)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Version 0.2.2 (5 December 2020)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

New options for shares test added

Functions affected: `apollo_sharesTest`

Detailed description: the user can now create pseudo-alternatives, grouping together existing alternatives

Backwards compatibility of code: new optional argument, no backwards compatibility issues

User can omit avail when all alternatives are available

Functions affected: `apollo_mnl`, `apollo_nl`, `apollo_cnl`, `apollo_dft`, `apollo_el`, `apollo_mdcev`, `apollo_mdcev`

Detailed description: the user can now omit the `avail` argument if all alternatives are available

Backwards compatibility of code: optional, no backwards compatibility issues

Improved counting of modelled outcomes

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: the output now counts modelled outcomes as opposed to rows in the data, which leads to different statistics for multi-component models

Backwards compatibility of code: outputs changed, with resulting changes in goodness-of-fit statistics

Optionally drop fixed parameters from output

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: the user can now optionally exclude fixed parameters from the output

Backwards compatibility of code: outputs changed

Optionally return only the model component when using `apollo_combineModels`

Functions affected: `apollo_combineModels`

Detailed description: the user can now optionally exclude subcomponents when calling `apollo_combineModels`

Backwards compatibility of code: new optional argument, no backwards compatibility issues

Individual IDs and observation numbers included in output of `apollo_bootstrap`

Functions affected: `apollo_bootstrap`

Detailed description: `apollo_bootstrap` now writes the `indivID` and `apollo_seq` to its file output .

Backwards compatibility of code: Changes to output.

Ability to include constraints in classical estimation

Functions affected: `apollo_estimate`

Detailed description: `apollo_estimate` now has an additional optional argument for constraints.

Backwards compatibility of code: New capability.

Version 0.2.3 (20 January 2021)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Version 0.2.4 (25 February 2021)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Monte Carlo replications for MDCEV can now be set by user

Functions affected: `apollo_prediction` with `apollo_mdcev`, `apollo_mdcnev`

Detailed description: the user can now set the number of Monte Carlo replications to use when predicting from MDCEV models

Backwards compatibility of code: new optional argument, no backwards compatibility issues

MDCEV predictions now include expenditure

Functions affected: `apollo_prediction` with `apollo_mdcev`, `apollo_mdcnev`

Detailed description: the output from `apollo_prediction` for MDCEV models now includes the expenditure

Backwards compatibility of code: new output, no backwards compatibility issues

Predictions now returned as `data.frame` instead of matrix

Functions affected: `apollo_prediction`

Detailed description: the output from `apollo_prediction` is now a `data.frame` instead of a matrix, which helps avoid issues with non-numeric IDs

Backwards compatibility of code: new output format, some possible backwards compatibility issues with examples

`apollo_sharesTest` now returns output

Functions affected: `apollo_sharesTest`

Detailed description: the output from `apollo_prediction` is now not just printed to screen, but also returned

Backwards compatibility of code: new output format, no backwards compatibility issues

Version 0.2.5 (31 July 2021)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

User can specify number of replications to use in predictions from MDCEV

Functions affected: `apollo_prediction` with `apollo_mdcev` and `apollo_mdcnev`

Detailed description: The Pinjari & Bhat forecasting algorithm uses replications, and the user can now provide this as an additional settings named `rep` in `prediction_settings`.

Backwards compatibility of code: new capability, no backwards compatibility issues

More detailed outputs for MDCEV predictions

Functions affected: `apollo_prediction` with `apollo_mdcev` and `apollo_mdcnev`

Detailed description: The prediction from MDCEV models now returns the mean and standard deviations of continuous demand (`cont_mean`, `cont_sd`), discrete demand (`disc_mean`, `disc_sd`), and expenditure (`expe_mean`, `exp_sd`).

Backwards compatibility of code: new output format, no backwards compatibility issues

Predictions with multiple component components

Functions affected: `apollo_prediction` with `apollo_combineModels`

Detailed description: `apollo_prediction` with multiple model components now returns a list of data.frame, one for each model component. Model components without prediction capabilities are omitted from the return. If `runs>1`, then a data.frame with the point estimates are given AND a 3-dim array with the repetitions as well (the array does not have the IDs)

Backwards compatibility of code: new output format, some ancillary code may need to be updated

Output directory can be set by the user

Functions affected: all functions reading and writing to files

Detailed description: the user can specify an optional argument `outputDirectory` in `apollo_control` which will be used for outputs.

Backwards compatibility of code: new capability, no backwards compatibility issues

Output returned by `apollo_sharesTest`

Functions affected: `apollo_sharesTest`

Detailed description: In addition to screen output, `apollo_sharesTest` also invisibly returns the output so it can be saved into a data.frame

Backwards compatibility of code: new capability, no backwards compatibility issues

Version 0.2.6 (8 November 2021)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Non-BFGS algorithms cannow also produce file with iterations

Functions affected: `apollo_estimate`

Detailed description: Non-BFGS algorithms can now also write iterations to file.

Backwards compatibility of code: new capability, no backwards compatibility issues

User can disable calculation of log-likelihood at constants

Functions affected: `apollo_control` settings

Detailed description: The user can now specify whether the log-likelihood at constants should be calculated, using the setting `calculateLLC` in `apollo_control`. This is set to TRUE by default

Backwards compatibility of code: new capability, no backwards compatibility issues

New function for latent class allocation model

Functions affected: `apollo_classAlloc`

Detailed description: A new function `apollo_classAlloc` has been included to compute class allocation probabilities for MNL

Backwards compatibility of code: updates to syntax, but old approach still works too

Syntax uses utilities instead of V

Functions affected: all model functions using utilities

Detailed description: The user can now use `utilities` inside the model settings instead of `V`

Backwards compatibility of code: old syntax still works too

ρ^2 at constants also reported

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: ρ^2 measures are now reported against a model with constants only, in addition to against a model with equal shares

Backwards compatibility of code: new output, no backwards compatibility issues

Average prediction added to output of `apollo_prediction` for discrete choice models

Functions affected: `apollo_prediction`

Detailed description: the use of `apollo_prediction` with discrete choice models now leads to the reporting of average predictions in addition to aggregate predictions

Backwards compatibility of code: new output, no backwards compatibility issues

Version 0.2.7 (26 January 2022)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Fractional logit model added to *Apollo*

Functions affected: `apollo_fmnl`

Detailed description: The user can now use the Fractional Multinomial Logit (FMNL) model

Backwards compatibility of code: new capability, no backwards compatibility issues

`apollo_deltaMethod` now allows for user-defined functions

Functions affected: `apollo_deltaMethod`

Detailed description: The user can now create any function of parameters and evaluate its standard error using the Delta method, and do so for multiple functions at the same time

Backwards compatibility of code: old syntax still works too

Syntax now uses classes instead of alternatives in class allocation model

Functions affected: `apollo_classAlloc`

Detailed description: The user can now use `classes` inside the settings for `apollo_classAlloc` instead of `alternatives`

Backwards compatibility of code: old syntax still works too

Version 0.2.8 (8 September 2022)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Single function for conditionals

Functions affected: `apollo_conditionals`

Detailed description: The user can now use the function `apollo_conditionals` for continuous mixture models or latent class, no longer needing `apollo_lcConditionals`

Backwards compatibility of code: old syntax still works too

Single function for unconditionals

Functions affected: `apollo_unconditionals`

Detailed description: The user can now use the function `apollo_unconditionals` for continuous mixture models or latent class, no longer needing `apollo_lcUnconditionals`

Backwards compatibility of code: old syntax still works too

Changes to saved model object when using Bayesian estimation

Functions affected: `apollo_saveOutput` with `apollo_control$HB=TRUE`

Detailed description: *Apollo* no longer includes `cmcLLout` and `cmcRLHout` in the saved model object - these were rarely if ever used and took up substantial space. If a user requires them, they are available in the model object in memory after estimation, and can be saved to file.

Backwards compatibility of code: no backwards compatibility issues

Model inputs now also saved with HB

Functions affected: `apollo_modelOutput` and `apollo_saveOutput` with `apollo_control$HB=TRUE`

Detailed description: The setting `printFunctions` now also works with Bayesian estimation.

Backwards compatibility of code: No backwards compatibility issues as old input still accepted and new inputs are optional.

Model object saved before covariance matrix calculation

Functions affected: `apollo_estimate`

Detailed description: If a model takes longer than 10 minutes to estimate, the model object is written to disk prior to the covariance matrix calculation.

Backwards compatibility of code: No backwards compatibility issues.

Ability to report *p*-values in Delta method calculations

Functions affected: `apollo_deltaMethod`

Detailed description: A new setting `printPVal` is now available for `apollo_deltaMethod`.

Backwards compatibility of code: No backwards compatibility issues as this is an optional setting.

Version 0.2.9 (13 May 2023)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Dropping pre-estimation diagnostics with RSGHB

Functions affected: `apollo_estimate` with `apollo_control$HB=TRUE`

Detailed description: The RSGHB setting `nodiagnostics` is set to TRUE by default

Backwards compatibility of code: no backwards compatibility issues for existing functions

Mixture models no longer require `apollo_control$mixing=TRUE`

Functions affected: `apollo_validateInputs`

Detailed description: The setting `apollo_control$mixing` is now set to TRUE automatically when `apollo_draws` is present

Backwards compatibility of code: no backwards compatibility issues for existing functions

More detailed reporting of properties of Hessian matrix

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: Additional properties about the maximum likelihood estimate are reported

Backwards compatibility of code: no backwards compatibility issues for existing functions

New function for reshaping long data

Functions affected: `apollo_longToWide`

Detailed description: A new function was added to translate data from long format to wide format

Backwards compatibility of code: no backwards compatibility issues for existing functions

BHHH standard errors saved and printed

Functions affected: `apollo_estimate`, `apollo_saveOutput`

Detailed description: BHHH standard errors are now saved in the `model` object and are also printed to screen at the end of estimation, prior to the calculation of the asymptotic variance-covariance matrix

Backwards compatibility of code: no backwards compatibility issues for existing functions

New capability to calculate standard errors for all ratios and differences for all pairs of parameters

Functions affected: `apollo_deltaMethod`

Detailed description: A new setting can be used with `apollo_deltaMethod` to compute standard errors for ratios and differences of all pairs of parameters at the same time

Backwards compatibility of code: no backwards compatibility issues for existing functions

RMSE calculation added to `apollo_outOfSample`

Functions affected: `apollo_outOfSample`

Detailed description: RMSE calculations are now reported when calling `apollo_outOfSample`

Backwards compatibility of code: no backwards compatibility issues for existing functions

New function for Random Regret Minimisation

Functions affected: `apollo_rrm`

Detailed description: New function no longer requiring manual coding of RRM

Backwards compatibility of code: no backwards compatibility issues for existing functions

New function for Extended Multiple Discrete Continuous model

Functions affected: `apollo_emdc`

Detailed description: New function for the extended multiple discrete continuous model of [Palma and Hess \(2022\)](#)

Backwards compatibility of code: no backwards compatibility issues for existing functions

Extensive changes to outputs from HB estimation

Functions affected: `apollo_modelOutput` and `apollo_saveOutput` with `apollo_control$HB==TRUE`

Detailed description: Extensive changes have been made, as follows:

- RSGHB log file no longer printed but equivalent outputs included in **Apollo** output file
- RSGHB component renamed:
 - A now called `HB_iterations_means`
 - B now called `HB_indiv_draws_means`
 - Bsd now called `HB_indiv_draws_sd`
 - C now called `HB_posterior_means`
 - Csd now called `HB_posterior_sd`
 - D now called `HB_iterations_covar`
 - F now called `HB_iterations_non_random`
- HB outputs at iteration level no longer automatically included in saved model object. Optionally included in model object by setting `saveOutput_settings$saveHBiterations=TRUE` (default=`FALSE`)
- Geweke test outputs saved under new names in model object, printing now controlled by new setting `saveOutput_settings$printhbconvergence`
- Individual-level draws still included by default in model object, but no longer automatically written to a separate file
- Individual level posteriors still included by default in model object, and automatically written to file, except if `saveOutput_settings$saveEst==FALSE`
- Summary for underlying Normals still saved in model object, but no longer written to separate file

Backwards compatibility of code: outputs changed, including names, potentially affecting old use of results

Version 0.3.0 (10 August 2023)

General

Various bug fixes, major improvements to efficiency, stability and reporting of user errors.

Added faster ‘simple’ method for calculating Hessian

Functions affected: `apollo_varcov` and `apollo_estimate`

Detailed description: A new setting `numDeriv_method` has been added with the option to set this to `simple` to use the `simple` option from the `numDeriv` package

Backwards compatibility of code: no backwards compatibility issues for existing functions

Made BGW the default estimation routine

Functions affected: `apollo_estimate`

Detailed description: *Apollo* now uses the BGW estimation routine by default

Backwards compatibility of code: no backwards compatibility issues for existing functions

Made *Apollo* model object S3 compatible

Functions affected: `apollo_estimate`

Detailed description: *Apollo* model objects are now S3 compatible, meaning output can also be viewed using `summary` and `print`

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.1 (12 October 2023)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Version 0.3.2 (15 April 2024)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Ability to not keep backup files in saving

Functions affected: `apollo_saveOutput`

Detailed description: A new setting `saveOld` has been added to `saveOutput_settings` to allow the user to disable the automatic renaming before overwriting of output files

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.3 (4 June 2024)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Fractional nested logit model added

Functions affected: `apollo_fnl`

Detailed description: A new model function has been included in *Apollo* to allow users to estimate a fractional nested logit model

Backwards compatibility of code: no backwards compatibility issues for existing functions

Analytic gradient validation set to FALSE by default

Functions affected: `apollo_estimate`

Detailed description: By default, the analytic gradient is no longer compared to the numerical one

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.4 (1 October 2024)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Second (scaled) round of estimation no longer carried out by default

Functions affected: `apollo_estimate`

Detailed description: By default, *Apollo* no longer automatically carries out a second round of estimation with rescaled parameters

Backwards compatibility of code: no backwards compatibility issues for existing functions

Added Tobit model

Functions affected: `apollo_tobit`

Detailed description: A new model function has been included in *Apollo* to allow users to estimate a tobit model

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.5 (12 March 2025)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Any user additions to `apollo_inputs` that should be split across cores now need to include an ID column

Functions affected: `apollo_makeCluster` via `apollo_estimate`

Detailed description: If a user includes additional objects into `apollo_inputs` to make them available during estimation, they will only be split across cores if they include an ID column

Backwards compatibility of code: Earlier implementations no longer compatible without adding an ID column

Syntax changed for `apollo_longToWide`

Functions affected: `apollo_longToWide`

Detailed description: The inputs to `apollo_longToWide` have been renamed to adopt *Apollo* naming conventions

Backwards compatibility of code: Earlier implementations no longer compatible

Starting values for within class models need to differ across latent classes

Functions affected: `apollo_lc`

Detailed description: *Apollo* no longer accepts starting values that would lead to the same initial probabilities across classes for a latent class model

Backwards compatibility of code: Earlier examples will no longer work if they used the same starting values

New function to set working directory

Functions affected: `apollo_setWorkDir`

Detailed description: a new function now allows users to set the working directory automatically in RStudio

Backwards compatibility of code: new function, with no backwards compatibility issues for existing functions

`modelName` now set automatically in RStudio

Functions affected: `apollo_validateInputs`

Detailed description: *Apollo* will now attempt to set the `modelName` automatically to the filename for users with RStudio

Backwards compatibility of code: new functionality, with no backwards compatibility issues for existing functions

R requirement changes to 4.3.0

Functions affected: installation of *Apollo*

Detailed description: newer versions of *Apollo* now require a minimum R version of 4.3.0

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.6 (19 September 2025)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Ability to output utilities

Functions affected: All model components, via `apollo_probabilities`

Detailed description: The `functionality` variable now has an additional option `utilities` which outputs the utilities for model components, e.g. at the MLE estimates.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Ability to make predictions with a user-provided vector of parameter values

Functions affected: `apollo_prediction`

Detailed description: Instead of providing a model object, a user can now pass a vector of parameter values to `apollo_prediction`.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Ability to simulate choices for discrete choice models

Functions affected: `apollo_prediction`

Detailed description: An additional setting `simChoice` allows a user to request predicted choices for discrete choice models when using `apollo_prediction`.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Ability to return random distributions at the observation rather than person level

Functions affected: `apollo_unconditionals`

Detailed description: An additional setting `obsLevel` allows a user to request that the draws for continuously distributed coefficients are returned at the observation rather than person level.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Ability to print/save BHHH covariance matrix

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: An additional setting `printBHHH` has been added for printing/saving results from the BHHH covariance matrix calculations.

Backwards compatibility of code: no backwards compatibility issues for existing functions

No covariance matrix calculated if BHHH matrix is singular

Functions affected: `apollo_estimate`

Detailed description: If the BHHH matrix is singular, no attempt will be made to calculate the full covariance matrix.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Name change for BHHH hessian matrix from BGW

Functions affected: `apollo_estimate`

Detailed description: The BHHH hessian from BGW is now saved as `model$BHHHhessian` instead of `model$BHHH_matrix`.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Default changed for printChange

Functions affected: `apollo_saveOutput`

Detailed description: The default for `printChange` is now set to FALSE.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Individual-level gradients saved in model object

Functions affected: `apollo_estimate`

Detailed description: For any model estimated using `bgw`, the individual-level gradients at convergence are now stored in the `model` object.

Backwards compatibility of code: new output

Parameter scaling for Hessians set to FALSE by default

Functions affected: `apollo_estimate`

Detailed description: By default, *Apollo* no longer uses scaling of the parameters for the covariance matrix calculation.

Backwards compatibility of code: no backwards compatibility issues for existing functions

Version 0.3.7 (13 March 2026)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

Seed set for HB

Functions affected: `apollo_estimate` with `apollo_control$HB==TRUE`

Detailed description: We now ensure that the seed set in `apollo_control` is also used for HB estimation.

Backwards compatibility of code: results produced with earlier versions might be slightly different

`apollo_validateInputs` recycles by default

Functions affected: `apollo_validateInputs`

Detailed description: The default setting for `recycle` as passed to `apollo_validateInputs` has been changed to `TRUE` meaning that *Apollo* will by default reuse objects it finds in an existing version of `apollo_inputs` as opposed to replacing them with the versions from the global environment.

Backwards compatibility of code: no clear backwards compatibility issues

`apollo_searchStart` gradient check fixed

Functions affected: `apollo_searchStart`

Detailed description: Earlier versions of this function had a bug in the checking of the gradient.

Backwards compatibility of code: results will be different after this bug fix

Unavailable alternatives have `utility=NA` when outputting utilities

Functions affected: `apollo_probabilities` with `functionality=="utilities"`

Detailed description: When returning utilities, any unavailable alternatives now have their utilities set to `NA` in the concerned rows.

Backwards compatibility of code: output changed for unavailable alternatives

BHHH standard errors reported when

`estimate_settings$hessianRoutine="none"`

Functions affected: `apollo_modelOutput` and `apollo_saveOutput`

Detailed description: When a user skips the covariance matrix calculation, the fallback BHHH standard errors are reported.

Backwards compatibility of code: no backwards compatibility issues

`successfulEstimation=FALSE` when iteration limit reached

Functions affected: `apollo_estimate`

Detailed description: Fixed a bug where, when estimation runs out of iterations, `successfulEstimation` was set to `TRUE`.

Backwards compatibility of code: bug fix, convergence status changed

Version 0.3.8 (17 May 2026)

General

Various bug fixes, minor improvements to efficiency, stability and reporting of user errors.

New setting to force covariance matrix calculation

Functions affected: `apollo_estimate` with

`estimateSettings$forceCovarianceMatrix==TRUE`

Detailed description: A user can now force the calculation of the covariance matrix even if estimation has failed.

Backwards compatibility of code: no backwards compatibility issues

Convergence flag added to `apollo_searchStart`

Functions affected: `apollo_searchStart`

Detailed description: An extra column has been added, containing 0/1 to indicate convergence.

Backwards compatibility of code: no backwards compatibility issues

`apollo_searchStart` now uses BGW

Functions affected: `apollo_searchStart`

Detailed description: The BFGS algorithm has been replaced with BGW.

Backwards compatibility of code: no backwards compatibility issues, although results may differ

RSGHB functions now merged into *Apollo*

Functions affected: `apollo_estimate` with `apollo_control$HB==TRUE`

Detailed description: As the RSGHB package has been archived on CRAN, the relevant functions are now directly coded into *Apollo*.

Backwards compatibility of code: no backwards compatibility issues

Convergence checks for BGW

Functions affected: `apollo_estimate` with

`estimateSettings$estimationRoutine=="bgw"`

Detailed description: Ability to run additional convergence checks when using BGW for estimation, with a second round of estimation carried out if necessary.

Backwards compatibility of code: no backwards compatibility issues

B

Data dictionaries

Tables B.1 to B.4 present data dictionaries for the four datasets made available with *Apollo*.

Table B.1: Data dictionary for `apollo_modeChoiceData`

Individuals | 500
Observations | 8,000

Variable	Description	Values
ID	Unique individual ID	1 to 500
RP	RP data identifier	1 for RP, 0 for SP
SP	SP data identifier	1 for SP, 0 for RP
RP_journey	Index for RP observations	1 to 2, NA for SP
SP_task	Index for SP observations	1 to 14, NA for RP
av_car	availability for alternative 1 (car)	1 for available, 0 for unavailable
av_bus	availability for alternative 2 (bus)	1 for available, 0 for unavailable
av_air	availability for alternative 3 (air)	1 for available, 0 for unavailable
av_rail	availability for alternative 4 (rail)	1 for available, 0 for unavailable
time_car	travel time (mins) for alternative 1 (car)	Min: 250, mean: 311.79, max: 390 (0 if not available)
cost_car	travel cost (£) for alternative 1 (car)	Min: 30, mean: 39.99, max: 50 (0 if not available)
time_bus	travel time (mins) for alternative 2 (bus)	Min: 300, mean: 370.29, max: 420 (0 if not available)
cost_bus	travel cost (£) for alternative 2 (bus)	Min: 15, mean: 25.02, max: 35 (0 if not available)
access_bus	access time (mins) for alternative 2 (bus)	Min: 5, mean: 15.02, max: 25 (0 if not available)
time_air	travel time (mins) for alternative 3 (air)	Min: 50, mean: 70.07, max: 90 (0 if not available)
cost_air	travel cost (£) for alternative 3 (air)	Min: 50, mean: 79.94, max: 110 (0 if not available)
access_air	access time (mins) for alternative 3 (air)	Min: 35, mean: 45.02, max: 55 (0 if not available)
service_air	service quality for alternative 3 (air)	1 for no-frills, 2 for wifi, 3 for food (0 if not used, RP data)
time_rail	travel time (mins) for alternative 4 (rail)	Min: 120, mean: 142.93, max: 170 (0 if not available)
cost_rail	travel cost (£) for alternative 4 (rail)	Min: 35, mean: 55.03, max: 75 (0 if not available)
access_rail	access time (mins) for alternative 4 (rail)	Min: 5, mean: 14.96, max: 25 (0 if not available)
service_rail	service quality for alternative 4 (rail)	1 for no-frills, 2 for wifi, 3 for food (0 if not used, RP data)
female	dummy variable for female individuals	1 for female, 0 otherwise
business	dummy variable for business trips	1 for business trips, 0 otherwise
income	income variable (£ per annum)	Min: 15,490, mean: 44,748.27, max: 74,891
choice	choice variable	1 for car, 2 for bus, 3 for air, 4 for rail

Table B.2: Data dictionary for `apollo_swissRouteChoiceData`

Individuals	388
Observations	3,492

Variable	Description	Values
ID	Unique individual ID	2,439 to 84,525
choice	choice variable	1 for alternative 1, 2 for alternative 2
tt1	travel time (mins) for alternative 1	Min: 2, mean: 52.59, max: 389
tc1	travel cost (CHF) for alternative 1	Min: 1, mean: 19.67, max: 206
hw1	headway (mins) for alternative 1	Min: 15, mean: 32.48, max: 60
ch1	interchanges for alternative 1	Min: 0, mean: 0.94, max: 2
tt2	travel time (mins) for alternative 2	Min: 2, mean: 52.47, max: 385
tc2	travel cost (CHF) for alternative 2	Min: 1, mean: 19.69, max: 268
hw2	headway (mins) for alternative 2	Min: 15, mean: 32.38, max: 60
ch2	interchanges for alternative 2	Min: 0, mean: 0.95, max: 2
hh_inc_abs	household income (CHF per annum)	Min: 10,000, mean: 76,507.73, max: 167,500
car_availability	car availability	1 for yes, 0 otherwise
commute	dummy variable for commute trips	1 for commute trips, 0 otherwise
shopping	dummy variable for shopping trips	1 for shopping trips, 0 otherwise
business	dummy variable for business trips	1 for business trips, 0 otherwise
leisure	dummy variable for leisure trips	1 for leisure trips, 0 otherwise

Table B.3: Data dictionary for `apollo_drugChoiceData`

Individuals	1,000
Observations	10,000

Variable	Description	Values
ID	Unique respondent ID	1 to 1,000
task	Index for SP choice tasks	1 to 10
best	first ranked alternative	1 to 4
second_pref	second ranked alternative	1 to 4
third_pref	third ranked alternative	1 to 4
worst	worst ranked alternative	1 to 4
brand_1	brand for first alternative	Artemis; Novum
country_1	country for first alternative	Switzerland; Denmark; USA
char_1	characteristics for first alternative	standard; fast acting; double strength
side_effects_1	rate of side effects for first alternative (out of 100,000)	Min: 1, mean: 37, max: 100
price_1	price (£) for first alternative	Min: 2.25, mean: 3.15, max: 4.5
brand_2	brand for second alternative	Artemis; Novum
country_2	country for second alternative	Switzerland; Denmark; USA
char_2	characteristics for second alternative	standard; fast acting; double strength
side_effects_2	rate of side effects for second alternative (out of 100,000)	Min: 1, mean: 37, max: 100
price_2	price (£) for second alternative	Min: 2.25, mean: 3.15, max: 4.5
brand_3	brand for third alternative	BestValue; Supermarket; PainAway
country_3	country for third alternative	USA; India; Russia; Brazil
char_3	characteristics for third alternative	standard; fast acting
side_effects_3	rate of side effects for third alternative (out of 100,000)	Min: 10, mean: 370, max: 1,000
price_3	price (£) for third alternative	Min: 0.75, mean: 1.75, max: 2.5
brand_4	brand for fourth alternative	BestValue; Supermarket; PainAway
country_4	country for fourth alternative	USA; India; Russia; Brazil
char_4	characteristics for fourth alternative	standard; fast acting
side_effects_4	rate of side effects for fourth alternative (out of 100,000)	Min: 10, mean: 370, max: 1,000
price_4	price (£) for fourth alternative	Min: 0.75, mean: 1.75, max: 2.5
regular_user	dummy variable for regular users	1 for regular users, 0 otherwise
university_educated	dummy variable for university educated	1 for university educated, 0 otherwise
over_50	dummy variable for age over 50 years	1 for age over 50 years, 0 otherwise
attitude_quality	Answer to <i>"I am concerned about the quality of drugs developed by unknown companies"</i>	Likert scale from 1 (strongly disagree) to 5 (strongly agree)
attitude_ingredients	Answer to <i>"I believe that ingredients are the same no matter what the brand"</i>	Likert scale from 1 (strongly disagree) to 5 (strongly agree)
attitude_patent	Answer to <i>"The original patent holders have valuable experience with their medicines"</i>	Likert scale from 1 (strongly disagree) to 5 (strongly agree)
attitude_dominance	Answer to <i>"I believe the dominance of big pharmaceutical companies is unhelpful"</i>	Likert scale from 1 (strongly disagree) to 5 (strongly agree)

Table B.4: Data dictionary for `apollo_timeUseData`

Individuals | 447
Observations | 2,826

Variable	Description	Values
indivID	Unique respondent ID	19209 to 9959342
day	Index of the day for the individual (day 1 excluded from data)	2 to 14
date	Date in format <code>yyyymmdd</code>	20161014 to 20170308
budget	Total amount of time registered during the day (in minutes)	1440 to 1440
t_a01	time spent dropping-off or picking up other people (in minutes)	0 to 1153
t_a02	time spent working (in minutes)	0 to 1425
t_a03	time spent on educational activities (in minutes)	0 to 1050
t_a04	time spent shopping (in minutes)	0 to 1434
t_a05	time spent on private business (in minutes)	0 to 1077
t_a06	time spent getting petrol (in minutes)	0 to 896
t_a07	time spent on social or leisure activities (in minutes)	0 to 1425
t_a08	time spent on vacation or on long (intercity) travel (in minutes)	0 to 828
t_a09	time spent doing exercise (in minutes)	0 to 1416
t_a10	time spent at home (in minutes)	0 to 1440
t_a11	time spent travelling (everyday travelling) (in minutes)	0 to 1182
t_a12	Non-allocated time (in minutes)	0 to 1160
female	dummy variable for female individuals	1 for female, 0 otherwise
age	age of the respondent (in years, approximate)	21 to 80
occ_full_time	dummy for respondents working full time	1 for respondents working full time, 0 otherwise
weekend	dummy for weekend days	1 for weekend, 0 otherwise



Index of example files

Table [C.1](#) presents an overview of the example files made available with *Apollo*.

Table C.1: Index of example files

Family	Name	Script	Output	Description	Database	Data	Dictionary
MNL	MNL_RP.r	.r	.txt	Simple MNL model on mode choice RP data	Mode choice	.csv	.pdf
	MNL_SP.r	.r	.txt	Simple MNL model on mode choice SP data	Mode choice	.csv	.pdf
	MNL_SP_effects.r	.r	.txt	Simple MNL model on mode choice SP data using effects coding	Mode choice	.csv	.pdf
	MNL_SP_WTP_space.r	.r	.txt	Simple MNL model on mode choice SP data in WTP space	Mode choice	.csv	.pdf
	MNL_SP_covariates.r	.r	.txt	MNL model with socio-demographics on mode choice SP data	Mode choice	.csv	.pdf
	MNL_RP_SP.r	.r	.txt	RP-SP model on mode choice data	Mode choice	.csv	.pdf
	MNL_iterative_coding.r	.r	.txt	MNL model using iterative coding for alternatives and attributes	Synthetic	n/a	n/a
GEV	NL_two_levels.r	.r	.txt	Two-level NL model with socio-demographics on mode choice SP data	Mode choice	.csv	.pdf
	NL_two_levels_constraints_on_lambda.r	.r	.txt	Two-level NL model with socio-demographics on mode choice SP data, with constraint on lambda	Mode choice	.csv	.pdf
	NL_three_levels.r	.r	.txt	Three-level NL model with socio-demographics on mode choice SP data	Mode choice	.csv	.pdf
	NL_three_levels_constraints_on_lambdas.r	.r	.txt	Three-level NL model with socio-demographics on mode choice SP data, with constraints on lambdas	Mode choice	.csv	.pdf
	CNL.r	.r	.txt	CNL model with socio-demographics on mode choice SP data	Mode choice	.csv	.pdf
	CNL_constraint_on_alphas.r	.r	.txt	CNL model with socio-demographics on mode choice SP data, with constraint on alphas	Mode choice	.csv	.pdf
	CNL_logistic_transform_for_alphas.r	.r	.txt	CNL model with socio-demographics on mode choice SP data, using logistic transform to ensure alphas meet constraints	Mode choice	.csv	.pdf
Non-RUM	RRM.r	.r	.txt	Simple RRM model on mode choice SP data	Mode choice	.csv	.pdf
	DFT_route_choice.r	.r	.txt	DFT model on Swiss route choice SP data	Swiss route choice	.csv	.pdf
	DFT_mode_choice.r	.r	.txt	DFT model on mode choice SP data	Mode choice	.csv	.pdf

Family	Name	Script	Output	Description	Database	Data	Dictionary
Ranking and rating	EL.r	.r	.txt	Exploded logit model on drug choice data	Drug choice	.csv	.pdf
	BW_EL.r	.r	.txt	Best-worst logit model on drug choice data, using exploded logit function	Drug choice	.csv	.pdf
	BW_joint_model.r	.r	.txt	Best-worst logit model on drug choice data (coded as two stages)	Drug choice	.csv	.pdf
	BW_joint_model_diff_params.r	.r	.txt	Best-worst logit model on drug choice data, using different parameters for best and worse choices	Drug choices	.csv	.pdf
	BW_simultaneous.r	.r	.txt	Best-worst model on drug choice data, simultaneous choice	Drug choices	.csv	.pdf
	OL.r	.r	.txt	Ordered logit model fitted to attitudinal question in drug choice data	Drug choice	.csv	.pdf
	OP.r	.r	.txt	Ordered probit model fitted to attitudinal question in drug choice data	Drug choice	.csv	.pdf
	normal_density.r	.r	.txt	Normal density function fitted to attitudinal question in drug choice data	Drug choice	.csv	.pdf
Discrete-continuous	FMNL.r	.r	.txt	Fractional MNL model on time use data	Time use	.csv	.pdf
	MDCEV_no_outside_good.r	.r	.txt	MDCEV model on time use data, alpha-gamma profile, no outside good and constants only in utilities	Time use	.csv	.pdf
	MDCEV_with_outside_good.r	.r	.txt	MDCEV model on time use data, alpha-gamma profile with outside good and socio-demographics	Time use	.csv	.pdf
	MDCNEV.r	.r	.txt	MDCNEV model on time use data, alpha-gamma profile with outside good and socio-demographics	Time use	.csv	.pdf
	eMDC_with_budget.r	.r	.txt	Extended MDC with complementarity and substitution, with observed budget and socio-demographics	Time use	.csv	.pdf
	eMDC_without_budget.r	.r	.txt	Extended MDC with complementarity and substitution, with unobserved budget and socio-demographics	Time use	.csv	.pdf

Family	Name	Script	Output	Description	Database	Data	Dictionary
Mixture models	MMNL_preference_space.r	.r	.txt	Mixed logit model on Swiss route choice data, uncorrelated Lognormals in preference space	Swiss route choice	.csv	.pdf
	MMNL_preference_space_correlated.r	.r	.txt	Mixed logit model on Swiss route choice data, correlated Lognormals in utility space	Swiss route choice	.csv	.pdf
	MMNL_wtp_space_inter_intra.r	.r	.txt	Mixed logit model on Swiss route choice data, WTP space with correlated and flexible distributions, inter and intra-individual heterogeneity	Swiss route choice	.csv	.pdf
	ECL_preference_space_heteroskedasticity.r	.r	.txt	Error components logit model on Swiss route choice data, uncorrelated Lognormals in preference space, with heteroskedasticity term	Swiss route choice	.csv	.pdf
	ECL_preference_space_panel_effect.r	.r	.txt	Error components logit model on Swiss route choice data, uncorrelated Lognormals in preference space, with panel effect term	Swiss route choice	.csv	.pdf
	DM.r	.r	.txt	Simple DM model on Swiss route choice data	Swiss route choice	.csv	.pdf
	LC_no_covariates.r	.r	.txt	Simple LC model on Swiss route choice data, no covariates in class allocation model	Swiss route choice	.csv	.pdf
	LC_with_covariates.r	.r	.txt	LC model with covariates in class allocation model on Swiss route choice data	Swiss route choice	.csv	.pdf
	LC_MMNL.r	.r	.txt	Latent class with continuous random parameters on Swiss route choice data	Swiss route choice	.csv	.pdf
Hybrid choice	Mixed_MDCEV.r	.r	.txt	Mixed MDCEV model on time use data, alpha-gamma profile, no outside good and random constants only in	Time use	.csv	.pdf
	Hybrid_with_continuous_measurement.r	.r	.txt	Hybrid choice model on drug choice data, using continuous measurement model for indicators	Drug choice	.csv	.pdf
	Hybrid_with_OL.r	.r	.txt	Hybrid choice model on drug choice data, using ordered measurement model for indicators	Drug choice	.csv	.pdf
	Hybrid_with_OL_bugged_example.r	.r	.txt	Hybrid choice model on drug choice data, using ordered measurement model for indicators - with bug for debugging	Drug choice	.csv	.pdf
	Hybrid_with_OL_and_MMNL.r	.r	.txt	Hybrid choice model on drug choice data, classical estimation	Drug choice	.csv	.pdf
	Hybrid_LC_with_OL.r	.r	.txt	Hybrid latent class choice model on drug choice data, using ordered measurement model for indicators	Drug choice	.csv	.pdf

Family	Name	Script	Output	Description	Database	Data	Dictionary
Alternative estimation approaches	HB_MNL.r	.r	.txt	Bayesian estimation of MNL model on mode choice SP data	Mode choice	.csv	.pdf
	HB_MMNL.r	.r	.txt	Bayesian estimation of Mixed Logit model on mode choice SP data	Mode choice	.csv	.pdf
	Hybrid_with_OL_and_MMNL_bayesian.r	.r	.txt	Hybrid choice model on drug choice data, bayesian estimation	Drug choice	.csv	.pdf
	EM_MMNL.r	.r	.txt	Mixed logit model on Swiss route choice data, correlated Lognormals in utility space, EM algorithm	Swiss route choice	.csv	.pdf
	EM_LC_no_covariates.r	.r	.txt	LC model with class allocation model on Swiss route choice data, EM algorithm	Swiss route choice	.csv	.pdf
	EM_LC_with_covariates.r	.r	.txt	LC model with class allocation model on Swiss route choice data, EM algorithm	Swiss route choice	.csv	.pdf

D

Detailed description of model object

Table [D.1](#) presents an overview of all elements inside a model object produced by *Apollo* during estimation, together with their description and for what kind of model it is generated.

Table D.1: Elements inside a model object estimated by *Apollo*

Element	Type	Description	for models
adjRho2_0	Numeric scalar	Adjusted ρ^2 for the whole model when all parameters are set to zero. Equal to NA if there are multiple components.	all models
adjRho2_C	Numeric scalar	Adjusted ρ^2 for the whole model when only constants are considered. Equal to NA if there are multiple components.	all models
AIC	Numeric scalar	Akaike Information Criterion for the whole model.	all models
apollo_beta	Named numeric vector	Starting value of parameters	all models
apollo_control	List	Main model settings, as validated by apollo_validateInputs	all models
apollo_draws	List of numeric matrix or 3-dim arrays	Contains the draws for models with mixing.	With mixing (NA otherwise)
apollo_fixed	Character vector	Name of fixed parameters. NULL for none.	all models
apollo_lcPars	Function	Function that groups parameters per latent class and calculate class allocation probabilities. As defined by the user.	With latent classes (NA otherwise)
apollo_lcPars_mod	Function	A modified version of apollo_lcPars, generated automatically by <i>Apollo</i> , for optimisation purposes.	all models
apollo_probabilities	Function	Function that calculates the likelihood of the whole model. As defined by the user.	all models
apollo_probabilities_mod	Function	A modified version of apollo_probabilities, generated automatically by <i>Apollo</i> , for optimisation purposes.	all models
apollo_randCoeff	Function	Function that generates the random coefficients used in the likelihood function. As defined by the user.	With mixing (NA otherwise)
apollo_randCoeff_mod	Function	A modified version of apollo_randCoeff, generated automatically by <i>Apollo</i> for optimisation purposes.	all models
avgCP	Numeric vector	Average likelihood for each individual at the estimated parameter values. As many elements as individuals in the sample.	all models
avgLL	Numeric vector	Average log-likelihood for each individual at the estimated parameter values. As many elements as individuals in the sample.	all models
betaStart	Named numeric vector	Starting value of parameters.	all models
betaStop	Named numeric vector	Value of parameters at the time the optimisation algorithm stopped (even if it failed to converge). It does not include the fixed parameters.	all models

Element	Type	Description	for models
bgw_settings	Named list	List of settings for BGW algorithm, as used during estimation.	only models estimated using BGW
BHHH_matrix	Numeric matrix	BHHH matrix as calculated by the optimisation algorithm. Can be used as an approximation of the Hessian.	all models estimated using maximum likelihood
bHHHcorrmat	Numeric matrix	BHHH correlation matrix of estimated parameters. Excludes fixed parameters.	all models
BHHHse	Named numeric vector	BHHH s.e. of estimated parameters. Includes fixed parameters.	all models
BHHHvarcov	Numeric matrix	BHHH covariance matrix of estimated parameters. Excludes fixed parameters.	all models
BIC	Numeric scalar	Bayesian Information criterion for the whole model.	all models
bootstrapSE	Numeric scalar	Number of bootstrap repetitions used to calculate the bootstrap standard errors.	with bootstrap s.e. (0 otherwise)
code	Numeric scalar	Success code from maxLik (and optim). 0 means succesful estimation.	all models estimated with Maximum Likelihood.
componentReport	List	Contains the report on each model component dependant variable as a list of character vectors.	all models
constraints	List	List describing the constraints used during estimation in maxLik format.	With constraints estimated using Maximum Likelihood
control	List	Settings of the maximum likelihood optimisation algorithm.	all models estimated with Maximum Likelihood.
corrmat	matrix	Correlation matrix of (non-fixed) estimated parameters	all models
eigValue	Numeric scalar	Maximum eigenvalue of the Hessian matrix at the parameter estimates	all models estimated with Maximum Likelihood.
estimate	Numeric vector	Value of estimated parameters (including fixed ones)	all models estimated with Maximum Likelihood.
estimationRoutine	Scalar character	Name of the estimation algorithm	all models estimated with Maximum Likelihood.
estimate_settings	Named list	Settings used in estimation	all models
finalLL	Numeric scalar	Final loglikelihood value at the estimated parameters.	all models estimated with Maximum Likelihood.
functionEvals	Numeric scalar	Number of function evaluations during estimation.	all models estimated using BGW.
fixed	Named logical vector	Only contains the names of non-fixed parameters, with a FALSE value associated to them. Used by maxLik, not Apollo.	all models estimated with BFGS and BHHH.

Element	Type	Description	for models
gradient	Named numerical vector	Gradient of the log-likelihood function at the estimated value of the parameters.	all models estimated with Maximum Likelihood.
gradientObs	Numerical matrix	Observation-level score matrix, i.e. the gradient of the log-likelihood function at the estimated parameter values at the observation level.	all models
hasAnalyticalGrad	Scalar logical (boolean)	TRUE if the estimation process used analytical gradients, FALSE otherwise.	all models estimated using Maximum Likelihood.
hessian	Numerical matrix	Hessian (second derivative) of the log-likelihood function.	all models
hessianEigenValue	Numeric vector	Eigenvalues associated to the Hessian.	all models with a covariance matrix
hessianMethodsAttempted	Character vector	Name of the methods attempted to calculate the Hessian.	all models
hessianMethodUsed	Scalar character	Name of the methods used to calculate the Hessian reported in model\$hessian.	all models
hessianScaling	Named numeric vector	Scaling used to calculate the Hessian. By default, they will be equal to the estimated value of the parameters.	all models
indLevelGradients	Matrix	Gradients at the person-parameter level at convergence.	all models estimated using bgw
iterations	Numeric scalar	Number of iterations of the maximum likelihood estimation according to maxLik (not Apollo).	all models estimated with Maximum Likelihood.
last.step	List	List describing last unsuccessful iteration, if model\$code=3 (i.e. if there was an error during maximum likelihood estimation).	all models estimated with Maximum Likelihood.
LL0	Numeric scalar	Log-likelihood of the null (equiprobable) model, if applicable (only for models with a single MNL, NL or CNL component)	all models. NA if not applicable.
LLC	Numeric scalar	Log-likelihood of the model with constants only, if applicable (only for discrete choice models)	all models. NA if not applicable.
LLout	Named numeric vector	Log-likelihood of each model component at the estimated parameter values. At the sample level.	all models
LLStart	Numeric scalar	Log-likelihood value at the starting value of the parameters.	all models
manualScaling	Scalar logical	TRUE if the user manually defined a scaling to be used during estimation. FALSE in other cases.	all models
maximum	Numeric scalar	Log-likelihood value at the estimated parameter values.	all models
message	Scalar character	Message describing convergence (successful or not), as generated by maxLik.	all models estimated with Maximum Likelihood.
modelTypeList	Character vector	Vector indicating the type of model (MNL, NL, etc.) of each model component.	all models
nFreeParams	Numeric scalar	Number of estimated parameters (omitting fixed parameters).	all models estimated using Maximum Likelihood.

Element	Type	Description	for models
nIndivs	Numeric scalar	Number of individuals in the database.	all models
nIter	Numeric scalar	Overall number of iterations necessary for convergence, as counted by Apollo.	all models estimated with Maximum Likelihood.
nIterPostscaling	Numeric scalar	Number of iterations necessary for convergence on the second round of estimation, i.e. after automatic scaling of parameters, as counted by Apollo.	all models estimated with Maximum Likelihood.
nIterPrescaling	Numeric scalar	Number of iterations necessary for convergence on the first round of estimation, i.e. without automatic scaling of parameters, as counted by Apollo.	all models estimated with Maximum Likelihood.
nObs	Numeric scalar	Number of observations in the database, counted as rows in the database.	all models
nObsTot	Numeric vector	Number of observations per model component, not counting excluding rows.	all models
numParams	Numeric scalar	Number of estimated parameters (omitting fixed parameters).	all models estimated using BGW.
numResids	Numeric scalar	Number of residuals used during estimation (same as number of observations if cross sectional, or individuals if panel data are used).	All models estimated using BGW.
objectiveFn	Function	Loglikelihood function generated by Apollo to mask apollo_probabilities from maxLik.	all models
rho0_C	Numeric scalar	ρ^2 for the whole model when all parameters are zero. Equal to NA if there are multiple components.	all models
rho2_C	Numeric scalar	ρ^2 for the whole model when only constants are considered. Equal to NA if there are multiple components.	all models
robcorrmat	Numeric matrix	Robust correlation matrix of estimated parameters. Excludes fixed parameters.	all models
robse	Named numeric vector	Robust s.e. of estimated parameters. Includes fixed parameters.	all models
robvarcov	Numeric matrix	Robust covariance matrix of estimated parameters. Excludes fixed parameters.	all models
scaleVec	Numeric vector	Scaling vector used by BGW.	all models estimated using BGW
scaling	Named numeric vector	Scaling parameters used during model estimation (does not include fixed parameters)	all models
se	Named numeric vector	Standard errors of estimated parameters. Includes fixed parameters.	all models

	Element	Type	Description	for models
	seBGW	Named numeric vector	Standard errors of estimated parameters calculated by BGW based on BHHH matrix. Excludes fixed parameters.	all models estimated using BGW
	startTime	Date object	POSIX date object recording the time when the full estimation processes started.	all models
	successfulEstimation	Logical (boolean) scalar	TRUE if estimation was successful, FALSE if not.	all models estimated with Maximum Likelihood
	timeEst	Numeric scalar	Time (in seconds) taken by the estimation algorithm (excludes pre and post-processing).	all models
	timeEstPostscaling	Numeric scalar	Time (in seconds) taken by estimation after automatic scaling of parameters.	all models
	timeEstPrescaling	Numeric scalar	Time (in seconds) taken by estimation before automatic scaling of parameters.	all models
	timePost	Numeric scalar	Time (in seconds) taken by the post-estimation processing (including the calculation of the covariance matrix).	all models
	timePre	Numeric scalar	Time (in seconds) taken by the pre-estimation processing (including validation and symbolic calculation of gradient).	all models
	timeTaken	Numeric scalar	Time (in seconds) taken by the full estimation process (pre and post-estimation calculations included).	all models
	tstatBGW	Named numeric vector	T-ratios for the estimated parameters (excluding fixed parameters) based on s.e. from BHHH matrix.	models estimated using BGW
	type	Scalar character	Type of maximisation (algorithm) as defined by maxLik.	all models estimated with Maximum Likelihood.
	varcov	Numeric matrix	Covariance matrix of estimated parameters. Excludes fixed parameters.	all models
	varcovBGW	Named numeric vector	Covariance matrix for the estimated parameters (excluding fixed parameters) based on s.e. from BHHH matrix.	models estimated using BGW
	vcHessianConditionNumber	Numeric scalar	Condition number of the Hessian (based on the BHHH matrix). If zero, it means the Hessian cannot be inverted.	models estimated using BGW
	vcVec	Numeric vector	Elements of the lower triangle of the covariance matrix (based on the BHHH matrix) as a vector, by row.	models estimated using BGW

Bibliography

- Abou-Zeid, M., Ben-Akiva, M., 2014. Hybrid choice models, in: Hess, S., Daly, A. (Eds.), *Handbook of Choice Modelling*. Edward Elgar. chapter 17, pp. 383–412.
- ALogit, 2016. ALOGIT 4.3. ALOGIT Software & Analysis Ltd. URL: www.alogit.com.
- Axhausen, K.W., Hess, S., König, A., Abay, G., Bates, J.J., Bierlaire, M., 2008. Income and distance elasticities of values of travel time savings: New swiss results author links open overlay panel. *Transport Policy* 15, 173–185.
- Ben-Akiva, M., Swait, J., 1986. The Akaike Likelihood Ratio Index. *Transportation Science* 20, 133–136.
- Berndt, E., Hall, B., Hall, R., Hausman, J., 1974. Estimation and inference in non-linear structural models. *Annals of Economic and Social Measurement* 3/4, 653–665.
- Bhat, C., 1997. An endogenous segmentation mode choice model with an application to intercity travel. *Transportation Science* 31, 34–48.
- Bhat, C.R., 2003. Simulation estimation of mixed discrete choice models using randomized and scrambled Halton sequences. *Transportation Research Part B* 37, 837–855.
- Bhat, C.R., 2008. The multiple discrete-continuous extreme value (mdcev) model: role of utility function parameters, identification considerations, and model extensions. *Transportation Research Part B: Methodological* 42, 274–303.
- Bierlaire, M., 2003. BIOGEME: a free package for the estimation of discrete choice models. *Proceedings of the 3rd Swiss Transport Research Conference, Monte Verità, Ascona*.
- Bierlaire, M., Thémans, M., Zufferey, N., 2010. A heuristic for nonlinear global optimization. *INFORMS Journal on Computing* 22, 59–70.
- Bradley, M.A., Daly, A., 1996. Estimation of logit choice models using mixed stated-preference and revealed-preference information, in: Stopher, P.R., Lee-Gosselin, M. (Eds.), *Understanding Travel Behaviour in an Era of Change*. Elsevier, Oxford. chapter 9, pp. 209–231.
- Broyden, C.G., 1970. The convergence of a class of double-rank minimization algorithms 1. general considerations. *IMA Journal of Applied Mathematics* 6, 76–90.
- Bunch, D.S., Gay, D.M., Welsch, R.E., 1993. Algorithm 717: Subroutines for maximum likelihood and quasi-likelihood estimation of parameters in nonlinear regression models. *ACM Trans. Math. Softw.* 19, 109–130. URL: <https://doi.org/10.1145/151271.151279>, doi:10.1145/151271.151279.
- Busemeyer, J.R., Townsend, J.T., 1992. Fundamental derivations from decision field theory. *Mathematical Social Sciences* 23, 255–282.

- Bussemeyer, J.R., Townsend, J.T., 1993. Decision field theory: a dynamic-cognitive approach to decision making in an uncertain environment. *Psychological Review* 100, 432.
- Calastri, C., Hess, S., Daly, A., Carrasco, J.A., 2017. Does the social context help with understanding and predicting the choice of activity type and duration? an application of the multiple discrete-continuous nested extreme value model to activity diary data. *Transportation Research Part A: Policy and Practice* 104, 1–20.
- Calastri, C., Crastes dit Sourd, R., Hess, S., 2019. We want it all: experiences from a survey seeking to capture social network structures, lifetime events and short-term travel activity planning. *Transportation* forthcoming.
- Chiou, L., Walker, J., 2007. Masking identification of discrete choice models under simulation methods. *Journal of Econometrics* 141, 683–703.
- Chorus, C., 2010. A new model of random regret minimization. *European Journal of Transport and Infrastructure Research* 10, 181–196.
- van Cranenburgh, S., Guevara, C.A., Chorus, C.G., 2015. New insights on random regret minimization models. *Transportation Research Part A: Policy and Practice* 74, 91–109. doi:[10.1016/j.tra.2015.01.008](https://doi.org/10.1016/j.tra.2015.01.008).
- Daly, A., 1987. Estimating Tree Logit models. *Transportation Research Part B* 21, 251–267.
- Daly, A., Hess, S., de Jong, G., 2012a. Calculating errors for measures derived from choice modelling estimates. *Transportation Research Part B* 46, 333–341.
- Daly, A., Zachary, S., 1978. Improved multiple choice models, in: Hensher, D.A., Dalvi, Q. (Eds.), *Identifying and Measuring the Determinants of Mode Choice*. Teakfields, London, pp. 335–357.
- Daly, A.J., Hess, S., Patruni, B., Potoglou, D., Rohr, C., 2012b. Using ordered attitudinal indicators in a latent variable choice model: A study of the impact of security on rail travel behaviour. *Transportation* 39, 267–297.
- Doornik, J.A., 2001. *Ox: An Object-Oriented Matrix Language*. Timberlake Consultants Press, London.
- Dumont, J., Keller, J., 2019. RSGHB: Functions for Hierarchical Bayesian Estimation: A Flexible Approach. URL: <https://CRAN.R-project.org/package=RSGHB>. r package version 1.2.1.
- Faure, H., Tezuka, S., 2000. Another random scrambling of digital (t,s)-sequences, in: Fang, K.T., Hickernell, F.J., Niederreiter, H. (Eds.), *Monte Carlo and Quasi-Monte Carlo Methods*. Springer, pp. 242–256.
- Fletcher, R., 1970. A new approach to variable metric algorithms. *The computer journal* 13, 317–322.
- Fosgerau, M., Mabit, S.L., 2013. Easy and flexible mixture distributions. *Economics Letters* 120, 206 – 210.

- Geweke, J., 1992. Evaluating the accuracy of sampling-based approaches to the calculations of posterior moments. *Bayesian statistics* 4, 641–649.
- Giergiczny, M., Dekker, T., Hess, S., Chintakayala, P., 2017. Testing the stability of utility parameters in repeated best, repeated best-worst and one-off best-worst studies. *European Journal of Transport and Infrastructure Research* 17, 457–476. URL: <http://eprints.whiterose.ac.uk/118496/>. © 2017, Author(s). Reproduced in accordance with the publisher’s self-archiving policy.
- Gilbert, P., Varadhan, R., 2016. numDeriv: Accurate Numerical Derivatives. URL: <https://CRAN.R-project.org/package=numDeriv>. r package version 2016.8-1.
- Goldfarb, D., 1970. A family of variable metric updates derived by variational means, v. 24. *Mathematics of Computation* .
- Greene, W.H., Hensher, D.A., 2013. Revealing additional dimensions of preference heterogeneity in a latent class mixed multinomial logit model. *Applied Economics* 45, 1897–1902.
- Halton, J., 1960. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik* 2, 84–90.
- Hancock, T.O., Hess, S., Choudhury, C.F., 2018. Decision field theory: Improvements to current methodology and comparisons with standard choice modelling techniques. *Transportation Research Part B: Methodological* 107, 18–40.
- Hancock, T.O., Hess, S., Choudhury, C.F., 2019. An accumulation of preference: two alternative dynamic models for understanding transport choices. Submitted .
- Henningsen, A., Toomet, O., 2011. maxlik: A package for maximum likelihood estimation in R. *Computational Statistics* 26, 443–458. URL: <http://dx.doi.org/10.1007/s00180-010-0217-1>, doi:10.1007/s00180-010-0217-1.
- Hensher, D.A., Louviere, J.J., Swait, J., 1998. Combining sources of preference data. *Journal of Econometrics* 89, 197–221.
- Hess, S., 2005. Advanced discrete choice models with applications to transport demand. Ph.D. thesis. Centre for Transport Studies, Imperial College London.
- Hess, S., 2014. Latent class structures: taste heterogeneity and beyond, in: *Handbook of choice modelling*. Edward Elgar Publishing Cheltenham, pp. 311–329.
- Hess, S., Bierlaire, M., Polak, J.W., 2007a. A systematic comparison of continuous and discrete mixture models. *European Transport* 36, 35–61.
- Hess, S., Daly, A., 2024. *Handbook of Choice Modelling*, second edition. Edward Elgar publishers, Cheltenham.
- Hess, S., Daly, A., Dekker, T., Cabral, M.O., Batley, R., 2017. A framework for capturing heterogeneity, heteroskedasticity, non-linearity, reference dependence and design artefacts in value of time research. *Transportation Research Part B: Methodological* 96, 126 – 149.
- Hess, S., Polak, J.W., Daly, A., Hyman, G., 2007b. Flexible Substitution Patterns in Models of Mode and Time of Day Choice: New evidence from the UK and the Netherlands. *Transportation* 34, 213–238.

- Hess, S., Rose, J.M., Hensher, D.A., 2008. Asymmetric preference formation in willingness to pay estimates in discrete choice models. *Transportation Research Part E* 44, 847–863.
- Hess, S., Stathopoulos, A., Daly, A.J., 2012. Allowing for heterogeneous decision rules in discrete choice models: an approach and four case studies. *Transportation* 39, 565–591.
- Hess, S., Train, K., 2011. Recovery of inter- and intra-personal heterogeneity using mixed logit models. *Transportation Research Part B* 45, 973–990.
- Hess, S., Train, K., 2017. Correlation and scale in mixed logit models. *Journal of Choice Modelling* 23, 1–8.
- Hess, S., Train, K., Polak, J.W., 2006. On the use of a Modified Latin Hypercube Sampling (MLHS) method in the estimation of a Mixed Logit model for vehicle choice. *Transportation Research Part B* 40, 147–163.
- Hotaling, J.M., Busemeyer, J.R., Li, J., 2010. Theoretical developments in decision field theory: comment on Tsetos, Usher, and Chater (2010). *Psychological Review* 117, 1294–1298.
- Huber, P., 1967. The behavior of maximum likelihood estimation under nonstandard conditions, in: LeCam, L., Neyman, J. (Eds.), *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*. University of California Press, pp. 221–233.
- Koppelman, F.S., Wen, C.H., 1998. Alternative Nested Logit Models: structure, properties and estimation. *Transportation Research Part B* 32, 289–298.
- Lancsar, E., Louviere, J., Donaldson, C., Currie, G., Burgess, L., 2013. Best worst discrete choice experiments in health: Methods and an application. *Social Science & Medicine* 76, 74 – 82. URL: <http://www.sciencedirect.com/science/article/pii/S0277953612007290>, doi:<https://doi.org/10.1016/j.socscimed.2012.10.007>.
- Lenk, P., 2014. Bayesian estimation of random utility models, in: *Handbook of Choice Modelling*. Edward Elgar Publishing. Chapters. chapter 20, pp. 457–497. URL: https://ideas.repec.org/h/elg/eechap/14820_20.html.
- Louviere, J.J., Woodworth, G., 1983. Design and analysis of simulated consumer choice and allocation experiments: A method based on aggregate data. *Journal of Marketing Research* 20, 350–367.
- Luce, R., 1959. *Individual choice behavior: a theoretical analysis*. J.Wiley and Sons, New York.
- McFadden, D., 1974. Conditional logit analysis of qualitative choice behaviour, in: Zarembka, P. (Ed.), *Frontiers in Econometrics*. Academic Press, New York, pp. 105–142.
- McFadden, D., 1978. Modelling the choice of residential location, in: Karlqvist, A., Lundqvist, L., Snickars, F., Weibull, J.W. (Eds.), *Spatial Interaction Theory and Planning Models*. North Holland, Amsterdam. chapter 25, pp. 75–96.
- McFadden, D., 2000. *Economic Choices*. Nobel Prize Lecture. URL: <https://www.nobelprize.org/uploads/2018/06/mcfadden-lecture.pdf>.

- McFadden, D., Train, K., 2000. Mixed MNL Models for discrete response. *Journal of Applied Econometrics* 15, 447–470.
- Owen, A.B., 1995. Randomly permuted (t,m,s)-nets and (t,s)-sequences, in: Niederreiter, H., Shiue, J.S. (Eds.), *Monte Carlo and Quasi-Monte Carlo Methods in Scientific Computing*. Springer, New York, pp. 351–368.
- Palma, D., 2016. Modelling wine consumer preferences using hybrid choice models: inclusion of intrinsic and extrinsic attributes. Ph.D. thesis. School of Engineering, Pontificia Universidad Católica de Chile.
- Palma, D., Hess, S., 2022. Extending the multiple discrete continuous (mdc) modelling framework to consider complementarity, substitution, and an unobserved budget. *Transportation Research Part B: Methodological* 161, 13–35. URL: <https://www.sciencedirect.com/science/article/pii/S0191261522000686>, doi:<https://doi.org/10.1016/j.trb.2022.04.005>.
- Papke, L.E., Wooldridge, J.M., 1996. Econometric methods for fractional response variables with an application to 401(k) plan participation rates. *Journal of Applied Econometrics* 11, 619–632. doi:[10.1002/\(sici\)1099-1255\(199611\)11:6<619::aid-jae418>3.0.co;2-1](https://doi.org/10.1002/(sici)1099-1255(199611)11:6<619::aid-jae418>3.0.co;2-1).
- Pinjari, A.R., Bhat, C., 2010a. A multiple discrete–continuous nested extreme value (mdcnev) model: formulation and application to non-worker activity time-use and timing behavior on weekdays. *Transportation Research Part B: Methodological* 44, 562–583.
- Pinjari, A.R., Bhat, C.R., 2010b. An efficient forecasting procedure for kuhn-tucker consumer demand model systems: application to residential energy consumption analysis. Technical paper, Department of Civil and Environmental Engineering, University of South Florida , 263–285.
- R Core Team, 2017. R: A Language and Environment for Statistical Computing. R Foundation for Statistical Computing. Vienna, Austria. URL: <https://www.R-project.org/>.
- Roe, R.M., Busemeyer, J.R., Townsend, J.T., 2001. Multialternative decision field theory: A dynamic connectionist model of decision making. *Psychological Review* 108, 370.
- RStudio Team, 2015. Rstudio: Integrated development for r. RStudio, Inc., Boston, MA URL <http://www.rstudio.com/>.
- Shanno, D.F., 1970. Conditioning of quasi-newton methods for function minimization. *Mathematics of computation* 24, 647–656.
- Sobol, I.M., 1967. On the distribution of points in a cube and the approximate evaluation of integrals. *Zhurnal Vychislitel’noi Matematiki i Matematicheskoi Fiziki* 7, 784–802.
- Train, K., 2009. *Discrete Choice Methods with Simulation*. second edition ed., Cambridge University Press, Cambridge, MA.
- Train, K., Weeks, M., 2005. Discrete choice models in preference space and willingness-to-pay space, in: Scarpa, R., Alberini, A. (Eds.), *Application of simulation methods in environmental and resource economics*. Springer, Dordrecht. chapter 1, pp. 1–16.

- Vovsha, P., 1997. Application of a Cross-Nested Logit model to mode choice in Tel Aviv, Israel, Metropolitan Area. *Transportation Research Record* 1607, 6–15.
- Walker, J.L., Ben-Akiva, M., Bolduc, D., 2007. Identification of parameters in normal error component logit-mixture (neclm) models. *Journal of Applied Econometrics* 22, 1095–1125.
- Wen, C.H., Koppelman, F.S., 2001. The Generalized Nested Logit Model. *Transportation Research Part B* 35, 627–641.
- Williams, H.C.W.L., 1977. On the Formulation of Travel Demand Models and Economic Evaluation Measures of User Benefit. *Environment & Planning A* 9, 285–344.
- Yáñez, M.F., Cherchi, E., Heydecker, B., Ortúzar, J. de D., 2011. On the treatment of repeated observations in panel data: Efficiency of mixed logit parameter estimates. *Networks and Spatial Economics* 11, 393–418.

Index: *Apollo* syntax

This section provides an index covering each *Apollo* function and key lists/elements. Functions are differentiated by being followed by (). Elements of lists are preceded by \$. For each function or list, a link to a syntax example is given where available. We also provide links to the use of the four datasets.

apollo_attach()	25	apollo_cnl()	47
syntax example	26	\$cnl_settings	48
apollo_avgInterDraws()	86	\$alternatives	48
bayesian estimation	112	\$savail	48
syntax example	86	\$cnlNests	48
apollo_avgIntraDraws()	86	\$cnlStructure	48
bayesian estimation	112	\$componentName	48, 94
syntax example	86	\$utilities	48
apollo_basTest()	135	rows	48
syntax example	135	\$nl_settings	
apollo_beta	22	\$choiceVar	48
syntax example	23	syntax example	50
apollo_bootstrap()	132	apollo_combineModels()	100
\$bootstrap_settings	132	P in hybrid	100
\$nRep	132	syntax example	102
\$samples	132	apollo_combineResults()	155
\$seed	132	\$combineResults_settings	155
syntax example	133	\$estimateDigits	155
interruption	132	\$modelNames	155
apollo_choiceAnalysis()	125	\$pDigits	155
\$choiceAnalysis_settings	125	\$printClassical	155
\$alternatives	126	\$printPVal	155
\$savail	126	\$printT1	155
\$choiceVar	126	\$sortByDate	155
\$explanators	126	\$tDigits	155
\$rows	126	apollo_conditionals() ...	150, 151, 153
syntax example	126, 127	limitations	150
apollo_classAlloc()	92	syntax example for latent class ...	154
\$classAlloc_settings	92	syntax example for mixed logit ...	152
\$savail	92	apollo_control	19
\$classes	92	\$HB	20
\$utilities	92	\$calculateLLC	20
rows	92	\$indivID	20

\$modelDescr	20
\$nCores	20, 83
\$noDiagnostics	20, 104
\$noValidation	20
\$outputDirectory	20
\$panelData	20
\$seed	20
\$weights	20
\$workInLogs	20, 158, 164
syntax example	19
apollo_deltaMethod()	147
\$deltaMethod_settings	147
\$allPairs	148
\$expression	148
\$printPVal	148
\$varcov	148
syntax example	148
apollo_detach()	27
syntax example	26
apollo_dft()	56
\$dft_settings	56
\$altStart	56
\$alternatives	56
\$attrScalings	56
\$attrValues	56
\$attrWeights	56
\$avail	56
\$choiceVars	56
\$componentName	56
\$procPars	56
\$rows	56
syntax example	57
apollo_draws	83
\$interDrawsType	83
\$interNDraws	84
\$interNormDraws	84
\$interUnifDraws	84
\$intraDrawsType	83
\$intraNDraws	84
\$intraNormDraws	84
\$intraUnifDraws	84
Halton draws	83, 84
mlhs draws	83
pmc draws	83
Sobol draws	83
Sobol-Faure-Tezuka draws	83
Sobol-Owen draws	83
Sobol-Owen-Faure-Tezuka draws ..	83
syntax example	83
user-generated draws	84
apollo_drugChoiceData	
example application ..	62, 64, 107, 157, 158
apollo_el()	61
\$el_settings	61
\$alternatives	61
\$avail	61
\$choiceVars	61
\$componentName	61
\$rows	61
\$scales	61
\$utilities	61
syntax example	62, 64
apollo_estimate()	30
\$estimate_settings	30
\$bgw_settings	31
\$bootstrapSE	31, 165
\$bootstrapSeed	31
\$constraints	31
\$estimationRoutine	31
\$hessianRoutine	31
\$maxIterations	31, 165
\$maxLik_settings	31
\$numDeriv_settings	31
\$printLevel	31
\$scaleAfterConvergence	31
\$scaleHessian	32
\$scaling	32
\$silent	32
\$umDeriv_method	31
\$validateGrad	32
\$writeIter	32
syntax example	33
apollo_firstRow()	151
syntax example	152, 154
apollo_fitsTest()	143
\$fitsTest_settings	144
\$subsamples	144
syntax example	144
apollo_fixed	22
syntax example	23
apollo_fmnl()	59, 160
\$fmnl_settings	59
\$alternatives	59

\$avail	59	apollo_lrTest()	134
\$choiceVars	59	syntax example	134
\$componentName	59	apollo_mdcev()	68
\$rows	59	\$mdcev_settings	68
\$utilities	59	\$alpha	68
syntax example	60	\$alternatives	68
apollo_HB	110	\$avail	68
\$constraintNorm	112	\$budget	68
\$fixedA	112	\$componentName	68
\$fixedD	112	\$continuousChoice	68
\$gFULLCV	112	\$cost	68
\$gINFOSKIP	112	\$gamma	68
\$gNCREP	112	\$nRep	68
\$gNEREP	112	\$outside	68
\$hbDist	112	\$rawPrediction	69
excluded arguments	112	\$rows	69
syntax example	111	\$sigma	69
apollo_initialise()	19	\$utilities	69
syntax example	19	syntax example	70
apollo_lc()	93	apollo_mdcnev()	72
\$lc_settings	94	\$mdcnev_settings	72
\$classProb	94	\$alpha	72
\$inClassProb	94	\$alternatives	72
P	94	\$avail	72
P in hybrid	100	\$budget	72
syntax example	93	\$componentName	72
apollo_lcEM()	116	\$continuousChoice	72
\$lcEM_settings	116	\$cost	72
\$EMmaxIterations	117, 122	\$gamma	72
\$postEM	117, 122	\$mdcnevNests	73
\$silent	117, 122	\$mdcnevStructure	73
\$stoppingCriterion	117, 122	\$outside	73
syntax example	119, 120, 123	\$rows	73
apollo_lcPars()	91	\$utilities	73
lcPars	91	syntax example	74
\$beta	91	apollo_mixEM()	121
\$pi_values	91, 94	\$mixEM_settings	122
syntax example	91	\$transforms	122
apollo_llCalc()	129	syntax example	124
syntax example	131	apollo_mnl()	27
apollo_loadModel()	128	\$mnl_settings	27
apollo_longToWide()	14	\$alternatives	28
altColumn	15	\$avail	28
altSpecAtts	15	\$choiceVar	28
choiceColumn	15	\$componentName	28
idColumn	15	\$utilities	28
obsColumn	15	rows	28

syntax example	26, 42, 86, 91, 96, 102, 107
apollo_modeChoiceData	
example application	26, 45, 50, 53, 57, 102, 111, 126, 136, 139–141, 143, 144
apollo_modelOutput()	35
\$modelOutput_settings	35
\$printBHHH	36
\$printChange	36
\$printClassical	36
\$printCorr	36
\$printCovar	36
\$printDataReport	36
\$printFixed	36
\$printFunctions	36
\$printHBconvergence	36
\$printHBiterations	36
\$printModelStructure	36
\$printOutliers	36
\$printPVal	36
\$printT1	36
syntax example	38
apollo_nl()	45
\$nl_settings	45
\$alternatives	46
\$avail	46
\$choiceVar	46
\$componentName	46
\$nlNests	46
\$nlStructure	46
\$utilities	46
rows	46
syntax example	45
apollo_normalDensity()	66
\$normalDensity_settings	66
\$componentName	66
\$mu	66
\$outcomeNormal	66
\$sigma	66
\$xNormal	66
\$rows	66
syntax example	108
apollo_ol()	64
\$ol_settings	65
\$coding	65
\$componentName	65
\$outcomeOrdered	65
\$rows	65, 106
\$tau	65
\$utility	65
syntax example	107
apollo_op()	66
\$op_settings	66
apollo_outOfSample()	145
\$outOfSample_settings	145
\$nRep	145
\$rmse	145
\$samples	145
\$validationSize	145
syntax example	146
apollo_panelProd()	29, 161
bayesian estimation	112
syntax example	26
apollo_prediction()	136
\$prediction_settings	136
\$modelComponent	137
\$nRep	137
\$runs	137
\$simChoice	137
output	138
prediction variability	137
syntax example	139, 140
syntax example with simulated	
choices	141
syntax example with user provided	
parameters	141
apollo_prepareProb()	29
syntax example	26
apollo_probabilities()	25
closure	29
functionality	77
initialisation	25
model definition	27
syntax example	26
apollo_randCoeff()	84
syntax example	85
apollo_readBeta()	23
apollo_beta	23
apollo_fixed	23
inputModelName	23
overwriteFixed	23
syntax example	24
apollo_rrm()	

\$rrm_settings	51	apollo_sink()	125
\$alternatives	51, 52	apollo_speedTest()	98
\$avail	51, 52	\$speedTest_settings	98
\$choiceVar	51, 52	\$nCoresTry	98
\$choiceset_scaling	52	\$nDrawsTry	98
\$componentName	51, 52	\$nRep	98
\$regret_inputs	52	syntax example	99
\$regret_scale	52	apollo_swissRouteChoiceData	
\$rows	51	example application .86, 93, 119, 123,	
\$rum_inputs	52	130, 133, 146, 152, 154	
rows	52	apollo_swissRouteChoice	
apollo_saveOutput()	35	example application	127
\$saveOutput_settings	35	apollo_timeUseData	
\$saveCorr	37	example application60, 70, 71, 74	
\$saveCov	37	apollo_unconditionals()	149
\$saveEst	37	\$obsLevel	149
\$saveHBiterations	37	syntax example for latent class ...	154
\$saveModelObject	37	syntax example for mixed logit ...	152
\$saveOld	37	apollo_validateInputs()	24
\$writeF12	37	syntax example	25
apollo_searchStart()	128, 163	apollo_weighting()	20, 30
\$searchStart_settings	129	functionality	
\$apolloBetaMax	129	estimate	77
\$apolloBetaMin	129	output	77
\$bfgsIter	129	utilities	136
\$dTest	129	syntax example	136
\$gTest	129	validate	77
\$llTest	129	zero_LL	77
\$maxStages	129	modelName	
\$nCandidates	129	\$modelName	20
\$smartStart	129	model	34
syntax example	130	\$estimates	34
apollo_setWorkDir()	19	\$robvarcov	34
syntax example	19	\$varcov	34
apollo_sharesTest()	142	syntax example	33
\$sharesTest_settings	142	print(model)	39
\$alternatives	142	syntax example	40
\$choiceVar	142	P	27, 29
\$modelComponent	142	joint models	100
\$newAltsOnly	142	syntax example	26
\$newAlts	142	summary(model)	39
\$subsamples	142	syntax example	41
syntax example	143		